



Cloud Computing

AA – 23/24

Saverio Mosti & Gabriele Pianigiani

Questo file è stato creato utilizzando appunti presi durante le lezioni, potrebbe contenere inesattezze; perciò, si consiglia di utilizzare questo documento come supporto e non come sostituto delle lezioni e del materiale fornito dai docenti. Buono studio!!!

Sommario

Capitolo 1 - Introduzione	9
Applicazioni distribuite.....	9
Sistemi di computer distribuiti	10
Cluster computing	10
Grid computing.....	11
Tecnologie Web-Based.....	12
Architettura orientata al server - SOA	12
Modello di Utility Computing.....	12
Capitolo 2 - Cloud computing	13
Definizione di Cloud Computing.....	13
Infrastruttura cloud	13
Servizio Cloud – XaaS	13
Cloud Business	14
Service Cloud Model	16
Vecchio approccio – Il modello Multi-User	17
Multi-Tenancy	17
Virtualizzazione	18
Vantaggi del cloud computing	23
Cloudification	24
Casi d'uso - Expedia	24
Casi d'uso - Startups.....	24
Casi d'uso - Sporadiche computazioni ad alte performance [HPC].....	24
Servizi in scala globale.....	24
Casi d'uso - Google search	24
Casi d'uso - Data collection.....	25
Casi d'uso - Content distribution	25
Sfide, rischi e contro della cloudification	25
Capitolo 3 – Modello NIST	27
Definizione di cloud computing	27
Caratteristiche fondamentali	27
Tipologia di servizio - Service Delivery Model.....	28
IaaS - Infrastructure as a Service.....	29
PaaS - Platform as a Service	29
SaaS - Software as a Service	30

Implementazione PaaS e SaaS	31
Modello di distribuzione - Deployment model.....	33
Capitolo 4 – Tecniche di virtualizzazione.....	35
Virtualizzazione delle risorse.....	36
Risorse non opzionali.....	36
Risorse opzionali.....	36
Recap di Multiprogrammazione	37
Cambio di contesto	37
Livelli di privilegio	38
Memoria virtuale.....	38
Interruzioni ed Eccezioni	41
Hypervisor – Virtual Machine Monitor - VMM.....	42
Hosted Approach	43
Bare Metal Hypervisor.....	43
Tecniche di virtualizzazione	45
Capitolo 5 - Full Virtualization.....	46
Emulazione delle istruzioni.....	46
Modello “Trap&Emulate”	48
Virtualizzazione della RAM	52
La virtual MMU e il GH Mapping	53
Il metodo “brute force” – Shadow page table	55
Virtualizzazione dei dispositivi I/O	58
Gestione delle richieste di interruzione dei dispositivi fisici	59
QEMU – Quick EMUlator	61
LAB - QEMU and Virsh	61
Capitolo 6 - Hardware Assisted Virtualization.....	62
Intel VMX	62
Operazioni di Intel VMX	64
Interrupt Remapping Table	66
Utilizzo della IRT	67
Virtual Machine Control Structure – VMCS	69
RAM Management.....	71
Tabella delle pagine estesa	71
I/O Management	73
Hardware passthrough.....	73
Interruzioni	75

Stati di una macchina virtuale	75
Posted Interrupt Descriptor Table (PID)	77
Dispositivi DMA	81
IOMMU	81
Virtualizzazione annidata (<i>nested virtualization</i>)	83
KVM – Kernel-Based Virtual Machine	84
Capitolo 7 – Paravirtualization	86
Hypercalls	87
Xen	89
Capitolo 8 - Lightweight Virtualization	90
Approccio di Linux - Containers	91
Namespace	92
Cgroups - Control groups	92
LAB - Docker	93
Architettura di Docker	93
Docker Hub	94
Processo di esecuzione di Docker	95
Capitolo 9 - Cloud Applications	97
Applicazioni Cloud Native	98
Requisiti di una applicazione cloud	98
Applicazione Two-Tier	99
Applicazione Multi-Tier	100
Architettura Service-Oriented (SOA)	102
Comunicazione tra i moduli	102
SOA Roles	103
Servizi Web	103
Web Service Description Language - WDSL	105
Simple Object Access Protocol - SOAP	108
REST – Representational State Transfer	110
Compute Cluster	113
High Availability Cluster – HAC	113
Load-Balancing Cluster - LBC	114
CIC - Compute intensive cluster	117
Modulo di frontend e di backend	119
Comunicazione diretta	119
Comunicazione indiretta	121

Sistema Publish-Subscribe	122
Sistema a coda di messaggi	123
LAB - RabbitMQ	127
Workflow di RabbitMQ	128
Tipologie di exchange	128
Replicazione dei dati	133
Modello del sistema	133
Comunicazione multicast	136
Primary-Backup replication	137
Active replication	138
Architettura gossip	139
Capitolo 10 - Bilanciamento globale del carico	144
Bilanciamento globale con DNS	144
DNS localizzato	145
VIP - Indirizzi IP virtuali (<i>Any Cast address</i>)	147
GRE - Incapsulamento dei pacchetti IP	148
Capitolo 11 - Sistemi geogr. distribuiti	149
Le sfide sulle reti su larga scala	149
Teorema CAP	150
Sistemi BASE	151
Architettura “ <i>Bayou</i> ”	153
Capitolo 12 - Piattaforme Cloud	156
Architettura di una piattaforma cloud	157
OpenStack	159
Moduli	160
Comunicazione tra moduli	163
Keystone	164
Nova	164
Glance	168
Neutron	170
OpenVSwitch - OVS	173
Network Node	176
Virtualizzazione della rete	177
VXLAN – Virtual eXtensible LAN	181
Cinder	183
Lab - OpenStack	186

Flavor	189
Capitolo 13 - Software development	199
Approccio Monolitico	199
Approccio a servizi	200
Team dev	200
Team ops	200
DevOps	201
NoOps	202
Service dependencies	203
Kubernetes	204
Utilizzo di Kubernetes	204
Kubernetes Cluster Architecture	206
Kubernetes - Application Deployment	207
Kubernetes - Gestione e manutenzione	209
Networking tra container	210
Autoscale	211
LAB – Kubernetes	212
Capitolo 14 - Cloud Storage	219
Block storage	220
Object storage	221
RAID – Redundant Array of Independent Disks	222
RAID 0	222
RAID 1	223
RAID 5	223
RAID 10	223
DFS - File System Distribuito	224
NFS – Network File System	224
GlusterFS	225
SAN - Storage Area Network	226
Ceph – Software Storage Solution	228
Architettura di Ceph	229
Algoritmo CRUSH	232
Piazzamento di un oggetto	233
Pools	234
Ridondanza dei MON	235
Calcolo dei Placement Groups	235

Recovering e Rebalancing	236
Ceph e OpenStack.....	236
STaaS – Storage as a Service.....	237
Laboratorio 0 – Architetture Parallele	238
Task Dependency Graph	239
Esempio	239
Granularità dei task.....	241
Laboratorio 1 - MapReduce.....	244
Parallelizzare i processi	244
Il framework di MapReduce	247
MapReduce Programming model.....	249
Hadoop	252
Design patterns per MapReduce	254
In-Mapper Combining.....	254
Pairs and stripes	256
Schemi per operazioni ad algebra relazionale	258
Stage chaining	260
Schema per moltiplicazione tra matrice e vettore.....	260
Schema per moltiplicazione tra matrice e matrice.....	261
Laboratorio 2 - Hadoop.....	263
HDFS – Hadoop Distributed File System	264
Architettura di HDFS	265
Lettura nel HDFS	266
Scrittura nel HDFS	267
Replicazione nel HDFS	270
Hadoop 1.0	271
Hadoop 2.0 – YARN - Yet Another Resource Negotiator.....	272
ResourceManager (RM).....	273
NodeManager (NM)	274
ApplicationMaster (AM).....	274
Esecuzione di un'Applicazione.....	276
Lo scheduler di YARN.....	277
Delay scheduling.....	279
Tolleranza ai guasti e ai fallimenti	281
Esecuzione speculativa.....	282
Shuffle and Sort	282

I limiti di MapReduce.....	283
Laboratorio 3 – Spark	284
Resilient Distributed Dataset – RDD	285
Lineage	286
Architettura di Spark.....	287
Processo Driver.....	288
Processo Esecutore	289
Modello di programmazione di Spark.....	289
Data flow – DAG	290
Trasformazione di un RDD.....	291
Action	293
Spark Job.....	299
Spark stage	300
Spark task	301
Persistenza degli RDD	302
Laboratorio 4 - Interfaccia REST	303
Swagger.....	304
Funzionalità e Componenti di Swagger.....	305
Utilizzo di Swagger nel Cloud Computing	306
Esempio di Specifica OpenAPI	307
Apache Tomcat.....	308
Schierare una interfaccia REST	308

Capitolo 1 - Introduzione

Una breve introduzione riguardo gli argomenti del corso.

Applicazioni distribuite

Applicazioni in cui il calcolo è distribuito tra più macchine al fine di aumentare la capacità, velocità e scalabilità dell'applicazione stessa. Per aumentare la capacità, intendiamo il diminuire l'effetto collo di bottiglia che può generare il calcolo centralizzato su un singolo nodo.

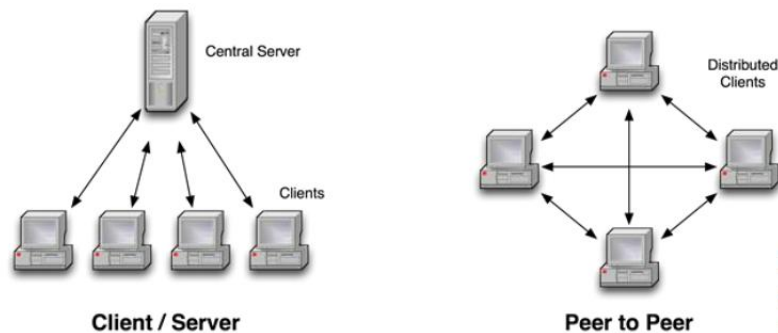


Figure 1 - Le due tipologie di applicazione più diffuse.

Parallel processing e parallel computing

La processazione in parallelo fu introdotta negli anni 80 e inizialmente con lo scopo di aumentare la capacità computazionale di un singolo PC: *più processori significa più programmi contemporaneamente*. Tuttavia, in seguito qualcuno ha pensato che potesse essere una buona idea far lavorare due PC sullo stesso compito allo stesso momento per completarlo in meno tempo.

Reti

Inizialmente la rete era una semplice connessione di pochi kilobyte. La rete è diventata essa stessa un collo di bottiglia.

Sistemi di computer distribuiti

Le applicazioni sono scritte in maniera distribuita, ossia come set di task indipendenti e quindi parallelizzabili.

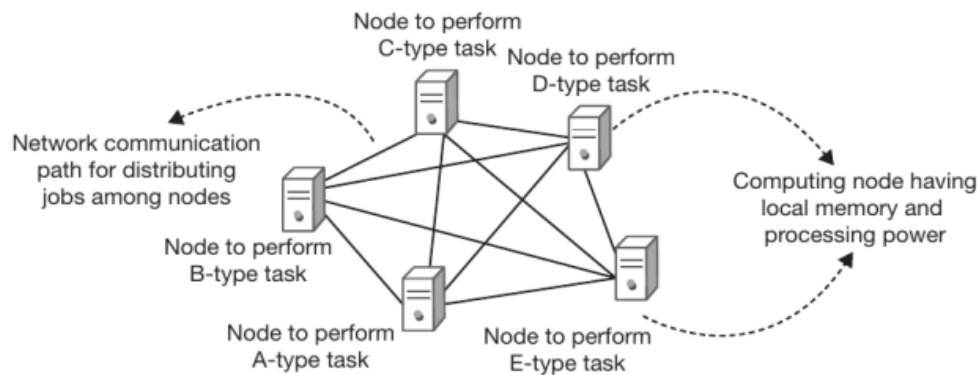


Figure 2 - Digramma di un sistema distribuito.

Cluster computing

Una evoluzione del calcolo distribuito: in questo approccio i computer sono raggruppati in un unico punto. Il cluster dall'esterno appare come un unico grande computer (*ma che in realtà è l'unione di tanti piccoli computer*).

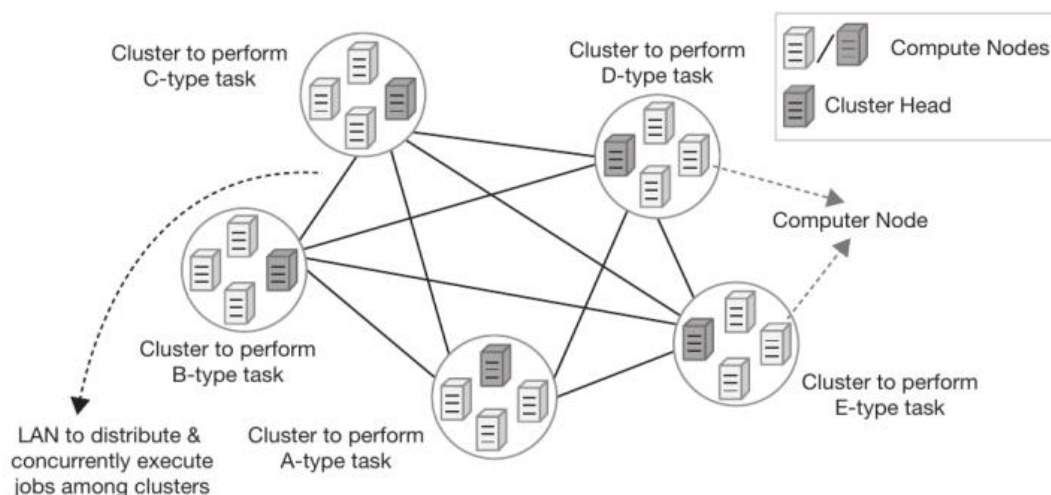


Figure 3 - Diagramma semplificato di un sistema che sfrutta il Cluster computing.

Da qui in avanti possiamo immaginare ogni cluster come un unico computer; quindi, ogni cluster ha lo scopo di eseguire un tipo specifico di task.

Vantaggi del clustering

Il clustering porta la ridondanza, ossia l'affidabilità: Se un computer del cluster (*non master*) si guasta, il cluster continua a funzionare correttamente ma con prestazioni leggermente degradate

Organizzazione di un cluster

I cluster sono solitamente organizzati in modo gerarchico: Dato un sistema di vari cluster, ne esiste uno speciale chiamato **cluster master**.

1. Il cluster master riceve dall'esterno un batch di task da compiere
2. Il cluster master (*secondo una sua logica*) inoltrerà i task ai vari sotto-cluster.
3. In ogni sotto-cluster è presente un cluster head (*ossia un computer speciale che si occupa di coordinare il cluster*); esso riceve il task dall'esterno e (*secondo una sua logica*) lo inoltra ai vari computer del cluster.

Il cluster master e il cluster head rappresentano dei single point of failure

Grid computing

I PC sono ancora divisi in cluster in base al tipo di task che devono eseguire, ma non c'è un cluster master o cluster head: i computer di ogni cluster si coordinano tra loro per decidere quale macchina eseguirà il task.

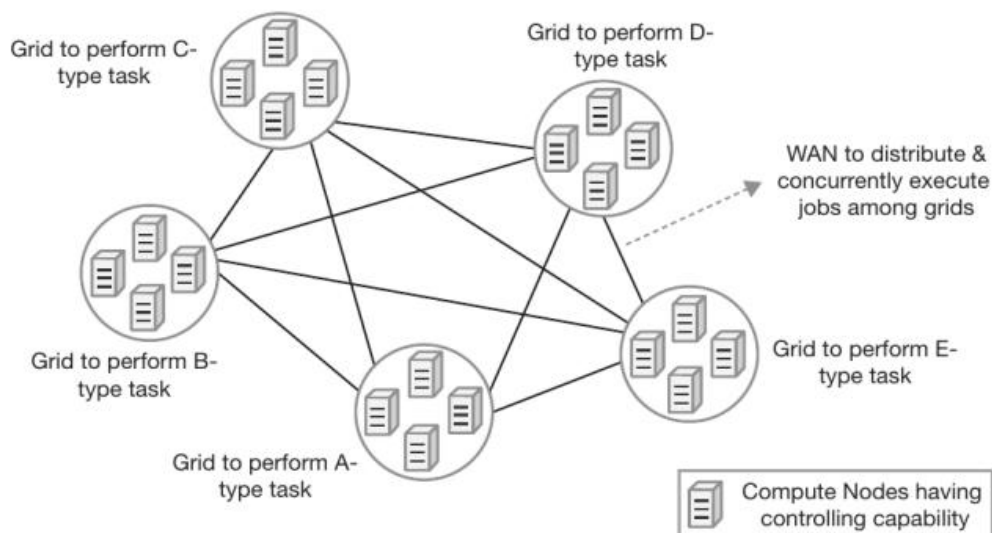


Figure 4 - Diagramma semplificato di un sistema che sfrutta il Grid computing.

Solitamente i PC all'interno del cluster sono collegati con una LAN ad alta velocità.

È più scalabile orizzontalmente rispetto al cluster computing (perché il master non può essere utilizzato per un numero infinito di PC).

Tuttavia, questo approccio non è sufficiente, presenta comunque molti inconvenienti:

- Per aggiungere una nuova macchina devo cambiare l'architettura.
- Il sistema non è fault tolerant: il guasto di un nodo può causare il guasto del sistema.
- L'hardware eterogeneo può essere un problema.

Tecnologie Web-Based

Le applicazioni Web possono essere distribuite su server molto diversi (*in hardware e software*) e devono fornire il servizio a utenti di tutto il mondo.

Architettura orientata al server - SOA

Tecnologia di progettazione software nata per realizzare applicazioni cloud: Per prima cosa scomponiamo il software principale in componenti più piccoli. Ogni componente è un servizio diverso che può essere implementato e distribuito in modo diverso. All'epoca questo modello era necessario, perché le applicazioni cloud sono molto complesse e devono gestire i big data.

Modello di Utility Computing

A un certo punto abbiamo avuto:

- Infrastruttura di elaborazione scalabile
- Virtualizzazione
- Ambiente di calcolo distribuito su diversi computer collegati tramite LAN veloce.
- SOA: Quindi anche il software è stato diviso in parti più piccole.
- Regole di programmazione.

Capitolo 2 - Cloud computing

Definizione di Cloud Computing

La definizione precisa di cloud computing è: *Un ambiente IT distinto progettato per fornire risorse **scalabili e misurate** accessibili tramite Internet.*

Tali risorse sono ad esempio: RAM, CPU, Hard Disk, etc.

Il cloud computing al giorno d'oggi è una parola che vuol dire tutto e niente come “intelligenza artificiale”.

Ad oggi il termine cloud è usato per riferirsi a due concetti diversi (*entrambi corretti*):

Infrastruttura cloud

L'infrastruttura cloud fornisce le risorse hardware e software utilizzate per implementare i servizi cloud a cui possono accedere più utenti contemporaneamente.

L'infrastruttura cloud è implementata in un data center (ossia uno spazio fisico dedicato per ospitare server e apparecchiature di rete).

Gli utenti possono accedere ai servizi cloud tramite la connessione internet, ad esempio utilizzando un browser (quindi protocollo HTTP/HTTPS).

Servizio Cloud – XaaS

Applicazione (software) eseguita da qualche parte nel mondo su un'infrastruttura cloud e i risultati sono forniti all'utente utilizzando Internet. Un servizio cloud può anche essere utilizzato da altri servizi cloud tramite un'apposita interfaccia.

I servizi cloud implementano un'arbitrariamente ampia gamma di funzioni; “*arbitrariamente ampia*” perché tutto può essere erogato come servizio seguendo questo paradigma.

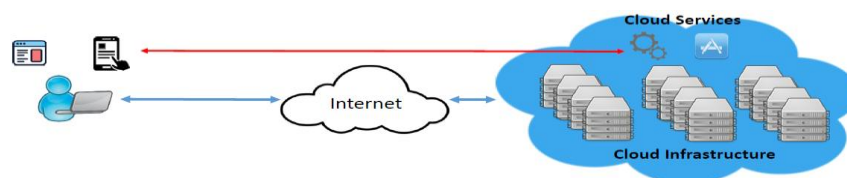


Figure 5 – Schema generale di uno XaaS

Esempio XaaS - Servizio di archiviazione file

L'infrastruttura cloud viene sfruttata per le sue risorse di archiviazione (quindi il disco, ossia la memoria persistente). L'utente può utilizzare le interfacce web (fornite dall'applicazione o

utilizzando il semplice browser) per caricare o scaricare i file. L'utente ha i file sia nel proprio dispositivo che nel cloud, tra questi file c'è un meccanismo di sincronizzazione.

L'utente paga per accedere all'archiviazione e il pagamento può essere fisso e mensile (come Google drive) oppure in base a quanta memoria il cliente sta occupando.

Esempio XaaS - Cloud ERP Service

L'ERP (*Enterprise Resource Planning*, è un software di gestione aziendale) viene implementato nel cloud e i dipendenti vi accedono tramite un'interfaccia web o un client dedicato da qualsiasi parte del mondo.

Cloud Business

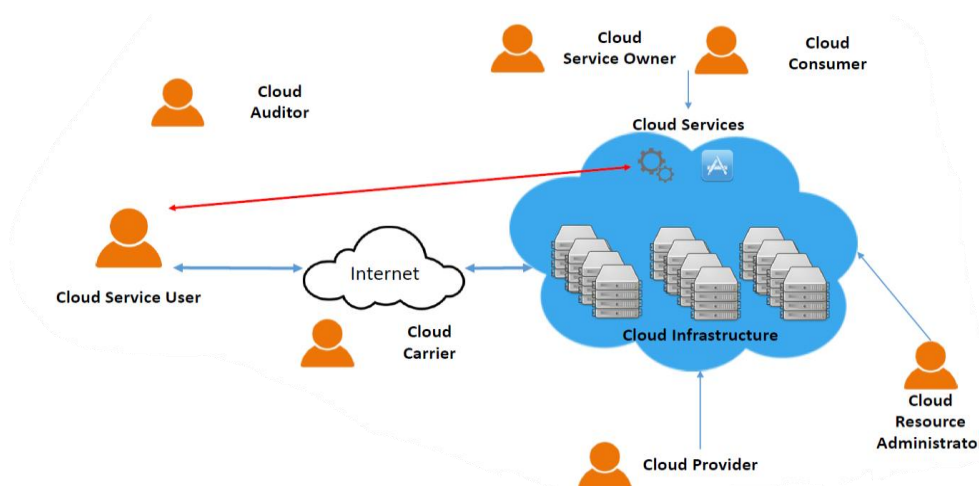


Figure 6 - Diagramma di un cloud business.

Cloud Provider

L'azienda che ha fatto il primo investimento; ossia quella che ha contribuito nella **creazione dell'infrastruttura cloud**. Vuole utilizzare la quantità minima di risorse per ogni servizio, nulla può essere sprecato.

Cloud Consumer

Il cliente del cloud provider, ossia chi acquista e accede alle risorse informatiche messe a disposizione dal cloud provider per offrire un servizio.

Il consumatore cloud può essere una persona, ma di solito è un'altra azienda che sfrutta tali risorse per creare uno o più servizi cloud.

Cloud Service Owner

Qualcuno che crea un servizio cloud in cima all'infrastruttura. Di solito questa figura e il cloud consumer sono la stessa cosa.

Cloud Service User

Il cliente del cloud consumer; anche detto “*consumatore finale*”.

Esso sfrutta l'accesso alle applicazioni e alle risorse cloud fornito dal proprietario del servizio cloud in cambio di un compenso monetario.

Cloud carrier

Ente terzo rispetto al servizio cloud. In pratica è colui che fornisce il servizio di connettività all'infrastruttura cloud: connette il data center al resto del mondo (*Internet*); ovvero lo ISP (*Tim, Vodafone, etc*).

Amministratore delle risorse cloud

Solitamente è un'entità interna al cloud provider (*nel cloud carrier*) gestisce le risorse dell'infrastruttura cloud.

Cloud Auditor

Ente terzo rispetto al servizio cloud. Il suo compito è quello di certificare la qualità del servizio e che tutto funzioni senza intoppi e che ogni servizio ottenga la giusta quantità (*accordata al contratto*) di risorse.

Service Cloud Broker

Figura opzionale che si interpone tra uno o più cloud providers e il consumatore.

Il broker compra le risorse dai cloud providers e le fornisce ai clienti. Il broker permette una comunicazione tra cliente e cloud provider e permettendo anche costi minori (*il broker affitta una grande quantità di risorse*).

Service Cloud Model

Il sistema usato dall'azienda proprietaria dell'infrastruttura cloud per fare profitto. Tutto si basa sulla **quantità risorse offerte** dal fornitore di servizi cloud.

In pratica, questo si traduce in due caratteristiche:

- Modello basato sull'utilità.
- Provisioning dinamico.

Modello basato sull'Utilità

Le risorse vengono offerte al proprietario del servizio cloud in cambio di un compenso monetario.

La fattura viene calcolata con un modello di prezzo detto “pay-per-use”: il proprietario del servizio cloud paga in modo direttamente proporzionale alla quantità di risorse che sta utilizzando al momento.

Per fare ciò è necessario misurare l'utilizzo delle risorse in ogni momento (*elaborazione, rete, RAM e storage*); per questo le risorse devono essere “*misurate*”.

Provisioning dinamico

Il provisioning delle risorse (*assegnamento, dislocamento, etc*) deve essere eseguito dinamicamente dal provider di servizi cloud al proprietario del servizio cloud.

La creazione delle risorse deve essere eseguita in automatico dall'infrastruttura cloud senza l'intervento umano; quindi, abbiamo anche bisogno di un sistema automatico.

Pertanto, se il proprietario del servizio cloud (*cliente*) richiede più risorse: basta che paghi e vengono assegnate dinamicamente in un breve lasso di tempo.

Questo meccanismo deve essere progettato per funzionare non solo su richiesta di un essere umano, ma anche sotto l'orchestrazione di un software (*l'infrastruttura cloud come vedremo deve fornire un'interfaccia per il provisioning delle risorse sia agli esseri umani che alle macchine*).

I fornitori di servizi cloud vendono risorse ad altre aziende, queste aziende utilizzano le risorse per fornire servizi ai propri clienti.

Vecchio approccio – Il modello Multi-User

In un sistema multi-user l'hardware di un server deve essere condiviso da più utenti contemporaneamente.

Il provider vuole vendere le proprie risorse a più clienti per massimizzare il guadagno: in pratica vendono la stessa infrastruttura a molteplici clienti, al costo di un degrado di prestazioni ovviamente.

Tutti gli utenti condividono la stessa macchina: stesso hardware, stesso sistema operativo, stesse impostazioni, etc.

Approccio multi-user nelle reti tradizionali

Le infrastrutture di rete tradizionali sono multi-user, il quale è un metodo usato per permettere a molteplici utenti di condividere il medesimo hardware e quindi permettere a due utenti di accedere ad una risorsa allo stesso momento.

C'è la necessità di un amministratore e gli altri utenti hanno un account limitato, hanno risorse e funzionalità limitate, accedono al sistema remoto, sono pienamente consapevoli che il sistema non è loro.

Multi-Tenancy

L'approccio multi-user non va più bene perché ogni cloud service provider ha bisogno di uno specifico sistema operativo e di una specifica configurazione.

Serve un sistema dove gli utenti di un cloud service provider accedono ad un sistema totalmente configurato dall'amministratore del cloud service provider.

Ma non conviene (*economicamente*) assegnare un server ad ogni server provider (*tenant*), per questo motivo è stata inventata la Multi-Tenancy, la quale rende l'infrastruttura molto più scalabile.

Definizione di Multi-Tenancy

Un'architettura in cui un'unica istanza di un software applicativo serve più distinti.

Ogni tenant opera con una porzione virtualmente separata dell'applicazione, ma condivide l'hardware con altri tenant sullo stesso server o cluster di server.

Questo modello offre una serie di vantaggi e caratteristiche chiave:

Condivisione delle risorse: Le risorse hardware e software sono condivise tra più tenant, aumentando l'efficienza e riducendo i costi operativi. Ogni tenant accede alle stesse risorse fisiche, come server, storage e reti, ma in modo isolato e sicuro.

Isolamento e sicurezza: Anche se i tenant condividono le stesse risorse fisiche, i loro dati e configurazioni sono mantenuti separati. I meccanismi di sicurezza e isolamento garantiscono che un tenant non possa accedere ai dati o interferire con le operazioni di un altro tenant.

Scalabilità e gestione centralizzata: Le applicazioni multi-tenant possono scalare facilmente per supportare un numero crescente di tenant senza richiedere modifiche significative all'infrastruttura. La gestione e la manutenzione sono centralizzate, semplificando gli aggiornamenti e le patch.

Personalizzazione limitata: Sebbene i tenant condividano la stessa applicazione, possono avere alcune opzioni di configurazione personalizzabili. Tuttavia, queste personalizzazioni sono generalmente limitate rispetto a un'architettura a tenant singolo, dove ogni cliente può avere un'istanza completamente separata del software.

Economia di scala: La condivisione delle risorse e la gestione centralizzata riducono i costi operativi e di manutenzione, permettendo ai fornitori di cloud di offrire servizi a prezzi più competitivi.

Virtualizzazione

La multi-tenancy si ottiene tramite la virtualizzazione dell'hardware, ossia mettendo un layer tra il cloud service owner e l'hardware.

La virtualizzazione crea una versione virtualizzata di una risorsa: CPU, RAM, disco, programmi, etc... se mettiamo assieme queste tre risorse virtuali otteniamo un server virtuale.

In pratica, da un server fisico possono ottenere molteplici server virtuali: ognuno con la sua CPU, RAM, disco e sistema operativo (*installato dal cliente stesso*) virtuali.

I clienti useranno i server virtuali come se fossero veri server fisici dedicati solo a loro, non accederanno alle risorse fisiche, ma accedono solo a quelle virtualizzate le quali funzioneranno allo stesso modo.

Per raggiungere questo obiettivo viene utilizzata la virtualizzazione dell'hardware, è il metodo per creare un server virtuale che viene fornito agli utenti ed è uguale al server reale, anche il sistema operativo può essere virtualizzato e si chiama **full operating system**, ogni ambiente creato tramite la virtualizzazione del sistema è chiamato **macchina virtuale**, è un ambiente informatico composto da diverse tecniche di virtualizzazione. È possibile eseguire più macchine virtuali sullo stesso hardware.

Hypervisor

Chiamati anche Virtual Machine Manager (VMM).

Essi sono appositi (*e molto complicati*) software come VirtualBox che implementano un livello di virtualizzazione dove diverse tecniche di virtualizzazione vengono implementate per creare un intero sistema operativo, in sostanza creano una macchina virtuale (*CPU, RAM e disco*

virtuali) in cui viene eseguito il **sistema operativo guest**. Al contrario, il sistema che ospita l'hypervisor è detto **sistema operativo host** (*nel caso ne fosse installato uno, spesso viene messo solo l'hypervisor*).

Il business model si basa sull'assegnare questi server virtuali ai service providers invece che le macchine virtuali.

Questo approccio è anche molto più scalabile, perché le macchine virtuali possono essere create dinamicamente.

Tecniche di virtualizzazione

La differenza tra le tre è il livello di astrazione e quindi il livello di overhead introdotto dalla virtualizzazione.

- **Full virtualization**: viene creato un ambiente che assomiglia ad un server completo.
- Para-virtualization.
- Operating system virtualization.

Queste tecniche sono differenti in approccio e overhead.

Ogni server in un data center esegue un **hypervisor**, e le macchine virtuali vengono create, il modello di business consiste nel fornire ai clienti le macchine virtuali e non l'hardware fisico a cui non accede nessun utente. Le macchine virtuali possono essere create quando un nuovo customer ha bisogno di risorse per creare una nuova applicazione, e l'hypervisor crea nuove macchine virtuali.

Il servizio è esposto in internet tramite la LAN:

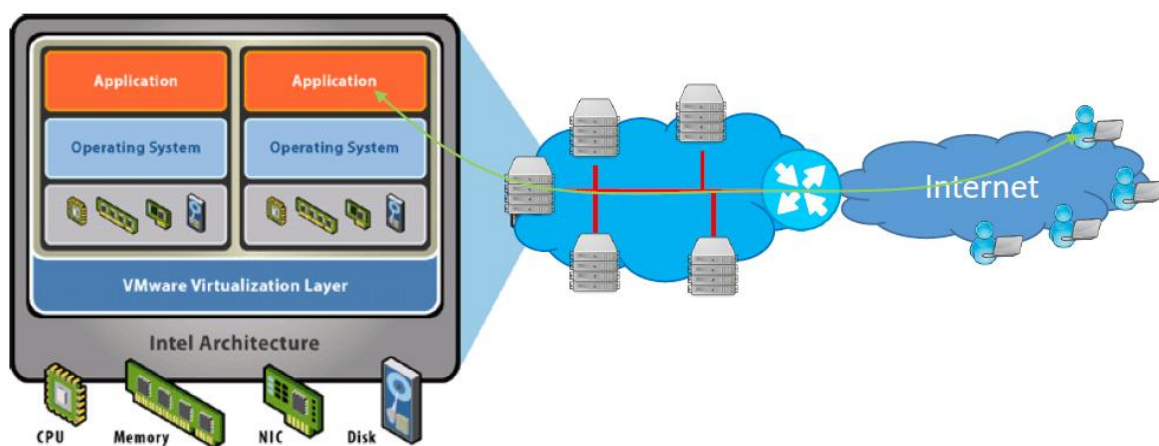


Figure 7 - Virtualizzazione

Tecniche di scalabilità

In ambiente cloud sono possibili due meccanismi di scalabilità:

Horizontal Scaling	Vertical Scaling
less expensive (through commodity hardware components)	more expensive (specialized servers)
IT resources instantly available	IT resources normally instantly available
resource replication and automated scaling	additional setup is normally needed
additional IT resources needed	no additional IT resources needed
not limited by hardware capacity	limited by maximum hardware capacity

Figure 8 - Confronto tra scaling orizzontale e verticale.

Horizontal scaling

Horizontal scaling consiste nell'aggiungere e rimuovere macchine virtuali. Quando il carico è troppo per una sola macchina virtuale, allora ne viene creata una nuova. Questo approccio di scaling rende l'applicazione più complessa, poiché è necessario modificarla per permettere una computazione distribuita su più macchine.

Vertical scaling

Vertical scaling riguarda l'aggiunta di nuove risorse alle macchine virtuali esistenti, quando aumentiamo la capacità lo indichiamo con scaling up, quando diminuiamo la capacità lo indichiamo con scaling down. Lo scaling verticale permette di migliorare le performance senza cambiare la logica dell'applicazione.

Esempio - SmartQuotes

L'idea imprenditoriale è risolvere delle frasi come servizio, gli utenti chiedono una frase al server e il sistema fornisce la risposta.

Gli utenti per solo 1 \$ al mese premono un pulsante e possono avere tutte le citazioni che vogliono.

Inizialmente l'azienda pensava che per implementare questo servizio in locale con due server, uno per esporre l'interfaccia agli utenti e uno per il database per archiviare le citazioni.

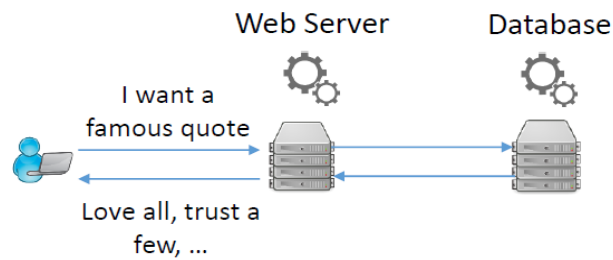


Figure 9 - Idea iniziale di SmartQuotes.

Supponiamo che un influencer abbia pubblicato su Instagram che questo servizio è fantastico (*Ferragni non è l'esempio più appropriato al momento, ma supponiamo che è accaduto*), molte persone iniziano ad abbonarsi servizio e invece di 10 utenti ci sono 10000 utenti, l'infrastruttura è la stessa, un server web e un server database, erano ok per 10 utenti, ma ora non lo sono.

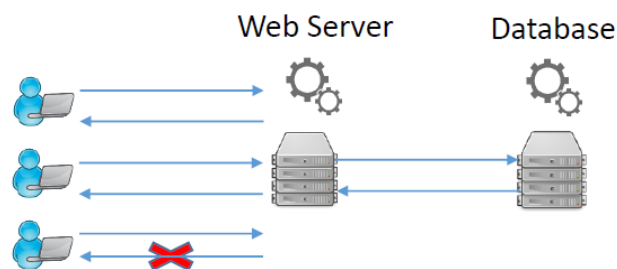


Figure 10 - Aumentano gli utenti e le richieste, il sistema va in saturazione.

Cosa succede se il sistema riceve tutte le richieste? Alcune di esse vengono ignorate dal sistema oppure il sistema mette le frasi in coda, ma in questo caso sarebbe troppo lento.

Quindi, in questo caso l'azienda assume due nuovi ingegneri e acquista due server aggiuntivi, e impiega 1/2 mesi per aggiornare il sistema, ma nel frattempo gli utenti si disiscrivono.

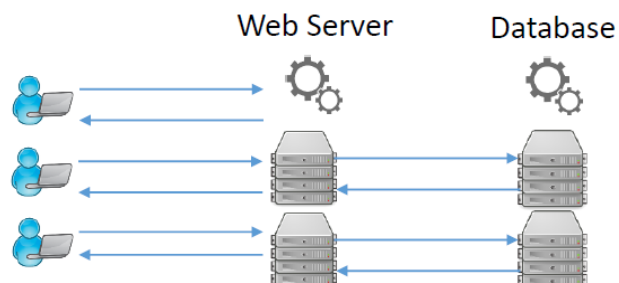


Figure 11 - Scaling orizzontale.

Cloud computing presenta vantaggi soprattutto perché è scalabile e utility-based, la scalabilità può aiutare ad aggiungere nuove risorse alla tua infrastruttura virtuale in pochi minuti senza uno sforzo significativo sugli investimenti nell'infrastruttura perché non ne hai una; quindi, scalabilità significa che puoi ottenere su richiesta più risorse di elaborazione.

Il secondo vantaggio è che è a consumo (*metered*), ovvero utility-based, alla fine del mese l'azienda riceve una fattura da pagare proporzionale al numero di utenti nel sistema.

Ora torniamo all'esempio precedente, i cloud forniscono un'elevata scalabilità che può essere configurata dinamicamente in modo automatico, potresti avere un software all'interno del servizio smart Quote che monitora la quantità di richieste che hai e se la quantità di richieste va sopra una certa soglia si può fare un provisioning dinamico in modo automatico senza intervento umano, se il numero di utenti diminuisce l'azienda può rimuovere alcune macchine virtuali e avere un'infrastruttura proporzionale al numero di utenti.

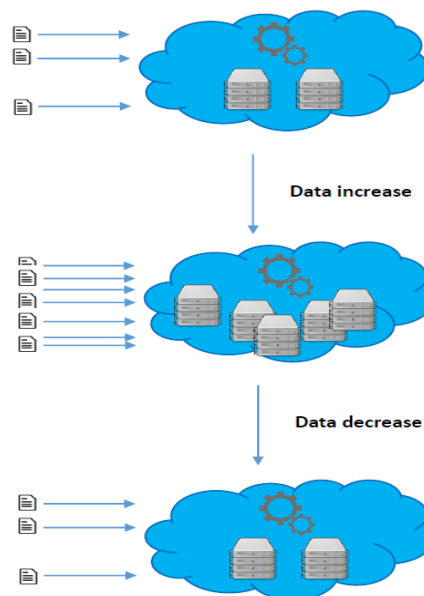


Figure 12 - Come la soluzione cloud si adatta al carico corrente.

Il secondo vantaggio è che non ci sono costi fissi, non c'è un'infrastruttura; quindi, il costo fisso è rimosso o è ridotto notevolmente, non serve un investimento iniziale perché non serve un'infrastruttura iniziale.

I costi variabili diminuiscono a loro volta: non hai il costo di manutenzione, perché l'infrastruttura non è tua; quindi, non hai bisogno di personale specializzato per mantenere e gestire l'infrastruttura.



Un altro vantaggio è la riduzione dei costi, per creare e fornire un servizio agli utenti un'azienda ha bisogno di risorse IT, se l'azienda acquista queste risorse da un fornitore cloud, hanno un costo, ma il prezzo è significativamente inferiore al costo che l'azienda avrebbe se avesse creato una propria infrastruttura per tre motivi:

- il fornitore di servizi cloud acquista hardware IT in blocco perché riduce i costi;
- l'infrastruttura è condivisa, il proprietario del servizio cloud è locatario dell'infrastruttura, le risorse fisiche sono condivise tra più utenti, nessuna risorsa viene sprecata, questo aiuta a ridurre il costo complessivo per risorsa;
- il fornitore di servizi cloud dispone di personale per gestire l'infrastruttura, il costo del personale è condiviso tra gli utenti.

Vantaggi del cloud computing

- **Minore responsabilità gestionale:** Le macchine sono dal cloud provider, quindi è sua responsabilità gestirle, ripararle, aggiornare, potenziare etc.
- **Minore responsabilità legale:** riguardo la gestione dei dati dei clienti, poiché “scarico” la responsabilità sul fornitore dell'infrastruttura cloud. Il cloud provider deve applicare tutte le misure di sicurezza, ridondanza in modo da proteggere la privacy dei clienti.
- **Qualità del servizio più elevata:** Il cloud provider ha il compito di creare e mantenere una grande e costosa infrastruttura; quindi, la qualità dell'infrastruttura (*hardware e software*) e la qualità delle persone che ci lavorano sono una priorità primaria.
- **Affidabilità maggiore:** I cloud provider hanno capacità gestionale infinitamente superiore rispetto ad un server “in casa”. Il sistema continuerà a funzionare correttamente anche in caso di guasti, disastri o altro.
- **Disponibilità maggiore:** I cloud provider sono attivi 24x7, quindi un servizio può rimanere attivo sempre.
- **Gestione software semplificata:** L'acquisto di licenze per software complessi (e costosi) è obbligatorio nella creazione di un'infrastruttura cloud. Usare un servizio cloud ti permette di non comprare queste licenze perché fornite in primo luogo dal cloud provider.
- **Indipendente dalla locazione:** Non mi importa dove siano localizzati i server, molto probabilmente non saprò mai dove sia il data center.

Cloudification

Concetto che si riferisce alla transizione dal local computing verso il cloud computing.

Le aziende spostano i loro servizi in cloud con pochissime modifiche (*moving as it is*) del codice (*semplicemente si cambia l'URL o l'indirizzo IP*).

Casi d'uso - Expedia

La cloudification punta a ridurre i costi fissi dovuti ad una infrastruttura locale (*energia elettrica, manutenzione, gestione, etc*). Nel 2017 Expedia (*la quale prima possedeva tutto in una infrastruttura locale*) spostò più dell'80% dei loro servizi in AWS.

Che benefici hanno ottenuto?

- Costi fissi dovuti all'infrastruttura ridotti a picco.
- Latenza ridotta, grazie all'infrastruttura aggiornata di AWS.

Casi d'uso - Startups

La categoria delle startup (*aziende piccole e con poca esperienza*) sono quelle che hanno giovato di più dal cloud computing. Poiché il cloud permette a piccole aziende di utilizzare una potente infrastruttura senza gli enormi costi iniziali di una infrastruttura locale. Aziende di questo tipo sono: Airbnb, Netflix, etc.

Casi d'uso - Sporadiche computazioni ad alte performance [HPC]

Casi in cui occorre calcolare complessi algoritmi in poco tempo: per farlo serve hardware potente e distribuito. Un'azienda di questo tipo è Pixar e i loro rendering 3D, per cui affittano data center per settimane.

Servizi in scala globale

Servire miliardi di utenti ogni giorno e possibilmente quasi contemporaneamente. Un servizio di questo tipo non può essere schierato in un unico punto, altrimenti la rete diventa un grande bottleneck (*la latenza Giappone-america è grande*). La soluzione è semplice: l'azienda replica il loro servizio in molteplici località (*possibilmente una per continente*) in modo da ridurre la latenza. I cloud provider offrono soluzioni per questo problema poiché possiedono molteplici data center in tutto il mondo.

Casi d'uso - Google search

Google indicizza circa 16 Exabytes (*16'000'000 TB*). Google gestisce circa 65'000 query al secondo, quindi 2 Triloni all'anno. Un solo datacenter non può gestire tutto questo: servono diversi punti di accesso in tutto il mondo.

Casi d'uso - Data collection

Applicazione che deve collezionare dati da dispositivi e utenti da tutto il mondo. Un esempio è GE Power, la quale produce macchinari per impianti di produzione elettrica su vasta scala. Sono macchinari molto complessi con controllori che inviano continuamente flussi di dati telemetrici al computer centrale. Questa quantità di dati proviene da tutto il mondo ed è tanta (*ovviamente*).

Casi d'uso - Content distribution

Applicazione che deve distribuire contenuto su svariati dispositivi eterogenei, con diverse capacità di rete, diversa tipologia di rete e sparsi su tutto il mondo, il tutto nel modo più efficiente e consistente possibile. Un esempio può essere Netflix (*video*), Spotify (*file audio*) o Microsoft Windows (*software updates*). Un esempio particolare è King, la software house che produsse Candy Crash. La compagnia deve fare in modo che quel gioco possa essere giocato su più dispositivi possibile con diverse tipologie di connettività (2G, 3G o 4G) e quindi diverse velocità di rete o diverse capacità computazionali. Per gestire questa eterogeneità devo avere una soluzione con molteplici cloud providers, ognuno adatto alla zona che devono servire.

Sfide, rischi e contro della cloudification

Di sicuro sono molti meno rispetto alla soluzione locale, però ci sono:

- **Connessione ad Internet obbligatoria:** Un'azienda con servizi in cloud deve sempre essere connessa; quindi, si aggiunge un costo fisso dovuto alla connessione ad Internet (*non è un grande contro, la connessione ad Internet l'avrebbe avuta comunque, però con una tariffa meno costosa*).
- **Richiesta di grande banda:** Serve un servizio di connettività potente in grado di trasportare un grande volume di dati.
 - È possibile mitigare questo problema, tramite una strategia molto particolare: Se voglio spostare petabyte verso il mio Cloud provider, mi conviene anzi consegnare fisicamente un Hard Drive che contiene tutto.
- **Portabilità limitata:** Il processo di standardizzazione delle tecnologie cloud è ancora indietro. Diversi cloud provider potrebbero (*molto spesso*) usare tecnologie molto diverse e spesso proprietarie e incompatibili tra loro; ciò rende difficoltoso il cambiare cloud provider dopo averne scelto uno, questo problema è detto “*lock-in*”.
 - Questo problema potrebbe essere mitigato grazie a SDN o altre tecniche di virtualizzazione.
- **Problemi legali:** I data center vengono costruiti in posti convenienti (*elettricità poco costosa, lavoro poco costoso, etc*), ma ogni nazione ha le sue normative che possono essere incompatibili con le normative del paese in cui voglio fornire il mio servizio, specialmente quando si parla di dati sensibili.
 - Ad esempio, le cartelle mediche devono essere memorizzate nella stessa nazione del paziente.
- **Sicurezza:** Una delle ragioni primarie per cui un'azienda non vorrebbe eseguire una cloudification. L'azienda vorrebbe tutto sotto il suo controllo, quali sono i rischi nel

trasportare i dati verso il cloud? Il canale di trasmissione è insicuro: I dati possono essere intercettati; quindi, ho bisogno di meccanismi di crittografia per proteggere la confidenzialità dei dati trasmessi.

Capitolo 3 – Modello NIST

Il modello NIST nel contesto del cloud computing si riferisce al framework di riferimento (*non esiste un vero e proprio standard*) e alla definizione di cloud computing. Questo modello è ampiamente accettato e utilizzato per comprendere e categorizzare i servizi di cloud computing.

Non ci sono vere e proprie standardizzazioni delle tecnologie cloud (*ogni cloud provider ha una soluzione proprietaria*), ma alcuni termini sono stati standardizzati.

Definizione di cloud computing

Il modello NIST spiega i tre pilastri fondamentali di una architettura cloud:

- Le **caratteristiche fondamentali** di una architettura cloud.
- Le tre **tipologie di servizio** che può offrire.
- I quattro **modelli di deployment** (*distribuzione*) dell'infrastruttura cloud.

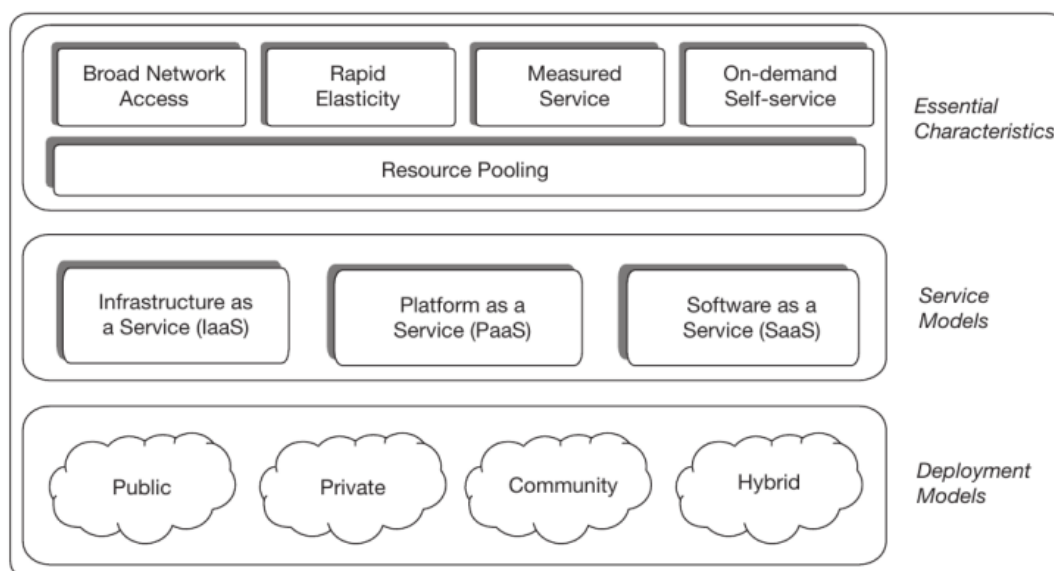


Figure 13 - Architettura di riferimento.

Caratteristiche fondamentali

Un infrastruttura cloud secondo il modello NIST deve:

- **Essere sempre accessibile** via rete internet.
- **Avere una alta elasticità** riguardo l'allocazione e de allocazione delle risorse.
- **Avere risorse misurabili:** L'utilizzo delle risorse deve essere misurato.
- **Allocazione di risorse On-Demand:** Le risorse devono essere allocate on-demand, quindi subito dopo la richiesta.
- **Gestione automatizzata delle risorse:** Le risorse computazionali dell'infrastruttura cloud devono essere tante, e gestite in modo remoto in modo da essere in grado di

soddisfare tutti i bisogni dei clienti contemporaneamente (*agli occhi dei clienti devono sembrare quasi infinite*).

Tipologia di servizio - Service Delivery Model

Definisce il tipo di servizi che ogni cloud provider dovrebbe poter offrire ai propri clienti. Rispetto a quelli mostrati sotto, ad oggi esistono nuovi modelli di servizio ma sono ancora acerbi. Quelli mostrati sotto sono modelli consolidati e tutto sommato sempre presenti nel mercato di oggi.

I modelli differiscono principalmente sul livello di astrazione dell'hardware fornita al cloud service provider.

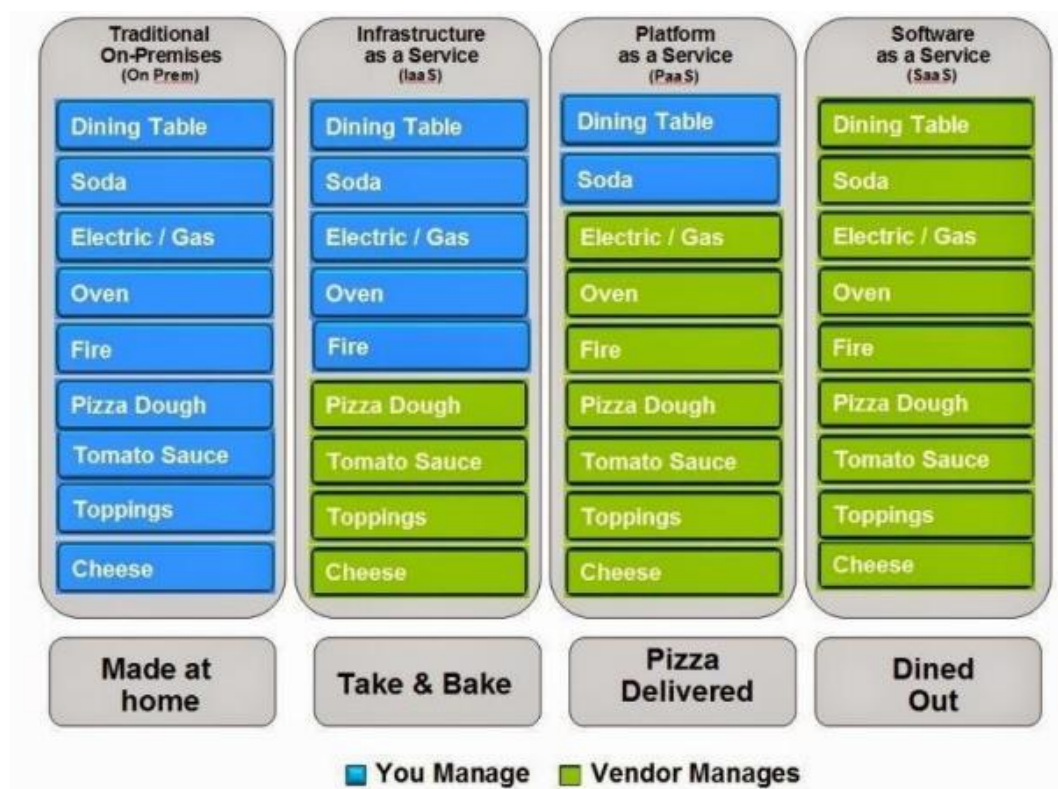


Figure 14 - Esempio di Pizza ad a Service.

IaaS - Infrastructure as a Service

Questo servizio consiste nel fornire delle macchine virtuali “vergini” al cliente, il quale potrà configurarle da remoto secondo i suoi bisogni come una normalissima macchina.

Ogni macchina virtuale avrà una CPU, un pò di RAM, alcuni dischi virtuali e un sistema operativo (*voluto dal cliente, il 99% delle volte un sistema operativo basato su Linux*).

Inoltre, il cliente deve configurare la sua macchina virtuale installando (*sopra il sistema operativo scelto*) i software necessari per creare il suo servizio. Il cloud manager invece ha il compito di installare l'hypervisor (VMM, *nell'immagine sotto è il “Layer of Virtualization”*) sulle macchine fisiche, gestire le macchine virtuali e ovviamente gestire l'hardware su cui girerà la macchina virtuale.

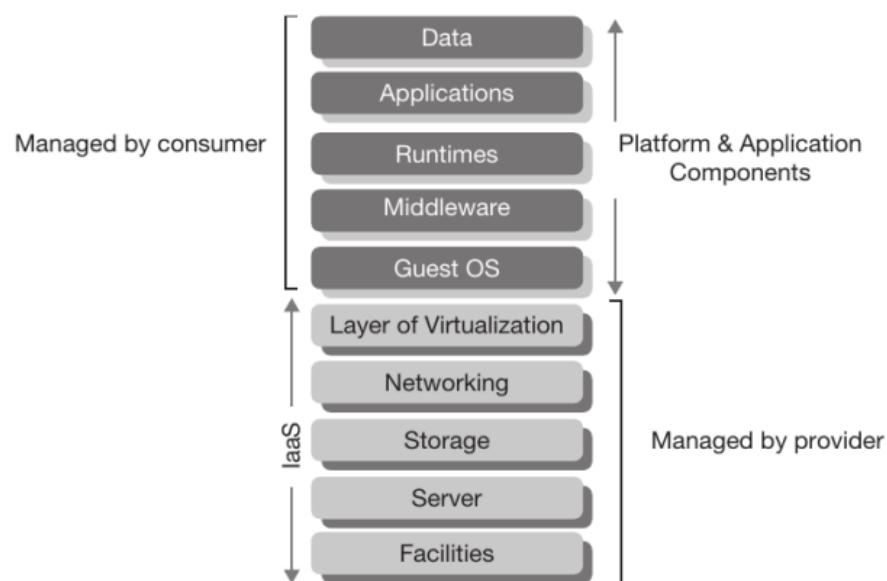


Figure 15 - Diagramma di un servizio IaaS.

In questo tipo di servizio Amazon (33%), Microsoft (16%) e Google (8%) sono i leader del settore.

PaaS - Platform as a Service

Il livello di astrazione cresce di molto rispetto a IaaS. PaaS non offre l'accesso alle macchine virtuali ma a piattaforma di sviluppo installate sopra le macchine virtuali.

Su queste piattaforme gli sviluppatori software possono creare le loro applicazioni usando le API fornite dalla piattaforma stessa.

Il cloud manager si occupa di installare il sistema operativo sulle macchine virtuali e di gestire il runtime.

Il customer si occupa solo di scrivere la logica dell'applicazione.

Il problema di questa opzione è il “**vendor Lock-In**”: L'applicazione si basa sulle API fornite dalla piattaforma sulla quale è stata scritta (*addirittura su uno specifico linguaggio di programmazione usato praticamente solo in quella piattaforma*), quindi quell'applicazione non può essere spostata a cuor leggero, poiché richiederebbe di modificare l'applicazione stessa in modo che non usi più quelle API.

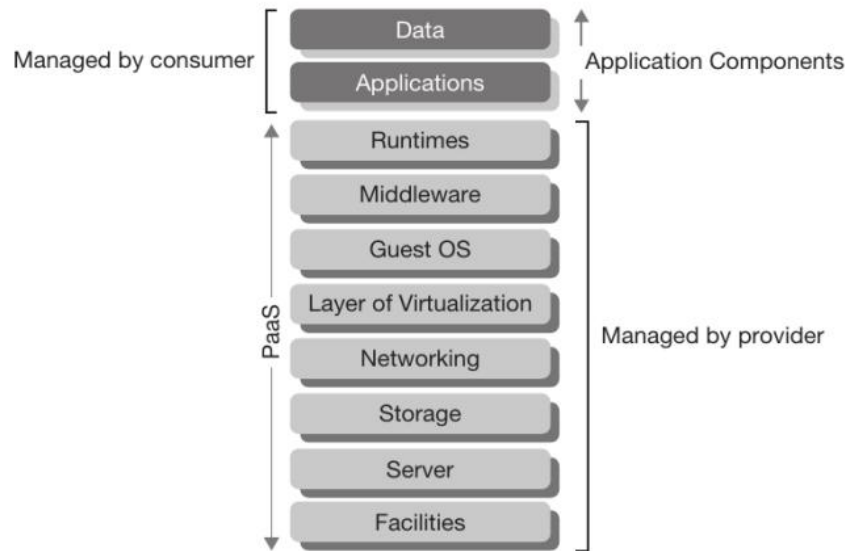


Figure 16 - Diagramma di un servizio PaaS

SaaS - Software as a Service

Il cloud provider gestisce tutto, logica dell'applicazione e dati. I clienti cosa fanno? Usano le applicazioni fornite (*ovviamente con pochissima possibilità di customizzazione da parte del consumatore del servizio*).

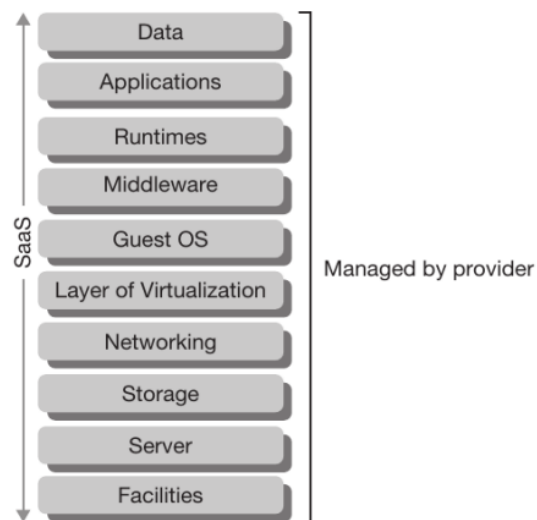


Figure 17 - Diagramma di un servizio SaaS

Implementazione PaaS e SaaS

Sia il servizio PaaS che SaaS si costruiscono sopra un servizio IaaS. E allo stesso modo, un servizio SaaS si costruisce sopra un servizio PaaS.

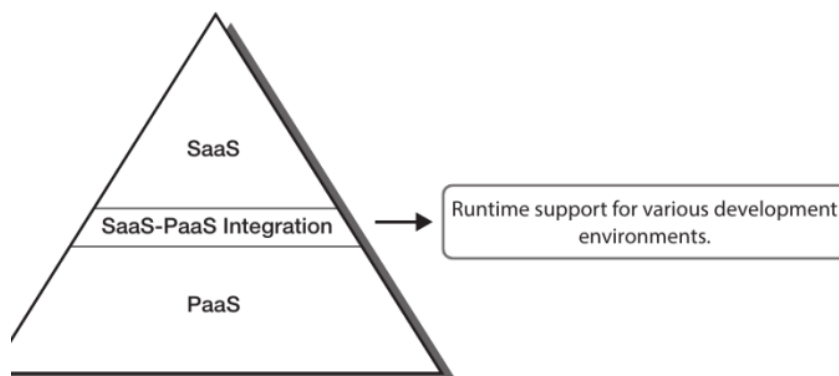


Figure 18 - Costruzione di un servizio PaaS e SaaS

Altri tipi di servizi

Ci sono altri tipi di servizi che derivano dai tre servizi fondamentali spiegati prima.

Storage as a service

I cloud provider offrono spazio per la memorizzazione;

Database as a service

I cloud provider offrono database e si occupano della loro gestione, manutenzione, configurazione ecc.;

Backup as a service

I cloud provider offrono funzionalità di backup che sono molto importanti per mantenere i dati in caso di disastri o altro;

Desktop as a service

I cloud provider offrono ambienti desktop personalizzati, ovvero offrono macchine virtuali da usare come personal computers.

Modello di distribuzione - Deployment model

Come l'infrastruttura cloud è schierata sull'infrastruttura fisica e come vengono definite le relazioni tra l'infrastruttura e i cloud service owner. Un'infrastruttura può essere creata, gestita e offerta in modi molti diversi; per fare ciò sono definiti quattro diversi modelli nel NIST.

Public Cloud

Il modello di deployment più popolare.

In questo modello promuove una alta multi-Tenancy ai suoi più alti livelli; infatti, è studiato per essere usato da moltissimi utenti contemporaneamente. È accessibile tramite rete pubblica e le sue risorse sono condivise tra molteplici consumatori.

Private Cloud

La gestione è eseguita da un'unica organizzazione per il suo uso interno. Di solito l'infrastruttura è localizzata all'interno della compagnia (*dentro l'edificio*), infatti questa soluzione è usata quando la compagnia ha dei bisogni davvero molto specifici; soprattutto riguardo la sicurezza per cui richiede il controllo completo sull'infrastruttura.

L'infrastruttura è costruita con tutte le funzionalità di un cloud computing system, ma l'accesso è ristretto solo ai membri dell'organizzazione

Rispetto al cloud pubblico è molto più costoso e impegnativo: l'organizzazione deve pagare tutto di propria tasca per creare e gestire l'infrastruttura.

Community Cloud

Queste infrastrutture cloud sono un ibrido tra cloud pubblici e privati: Permette a svariate organizzazioni (*appunto una comunità*) di accedere alle risorse cloud; ossia è un cloud privato ma condiviso tra queste organizzazioni.

I costi fissi di creazione sono divisi tra gli utilizzatori (*quindi minori di un cloud privato*) e i costi variabili di gestione sono stabiliti tramite tariffe pay-per-use.

Hybrid cloud

Un'altra soluzione nel mezzo tra un cloud privato e un cloud pubblico. I cloud ibridi hanno una parte pubblica (*situato in una infrastruttura pubblica*) e una parte privata.

Ognuna delle due parti può offrire servizi diversi: Ad esempio la parte privata gestirà dati sensibili e quindi attuerà protocolli e tecnologie più indicate a gestire attacchi esterni.

I due cloud ovviamente comunicano tra loro attraverso una certa modalità. Di solito la maggior parte della logica e dai dati è schierata sul cloud pubblico in modo da abbattere i costi; Il cloud privato viene usato solo per i dati molto sensibili.

Come scegliere il modello di deployment?

Parameters\Type	Public Cloud	Private Cloud	Hybrid Cloud	Community Cloud
Description	In public cloud, services are available for public users.	Private cloud is build up with existing private infrastructure. This type of cloud has some authentic users who can dynamically provision the resources.	Hybrid cloud is a heterogeneous distributed system, resulting from a private cloud, which incorporates different types of services and resources from public clouds.	Different types of cloud are integrated together to meet a common or particular need for some organizations.
Scalability	Very High	Limited	Very High	Limited
Reliability	Moderate	Very High	Medium to High	Very High
Security	Totally Depends on service provider	High class security	Secure	Secure
Performance	Low to medium	Good	Good	Very Good
Cost	Cheaper	High Cost	Costly	Costly
Examples	Amazon EC2, Google AppEngine	VMWare, Microsoft, KVM, Xen	IBM, HP, VMWare vCloud, Eucalyptus	SolaS Community Cloud, VMWare

Figure 19 - Differenze tra i vari modelli di distribuzione.

Dipende da vari fattori, di solito ogni business-model ha bisogni speciali; quindi, serve un'analisi dei requisiti assieme ad un analisi dei costi-benefici.

Dopo aver analizzato i requisiti più importanti si sceglie il modello appropriato:

- Nel 99% dei casi il cloud pubblico è la soluzione migliore: Il meno costoso e il più affidabile poiché la gestione e la sicurezza sono affidate ad un azienda specializzata.
- Il cloud privato (o la community) serve quando l'azienda ha bisogni seri riguardo la privacy e la sicurezza (e ovviamente se ha tanti soldi).

Capitolo 4 – Tecniche di virtualizzazione

La virtualizzazione riguarda la rappresentazione di risorse computazionali in maniera simulata. Le risorse virtuali emulano perfettamente le risorse fisiche e possono essere usate alla stessa maniera dagli utenti come se fossero risorse fisiche.

Ossia si tratta di introdurre un layer tra le risorse fisiche e il sistema operativo; questa cosa si fa tramite un programma speciale chiamato “*Hypervisor*” o “*VMM*”, il quale crea una rappresentazione virtuale delle risorse fisiche.

Detto in maniera più specifica: Gli utenti (*cloud service owners o cloud consumers*) vengono assegnate risorse virtuali, le quali sono completamente scollegate da quelle fisiche. Alla fine, sopra una macchina fisica vengono create ed eseguite svariate macchine virtuali indipendenti.

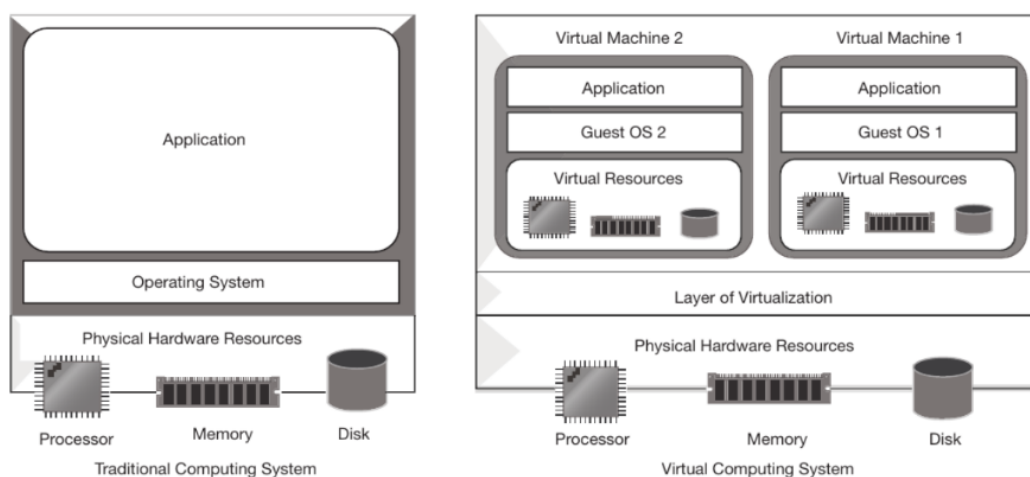


Figure 20 - Differenza tra una macchina tradizionale (sinistra) e una macchina su cui è stato applicato un layer di virtualizzazione (destra).

Vantaggi della virtualizzazione

- **Aumenta l'utilizzazione complessiva** delle risorse, quindi porta ad un costo minore dell'hardware necessario.
- **Semplifica l'amministrazione** del sistema, il livello delle macchine virtuali è ben separato da quello della macchina fisica.
- **Aumenta la tolleranza ai guasti**: il guasto di una macchina virtuale si recupera riavviandola, senza interrompere le altre.
- **Riduce il tempo necessario per la manutenzione**: sistemare una macchina virtuale richiede un tempo nullo: basta riavviarla o alla peggio spostarla (*migrazione*) in una macchina fisica diversa senza installare o configurare un intero sistema.
- **Migliora la sicurezza**: Ogni macchina virtuale è isolata dall'altra.

Svantaggi della virtualizzazione

- **La macchina fisica è un single point of failure per tutte le macchine virtuali su di essa** (*mitigato dalla migrazione o dagli snapshot/backups che permettono di far ripartire istantaneamente una macchina virtuale su un'altra macchina fisica*).
- **Performance peggiori in termini di latenza** (*mitigato dal supporto hardware, il quale ha ridotto la latenza rispetto ad un sistema equivalente senza virtualizzazione quasi a 0*).

Virtualizzazione delle risorse

Ogni tipo di risorsa (*dalle risorse necessarie come la RAM alle risorse opzionali come le periferiche*) può essere virtualizzata, ovviamente mediante strategie diverse. Componendo diverse risorse virtuali posso creare delle macchine virtuali su cui installare ed eseguire programmi. Prima di tutto, dividiamo le risorse in due categorie: opzionali e non opzionali.

Risorse non opzionali

Una risorsa virtuale è di questo tipo funziona se richiede che esista una risorsa fisica equivalente. Ad esempio: non posso eseguire un programma se non ho almeno una CPU e un pò di RAM.

Questa regola non va interpretata in modo errato: Da una CPU fisica posso ottenere tre CPU virtuali; oppure da un processore a 64 bit posso ottenere un processore virtuale a 32 bit (*la struttura della risorsa virtuale è diversa da quella fisica*). Ma senza una scheda di rete fisica, non posso virtualizzarne nessuna.

Risorse opzionali

Risorse che possono anche essere simulate tramite software, quindi non richiedono una controparte fisica. Un esempio sono le periferiche (*eccetto la scheda di rete*), per inviare messaggi c'è bisogno che ci sia una interfaccia di rete reale.

Recap di Multiprogrammazione

Capacità di un sistema di eseguire molteplici programmi allo stesso tempo nonostante il sistema stesso non è capace di eseguire due programmi contemporaneamente (*hardware concorrente*).

Dobbiamo dare l'impressione di eseguire più programmi allo stesso tempo, e ogni programma deve avere l'impressione di essere l'unico ad essere eseguito.

Ogni processo, vede una CPU virtuale chiamata vCPU; ogni vCPU è dedicata all'esecuzione di un singolo processo. Chiameremo VCPU la struttura dati che rappresenta lo stato del processore mentre esegue uno specifico processo.

Quando un processo ha finito il turno di utilizzo della CPU fisica, il suo stato viene salvato in memoria (*lo stato della sua vCPU*) per essere poi ripristinato poco dopo.

Cambio di contesto

Operazione eseguita dal sistema operativo per assegnare la CPU ad un altro processo in attesa (vCPU).

1. Il sistema entra in modalità privilegiata.
2. Si salva lo stato della CPU Host (*lo stato del processo in esecuzione*) e lo carica nella relativa rappresentazione vCPU.
3. Seleziona una delle vCPU di un processo in attesa e carica il suo stato nei registri.
4. Esegue una istruzione per saltare all'indirizzo memorizzato nel registro RET e ritorna in modalità non privilegiata.

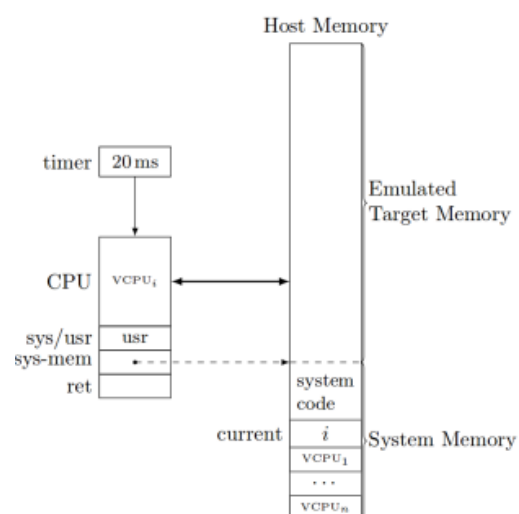


Figure 21 – Diagramma semplificato che mostra un cambio di contesto.

Livelli di privilegio

I processori hanno due livelli di privilegio differenti:

User mode

Il processore protegge l'accesso alle istruzioni privilegiate; quindi, solo un sottoinsieme d'istruzioni possono essere eseguite liberamente.

Usato quando un'applicazione è in esecuzione.

System (kernel) mode

Il sistema ha accesso a tutte le istruzioni nel set del processore.

Riservato solo alle routine ed alle funzioni del sistema operativo e attivato quando il sistema operativo assume il controllo per implementare funzionalità come eseguire un driver, gestire un dato di input/output ecc.

Istruzioni privilegiate

Dividere un sistema in user mode e kernel mode è una semplificazione, perchè un processore di solito implementa quattro livelli di privilegio, assimilabili ad un anello. Un anello interno ha un livello di privilegio superiore a quello esterno.

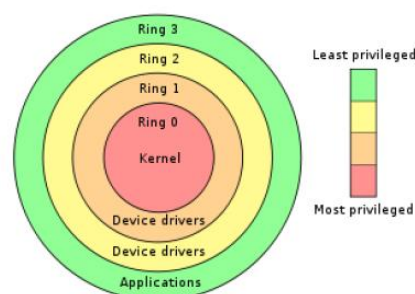


Figure 22 - Anello che assegna il livello di privilegio ad ogni tipo di operazione.

Memoria virtuale

Meccanismo che permette ad un processo di separarsi dalla memoria fisica. Questo meccanismo cambia il modo in cui il processo accede agli indirizzi, in particolare, ad ogni processo sembra che la memoria a lui assegnata sia:

1. Tutta, ossia tutto lo spazio di indirizzamento possibile.
2. Inizia dall'indirizzo 0.

In realtà ogni processo lavora con indirizzi di memoria virtuali, che poi tramite un componente hardware (*Memory Management Unit*, detta anche *MMU*) verranno tradotti nei relativi indirizzi fisici.

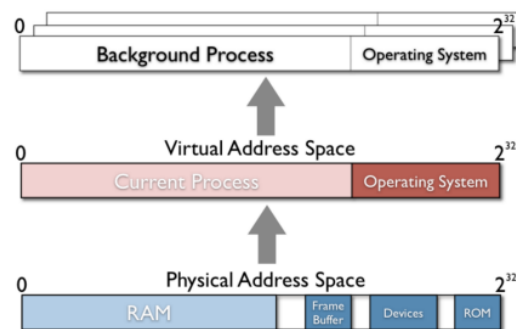


Figure 23 - Memoria virtuale di un processo.

Da come si vede in figura, lo spazio di indirizzamento di un processo può venir diviso nella memoria fisica effettiva ma dal punto di vista del processo non è così.

MMU e Paginazione

Il sistema operativo ha il controllo della RAM, la RAM viene suddivisa in segmenti, questi segmenti vengono assegnati alle applicazioni. Gli indirizzi virtuali devono essere tradotti in indirizzi fisici perché il processore può gestire solo gli indirizzi fisici, questa traduzione è effettuata da un componente hardware detto MMU. Per fare ciò la MMU utilizza una tabella chiamata “*page table*”. Questa tabella è gestita dal sistema operativo.

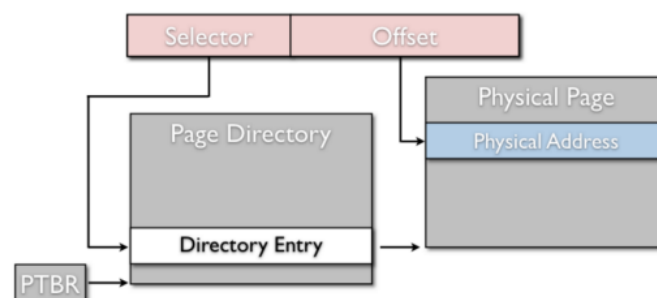


Figure 24 - Page table ad un livello.

La memoria primaria è organizzata in pagine di equa dimensione (4 KB). Ogni indirizzo virtuale è diviso in due parti:

- **Selettore:** Calore che seleziona la pagina virtuale in cui si trova l'indirizzo.
- **Offset:** L'indirizzo all'interno della pagina indicata dal selettore.

La MMU si occupa di tradurre il selettore della pagina virtuale nel selettore della pagina fisica corrispondente, lasciando l'offset inalterato.

La traduzione avviene tramite una tabella memorizzata in RAM. La tabella (*detta anche “tabella delle pagine”*) è gestita dal sistema operativo e ogni processo ne ha una; l'indirizzo iniziale di questa tabella è contenuto nel registro **PTBR** (*Page Table Base Register*) del processore.

La page table è organizzata in pagine, ha un'organizzazione multi-level, aggiunge molteplici layer di traduzione di indirizzi (*la level 3 traduce l'output della level 4 e così via*).

Presenta molte directory di pagina connesse l'une alle altre, tramite questa catena di pagine (o page directories), la MMU può ottenere tutte le informazioni richieste per tradurre un indirizzo virtuale in un indirizzo fisico.

La creazione e la gestione delle page table è assegnata e gestita dal SO.

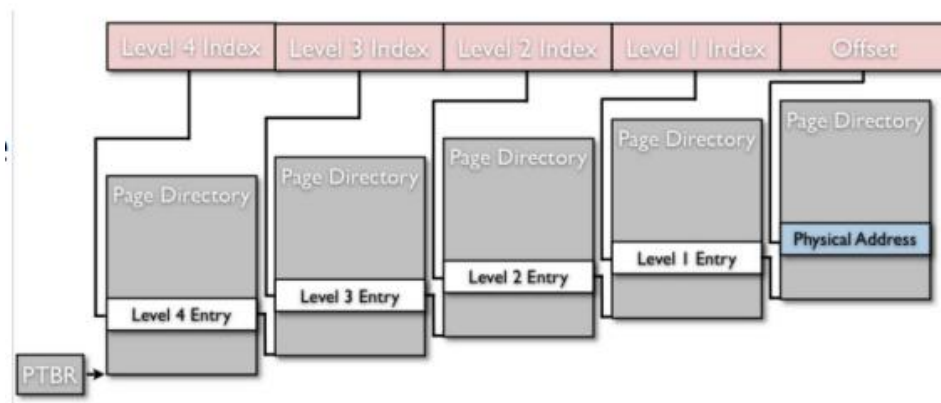


Figure 25 - Page table multi-livello.

Segmentazione

Spesso lo spazio di indirizzamento virtuale può superare la capacità massima della memoria primaria fisica, in tal caso le pagine in eccesso vengono caricate in memoria secondaria (*swapping*).

Quando un processo vuole accedere ad una pagina “*swappata*”, si genera un **page fault** e la pagina viene caricata di nuovo in RAM e la page table viene aggiornata di conseguenza.

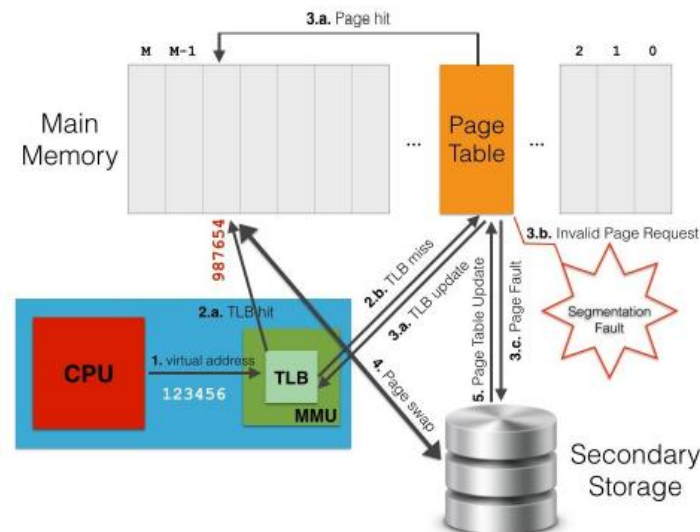


Figure 26 – Diagramma che mostra un page fault.

Interruzioni ed Eccezioni

Entrambi sono meccanismi per notificare al sistema operativo che è necessario un immediato intervento durante l'esecuzione di un processo.

Le interruzioni sono eventi asincroni (*la periferica ha prodotto dati, derivano da un device*), mentre le eccezioni sono eventi interni e sincroni (*page fault, errori di esecuzione, divisione per 0, etc*). Entrambi i meccanismi causano un'alterazione del normale flusso di esecuzione per eseguire una funzione (*handler*) del kernel.

Un'eccezione è detta anche trap, questa può essere chiamata anche da software tramite l'istruzione int.

Per fare ciò avviene una transizione (cambio di contesto) verso il kernel mode (modalità privilegiata) e al termine dell'esecuzione della funzione (handler, la quale è definita dal sistema operativo), si ritorna all'user mode e restituisce il controllo ad una applicazione che può essere portata avanti.

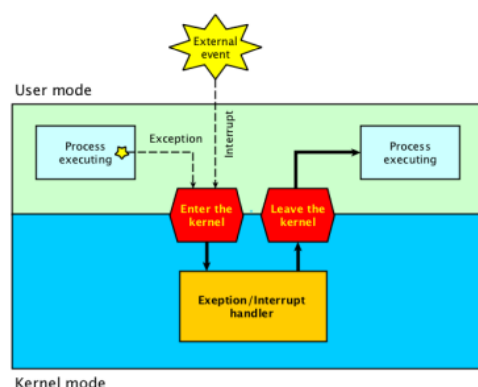


Figure 27 - Cambio di contesto in caso di una interruzione/eccezione.

Interrupt Descriptor Table - IDT

Ogni interruzione è identificata da un codice e da un handler di gestione (*indirizzo verso la funzione*). Il legame tra queste due cose è mostrato dalla Interrupt Descriptor Table (IDT) (anche detta Interrupt Descriptor Vector IDV).

Contiene una lista di indirizzi di memoria, i quali rappresentano la prima istruzione dell'handler associato con uno specifico interrupt. Da notare che contiene le entrate sia per le interruzioni che per le eccezioni.

Questa tabella (*come la page table*) si trova in RAM ed è popolata dal sistema operativo in fase di bootstrap.

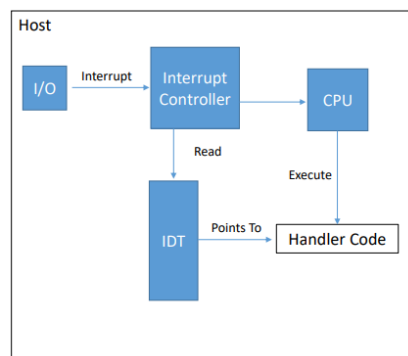


Figure 28 – Utilizzo della Interrupt Descriptor table.

Hypervisor – Virtual Machine Monitor - VMM

È il software responsabile di creare l'ambiente virtuale (*più precisamente di creare e gestire le risorse virtuali*) nel quale le macchine virtuali girano, ossia ha il compito di creare il layer virtuale. La virtualizzazione tramite hypervisor si divide in due tipologie:

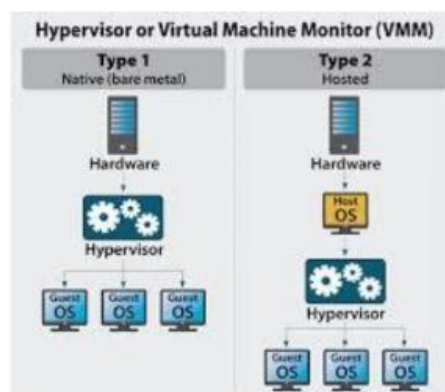


Figure 29 - Le due tipologie di hypervisor.

La macchina fisica è chiamata “*host*” e la macchina virtuale è chiamata “*guest*”. Ogni macchina guest è indipendente dalle altre macchine guest che girano sulla stessa macchina fisica.

Hosted Approach

La macchina fisica possiede un normale sistema operativo e l'hypervisor è un programma come tanti altri installati sulla macchina host.

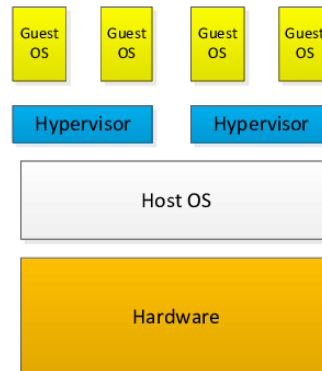


Figure 30 - Hosted hypervisor.

Esempi di hypervisor di questo tipo sono: VMware e VirtualBox. Il sistema operativo host ha il compito di sempre, ossia quello di gestire i componenti hardware e le loro interazioni: ciò semplifica molto i compiti dell'hypervisor e gli consente di avere una più larga compatibilità poiché non deve preoccuparsi dell'hardware e dei driver.

Il problema di questi hypervisor è anche la sua benedizione, non avendo il controllo completo sull'hardware le performance sono peggiori. Inoltre, l'hypervisor non ha accesso diretto alle periferiche e in generale all'hardware sottostate.

Bare Metal Hypervisor

La macchina fisica non possiede un normale sistema operativo e l'hypervisor è installato direttamente sulla macchina.

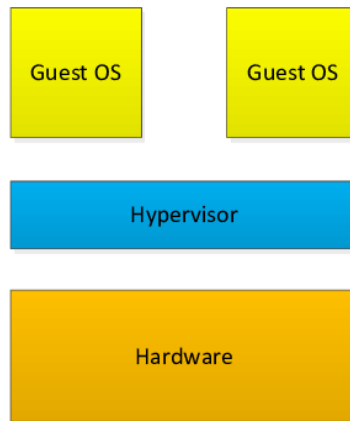


Figure 31 - Bare metal hypervisor.

Esempi di hypervisor di questo tipo sono: VMware ESX, Microsoft Hyper-V.

In questo caso l'hypervisor non è un normale software ma si comporta più come un sistema operativo (*gestione della memoria fisica, gestione della CPU, etc*): ciò li rende leggermente più difficili da sviluppare ma molto più performanti e scalabili.

Questo tipo di hypervisor è ideale per data centers, poiché è importante massimizzare le performance e l'utilizzazione delle risorse.

Tra i vantaggi troviamo:

- Semplicità di utilizzo.
- Pieno controllo sull'hardware e quindi performance poiché minimizza l'overhead e la quantità di risorse sprecate.

Il principale svantaggio è la più stretta compatibilità di dispositivi, architetture e periferiche; questo perché questi hypervisor sono ottimizzati per i più famosi dispositivi.

Tecniche di virtualizzazione

Vediamo ora quali sono i requisiti necessari ad un software di virtualizzazione per essere corretto.

Equivalenza

Un sistema operativo che gira su una macchina virtuale deve esibire un comportamento identico a quello dimostrato su una macchina fisica.

Controllo delle risorse

L'hypervisor deve avere accesso delle risorse fisiche e il sistema operativo guest deve avere il controllo completo delle risorse virtuali a lui assegnate dal VMM.

Efficienza

La maggior parte delle istruzioni della macchina virtuali devono poter essere eseguite dall'hardware fisico senza una traduzione da parte dell'hypervisor (*l'istruzione set usato dal sistema virtuale deve essere lo stesso di quello usato dalla macchina fisica*).

Capitolo 5 - Full Virtualization

L'obiettivo è quello di permettere l'esecuzione di un sistema operativo guest sopra di essa, senza che quest'ultimo sia in grado di capire di essere su una macchina virtuale, quindi permettergli di essere eseguito senza apportare nessuna modifica a quest'ultimo. Il sistema operativo guest crede di essere in esecuzione su risorse fisiche reali.

Per fare ciò l'hypervisor deve creare una esatta copia virtuale delle risorse e permettere al sistema operativo guest di gestirle. L'hypervisor ha la responsabilità di gestire tutte le richieste del sistema operativo guest verso l'hardware.

Il sistema operativo guest rimane completamente isolato dalle risorse fisiche grazie all'hypervisor che invece espone le loro rappresentazioni virtuali.

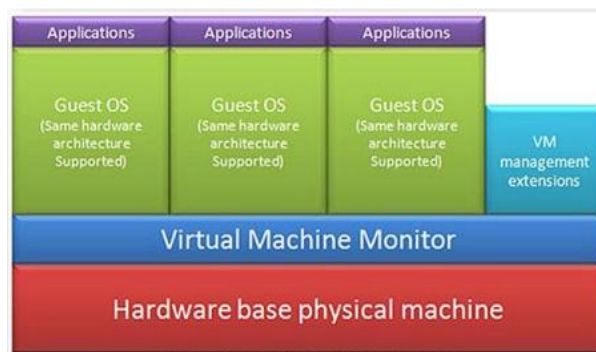


Figure 32 – Diagramma che mostra la full virtualization.

Il problema di questa strategia sono le performance, il dialogo tra hypervisor e sistema operativo guest è lentissimo.

Emulazione delle istruzioni

L'emulazione è il processo che consiste nel creare un sistema con lo scopo di imitare perfettamente un altro sistema, in tutti i suoi componenti.

L'emulatore è in grado di riprodurre lo stesso instruction set (*quindi anche l'architettura*) del sistema emulato. In questo modo, il codice sviluppato per il sistema emulato può essere eseguito sull'emulatore senza problemi.

Ad esempio: un computer Apple con architettura M3 ha un processore ARM; per far girare un programma di windows ha bisogno di un emulatore e non di un software di virtualizzazione, poiché si richiede la simulazione di un'intera architettura diversa.

Differenza tra emulazione e virtualizzazione

Nel caso di un virtualizzatore, l'istruzione set usato dal sistema virtualizzato e l'hardware effettivo è lo stesso; perciò, la virtual machine può essere eseguita direttamente in un vero

sistema operativo, non è richiesta una traduzione delle istruzioni come invece avviene in una emulazione.

Questa differenza comporta un minore overhead nella virtualizzazione (*è molto vicina alla velocità nativa*). Quindi la virtualizzazione è molto più veloce dell'emulazione.

Invece, nel caso di un emulatore, l'architettura del sistema emulato e del sistema emulatore può essere diversa, ciò porta a dover tradurre le istruzioni del sistema emulato per essere effettivamente eseguite sul sistema emulatore.

L'emulatore consente la simulazione dell'hardware completo, in software, quindi creando un ambiente per l'esecuzione di un sistema operativo.

L'emulatore deve implementare ogni componente del sistema emulato, ciò lo rende più complesso di un Hypervisor, il quale deve implementare solo una parte di esse, quelle che necessita per fare le stesse cose; invece, l'emulatore deve farle anche allo stesso modo.

Infatti, un emulatore simula sia l'hardware che il software di un sistema. Ci sono due modi per implementare un emulatore:

Binary translation (recompilation)

Il codice binario prodotto dal sistema emulato viene **tradotto** (*ricompilato*) per essere eseguito sull'emulatore (la ricompilazione può essere statica o dinamica).

La tecnica che permette di avere le performance migliori ma è il modo più complicato da realizzare.

Interpretation

Ogni istruzione del sistema emulato è **interpretata** dall'emulatore ogni volta che viene incontrata, in modo che possa essere eseguita in maniera equivalente. Questo metodo è più facile da implementare ma comporta un maggiore delay.

Modello “Trap&Emulate”

Prima di vedere questa tecnica dobbiamo capire come funziona l'esecuzione del sistema operativo guest. Il sistema operativo guest è in esecuzione in modalità utente, non può essere eseguito in modalità kernel, perchè in questa modalità è il VMM che viene eseguito, però all'interno di una VM è presente una sorta di codice privilegiato, ovvero il kernel della VM stessa.

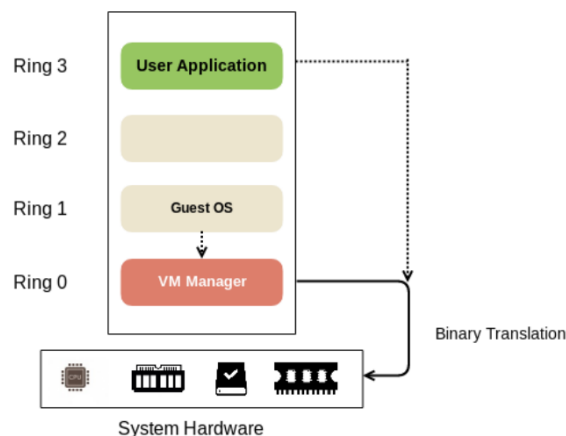


Figure 33 – Organizzazione in livelli nella full virtualization

La tecnica trap & emulate è utilizzata nella full virtualization per gestire l'esecuzione di istruzioni privilegiate (*quelle che non possono essere eseguite direttamente dall'utente*) da parte delle macchine virtuali.

Questo modello permette al sistema di virtualizzazione di intercettare le istruzioni privilegiate (*trap*) e di emularne il comportamento appropriato in modo sicuro. Questo è essenziale per creare un ambiente di virtualizzazione sicuro e isolato in cui più sistemi operativi possono coesistere su una singola macchina fisica.

Ogni volta che un'istruzione privilegiata viene eseguita nella VM, una trap viene sollevata automaticamente dall'hardware, questo accade quando includiamo un'istruzione privilegiata nel programma.

In figura (privilege instruction non è in guest OS, ma nel kernel) vediamo il meccanismo che deve essere adottato ogni volta che una VM vuole eseguire un'istruzione privilegiata, questo

accade durante l'esecuzione del codice del kernel del sistema operativo, questo meccanismo è effettuato nel seguente modo:

- L'hypervisor prende il controllo del flusso di esecuzione;
- L'hypervisor emula il comportamento dell'istruzione privilegiata, emulandola rende lo stato della VM come se quell'istruzione privilegiata fosse stato eseguito;
- Successivamente il VMM restituisce il controllo alla VM che restituisce la sua esecuzione all'istruzione successiva dell'istruzione privilegiata.

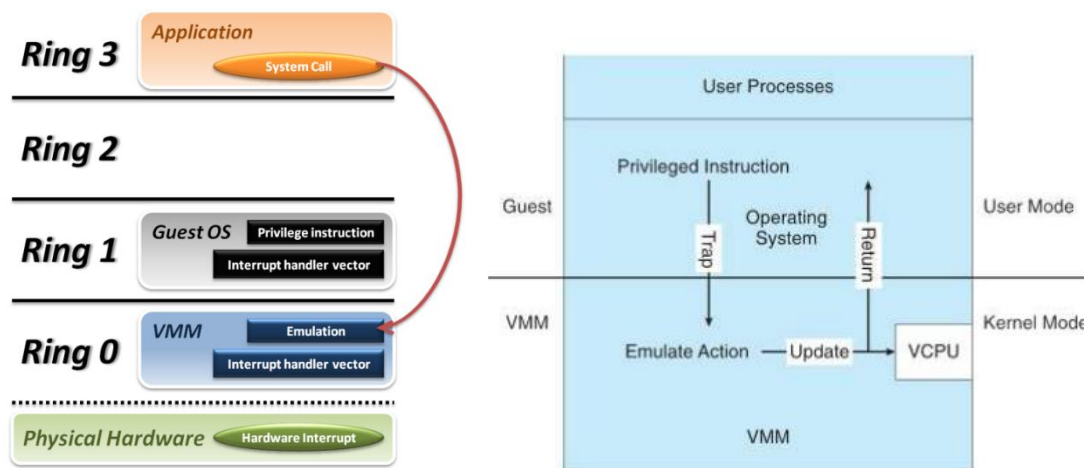


Figure 34 - Modello "trap&emulate"

Se il set di istruzioni privilegiate è limitato, le performance non sono degradate perché il numero di cambi di contesto è minore (*questo problema lo mitigheremo con il supporto hardware*).

1. Una macchina virtuale tenta di eseguire un'istruzione privilegiata.

Come, ad esempio, tutte le istruzioni che hanno a che fare con lo I/O o quelle che manipolano la MMU.

2. Il processore rileva questa situazione e genera un'eccezione (**trap**).
3. La CPU fisica non riceve la trap, ma viene direttamente intercettata dal VMM.

Quando viene generata una trap, il VMM la riconosce e prende il controllo del processore.

4. Il VMM emula il l'handler associato alla trap dal sistema operativo guest (**emulate**) che replica lo stesso risultato che si avrebbe in un contesto non virtualizzato.

Invece di lasciare che l'handler del sistema operativo guest gestisca l'operazione, il VMM interviene. Il VMM la traduce in una serie di operazioni equivalenti che possono essere eseguite in modo sicuro (*ovvero senza compromettere altre macchine virtuali*) nel contesto della macchina virtuale ospite.

5. Dopo aver emulato correttamente l'istruzione o l'accesso alla risorsa, il VMM restituisce il controllo al sistema operativo guest.

Prima di vedere un esempio dobbiamo saper che il VMM gestisce la host IDT, mentre il sistema operativo guest gestisce la propria guest IDT che non è collegata all'hardware reale.

Esempio – Istruzione INT

Non tutte le istruzioni privilegiate sono semplici da emulare, ad esempio, l'istruzione INT porta all'avvio di una system call: la quale causa un cambio di contesto nel processore.

1. L'applicazione eseguita nel sistema operativo guest genera una interruzione.

Quando si verifica la trap, il processore sospende l'esecuzione della VM e trasferisce il controllo al VMM. Il VMM intercetta l'interruzione e analizza il tipo di interruzione richiesta. Il numero dell'interruzione viene passato come parametro e indica quale servizio o routine di interruzione deve essere eseguita.

2. Il VMM deve capire che l'interruzione è stata generata all'interno della VM e cerca l'handler relativo all'interruzione contenuto nella IDT gestita dal VMM.

A questo punto il VMM emula l'esecuzione della routine di interruzione selezionata dal guest operating system. Questo può includere l'esecuzione di codice che simula il comportamento della routine di interruzione del sistema operativo guest.

3. Manipolazione dello Stato della VM.

Se la routine di interruzione coinvolge la manipolazione dei registri della VM, lo stato della VM viene aggiornato di conseguenza. Ad esempio, una chiamata di sistema potrebbe restituire un valore in un registro specifico.

4. Ripristino dello Stato della VM.

Dopo aver emulato l'interruzione, il VMM ripristina lo stato della VM salvato in precedenza, in modo che la VM possa continuare la sua esecuzione dal punto successivo all'istruzione `INT`.

5. Ritorno alla VM.

Il VMM esegue un cambio di contesto di ritorno, restituendo il controllo alla VM con lo stato aggiornato.

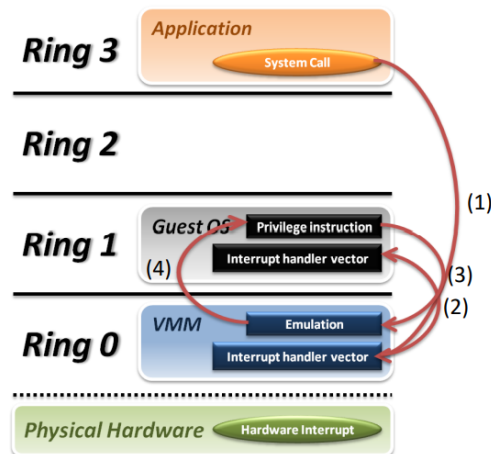


Figure 35 - Emulazione della INT.

Nota molto importante: in questo meccanismo si hanno due cambi di contesto:

1. Da applicazione guest a VMM.
2. Da VMM al codice del kernel del guest operating system (in questo caso il livello di privilegio è il più alto).

Esempio – Lettura I/O

1. Esecuzione del Codice nella VM.

La VM esegue il suo codice come se fosse in esecuzione su hardware fisico. Quando il codice della VM tenta di leggere un dato dalla periferica, esegue un'istruzione privilegiata che non può essere eseguita direttamente dalla VM (*ad esempio, un'istruzione IN per leggere da una porta I/O*).

2. Trap dell'Istruzione Privilegiata.

Poiché la VM non ha i privilegi per eseguire direttamente l'istruzione, il processore genera un'eccezione o una trap. Questo evento viene catturato dal VMM.

3. Cambio di Contesto dalla VM al VMM.

Quando si verifica una trap, il controllo passa dalla VM al VMM. Questo comporta un cambio di contesto, dove lo stato della CPU (*come i registri, lo stato della memoria, ecc.*) viene salvato per la VM e viene caricato lo stato (*la vCPU*) del processo relativo al VMM.

4. Emulazione dell'istruzione.

Il VMM esamina l'istruzione che ha causato la trap e determina l'azione appropriata. In questo caso, il VMM emula la lettura del dato dalla periferica. Questo potrebbe comportare l'accesso alla periferica fisica o a un dispositivo virtualizzato.

5. Lettura del dato.

Il VMM legge il dato dalla periferica fisica o dal dispositivo virtualizzato. Questo potrebbe comportare una chiamata di sistema o un'interazione diretta con l'hardware, a seconda della configurazione del sistema e del tipo di periferica.

6. Aggiornamento dello stato della VM.

Dopo aver ottenuto il dato, il VMM aggiorna lo stato della VM per riflettere il risultato dell'operazione. Ad esempio, il dato letto viene posto nel registro appropriato della VM.

7. Ritorno alla VM.

Il VMM esegue un cambio di contesto di ritorno, ripristinando lo stato della CPU per la VM e riprendendo l'esecuzione del codice della VM immediatamente dopo l'istruzione che ha causato la trap.

8. Continuazione dell'esecuzione.

La VM continua la sua esecuzione, ora con il dato richiesto ottenuto dalla periferica.

Virtualizzazione della RAM

La virtualizzazione della memoria RAM è una tecnica che permette a ogni macchina virtuale di avere la propria visione indipendente della memoria, come se fosse l'unica entità che utilizza le risorse di memoria fisica. Questo è realizzato tramite il VMM, che gestisce le mappature tra la memoria virtuale utilizzata dalle VM e la memoria fisica effettiva del sistema. Ovvero la VM ha l'impressione di poter accedere direttamente alla RAM fisica, in realtà solo il VMM ne ha il controllo.

La memoria della macchina virtuale guest è ottenuta usando svariate parti della memoria fisica del sistema host (*non per forza contigue*).

Caso di un VMM di tipo “hosted”

Parte della memoria fisica della macchina host è riservata al processo VMM.

Il sistema operativo guest ha accesso solamente ad una parte della memoria fisica assegnata al VMM; comunque mantenendo le funzionalità della memoria virtuale (*ossia dal punto di vista dei programmi eseguiti nella macchina virtuale guest, ogni indirizzo di memoria deve partire dallo 0*). Un range d'indirizzi fisici viene assegnato ad una VM, quindi la virtual RAM assegnato a quella VM è mappato in quel determinato spazio fisico in RAM.

Caso di un VMM di tipo “bare metal”

Il VMM è anche il sistema operativo host; quindi, ha a sua disposizione tutta la memoria fisica.

Traduzione nel sistema operativo guest

Il sistema operativo guest gestisce il proprio spazio di indirizzamento virtuale. Ogni processo che gira sul sistema guest ha il suo spazio di indirizzamento ricavato dallo spazio di indirizzamento assegnato al sistema guest.

Il sistema operativo guest gestisce la traduzione da *guest virtual address* a *guest physical address*; come farebbe in un contesto non virtualizzato.

Il *guest physical address* non è utilizzabile nella memoria fisica del sistema host, perché appartiene alla memoria assegnata alla macchina guest dal VMM.

Serve un'altra traduzione da *guest physical address* a *host physical address* (*indirizzo fisico effettivo*), in modo da poter eseguire l'istruzione.

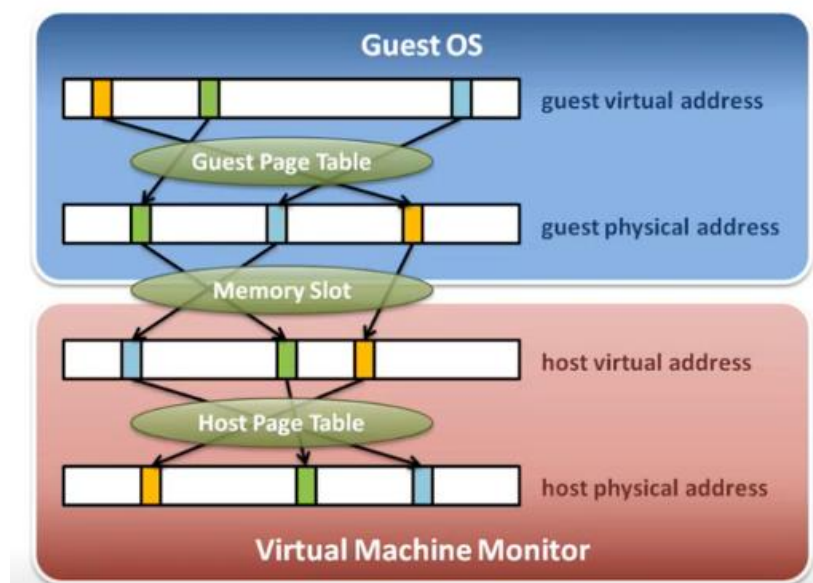


Figure 36 – La traduzione in due step degli indirizzi.

Come si vede in figura sopra ho una traduzione in due step, i quali possono essere eseguiti direttamente dalla MMU host con del supporto hardware.

Questa doppia traduzione non può essere gestita tramite software, altrimenti l'overhead sarebbe altissimo a causa dei cambi di contesto da e verso il VMM.

La virtual MMU e il GH Mapping

Per effettuare questa doppia traduzione sfruttiamo del supporto hardware: si cambia la configurazione della MMU (*ossia la struttura dati al suo interno*). Nella MMU tradizionale, per

ogni processo, sono contenute le informazioni per tradurre un indirizzo virtuale al relativo indirizzo fisico.

Ogni VM ha una propria virtual MMU, il guest OS gestisce la funzione G, la quale è implementata nella page table.

Serve però una seconda funzione H, gestita dal VMM, questa funzione si occupa della traduzione tra guest physical address a host physical address. Serve una page table aggiuntiva per la doppia traduzione.

La MMU virtuale deve avere due funzioni, usate sempre in sequenza e con la seconda trasparente dal punto di vista del sistema operativo guest:

1. Il G mapping mappa un *guest virtual address* V in un *guest physical address* F .
2. Il H mapping traduce un *guest physical address* F in un *host physical address* F' .

L'effetto combinato è questo: ogni volta che il software guest prova ad accedere ad un *virtual guest address* V , accederà invece ad *host physical address* F' .

Il VMM eseguirà la Trap&Emulate per entrambe queste azioni.

Perché tenere separate le funzioni? Per avere la massima flessibilità:

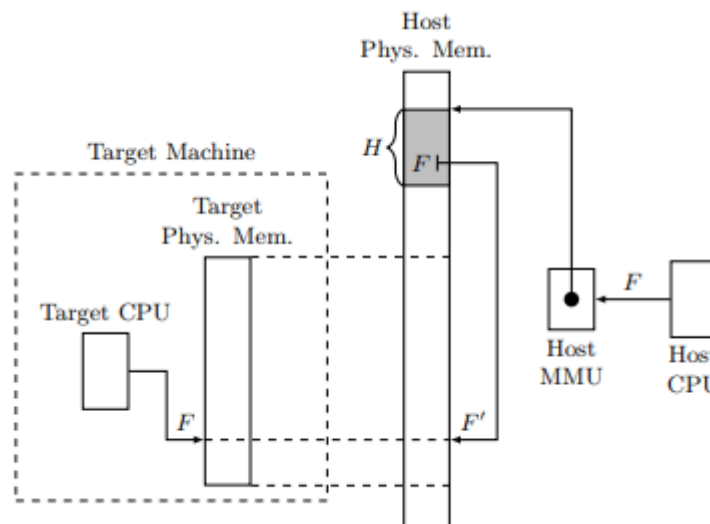


Figure 37 - La virtual MMU.

Di norma è il VMM a gestire la traduzione H. Ma come realizzare questa traduzione H?

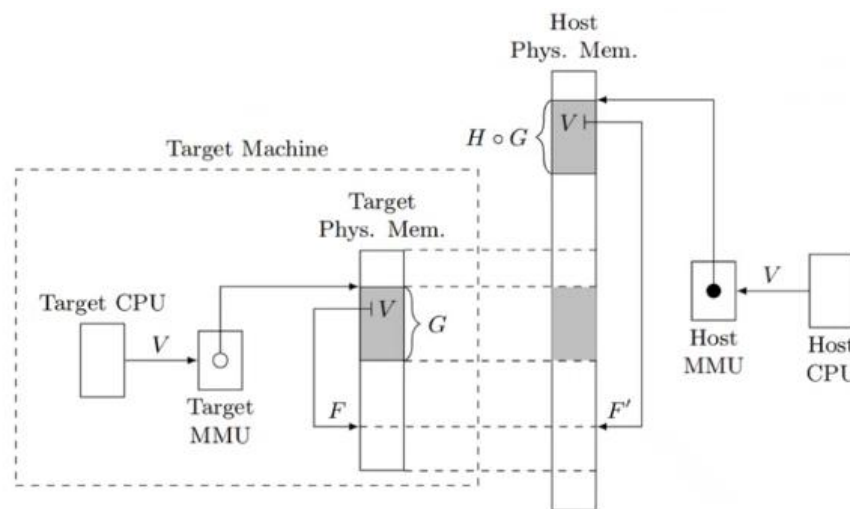


Figure 38 - Virtual MMU e il GH mapping, notare come sia la tabella G che la tabella H sono presenti in memoria RAM.

Il metodo “*brute force*” – Shadow page table

Terremo due tabelle delle pagine: la prima è gestita dal sistema operativo guest (*che esegue la funzione G*), ma la seconda (*che esegue la funzione H*) è gestita dal VMM.

Questo metodo applica delle modifiche alle page table dei guest OS, ovvero il VMM modifica le page table delle VM in modo da implementare la funzione H .

Con questo metodo il VMM cerca di aggiornare la tabella delle pagine host (*H mapping*) subito dopo che il sistema operativo guest ha modificato la sua guest page table (*G mapping*). Le modifiche che il sistema operativo guest può fare sono:

- Cambiare il puntatore PTBR alla MMU (*accade ad ogni cambio di processo*).
- Cambiare le righe della tabella delle pagine (ovvero *cambiare la parte grigia della memoria fisica target, in figura sotto: corrispondente alla zona grigia della parte bassa della memoria fisica host*).

La shadow page table

Nel metodo brute force, il VMM modifica la tabella delle pagine guest in modo da aggiungere un nuovo livello detto **shadow page table**.

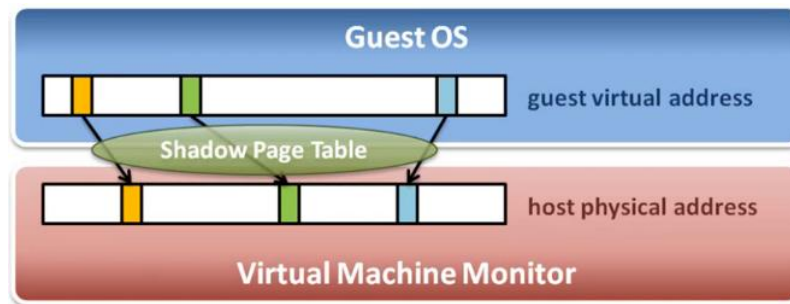


Figure 39 - Metodo brute force.

La shadow table effettivamente implementa entrambe le funzioni G ed H in cascata, l'implementazione della funzione H avviene in un modo trasparente rispetto al sistema operativo guest, il quale sa solo di aver modificato la sua funzione G.

Il VMM modifica la guest page table sostituendola con la shadow page table (calcolata basandosi sulla guest page table). Quindi il sistema operativo guest non si accorge della modifica e quindi traduce il guest virtual address direttamente in host physical address.

In caso di traduzione di un indirizzo cosa succede? Il VMM non viene interpellato.

La shadow table è protetta in scrittura, quindi, come detto prima, ogni possibile modifica causa un cambio di contesto verso il VMM.

- Cambiare il puntatore alla MMU (ovvero un cambiamento nel PTBR), poiché il sistema operativo guest vuole cambiare tabella delle pagine (*accade ad ogni cambio di processo*).
- Cambiare le righe (entry) della tabella, quindi vuole cambiare il mapping di certe aree.

Il VMM eseguirà la Trap&Emulate per entrambe queste azioni.

- Se la modifica è del primo tipo: la tabella nuova viene analizzata e la relativa shadow table viene creata.
- Se la modifica è del secondo tipo: oltre a eseguire le azioni volute aggiornerà anche la shadow table di conseguenza.

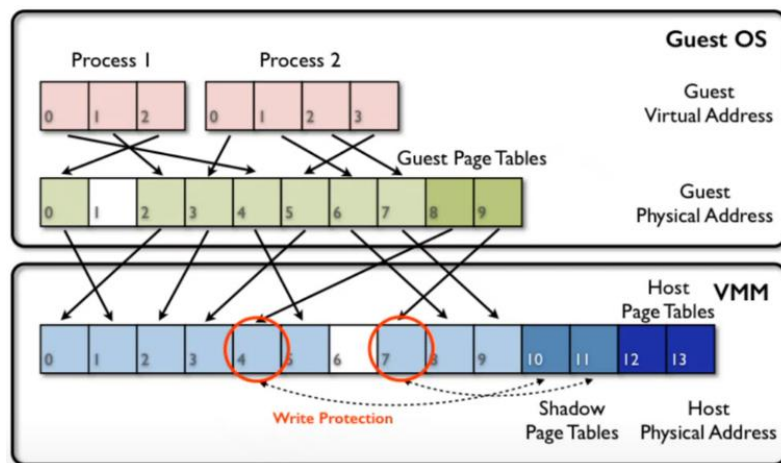


Figure 40 - La shadow table.

L'overhead introdotto da questo livello addizionale si ha quando la page table viene modificata dal guest OS (*a causa dei cambi di contesto verso il VMM*), ma è nullo quando si deve tradurre un indirizzo da guest virtual address ad host physical address, poiché è fatta dalla MMU che è in hardware.

Utilizzo della shadow table

1. **Accesso alla Memoria:** Quando un processo in una macchina virtuale accede a un indirizzo virtuale, questo deve essere tradotto in un indirizzo fisico reale.
2. **Intercettazione:** Il VMM intercetta le operazioni di traduzione della memoria effettuate dal sistema operativo guest.

Viene applicato il modello Trap&Emulate.

3. **Traduzione:** Il VMM utilizza la shadow table per tradurre il *guest physical address* direttamente in un *host physical address*.

Aggiornamento delle Tabelle

Quando il sistema operativo guest modifica le proprie tabelle delle pagine (*ad esempio, per gestire la memoria virtuale*), il VMM aggiorna le shadow page tables per riflettere questi cambiamenti.

L'aggiornamento causa l'applicazione del modello Trap&Emulate.

Prestazioni della shadow table

Questo metodo è veloce quando le applicazioni sono eseguite nella VM, però introduce un grande overhead visto che richiede numerosi cambi di contesto per aggiornare continuamente il livello shadow delle page tabel, ogni volta che queste vengono modificate.

L'uso delle shadow page tables introduce un certo overhead, poiché il VMM deve mantenere e sincronizzare queste tabelle. Tuttavia, sono cruciali per garantire l'isolamento e la sicurezza delle VM, prevenendo accessi non autorizzati alla memoria fisica.

Virtualizzazione dei dispositivi I/O

I dispositivi I/O richiedono spesso delle istruzioni privilegiate per poter comunicare con loro.

Il VMM deve dare l'illusione al sistema operativo guest che sia effettivamente in controllo dell'hardware dei dispositivi e che possa accedervi direttamente (*la shadow table è trasparente ad esso*).

Per dare questa illusione il VMM deve emulare il vero hardware del dispositivo creando un set di strutture dati (*rappresentazioni virtuali*) in memoria primaria dette device models.

All'atto pratico il VMM si comporta da controller delle periferiche in modo da tradurre i comandi inviati alle rappresentazioni virtuali in veri comandi per le vere periferiche.

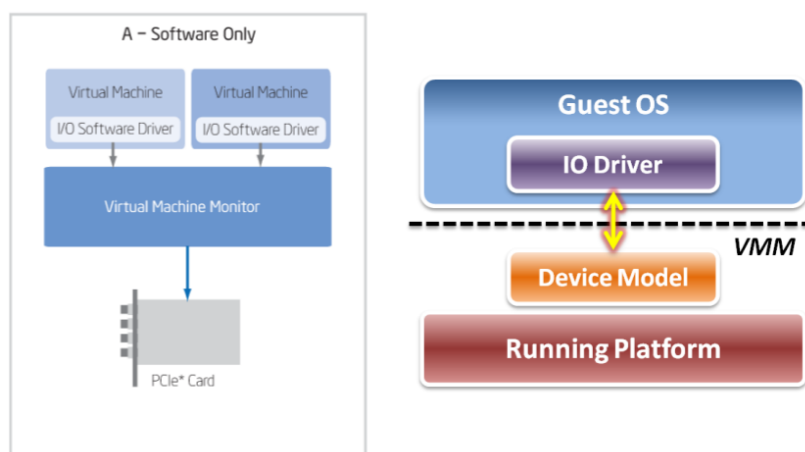


Figure 41 - Virtualizzazione dei dispositivi I/O, a sinistra un dispositivo che non può essere completamente emulato tramite software; a destra il caso opposto.

Dipendentemente dalla periferica, il VMM può anche emularla completamente tramite software (ad esempio un mouse, una tastiera o una stampante che in realtà crea un file pdf e non una vera stampa).

Una scheda di rete invece non può essere puramente software, in quanto richiede hardware corrispondente.

Queste rappresentazioni virtuali (*device models*) sono accedute dal sistema operativo guest tramite istruzioni privilegiate:

- Il VMM legge o aggiorna il device model ogni volta che il sistema operativo guest scrive o legge da esso.

- Il VMM vede le periferiche fisiche e per questo può tradurre le istruzioni I/O eseguite dal sistema operativo guest sui device model e inviare comandi equivalenti alle periferiche fisiche.

Gestione delle richieste di interruzione dei dispositivi fisici

Il VMM deve gestire le richieste di interruzione provenienti dalle periferiche e deve gestirle in maniera trasparente rispetto al sistema operativo guest (*ossia deve sembrare che il VMM non ci sia*).

Host Interrupt Descriptor Table – host IDT

Solo il VMM gestisce la tabella chiamata host Interrupt Description Table (*host IDT*) normalmente inaccessibile al sistema operativo guest. Gestisce anche il kernel code. La CPU host utilizza la host IDT per gestire le richieste di interruzione provenienti dalle periferiche fisiche.

Questa tabella è ovviamente inaccessibile al sistema operativo guest.

Guest Interrupt Descriptor Table – guest IDT

Ogni macchina virtuale ha la sua guest Interrupt Description Table (*guest IDT*), gestita dal sistema operativo guest e che risiede nello spazio di memoria del sistema operativo guest. Ogni entrata della tabella punta ad un handler che il kernel del guest OS ha allocato per la gestione della periferica.

Workflow

Ogni volta che il VMM riceve una richiesta di interruzione dal dispositivo fisico, emulerà (*tramite software*) la relativa richiesta di interruzione create nel sistema operativo guest, in modo da attivare gli handler del sistema operativo guest.

Quest'operazione è semplice ma è costosa in termini di overhead, perché richiede vari step: In questo semplice esempio consideriamo una tastiera virtuale che prende gli input da una vera tastiera fisica.

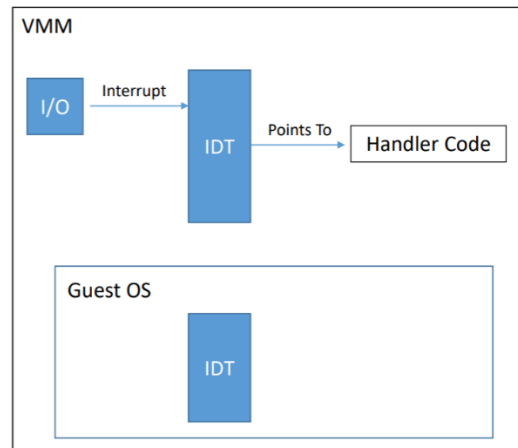


Figure 42 - La richiesta di interruzione del dispositivo viene catturata dal VMM, in particolare vediamo la Host IDT e la Guest IDT.

Ogni volta che il dispositivo virtuale genera un evento, quindi genera un'interruzione nella VM, il VMM deve emulare la ricezione di un'interruzione nella VM come se fosse stata rilevata dal processore. L'interruzione potrebbe provenire da un dispositivo completamente emulato, ad esempio nel caso di un timer, oppure da un dispositivo fisico come una tastiera.

1. **Interruzione:** La tastiera fisica invia dati.
2. Il controller della tastiera invia una richiesta di interruzione alla CPU host.
3. **Intercettazione:** Avviene il primo cambio di contesto (*verso il VMM*).

Nel punto 3 avviene il Trap&Emulate.

4. Il VMM determina quali macchine virtuali devono ricevere questa interruzione.
5. Il VMM scorre la host IDT e individua l'handler da eseguire (*chiamato VMM code*).
6. Il codice dell'handler del VMM viene avviato.
7. Il VMM code dalla Guest IDT legge il codice necessario X di un'interruzione equivalente dentro la macchina virtuale.
8. Avviene il secondo cambio di contesto (*verso il sistema operativo guest*).
 - a. Viene salvato lo stato corrente.
 - b. Si cambia l'istruzione pointer al valore della prima istruzione del codice di interruzione. (INT X)
 - c. Viene ridato il controllo alla macchina guest.
 - d. Viene eseguito il codice di interruzione dato che l'istruzione pointer punta quello.

Nel punto 8 si vede come il VMM inietta un'interruzione della tastiera virtuale nella macchina virtuale attiva. Questo fa sì che la macchina virtuale "*creda*" di aver ricevuto un'interruzione dalla tastiera hardware.

9. Il sistema operativo guest riconosce la richiesta di interruzione e come da norma scorre la sua guest IDT e individua l'handler associato alla richiesta di interruzione lanciata (X).
10. Viene eseguito l'handler all'interno della macchina virtuale.
11. Viene ridato il controllo all'applicazione eseguita nella macchina virtuale.

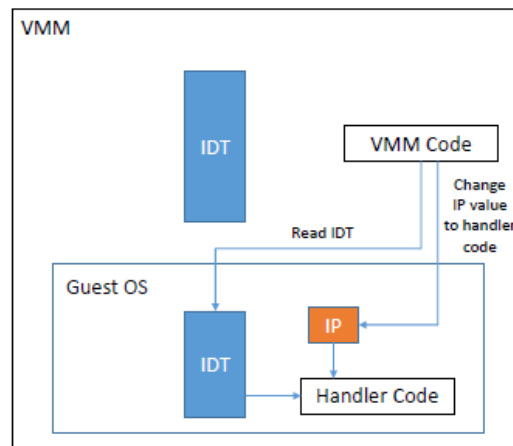


Figure 43 - Gestione di una richiesta di interruzione nella macchina guest.

Cosa avviene se il sistema operativo guest disabilita le interruzioni (*magari perché sta già eseguendo un handler*)? In questo caso il VMM deve aspettare che il sistema operativo guest riattivi le interruzioni.

Workflow - Hardware Interrupt

1. La VM è in esecuzione, accade qualcosa;
2. L'hardware interrompe l'esecuzione;
3. Si ha un cambio di contesto da user a kernel;
4. L'hardware controlla la IDT del VMM ed esegue l'handler;
5. Se c'è un dispositivo virtuale che emula uno fisico che ha triggerato l'interruzione, bisogna ripetere gli step precedenti ed emulare gli handler dentro il guest OS

QEMU – Quick EMUlator

QEMU è un emulatore open-source che esegue una hardware virtualization. Software popolare, che esegue simulazione ed emulazione di architetture (*ARM, Spark, etc...*).

Se l'istruzione set di guest e host sono diversi: QEMU si comporta come emulatore (*lento*), altrimenti come simulatore (*veloce*).

LAB - QEMU and Virsh

- Per prima cosa controlliamo se il supporto hardware è abilitato installando il comando **cpu-checker**.
- Adesso installiamo l'hypervisor **KVM** e **libvirt** il quale include Virsh che consente di gestire le VM sopra KVM.

- Aggiungiamo il nostro utente al gruppo libvirt con il comando ***sudo adduser 'id -un' libvirt***. Questo comando è opzionale dato che lavoriamo in root.
- Per vedere le macchine virtuali correntemente in esecuzione eseguiamo il comando ***virsh list***.
- Per vedere le risorse disponibili nel sistema eseguiamo il comando ***virsh nodeinfo***.
- Per creare una VM ci sono vari metodi, tramite un comando in cui specifichiamo le caratteristiche della macchina, oppure crearla tramite immagini preconfigurate che troviamo in rete (noi scarichiamo la VM con **Cirros**).
- Per distruggere una VM è possibile utilizzare i comandi ***virsh destroy Ubuntu*** (non la distrugge, la stoppa e basta) o ***virsh undefine Ubuntu*** (la distrugge).
- La VM può essere fermata o sospesa con i comandi ***virsh shutdown Cirros***, ***virsh suspend Cirros***, ***virsh autostart Cirros*** (imposta l'autostart al boot dell'host OS).
- Le VM in virsh sono connesse ad una rete virtuale creata all'interno delle VM, questa rete è creata utilizzando un **Linux bridge**.
- Con ***brctl show*** eseguito non sulla macchina Cirros è possibile visualizzare il bridge.
- Con il comando ***virsh net-list*** vengono mostrate tutte le reti connesse.
- Con il comando ***virsh net-dumpxml default*** è possibile visualizzare la configurazione di rete.
- È possibile creare nuove configurazioni di rete:
 - Scriviamo le configurazioni in un file XML.
 - Definiamo la nuova rete partendo da questo file (***virsh net-define net.xml***).
 - Impostiamo l'avvio automatico della rete (***virsh net-autostart net***).
 - Avviamo la rete (***virsh net-start net***).
 - Creiamo una nuova NIC virtuale e la colleghiamo alla rete.
 - Configuriamo la NIC nella VM.
- Adesso vogliamo creare un secondo storage e connetterlo alla VM.
- Tutti gli storage sono contenuti in uno **storage pool**.
- Nella VM Ubuntu creiamo un nuovo volume e lo colleghiamo alla VM.
- Nella VM Cirros creiamo le partizioni, formattiamo le partizioni, montiamo e utilizziamo la partizione.
- Infine, puliamo la VM.

Capitolo 6 - Hardware Assisted Virtualization

Non è un nuovo tipo di virtualizzazione, ma è una serie di tecniche che permettono alla full virtualization di andare molto più veloce.

Intel VMX

La tecnologia VMX introduce due nuove modalità operative: **root** e **non-root**. Queste due modalità sono **ortogonali** rispetto a quelle già esistenti. Con queste due nuove modalità è possibile definire quattro combinazioni:

- **root/system**

- **root**/user
- **non-root**/system
- **non-root**/user

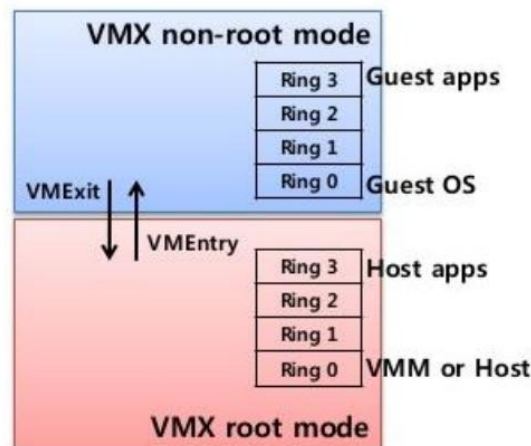


Figure 44 – Diagramma che mostra le due modalità e i quattro ruoli che può avere un processo.

Le due modalità **root** sono progettate per eseguire il codice del VMM e del sistema operativo host.

Le due modalità **non-root** sono progettate per il codice della macchina virtuale, nello specifico per eseguire il software guest all'interno della macchina virtuale

Il nostro obiettivo è quello di ottimizzare i cambi di contesto: fare in modo che per alcune istruzioni privilegiate non serva un cambio di contesto verso il VMM ma la macchina virtuale le possa eseguire direttamente sull'hardware host.

Un altro nostro obiettivo è fare in modo che ogni volta che la macchina virtuale esegue un'istruzione, l'hardware eseguirà un controllo (*senza cambio di contesto*) che verifichi la correttezza di quella esecuzione.

Il VMM può specificare un set di istruzioni privilegiate che possono essere eseguite all'interno della VM senza richiedere cambio di contesto ed emulazione.

Se il VMM invece vuole il controllo su un set d'istruzioni, quella lista verrà esclusa dalle istruzioni che possono essere eseguite nella VM autonomamente senza richiedere cambio di contesto ed emulazione.

Nel caso in cui il sistema operativo guest provi ad eseguire istruzioni vietate, si occorrerà alla normale procedura di Trap&Emulate.

Modalità Root (Root Mode)

Quando una CPU è in modalità **root**, il sistema operativo host (*hypervisor*) ha accesso completo a tutte le risorse fisiche del sistema.

In questa modalità, il VMM è responsabile della gestione delle risorse hardware, inclusa la distribuzione delle risorse alle macchine virtuali ospitate e l'arbitraggio dei conflitti di accesso alle risorse fisiche.

Il VMM esegue i cosiddetti "*hypervisor calls*" per controllare e coordinare l'esecuzione delle macchine virtuali ospitate.

Modalità Non-Root (Non-Root Mode)

La modalità **non-root** permette alla macchina virtuale di:

- Eseguire istruzioni privilegiate senza cambio di contesto (*solo alcune istruzioni privilegiate, decise dal VMM*).
- Gestire direttamente alcune interruzioni (*senza la traduzione da parte del VMM*).

Quando una CPU è in modalità **non-root**, esegue il codice delle macchine virtuali.

In questa modalità, le macchine virtuali operano come se fossero eseguite su hardware fisico dedicato, senza avere consapevolezza della presenza dell'hypervisor o delle altre macchine virtuali.

Modalità Root/User

Questo livello è stato introdotto per supportare l'esecuzione di applicazioni in un sistema, al di fuori delle VM, ovvero eseguire funzionalità al di fuori di una VM e che non vengono eseguite al livello più alto di privilegio.

Operazioni di Intel VMX

L'introduzione delle due modalità e queste nuove istruzioni ha migliorato molto la virtualizzazione, consentendo alla macchina virtuale di eseguire istruzioni privilegiate senza subire un cambio di contesto. Si vuole eseguire il più possibile una VM effettuando il minimo numero di cambi di contesto verso il VMM.

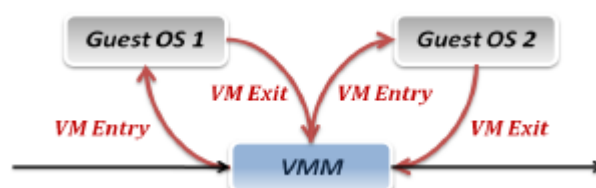


Figure 45- Le operazioni fondamentali di Intel VMX.

VMEXIT

Eseguibile in modalità **non-root**. Fa uscire la CPU dalla modalità **non-root** e riporta il controllo all'hypervisor, quindi in modalità **root**.

Una `VMEXIT` può essere causata da diversi eventi:

- L'esecuzione di istruzioni sensibili alla virtualizzazione.
- L'accesso a risorse condivise.
- Richieste di interruzione.
- Eccezioni hardware o altre condizioni che richiedono l'intervento dell'hypervisor.

Quando si verifica una `VMEXIT`, la CPU salva lo stato corrente della macchina virtuale, incluso il contenuto dei registri e altre informazioni pertinenti, e passa il controllo all'hypervisor.

L'hypervisor è quindi responsabile di gestire l'evento che ha causato la `VMEXIT`, ad esempio gestendo le operazioni I/O, emulando l'istruzione non supportata o rispondendo a richieste di interruzione.

Dopo aver gestito l'evento, l'hypervisor può decidere di eseguire un'altra macchina virtuale, ripristinando il suo stato e utilizzando l'istruzione `VMLAUNCH`.

VMENTRY

Eseguibile solo in modalità **root**. Viene eseguita dall'hypervisor per far entrare la CPU nella modalità di esecuzione della macchina virtuale (*modalità non-root*) e avviare l'esecuzione del codice della macchina virtuale scelta.

Prima di eseguire `VMENTRY`, l'hypervisor deve configurare correttamente lo stato della CPU per la macchina virtuale ospite, incluso il caricamento delle strutture di controllo della VM e l'impostazione dei registri appropriati.

`VMENTRY` avvia quindi il ciclo di esecuzione della macchina virtuale, durante il quale la CPU esegue le istruzioni della macchina virtuale fino a quando non si verifica una `VMEXIT`.

VMRESUME

Eseguibile solo in modalità **root**. Il VMM riprende l'esecuzione di una macchina virtuale precedentemente stoppata. Questa istruzione viene utilizzata per restituire il controllo alla VM in seguito ad un cambio di contesto.

Dopo che una macchina virtuale ha eseguito una `VMEXIT`, il VMM può decidere di mettere in pausa temporaneamente la macchina virtuale; Quando è pronta per riprendere l'esecuzione della macchina virtuale, l'hypervisor esegue l'istruzione `VMRESUME`. Ad esempio, al completamento di un evento di I/O o un'operazione di sincronizzazione.

In breve, la `VMRESUME` ripristina lo stato della CPU alla macchina virtuale e riprende l'esecuzione delle istruzioni dalla posizione in cui è stata interrotta.

Dopo l'esecuzione di `VMRESUME`, la CPU torna alla modalità `non-root` e continua ad eseguire le istruzioni della macchina virtuale.

`VMLAUNCH`

Eseguibile solo in modalità `root`. Esegue una nuova macchina virtuale, questa istruzione causa il passaggio dalla modalità `root` alla modalità `non-root`.

Quindi dopo questa istruzione la CPU host eseguirà le istruzioni della macchina virtuale.

Interrupt Remapping Table

Come modifichiamo l'architettura per permettere ad una macchina virtuale di gestire direttamente le interruzioni (*le quali sono inviate al sistema operativo host*) senza effettuare un cambio di contesto con il VMM? (*quindi senza una VMEXIT*).

In parole povere, permette di inoltrare tutte le interruzioni hardware direttamente alla macchina virtuale. Questa operazione è necessaria per ridurre il numero di cambi di contesto.

La IDT ha una entry per ogni possibile tipo di richiesta di interruzione possibile che si può avere nel sistema. Ogni entry associa la richiesta di interruzione fisica (*quella effettivamente inviata dal dispositivo*) alla relativa richiesta di interruzione da utilizzare nella VM.

Ogni entry della IRT ha un puntatore verso la IDT di una VM se si vuole instradare la richiesta d'interruzione direttamente verso la VM, ovvero se si vuole far eseguire dall'hardware direttamente l'handler selezionato dalla VM. Una entry della IRT altrimenti può puntare direttamente alla IDT del VMM nel caso in cui si voglia che l'hypervisor prenda il controllo.

La IDT del VMM è completamente separata dall'hardware (*è in memoria RAM*), ed è quello il vero handler che va chiamato, poiché deve interagire con il dispositivo fisico.

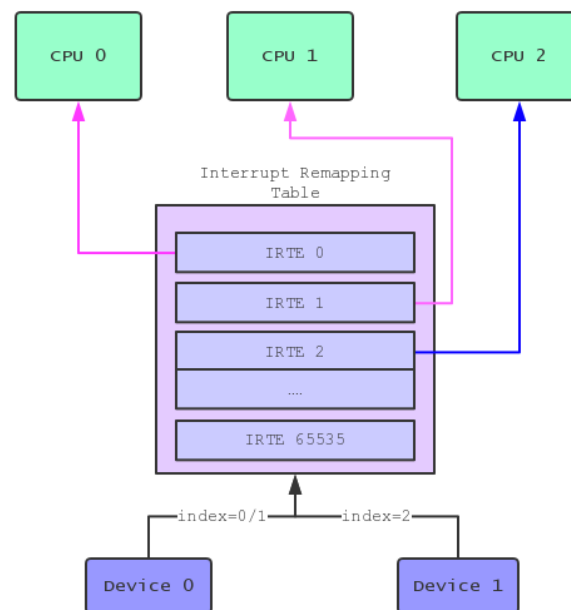


Figure 46- Diagramma semplificato dell' interrupt remapping table.

La IRT consente di gestire le interruzioni in modo da garantire che le richieste inviare da dispositivi hardware verso macchine virtuali siano correttamente instradate e gestite (*ossia gestite dai relativi handler della macchina guest*) senza interferire con altre macchine virtuali o senza interpellare il VMM.

Gestione della IRT

La IRT è una struttura dati riempita e gestita dinamicamente dal VMM; ogni riga della IRT punta ad una riga della host IDT (*la quale contiene l'indirizzo del handler da eseguire*).

Gestione dei vettori di interruzione fisici

L'hypervisor ottiene i vettori di interruzione direttamente dal hardware sottostante. Questi vettori sono gli indirizzi fisici nella IDT che identificano i diversi dispositivi I/O.

Gestione dei vettori di interruzione virtuali

I vettori di interruzione virtuali, d'altra parte, sono gli indirizzi che l'hypervisor assegna a ciascuna macchina virtuale per mappare le interruzioni fisiche alle interruzioni virtuali. Questi vettori sono unici per ogni macchina virtuale.

Utilizzo della IRT

Quando si verifica un'interruzione hardware la CPU host consulta la IRT (*precedentemente compilata dal VMM*): L'unico compito del VMM è quello di tenere aggiornata la IRT.

1. **Interruzione:** Un dispositivo fisico genera un'interruzione.

2. **Intercettazione da parte del Hardware di Remapping:** L'interruzione arriva alla CPU, ma prima di essere gestita dal VMM, viene intercettata dal hardware di remapping delle interruzioni.
3. **Consultazione della IRT:** L'hardware consulta la IRT, che contiene le mappature tra le interruzioni hardware e le macchine virtuali destinatarie.
4. **Rimappatura dell'interruzione:** Per ogni macchina virtuale destinataria, l'hardware rimappa l'interruzione a un identificatore di interruzione virtuale che è specifico per la VM destinataria.
5. **Iniezione dell'interruzione:** L'hardware consegna l'interruzione virtuale alle macchine virtuali appropriate.

Quando l'interruzione arriva alla macchina virtuale, questa la gestisce come farebbe normalmente su un sistema fisico non virtualizzato. Tutto completamente implementato in hardware, perciò il VMM non viene interpellato.

Esempio utilizzo IRT - Istruzioni INT & IRET

Con questa tecnica l'esecuzione di istruzioni privilegiate è molto più semplice ed efficiente. Nello specifico le istruzioni INT e IRET possono essere eseguite senza richiamare il VMM, questo perché non è più necessario emularle.

1. L'applicazione guest (*che gira in modalità non-root*) richiama la INT.

Se la INT è incisa nel set di istruzioni eseguibili direttamente, allora si può procedere senza il cambio di contesto verso il VMM.

2. La CPU tramite la IRT prende l'handler virtuale relativo alla richiesta di interruzione inviata dalla macchina guest.
3. Viene eseguito l'handler della macchina virtuale relativo alla INT.
4. Quando l'handler della INT termina avrà una IRET.

Se la IRET è incisa nel set di istruzioni eseguibili direttamente, allora si può procedere senza il cambio di contesto verso il VMM.

5. La CPU tramite la IRT prende l'handler virtuale relativo alla richiesta di interruzione inviata dalla macchina guest.
6. Viene eseguito l'handler della macchina virtuale relativo alla IRET.

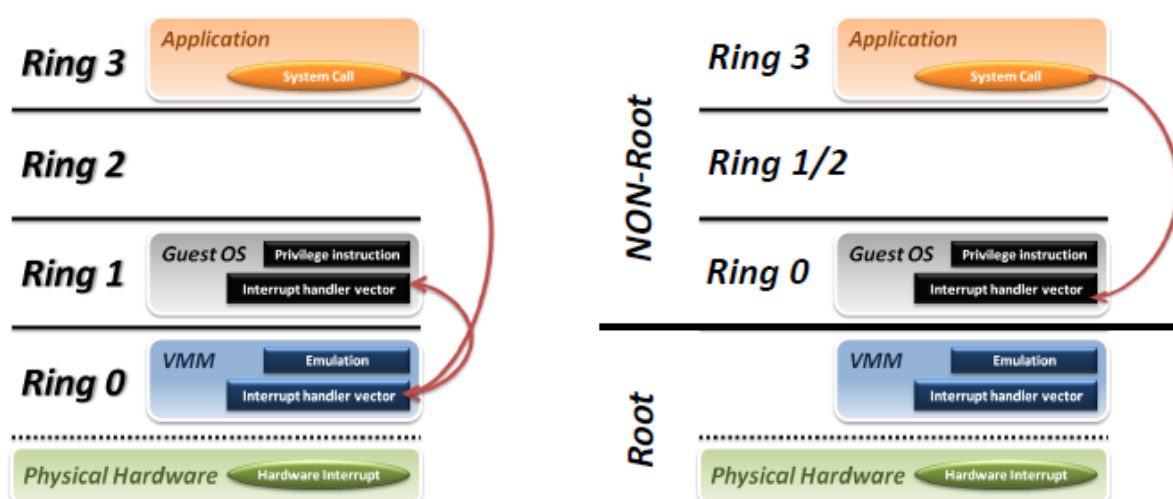


Figure 47 - La differenza tra il "prima" e il "dopo" l'introduzione della IRT.

Nel metodo senza supporto hardware si richiedeva che fosse il VMM a emulare la richiesta di interruzione per scatenare l'attivazione dell'handler della macchina virtuale voluto.

In particolare, quando l'applicazione in esecuzione nella VM invoca una system call (ad esempio l'istruzione INT), la INT triggera l'esecuzione di un handler che può essere l'handler associato a quella specifica interruzione nella IDT della VM. Questo handler simulerà l'esecuzione del corrispondente handler all'interno della VM. Inoltre, l'handler contiene istruzioni privilegiate, quindi ci sono continui cambi di contesto.

Con il supporto hardware c'è solo un cambio di contesto, ed è lo stesso numero di cambi di contesto che si hanno in un sistema reale.

In figura vediamo che il codice del VMM è eseguito in modalità root/system, ovvero il livello con massimo privilegio, mentre il guest OS è eseguito in modalità non-root, in particolare viene eseguito nella modalità non-root/system, mentre in non-root/user vengono eseguite le applicazioni guest, questo livello è quello di privilegio minimo.

Virtual Machine Control Structure – VMCS

Intel VMX aggiunge una nuova struttura chiamata "Virtual Machine Control Structure" (VMCS) che contiene tutte le informazioni necessarie per gestire la modalità **non-root** (contiene le informazioni dello stato della macchina virtuale, quindi il valore dei registri). Ovvero contiene il set di istruzioni che si possono eseguire senza cambio di contesto verso il VMM.

Il VMM può gestire diversi VCMS in genere uno per ogni processore di ogni macchina virtuale. Tuttavia, solo un VMCS alla volta è quello corrente sul processore fisico.

Il processore fisico ha un apposito registro (*supporto hardware*) che punta al VMCS corrente e tutte le istruzioni della macchina virtuale si riferiscono al VMCS corrente.

Il supporto hardware non finisce al semplice registro, anzi si fornisce un set istruzioni che vanno a manipolare questa struttura in modo molto efficiente: ad esempio il VMM può utilizzare l'istruzione `VMPTRLD` per caricare l'indirizzo di un VMCS nel registro, rendendo tale VMCS quello corrente. Questa struttura dati è letta e aggiornata molte volte (*ogni volta che ho un cambio di contesto*).

La struttura VMCS include:

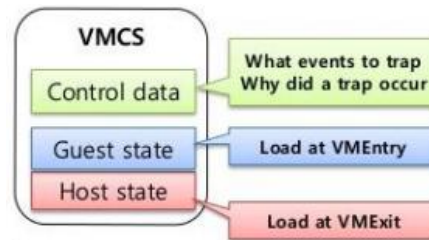


Figure 48 - Contenuto del VMCS.

- **Guest state**
 - Lo stato corrente della CPU virtuale della macchina virtuale, necessario per riprendere l'esecuzione di una macchina virtuale dopo un cambio di contesto.
 - Contiene anche l'istruzione pointer alla prima istruzione da eseguire in caso di ripartenza.
 - È lo stato da ripristinare in caso di `VMLAUNCH`.
- **Host state**
 - Stato del processore host prima del `VMLAUNCH` della macchina virtuale; può contenere lo stato del VMM prima del lancio della macchina virtuale.
 - È lo stato da ripristinare in caso di `VMEXIT`.
 - Contiene la prima istruzione da eseguire quando il VMM riprende il controllo.
- **VM Execution Control**
 - Specifica il tipo di operazioni che sono permesse durante la modalità **non-root**.
 - Si usa per effettuare i controlli prima di ogni istruzione privilegiata.
 - Contiene un bitmap il quale serve per capire se un'istruzione può essere eseguita in root mode oppure occorre effettuare una `VMEXIT`.
 - Eventuali infrazioni causano una `VMEXIT`.
- **VM Enter Control**
 - Flags che determinano comportamenti opzionali per la transizione da **root** a **non-root**.
 - Lo scopo di questa struttura è quello di simulare interruzioni esterne o per iniettare eccezioni, se uno dei flag è settato, per dire al processore di simulare un'interruzione.
 - Devono essere eseguite le seguenti operazioni:
 - Salva lo stato sullo stack del guest;
 - Cerca la tabella dei descrittori degli interrupt del guest per determinare l'indirizzo dell'interrupt handler;

- Modifica i registri della CPU per eseguire l'handler.
- **VM Exit Control**
 - Flags che determinano comportamenti opzionali per la transizione da **non-root** a **root**.
- **VM Exit Reason**
 - Contiene informazioni relative al motivo dell'ultimo evento `VMEXIT`.

RAM Management

Ogni sistema operativo guest è convinto di essere in controllo completo della RAM fornita dal VMM, questa RAM pseudo-fisica deve essere sembrare contigua e iniziare dall'indirizzo 0. Si vuole avere un meccanismo dentro il VMM che possa assegnare porzioni della memoria fisica della macchina alle guest physical memory delle macchine virtuali.

Ciò è possibile grazie ad ulteriore supporto hardware, in particolare serve un'estensione della MMU: ora esegue due funzioni di traduzione una gestita dal sistema operativo guest e l'altra dal VMM. L'hardware della MMU deve poter ospitare due tabelle di traduzione delle pagine.

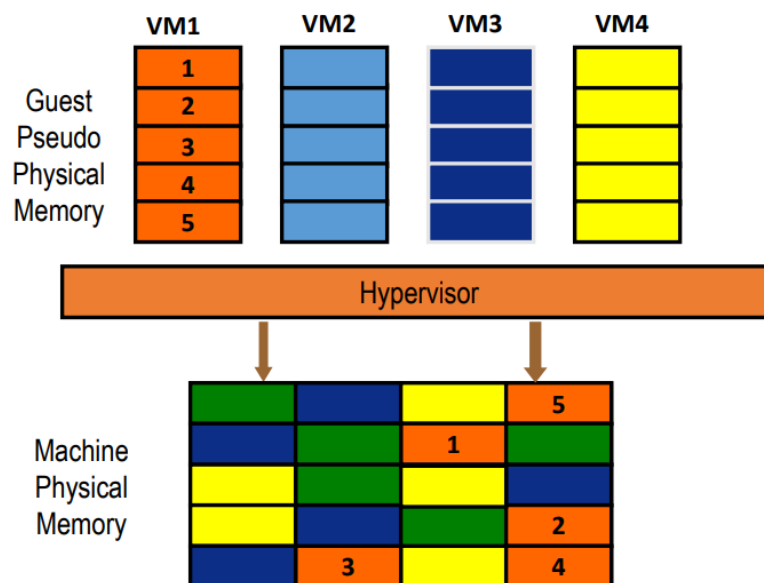


Figure 349 - Obiettivo del VMM riguardo la memoria primaria.

Tabella delle pagine estesa

Il metodo brute force ha uno svantaggio significativo, ovvero le shadow table necessitano di essere introdotte nelle page table delle VM e queste page table vanno aggiornate ogni volta che il guest OS effettua una modifica, effettuare una modifica comporta un cambio di contesto.

Vogliamo modificare la MMU in modo da implementare questi due step di traduzione in hardware, ovvero mappare G ed H direttamente nella MMU.

Grazie a questa estensione non serve che la macchina virtuale subisca un `VMEXIT` ogni volta che necessita di modificare la tabella delle pagine guest.

Gli svantaggi sono l'aumento della complessità circuitale della MMU e un pò di overhead in più nella traduzione di un indirizzo. Grazie a dei meccanismi di cache il secondo svantaggio viene mitigato.

La MMU host ora contiene due puntatori:

- Uno alle tabelle delle pagine delle macchine virtuali G .
- Uno alle tabelle delle pagine del sistema operativo host H .

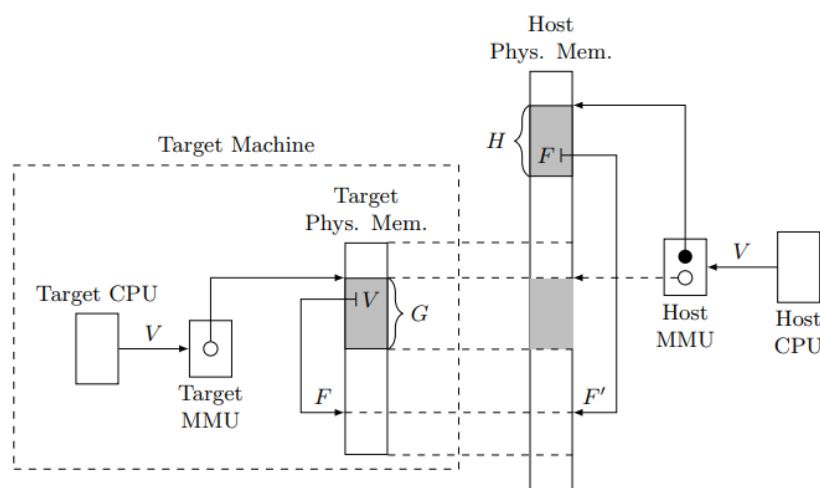


Figure 50 - Tabella delle pagine estesa, notare i due puntatori nella Host MMU.

La tabella delle pagine estesa ha lo scopo di applicare la funzione G e H in modo sequenziale. Ora la MMU host ha due puntatori uno che punta alla tabella delle pagine host e uno che punta alla tabella delle pagine guest.

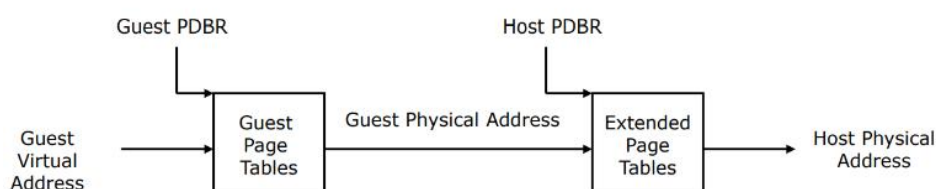


Figure 51 - Workflow della tabella delle pagine estesa.

Come vedremo a breve, con questo supporto hardware non serve fare `VMEXIT` ad ogni modifica di un indirizzo.

Questo metodo ha un overhead nella traduzione perché richiede l'accesso a due tabelle, la guest page table e la extended page table. Il numero dei livelli delle due page table è solitamente maggiore della shadow page table.

Ad ogni accesso alla memoria: La MMU host esegue (*in hardware*):

1. La traduzione G, ottenendo l'indirizzo fisico del guest.
2. La traduzione H, ottenendo l'indirizzo fisico dell'host dall'indirizzo fisico del guest.

Per mitigare questo problema di overhead nella traduzione degli indirizzi si ricorre al caching delle traduzioni (*TLB*).

I/O Management

Dobbiamo fare in modo che le nostre macchine virtuali possano accedere ai dispositivi I/O nel modo più efficiente possibile.

Le operazioni di I/O si possono ridurre a due tipi: Funzione che triggera una funzione o che fornisce dei dati alla periferica (*quindi trasferimento dal processore verso lo spazio di I/O in RAM*) e viceversa, ovvero la periferica fornisce all'applicazione in esecuzione o al sistema operativo, dei dati.

In genere l'I/O è gestito tramite emulazione, quindi l'hypervisor espone delle rappresentazioni virtuali e le macchine virtuali interagisce con quest'ultime, questo comporta un grande spreco di tempo. Questo può essere accettabile per alcune periferiche per le quali il guest OS non necessita grande velocità.

Hardware passthrough

Permettere ad un dispositivo di essere usato direttamente dalla macchina virtuale senza fare cambi di contesto, traduzioni o interventi del VMM. Quindi riducendo molto l'overhead.

Ovviamente ciò porta al problema dell'accesso esclusivo sulla periferica fisica (*unico svantaggio di questa strategia, nel caso delle periferiche emulate ogni macchina virtuale ha la sua*). Ovvero solo una VM può accedere alla periferica fisica in un certo momento.

Esempi in cui questa strategia è fondamentale?

- **NIC:** Ogni macchina fisica ne ha sempre solo una e di solito è anche un collo di bottiglia, meglio cercare di non rallentarla più del necessario.
- **GPU:** Necessaria per gli acceleratori che sfruttano l'intelligenza artificiale, il problema è in termini di performance e ogni costruttore ha i suoi driver, la sua architettura, quindi è difficile virtualizzare, ognuno ha i suoi requisiti.

Ovviamente per avere questo approccio è necessario del supporto hardware. Nota che questi meccanismi non valgono (*e non sono fisicamente implementati*) per tutte le periferiche, ma solo per quelle necessarie e quindi non totalmente emulate tramite software.

Per abilitare l'hardware passthrough abbiamo bisogno di:

- **Mapping diretto** dell'I/O fisico nella memoria virtuale della macchina virtuale, in modo che la macchina virtuale possa scrivere e leggere direttamente dai registri della periferica.
- **Assegnamento diretto** delle interruzioni alla macchina virtuale, questa interruzione è direttamente gestita senza un cambio di contesto verso il VMM.

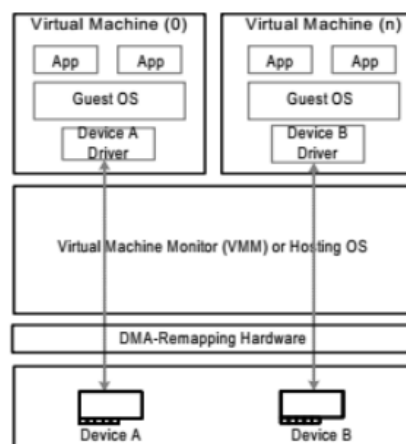


Figure 52 - Assegnamento diretto.

Direct mapping dello spazio I/O – I/O Bitmap

La macchina virtuale può effettuare operazioni di lettura e scrittura nello spazio di I/O del dispositivo senza cambi di contesto o interventi del VMM.

Bisogna fare una nota: non tutti i dispositivi saranno accessibili in questo modo ma richiederanno lo schema Trap&Emulate. Per i dispositivi per cui permettiamo questa cosa definiamo una struttura dati chiamata I/O Bitmap, questa struttura è inclusa in VMCS.

È responsabilità del VMM di inizializzare il bitmap per tutte le macchine virtuali.

I/O Bitmap ha un bit per ogni possibile indirizzo dello spazio di I/O e server per regolarne l'accesso (*1 significa che l'accesso è consentito senza cambio di contesto*). Di solito il Bitmap ha dimensione di qualche Kilobyte.

In modalità **non-root**, quando viene eseguita un'operazione nello spazio I/O, la CPU per prima cosa controllerà la Bitmap per tutti gli indirizzi usati dalle istruzioni in e out.

In base al valore del suddetto bit la CPU potrebbe completare l'operazione o causare un **VMEXIT** (*magari il dato non è pronto*). Ad esempio, in modalità non root, la CPU controlla prima il bitmap, se è settato a 1, viene completata un'operazione di lettura/scrittura, altrimenti una VM restituisce il controllo al VMM.

Nel caso in cui sia permesso: Il supporto hardware, tramite la IRT, è capace di eseguire l'handler del sistema operativo guest corrispondente. Tutto senza interpellare il VMM.

Interruzioni

VMX consente due opzioni per la gestione delle interruzioni:

- L'interruzione causa una `VMEXIT`, quindi l'interruzione viene gestita dal VMM.
- L'interruzione è gestita direttamente dalla VM senza `VMEXIT`.

Nel caso di hardware passthrough è adottata la seconda opzione, l'interruzione viene gestita direttamente dalla VM, c'è un cambio di contesto all'interno della VM ovvero l'handler dell'interruzione in quella VM viene eseguito direttamente senza intervento del VMM.

L'hardware deve essere in grado di accedere direttamente all'IVT del guest OS ed eseguire il corrispondente handler del guest OS. Il guest OS gestisce la sua IVT tramite la IRT

Ma cosa succede se la macchina virtuale non è in esecuzione?

Stati di una macchina virtuale

Ma manca qualcosa...cosa succede se ricevo un interrupt e la macchina virtuale non è in esecuzione?

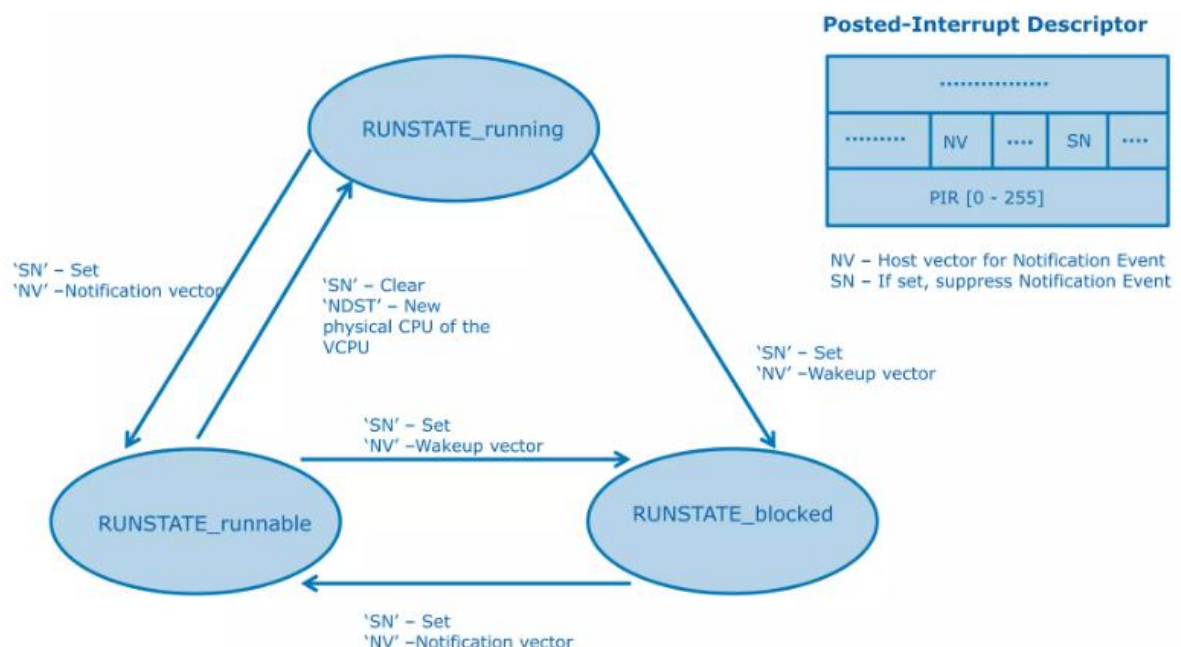


Figure 53 - Diagramma di stato di una macchina virtuale, "blocked" è "halted", invece "runnable" è "ready"..

Il sistema passthrough necessita di poter fare le sue cose anche quando la macchina virtuale non è in esecuzione ossia quando la macchina si trova in stato `Altd`.

Running State

La CPU virtuale sta eseguendo codice usando la CPU host. In questo caso l'interruzione viene direttamente gestita dal processore host grazie al meccanismo passthrough esposto prima senza l'intervento del VMM.

Ready State

La CPU virtuale eseguirà istruzioni non appena avrà la CPU host. La macchina virtuale sta attendendo di essere “*schedulata*” dal VMM.

In questo caso l'interruzione viene memorizzata da qualche parte per essere gestita in un secondo momento (*quando la macchina virtuale ritorna in stato running, in quel caso verrà ricevuta dalla macchina virtuale come se l'avesse ricevuta in running state*); l'intervento del VMM è necessario per la memorizzazione.

Halted State

La macchina virtuale è ferma (*come se avesse la spina scollegata*) non può essere eseguita dal VMM. L'esecuzione può essere ripresa andando in ready in caso di interruzione.

È lo stato in cui la VM viene messa quando il kernel del guest OS esegue l'istruzione *hlt*. Questo stato è l'equivalente di un PC spento. L'hypervisor, per svegliare la macchina, richiede un input esterno, come ad esempio l'utente che preme un bottone in un'interfaccia di gestione, oppure può essere la ricezione di un'interruzione. Quindi la VM passa da halted a ready in seguito ad un'interruzione.

In caso di interruzione, quest'ultima deve essere memorizzata e la macchina virtuale deve essere portata in stato di ready.

In questo caso l'interruzione viene memorizzata da qualche e la macchina virtuale viene messa in stato ready; l'interruzione verrà gestita nello stato running; l'intervento del VMM è necessario per la memorizzazione e per il passaggio di stato.

Nel caso in cui la macchina virtuale sia in stato halted, la IRT non punterà all'handler da equire ma ad una struttura dati chiamata Posted Interrupt Descriptor Table.

Il meccanismo che consente di memorizzare un'interruzione e posticipare la sua esecuzione è detto posted interrupt mechanism, il quale consente di memorizzare i segnali d'interruzione e notificare la VM di questi segnali.

Quando la VM recupera la sua esecuzione, questo meccanismo fornisce una struttura dati addizionale, il posted interrupt descriptor (PID).

Posted Interrupt Descriptor Table (PID)

Struttura dati che contiene tutti i dati per poter implementare il sistema di gestione delle interruzioni posticipato.

Il PID è usato nella IRT: ogni riga della IRT punta ad un interrupt vector (*se l'interruzione deve essere gestita dal VMM*) oppure al PID il quale punta all'interrupt vector (*se la macchina virtuale è attiva*). Quindi la IRT non punta più ad un handler.

Ogni volta che viene generata una richiesta di interruzione, la IRT decide quale interrupt vector o PID gestirà l'interruzione.

Il PID è contenuto nel VMCS e contiene.

Posted Interrupt Request (PIR)

Struttura che contiene un bit per ogni possibile vettore delle interruzioni è qua che le interruzioni sono “*postate*” settando il relativo bit a 1.

Suppress Notification (SN)

Flag che determina se il controller *dopo* il posting dell'interruzione nel PIR debba anche notificare la CPU (SN = 0) o no (SN = 1).

Notification Vector (NV)

In caso di notifica alla CPU, in questo campo troverò l'indirizzo del vettore delle interruzioni che deve essere usato. Questo campo contiene il vettore delle interruzioni visto dal processore. Serve per notificare le interruzioni alla VCPU della VM.

Aggiornamento del PID

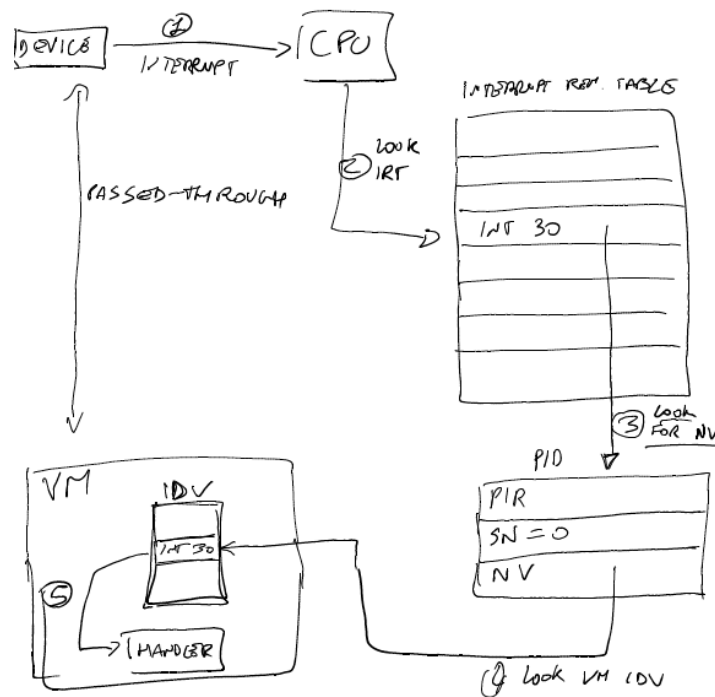
- Quando la VM va in **running**, il VMM mette SN = 0 e a NV assegna l'**Active Notification Vector** che punta alla IDT della VM.
- Quando la VM va in **ready**, il VMM mette SN = 1.
- Quando la VM va in **halted**, il VMM mette SN = 0 e a NV assegna il **Wakeup Notification Vector** che sveglia la VM.

Quando lo stato della VM entra in running, deve controllare anche il PIR, se ci sono dei bit settati, deve settare il NV conseguentemente quando entra nella VM. In questo modo il processore potrà gestire le interruzioni ricevute mentre la VM non era in esecuzione.

Macchina virtuale in “running”

In tutti i 5 i punti, il VMM non viene mai interpellato.

1. Avviene la richiesta di interruzione da parte di un device.
2. La CPU guarda la IRT e legge l'indirizzo del PID.
3. Il SN = 0 allora la macchina è "running" e non c'è soppressione delle interruzioni.
4. Si guarda il valore di NV.
5. Dato che la macchina è in stato di "running", il NV punta alla IDT della macchina virtuale.
6. Si notifica l'interruzione usando l'interrupt descriptor vector e quindi l'handler del sistema operativo guest viene chiamato.



ACTIONS FROM 1 TO 5 EXECUTED IN
SERVERS WITHOUT THE NEED
FOR A CONTEXT SWITCH FOR
VMM INTERVENTION

Figure 54 . Workflow se la macchina virtuale è in stato di running.

Macchina virtuale in “ready”

1. Avviene la richiesta di interruzione.
2. La CPU guarda la IRT e legge l'indirizzo del PID.
3. Setta a 1 il relativo bit del PIR.
4. SN = 1 allora la macchina è in stato di “ready”.
5. Viene messo ad 1 il bit corrispondente nel PIR.

I seguenti step vengono effettuati dopo che la VM va in “running”.

Dopo un pò la macchina virtuale va in stato di “running”: Il VMM setta SN a 0 e avvalora NV con l'indirizzo che punta allo IDV della macchina virtuale.

Se alla transizione verso running c'è un qualche bit nel PIR a 1, allora deve avvalorare NV in modo opportuno prima di mandarla in esecuzione.

6. Grazie al valore di NV si accede alla IDT della macchina virtuale, per notificare l'interruzione.
7. Si notifica l'interruzione, l'handler del sistema operativo guest viene chiamato.

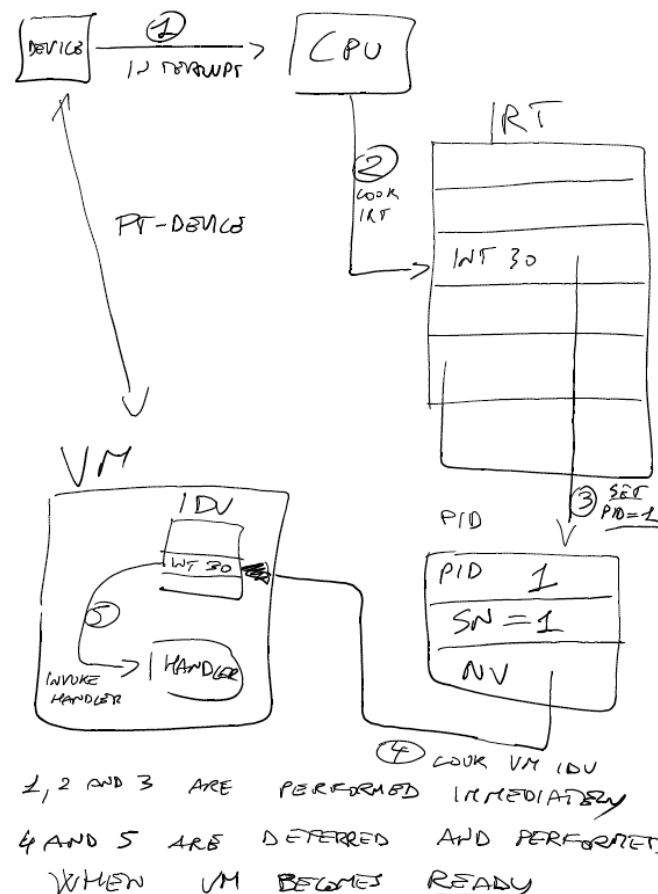


Figure 55 - Workflow se la macchina virtuale è in stato di ready.

Macchina virtuale in "halted"

1. Avviene la richiesta di interruzione.
2. La CPU guarda la IRT e legge l'indirizzo del PID.

Se SN = 0 allora la macchina è "running" o "halted"

3. Si setta il relativo bit del PIR a 1 e si guarda il valore di NV.

NV punta allo Wakeup Notification Vector (WNV) della macchina virtuale.

4. Si utilizza il WNV per svegliare la macchina virtuale.
5. La macchina virtuale passa allo stato di "ready".

Quindi il VMM setta SN = 1 e NV all'indirizzo dello IDV.

Quando la VM va da "ready" a "running" avvengono gli stessi passaggi del caso precedente, SN viene messo = 0 e la prima istruzione da cui la VM viene riattivata è la prima istruzione dell'handler dell'interruzione.

6. Grazie al valore di NV si accede alla IDT della macchina virtuale, per notificare l'interruzione.
7. Si notifica l'interruzione, l'handler del sistema operativo guest viene chiamato.

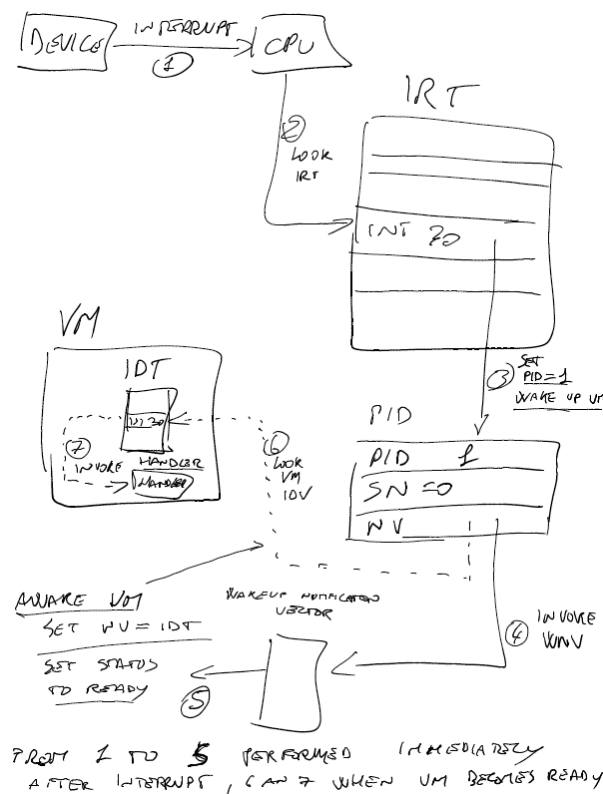


Figure 56 - Workflow se la macchina virtuale è in stato di halted.

Dispositivi DMA

Dispositivi che possono leggere dati direttamente dalla RAM senza interpellare la CPU; tutto grazie al supporto hardware (periferica) chiamato controllore DMA.

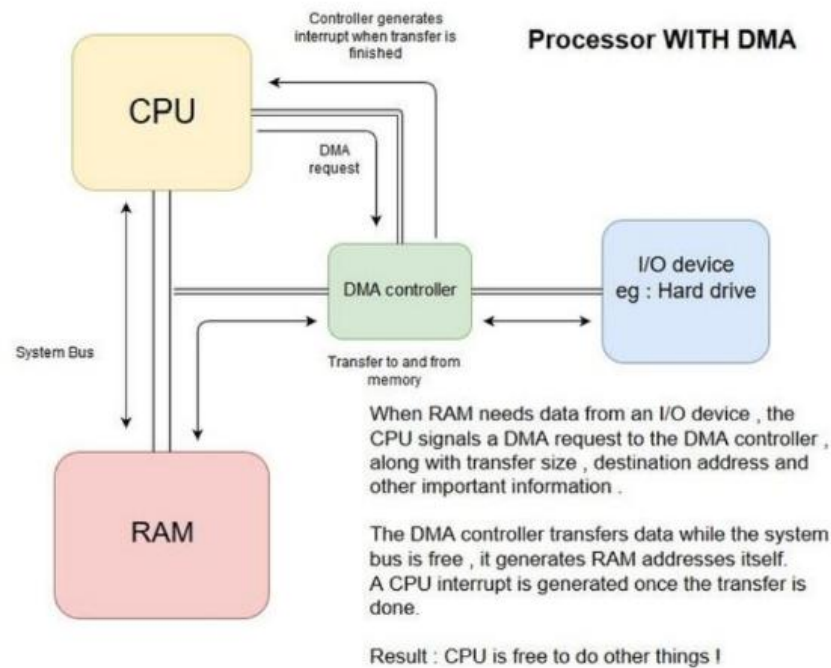


Figure 57 - Funzionamento del DMA.

I dispositivi di I/O richiedono una porzione di RAM proveniente dal sistema, il sistema fornisce indirizzi virtuali, la periferica riceve indirizzi virtuali e deve tradurli in indirizzi reali. Per questo è stata introdotta la **IOMMU**, modulo responsabile di tradurre gli indirizzi virtuali in fisici per conto del DMA. Anche qui l'IOMMU deve essere modificata in modo da eseguire due funzioni F e G per tradurre gli indirizzi.

IOMMU

Con lo scopo di tradurre indirizzi virtuali in indirizzi fisici, i dispositivi DMA usano un circuito della MMU apposta per lo I/O.

La IOMMU è responsabile di tradurre gli indirizzi virtuali dello spazio I/O in indirizzi fisici dello spazio I/O.

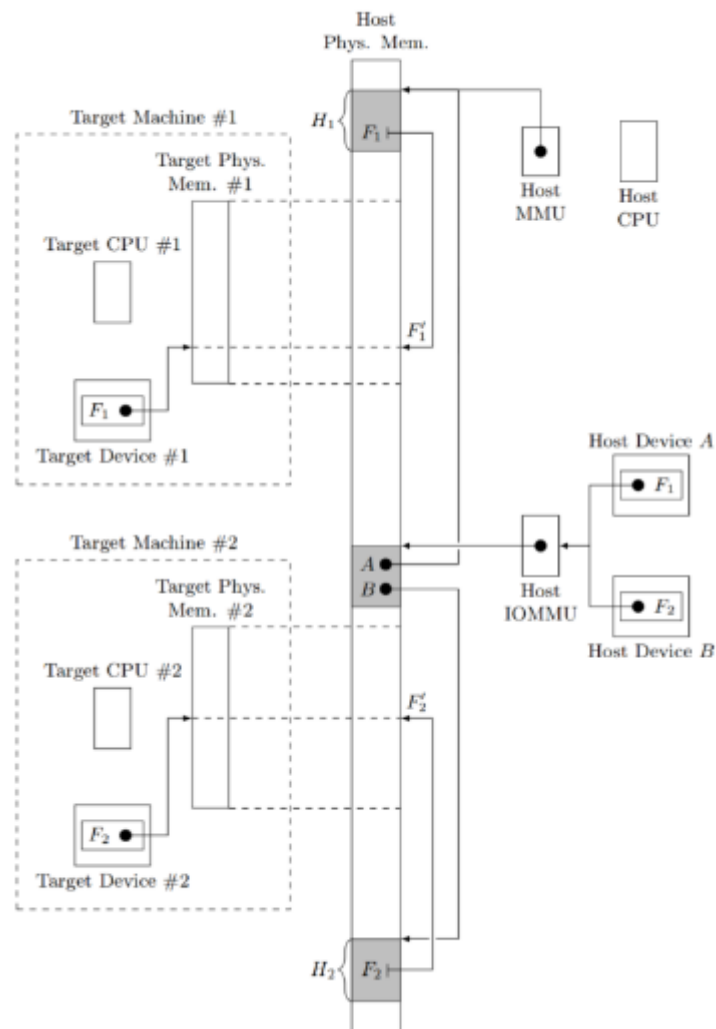


Figure 58 - IOMMU e virtualizzazione.

Virtualizzazione annidata (*nested virtualization*)

Una macchina virtuale può essere eseguita all'interno di un'altra macchina virtuale, basta installare un VMM (*guest hypervisor*) all'interno di una macchina virtuale in modo da generare macchine virtuali all'interno di essa.

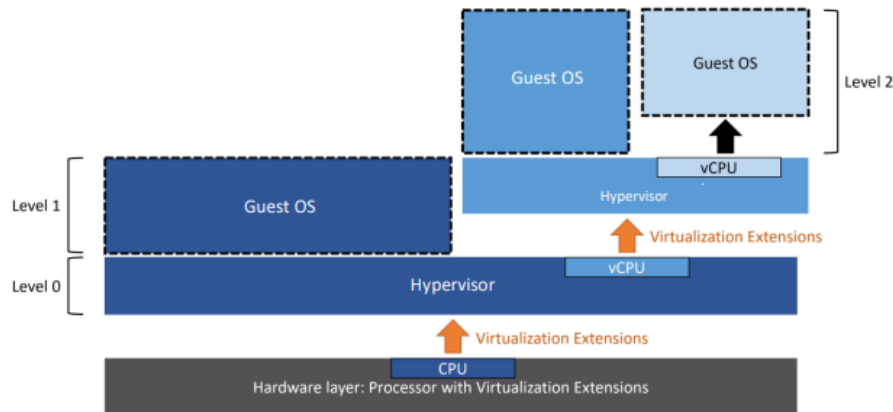


Figure 59 – Diagramma che mostra la virtualizzazione innestata.

La macchina annidata (*in realtà il VMM guest*) non può usare il supporto hardware, deve usare sempre l'approccio Trap&Emulate, quindi mostrare prestazioni molto basse.

Come possiamo eliminare questo overhead? Assumiamo che il VMM guest e il VMM host sono dello stesso tipo (*lo stesso software*) e che i due VMM possano comunicare senza problemi. In questo modo possiamo migliorare le prestazioni.

1. Il primo VMM è veloce perché ha il supporto hardware.
2. Il primo VMM crea una macchina virtuale.
3. Il sistema operativo guest avvia un altro VMM (*il VMM guest*) per creare la macchina virtuale innestata.

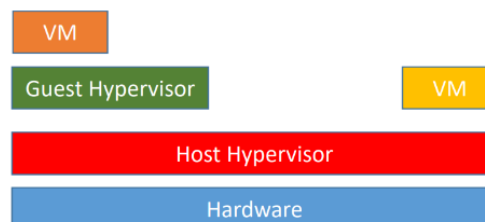


Figure 60 – Diagramma semplificato che mostra la virtualizzazione innestata.

In breve, si cerca di riportare tutto ad un singolo livello di virtualizzazione, ovvero la macchina virtuale che è sopra l'hypervisor guest viene messa sopra l'hypervisor host, in modo che possa accedere alle ottimizzazioni hardware. Ovvero quando l'utente chiede al guest hypervisor di creare una VM, il guest hypervisor chiede all'host hypervisor di creare una VM.

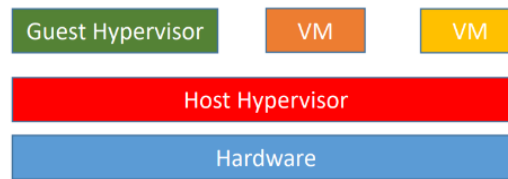


Figure 61 – Effetti del supporto alla virtualizzazione innestata

KVM – Kernel-Based Virtual Machine

È il software più famoso che implementa la full virtualization. Soluzione di virtualizzazione basata su kernel per il sistema operativo Linux introdotta nel 2007. Consente di trasformare Linux in un hypervisor, consentendo così di eseguire più macchine virtuali isolate sullo stesso hardware fisico.

KVM è integrato come modulo del kernel, permettendo a Linux di diventare un hypervisor. Utilizza le funzionalità del kernel per gestire le VM, sfruttando l'efficienza e la sicurezza del kernel stesso, rendendolo una scelta efficiente e potente per la virtualizzazione.

Le VM eseguite su KVM sono isolate l'una dall'altra e dall'host. Questo isolamento garantisce che un problema in una VM non influenzi le altre o l'host.

KVM consente la gestione dinamica delle risorse come CPU, memoria e dispositivi I/O, consentendo il provisioning e la deallocazione flessibile delle risorse alle VM in base alle esigenze.

KVM viene spesso utilizzato insieme a QEMU per fornire emulazione completa dell'hardware e gestire le VM.

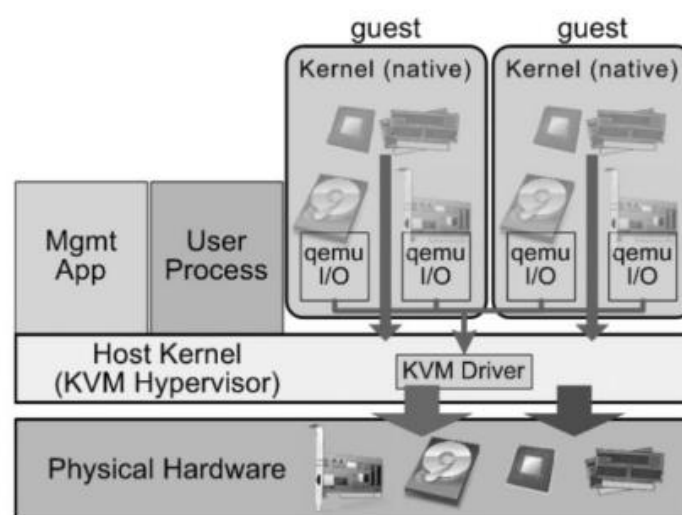


Figure 62 - Architettura di KVM.

Laboratorio di KVM

1. Abbiamo installato le componenti di KVM, recuperato le librerie, accertati del supporto hardware.
2. Abbiamo recuperato delle immagini da qualche repository online.
3. Lanciato le macchine virtuali.
4. Abbiamo anche interrotto (*non distrutto*) le macchine virtuali.
5. Con KVM si può creare delle immagini personalizzate.

Capitolo 7 – Paravirtualization

Molto usata prima del supporto hardware (*ora molto meno popolare*) ed ha l'obiettivo di ridurre l'overhead rispetto alla full virtualization. È un tipo di lightweight virtualization, non è più utilizzata al giorno d'oggi.

Il VMM è creato con l'assunzione che il sistema operativo guest può essere modificato ed è conscio di essere eseguito su una macchina virtuale; le modifiche in questione fanno riferimento soprattutto al kernel (*le applicazioni nel sistema guest sono unaware della virtualizzazione*).

Nella paravirtualization una porzione dei task di gestione della virtualizzazione è trasferita dall'hypervisor al sistema operativo guest.

Nel caso di full virtualization (senza supporto hardware) ci sono numerosi cambi di contesto tra VM e VMM. Inoltre, l'emulazione è effettuata frequentemente perché è necessario emulare tante istruzioni quante non possono essere emulate direttamente dal sistema operativo guest.

- Accelerare il tutto e rendere meno costose le operazioni.
- Implementare funzionalità in modo che non richiedano l'emulazione e quindi un cambio di contesto verso il VMM.
- Evitare che esegua direttamente operazioni che non può fare.

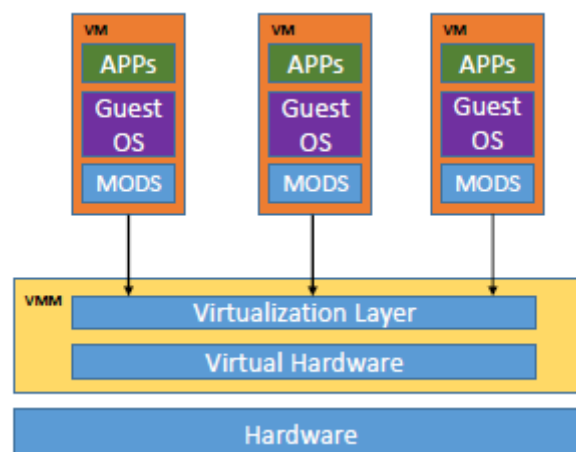


Figure 353 - Diagramma che mostra l'architettura realizzata da un sistema che sfrutta la paravirtualization.

Il risultato è questa implementazione mostrata in figura sopra, abbiamo il VMM che viene eseguito in modalità sistema, mentre le applicazioni e la VM vengono eseguite in modalità user. L'hypervisor ha un componente centrale "Virtual Hardware" ed un componente "Virtualization Layer" che espone un insieme di API.

Hypercalls

Nella para virtualizzazione, vengono utilizzate le API (*chiamate Hypercalls*) esposte dal VMM e richiamabili dal sistema operativo guest. Chiamando queste API avverrà un solo cambio di contesto verso il VMM dove verrà eseguita l'operazione voluta.

Ogni sistema operativo guest deve conoscere su quale hypervisor viene eseguito, questo perché in base al VMM il sistema operativo guest viene modificato in modo da poter interagire con tale hypervisor.

Il kernel del sistema operativo guest è modificato in modo da chiamare le hypercalls, invece di tentare di eseguire istruzioni privilegiate.

L'hypercall viene eseguita completamente nello spazio kernel (*livello massimo di privilegio*), quindi c'è solo un cambio di contesto, mentre nella trap emulation, avevamo numerosi cambi di contesto.

Una volta chiamata una hypercall, il VMM è responsabile per implementare funzionalità come un'operazione di I/O. In questo caso abbiamo ancora un cambio di contesto, però ne abbiamo solo uno e non multipli.

Il VMM effettua ancora emulazioni, però questa volta l'hardware virtualizzato è acceduto dal guest OS tramite le API esposte dall'hypervisor e non tramite istruzioni privilegiate che richiedono numerosi cambi di contesto.

In figura vediamo che siamo in un'architettura con due livelli (oppure 4 se abbiamo i vari ring). Le applicazioni e funzionalità della VM sono in esecuzione in modalità user. Mentre l'hypervisor è in esecuzione in modalità user.

Una parte del codice del guest OS è eseguito come ring 0, perché alcune funzionalità implementate nel kernel sono implementate tramite le hypercall. Le hypercall sono eseguite in modalità sistema. Nella paravirtualization c'è solo un cambio di contesto da user a kernel e successivamente l'hypercall viene eseguita.

Nella full virtualization un'applicazione guest esegue un'istruzione privilegiata, avveniva un cambio di contesto, una volta eseguita l'istruzione privilegiata veniva restituito il controllo alla VM che potrebbe eseguire nuove istruzioni privilegiate che comportano altri cambi di contesto.

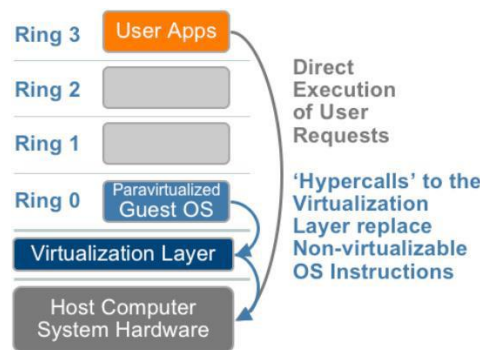


Figure 364 – Diagramma che mostra il workflow di una hypercall.

Workflow di una hypercall

1. Il sistema operativo per eseguire una istruzione privilegiata richiama una hypercall.
2. Il controllo viene passato all'hypervisor, il quale esegue il codice relativo alla hypercall chiamata.
3. Viene restituito il controllo al sistema operativo guest.

Vantaggi nella paravirtualization

- Consente al guest OS di effettuare chiamate per comunicare direttamente con il VMM senza emulazione di istruzioni. Viene ridotto l'overhead dell'hypervisor rispetto alla full virtualization.
- Non richiede supporto hardware specifico.
- Il sistema non è limitato dai device driver forniti dal livello di virtualizzazione, l'hypervisor può sfruttare i device driver del sistema operativo guest.

Svantaggi nella paravirtualization

- Versioni non modificate di un sistema operativo non sono compatibili con la paravirtualization (le modifiche sono possibili in OS open source e non in OS proprietari come Windows).
- Sistemi operativi chiusi non possono essere supportati.
- La sicurezza è compromessa visto che il guest OS ha più controllo dell'hardware sottostante.

I vantaggi di questo approccio consistono nel miglioramento delle prestazioni e non richiede un supporto hardware specifico. Gli svantaggi sono palesi: non tutti i sistemi operativi sono compatibili perché richiedono delle modifiche radicali al kernel.

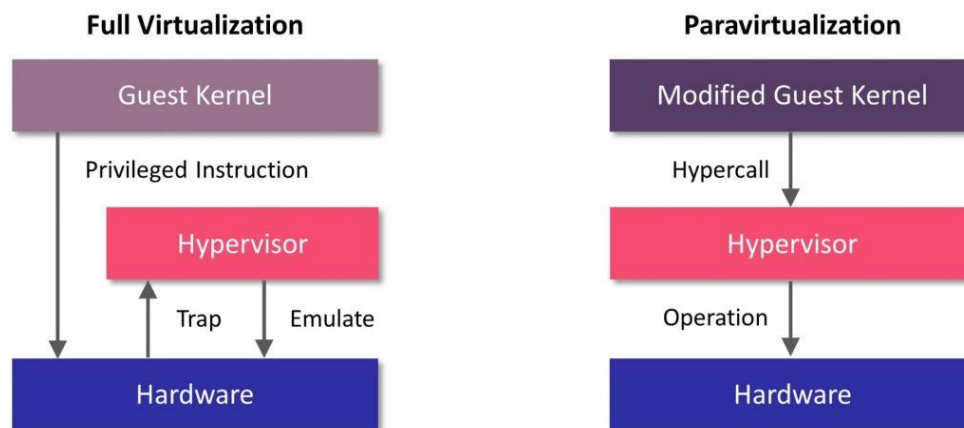


Figure 375 – Diagramma che mostra la differenza tra la full virtualization (sinistra) e la para virtualization (destra).

Xen

Uno dei VMM più famosi per la para virtualizzazione (*creato per supportare sistemi basati su Linux*), viene usato ancora oggi e le versioni recenti supportano anche full virtualization. Xen consente di creare più macchine virtuali contemporaneamente; di default (*dall'avvio*) nel sistema c'è sempre una macchina virtuale chiamata Domain0.

Domain0 è acceduta dall'utente, il quale tramite linea di comando può creare/distuggere altre macchine virtuali.

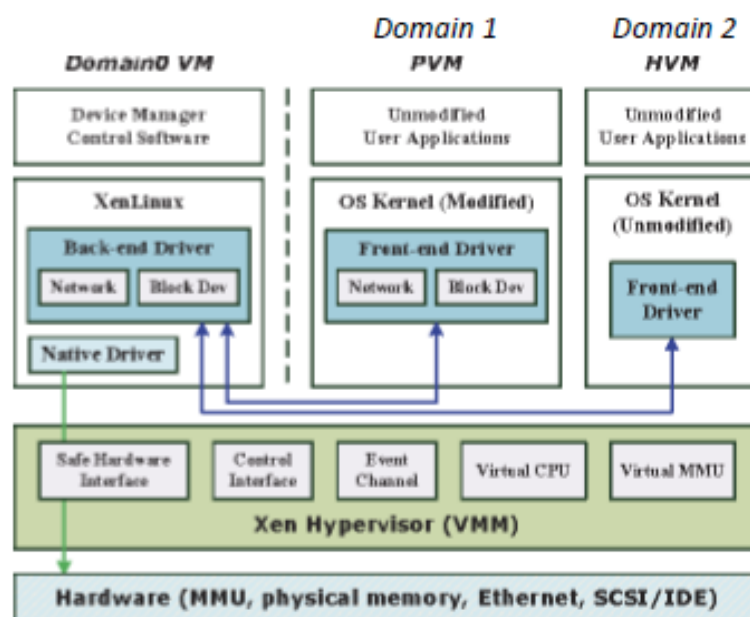


Figure 38 – Diagramma che mostra l'architettura di Xen.

Capitolo 8 - Lightweight Virtualization

Nonostante l'introduzione del supporto hardware, eseguire un'applicazione sopra una VM comporta un overhead compreso tra il 5% ed il 10% rispetto ad una macchina fisica.

Molte volte necessitiamo solo una porzione dei vantaggi forniti dalla virtualizzazione. A volte vorrei che due applicazioni vengano eseguite sulla stessa macchina fisica ma in modo completamente isolato. Non mi serve avere un'intera macchina virtuale separata, ma solo isolamento delle applicazioni.

Avere VM comporta un overhead per l'hypervisor e per il guest OS in cui le applicazioni vengono eseguite.

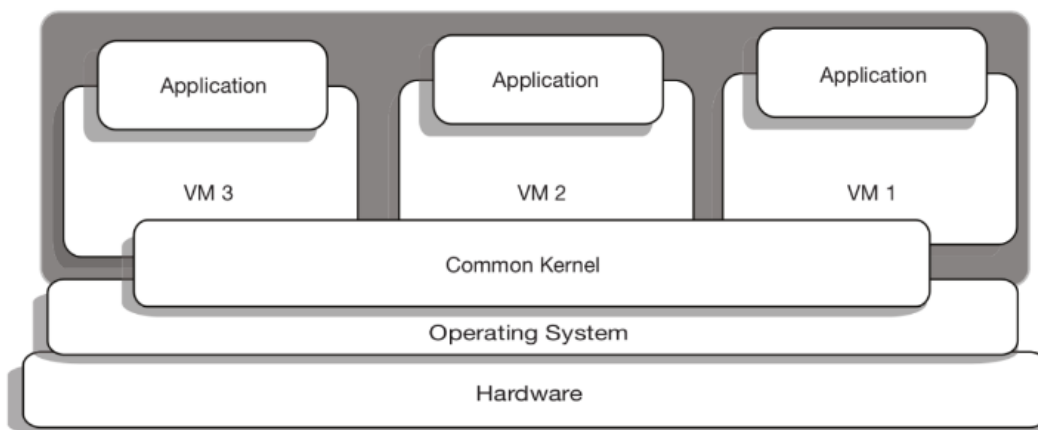


Figure 397 – Diagramma che mostra l'architettura di un sistema che usa la lightweight virtualization.

Questa tecnica è stata introdotta dopo il supporto hardware, dato che in certi contesti l'uso di un'intera macchina virtuale risulta eccessivo.

Un sistema operativo che supporta la lightweight virtualization ha la capacità di creare ambienti virtuali in modo dinamico in cui eseguire le applicazioni in modo isolato (*le applicazioni non possono interagire direttamente*).

Lo scopo di questa virtualizzazione è avere isolamento, istanziamento dinamico e un ambiente autocontenuto (nessuno può interagire con questo ambiente).

In questo caso il sistema operativo host (*il kernel*) è considerato un hypervisor, quindi viene rimosso completamente un hypervisor separato.

Pro di questa tipologia di virtualizzazione

- **Meno overhead:** poiché non devo eseguire un intero sistema operativo guest o virtualizzazione dell'hardware.
- **Più scalabile:** A parità di hardware, con questa tecnica posso gestire molte più macchine virtuali.

Contro di questa tipologia di virtualizzazione

- **Limiti sul sistema operativo guest:** Tutte le macchine virtuali devono condividere lo stesso kernel (*ad oggi la stragrande maggioranza delle macchine virtuali hanno il kernel di Linux, quindi questo punto non ha rilevanza*)
- **Limiti sul sistema operativo host** Non tutti i sistemi operativi forniscono queste funzionalità (*Linux supporta la lightweight virtualization, quindi questo punto non ha rilevanza*).

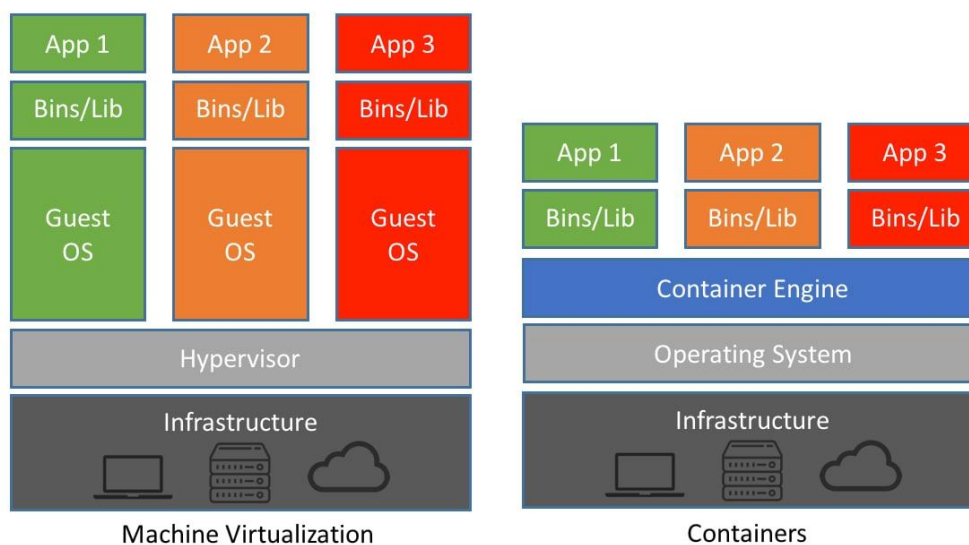


Figure 408 – Diagramma che mostra la differenza tra lightweight virtualization (destra) e full virtualization (sinistra).

Approccio di Linux - Containers

I containers sono ambienti virtuali in cui eseguire applicazioni in modo isolato. Le applicazioni all'interno di un container non possono comunicare con le altre applicazioni e possono usare solo uno specifico pool di risorse a loro assegnate.

Spesso non ci serve creare un'intera macchina virtuale, ma ci basta solo l'isolamento.

Un container non è una macchina virtuale, i processi che girano all'interno di un container sono normalissimi processi che girano sul sistema operativo host, ma che semplicemente non hanno modo di vedere nulla al di fuori del container.

In un container posso anche eseguire un intero sistema operativo (*in fin dei conti, un sistema operativo è un processo*), con il vincolo che abbia lo stesso kernel di Linux.

Un container causa un overhead trascurabile (*praticamente nullo*) e sono quelli più usati nell'ambiente cloud (es. *Docker*). Non c'è un overhead ridotto, non c'è nessun overhead, questo è il vantaggio principale.

I container di Linux sono implementati usando principalmente due feature:

- **Namespace:** Realizza gli ambienti isolati per le applicazioni.
- **Control groups:** Fa in modo che le applicazioni accedano solo alle risorse a cui possono accedere.

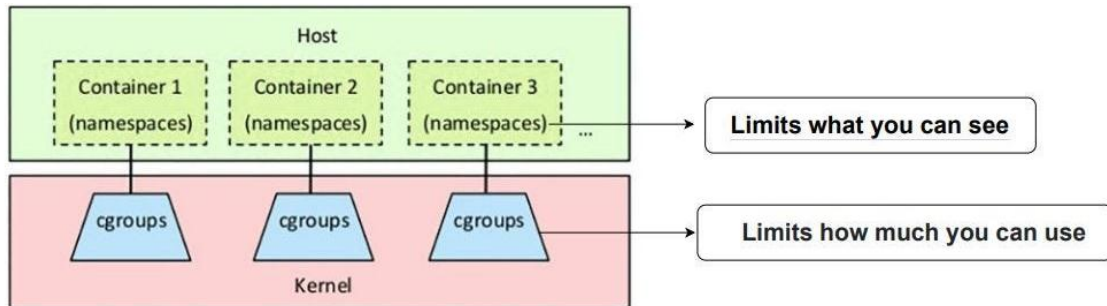


Figure 419 - Diagramma che mostra cosa fanno i namespace e cosa fanno i control group.

Namespace

Meccanismo (*insieme di tecniche*) progettato per isolare un gruppo di risorse del sistema host, in modo da nasconderle da certi processi.

Questa tecnica utilizza una funzionalità molto vecchia di Linux (*chroot()*) che fu creata per nascondere una parte del file system da un gruppo di processi.

Possono essere definiti vari tipi di namespace e non solo namespace per il file system, ad esempio, namespace di rete che possono permettere l'accesso ad un processo solo a particolari schede di rete rispetto ad altre.

Cgroups - Control groups

I namespace non sono sufficienti per fare in modo che due processi non possano interferire tra loro se sono in due container differenti, come nel caso in cui un processo voglia prendersi più risorse di quanto dovrebbe.

I control groups permettono un “*fine grain control*” sulla quantità di risorse usate da ogni processo che appartiene ad un certo container.

Il control group è creato dall'utente root, è applicato ad uno specifico gruppo di processi e definisce il set di risorse disponibili per quel gruppo di processi.

Un control group ad esempio può definire:

- Massima quantità di RAM utilizzabile.
- Limite ai core della CPU utilizzabili.
- Massimo numero di file aperti.
- Massimo tempo di utilizzo della CPU.
- Priorità nelle operazioni di I/O.

Non si può scappare, i processi di quel gruppo non possono prendere più di quel set, a prescindere da quanti processi figli creano (*i quali ereditano lo stesso gruppo del processo padre*).

LAB - Docker

In aggiunta i container possono essere usati per la distribuzione di software. Un esempio di piattaforma che permette una cosa del genere è Docker. Docker è uno strumento molto potente che semplifica il processo di distribuzione (*deployment*) di un'applicazione.

Docker garantisce che le applicazioni funzionino in modo prevedibile in qualsiasi ambiente.



Figure 70 - Logo di Docker.

Architettura di Docker

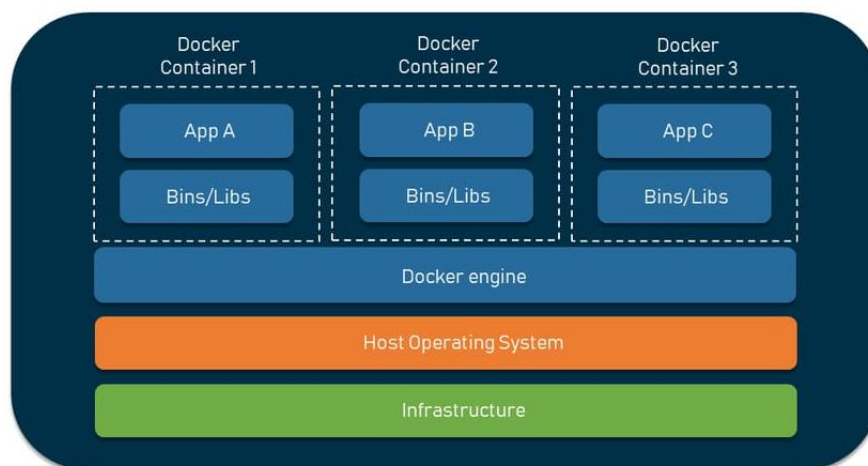


Figure 421 – Diagramma semplificato che mostra l'architettura di un sistema che sfrutta Docker.

Docker è una piattaforma che consente di automatizzare il deployment di applicazioni all'interno di container.

In altre parole, è uno strumento di orchestrazione locale dei container all'interno di una macchina. Consente agli sviluppatori di creare immagini.

Immagini Docker (*Docker Images*)

Un'immagine Docker è un grosso pacchetto che include tutto il necessario per eseguire un'applicazione:

- Il codice dell'applicazione.
- Le librerie e le dipendenze necessarie.
- Il runtime.
- File di configurazione.
- E altro ancora.

Le immagini sono create tramite un file chiamato “*Dockerfile*”, che contiene, oltre alle informazioni dette prima, le istruzioni per assemblare l'immagine. Le immagini devono essere in un'altra VM.

Container Docker

Un container è un'istanza in esecuzione di un'immagine Docker. I container sono isolati dal sistema operativo host e da altri container di Docker, ma possono condividere risorse come il networking e il filesystem.

Questo isolamento garantisce che l'applicazione venga eseguita sempre nello stesso modo, indipendentemente dall'ambiente e dalle altre applicazioni installate.

Docker Engine

Docker Engine è il motore di runtime di Docker che gestisce le immagini e i container. Esso consente di eseguire comandi come il build di immagini, la creazione di container e la gestione delle reti e dei volumi.

Docker Hub

I registri sono archivi centralizzati per le immagini Docker disponibili per il deployment, contiene tutte le immagini preconfigurate per uno specifico software.

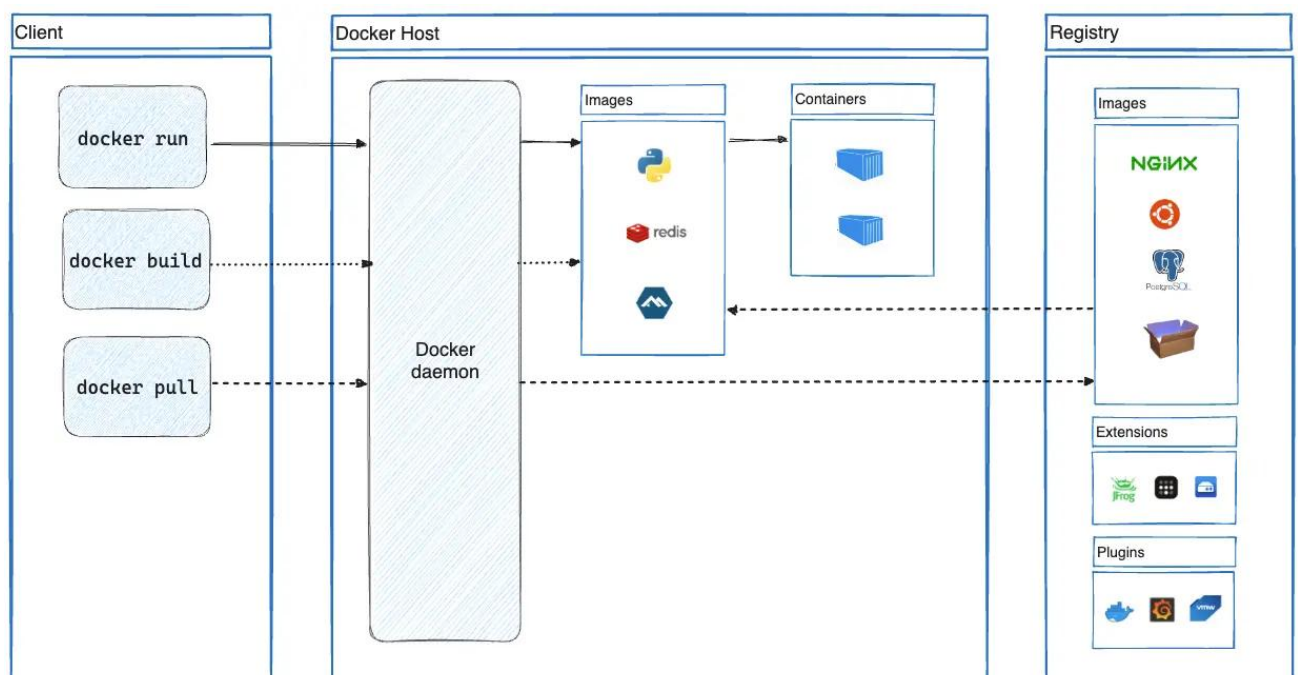


Figure 432 – Diagramma che mostra l'architettura di Docker.

Il più noto è Docker Hub, un servizio cloud pubblico che permette agli utenti di scaricare e caricare immagini. È possibile anche configurare registri privati.

Un host che sta eseguendo il Docker engine può in ogni momento scaricare una o più immagini da un registro ed eseguirle senza preoccuparsi di niente e senza cambiamenti dell'immagine.

Su questi registri, gli sviluppatori possono caricare i container e altri sviluppatori possono scaricarli e installarli sulle loro macchine.

Processo di esecuzione di Docker

1 - Creazione di un Dockerfile

Scriviamo un programma in Python e creiamo un'immagine che ospita il programma di default, in Docker è possibile creare l'immagine personalizzata da una già esistente, bisogna creare un file e in base ad alcune direttive possiamo personalizzare l'immagine.

Scrivi un Dockerfile che descrive come costruire l'immagine.

Include istruzioni come `FROM` (*per specificare l'immagine base*), `COPY` (*per copiare file all'interno dell'immagine*), e `RUN` (*per eseguire comandi all'interno dell'immagine*).

```
# Start from an official ubuntu image
FROM ubuntu
# Specify the command to be run inside the container at installation
RUN apt-get update && apt-get install -y python3
# Add some file to the image from the host
ADD ./my_program.py /root/
# Run at startup the following command
CMD ["/usr/bin/python3", "/root/my_program.py"]
```

2 - Build dell'Immagine

Usa il comando `docker build -t custom-ubuntu .` per creare l'immagine Docker dal Dockerfile. Nel comando `custom-ubuntu` è il nome dell'immagine personalizzata che vogliamo creare, mentre l'ultimo parametro è il percorso della cartella in cui il Dockerfile è memorizzato (come ultimo parametro mettiamo il punto dato che eseguiamo il comando nella stessa cartella in cui c'è il Dockerfile).

3 - Avvio di un Container

Usa `docker run` per avviare un container dall'immagine creata. Puoi specificare opzioni come il mapping delle porte, la gestione dei volumi e le variabili d'ambiente.

Ad esempio, `docker run hello-world` esegue un container predefinito chiamato `hello-world`.

Per eseguire l'immagine base di ubuntu bisogna inserire `-t`, questo sposta il controllo sull'ambiente virtuale appena creato (`docker run -t ubuntu`).

4 – Ricerca delle immagini

Con il comando `docker search ubuntu` restituisce tutte le immagini nella directory.

Con il comando `docker images` vengono mostrate le immagini disponibili in locale.

5 - Gestione dei Container

Una volta avviato, puoi gestire il container con comandi come ``docker stop``, ``docker start``, ``docker restart``, e ``docker rm ID``.

Con `docker ps -a` vengono mostrati tutti i container, anche quelli non in esecuzione ().

Docker networking

Ogni container ha un adattatore di rete che è connesso ad una rete virtuale locale, il sistema crea un bridge e tutti gli adattatori sono connessi a quel bridge. Quindi tutti i container connessi al bridge possono comunicare tra loro, solitamente vogliamo configurare almeno un container per esporre i suoi servizi in internet.

Se vogliamo esporre un servizio creiamo un'immagine customizzata, se si vuole esporre un'applicazione fornita da un server web, bisogna inserire il comando *EXPOSE 80* nel Dockerfile.

A questo punto ripetiamo i comandi come nel caso precedente del programma Python (build). Eseguiamo il container ed esponiamo la porta pubblicamente, eseguiamo il server in background ed infine stoppiamo.

Se vogliamo implementare la comunicazione tra container eseguiamo il comando `docker network inspect bridge` per mostrare gli IP dei container, a questo punto è possibile provare il comando `ping` con l'indirizzo di un container.

Capitolo 9 - Cloud Applications

Le applicazioni tradizionali si dicono tali se sono state pensate per essere eseguite sulla tradizionale architettura client-server:

- Il server fisico risponde alle richieste di client secondo una sua logica.
- I dati sono memorizzati sul disco del server fisico.

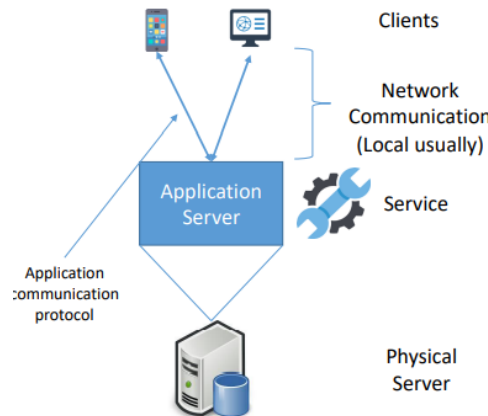


Figure 443 - Struttura di una applicazione client-server tradizionale.

La cloudification ha come primo step quello di portare le applicazioni e i servizi sul cloud. Ossia quello di spostare le applicazioni così come sono (*as they are*) direttamente su una infrastruttura cloud.

L'architettura del sistema rimane la stessa ma ora i servizi sono installati su delle macchine virtuali.

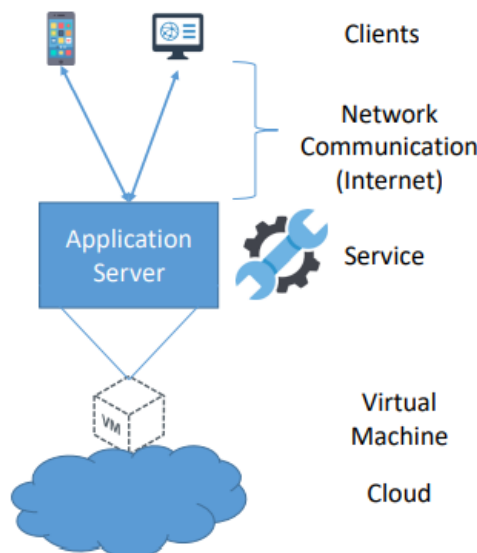


Figure 454 - Applicazione client-server con aggiunto un layer di virtualizzazione.

Applicazioni Cloud Native

La cloudification non implica per forza la creazione di applicazioni cloud “*native*” (migra nel cloud applicazioni già esistenti), ossia progettate in modo da massimizzare la rendita dell’architettura sottostante. Ciò non toglie che non sia una cosa buona! perché una applicazione cloud non nativa non sfrutta la piena potenzialità di una infrastruttura cloud.

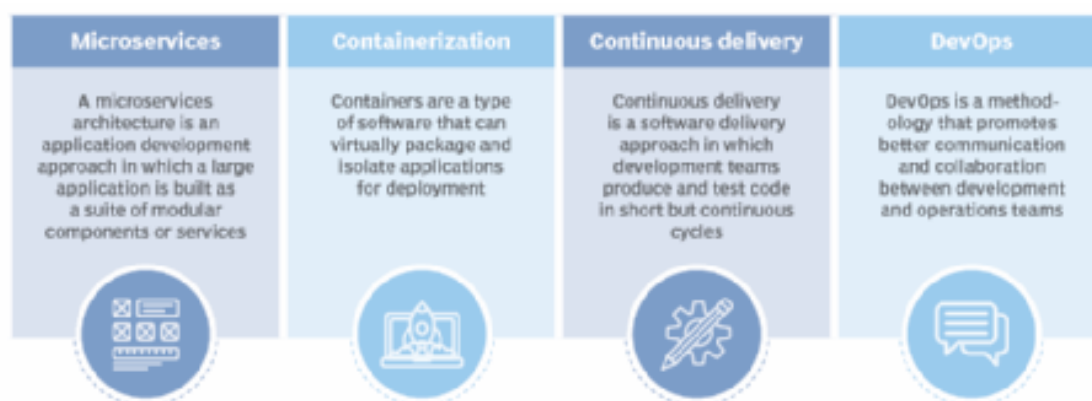


Figure 465 - I quattro principi su cui si basa lo sviluppo di applicazioni cloud native (Continuous Delivery → CI/CD).

Un’applicazione nativa per il cloud è organizzata in livelli (*tier*), ogni tier esegue uno specifico insieme di task, ed ognuno di essi è implementato in una o più macchine virtuali.

Un applicazione cloud è un sistema distribuito, composto da diversi componenti che interagiscono tra di loro con lo scopo complessivo di fornire un servizio all’utente.

L’implementazione e l’integrazione di ognuna di queste componenti software è un compito complesso che può risultare molto costoso in termini di tempo e quindi anche di soldi. Inoltre, una applicazione cloud viene aggiornata nel tempo aggiungendo nuovi componenti o rimuovendone altri.

Requisiti di una applicazione cloud

Ogni applicazione cloud ha alcuni bisogni in comune:

Interoperabilità

Ogni componente di una cloud application deve interagire con gli altri componenti.

Per assicurare questa proprietà il software di ogni componente deve esporre una interfaccia standard. Ciò nasconde il funzionamento intrinseco del componente, quindi facilitando eventuali cambiamenti all’implementazione del componente.

Alta Scalabilità Orizzontale

Deve essere facile potenziare l'infrastruttura aggiungendo componenti che eseguono nuove funzionalità dinamicamente e senza sforzo. L'architettura deve poter accogliere i nuovi componenti senza compromettere il lavoro degli altri.

Composability

Un ambiente cloud è un ambiente dinamico che può cambiare rapidamente a causa di fallimento dei componenti, creazione/distruzione di VM. Il software deve essere progettato in modo da poter essere utilizzato con componenti differenti senza cambiamenti o con cambiamenti minimi.

In altre parole, l'applicazione è costituita da componenti indipendenti che possono essere interscambiabili tra loro nel caso eseguissero la stessa funzione, e questa cosa deve essere fatta con il minor numero di cambiamenti possibili.

Loose coupling (accoppiamento "lento"): Proprietà del sistema per cui è possibile aggiungere o rimuovere servizi senza sforzo.

Vantaggi SOA

- Implementazione di un servizio è nascosta, quindi è possibile modificarla senza influenzare altri servizi.
- Loose coupling.
- Composability.
- Comunicazione tra servizi è effettuata in un modo **stateless**: ogni modulo tratta le transazioni separatamente, in modo indipendente da ogni altra transazione.

Applicazione Two-Tier

Le funzionalità di una applicazione cloud nativa si dividono in livelli e ogni livello si occupa di uno specifico set di task e ognuno di essi è eseguito in una diversa macchina virtuale. Un esempio di una semplicissima divisione a due livelli può essere:

- **Server Applicativo:** Gestisce la logica dell'applicazione.
- **Database:** Contiene i dati dell'applicazione.

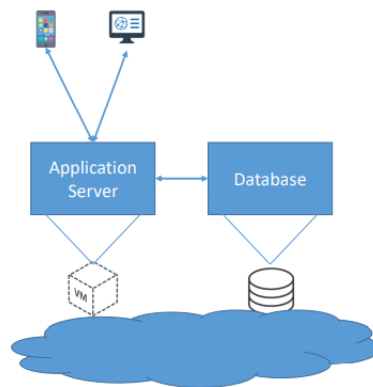


Figure 476 – Esempio di applicazione a due livelli.

Applicazione Multi-Tier

Un'applicazione molto complessa può essere divisa in un numero arbitrariamente alto di livelli: in questo caso si dice che si sta parlando di una applicazione *multi-tier* (*N-Tier*).

Ogni livello è realizzato con varie macchine virtuali che eseguono lo stesso codice o anche diverso. L'ultimo tier è spesso detto "*storage tier*": esso contiene tecnologia di tipo "*cloud storage*", ossia tecnologia progettata per gestire una grande quantità di dati (*big data*).

Se una delle VM nell'application tier riceve una richiesta dal cliente che richiede accesso al database tier, contatta una delle VM del database tier, tale VM recupera i dati interagendo con lo storage tier.

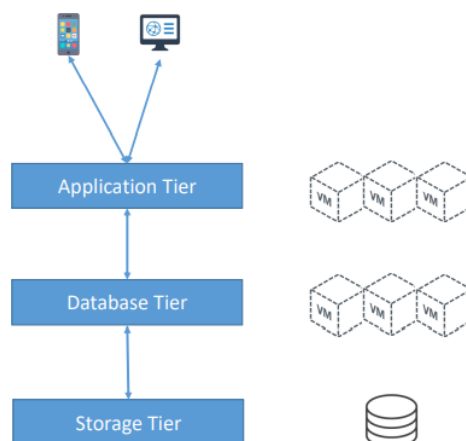


Figure 487 – Esempio di applicazione multi-tier.

Ogni volta che si ha una richiesta di un utente avviene una **catena di interazioni** tra i livelli dell'applicazione cloud.

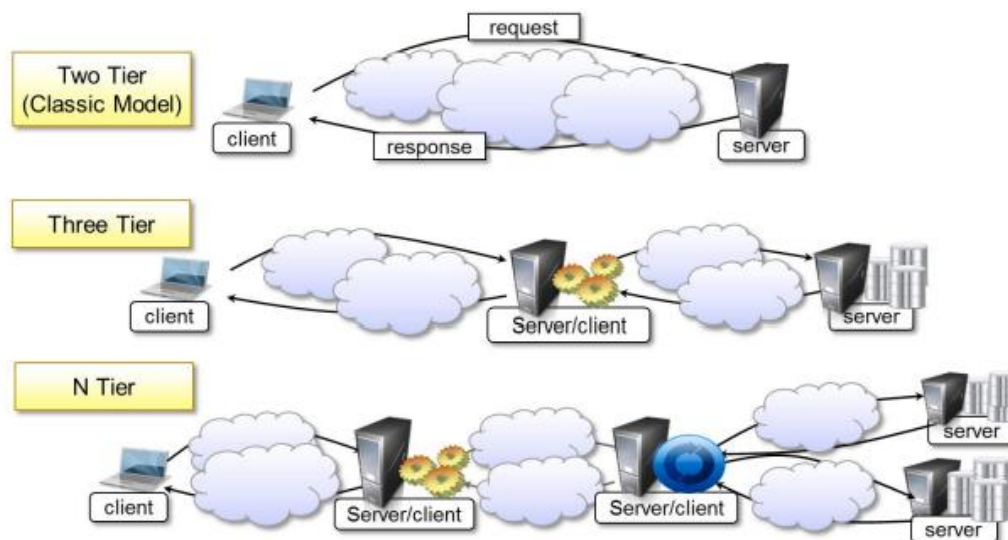


Figure 498 – Esempio semplificato che mostra una catena di interazioni in una applicazione cloud a molti livelli.

La catena di interazioni è sviluppata attraverso varie interazioni Client-Server dove un nodo può fare sia da server che da client. I server nei vari livelli comunicano tramite interazioni client-server o altre forme, sono connessi tramite una LAN.

Architettura Service-Oriented (SOA)

Emersa in anni recenti, SOA è una metodologia (*non un tool o un framework*) per progettare applicazioni cloud.

L'idea di base è la seguente: Un applicazione cloud è costruita sulla base di una collezione di servizi (*micro-servizi*) e ognuno di essi deriva da un componente software; ognuno di questi servizi comunica con altri servizi attraverso message passing.

Il risultato è che le applicazioni sono altamente modulari offrendo molta flessibilità: in particolare, ogni modulo può essere implementato indipendentemente dagli altri (*anche con un linguaggio di programmazione diverso*) e può essere installato su una macchina virtuale separata.

Il punto di SOA è quello di eliminare le dipendenze implementative e tecnologiche.

Comunicazione tra i moduli

I servizi dell'applicazione (moduli) comunicano tramite passaggio di messaggi. Ogni messaggio segue un formato ben preciso e concordato tra tutti i moduli (*ad esempio una stringa JSON*). Ogni componente gestirà i messaggi ricevuti a modo suo secondo la propria logica (*la logica interna del modulo è nascosta agli altri moduli*).

Il formato del messaggio è molto importante perché definisce il servizio e il pattern di interazione per invocare una certa funzionalità.

Questo approccio è il migliore, perché tutta la logica interna (*compreso il linguaggio di programmazione*) del componente può cambiare del tutto ma se il formato rimane uguale allora gli altri componenti non devono essere cambiati.

Remote Method Invocation (RMI)

Ogni modulo espone all'esterno dei metodi che possono essere richiamati da altri moduli. Questo approccio però non semplifica la vita dei programmatori in quanto limita l'implementazione interna dei moduli.

SOA Roles

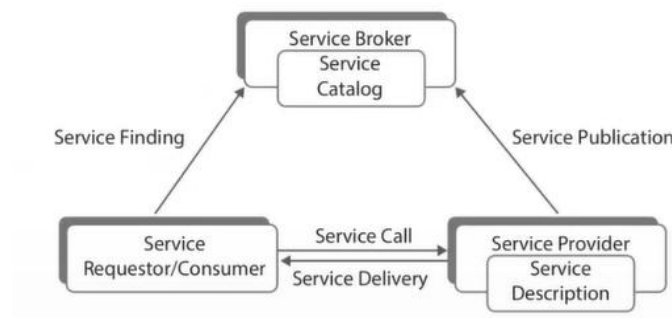


Figure 509 - SOA roles e come interagiscono con il broker.

Consumatori & Produttori

In una interazione SOA si hanno due ruoli:

- **Consumatore:** Invia una richiesta per un servizio tramite un messaggio ad un altro modulo.
- **Produttore:** Riceve una richiesta di servizio tramite un messaggio e invia al consumer una risposta con il risultato aspettato.

Un servizio può essere nessuno, uno di essi o entrambi in ogni momento, dipendentemente dalla situazione.

Broker

Il consumatore e il produttore in genere non comunicano in modo diretto, ma inizialmente il consumatore deve poter conoscere l'indirizzo e il descrittore del provider.

Per risolvere questo problema ci pensa il broker: ossia un modulo indipendente che ha il duplice compito di catalogare tutti i servizi offerti dai provider e informare i consumatori riguardo al come raggiungerli e il formato di messaggio necessario.

1. Il consumatore prima di contattare il produttore contatta il broker con un messaggio di "*Service Finding*".
2. Nel caso in cui ci sia stato un provider che in precedenza ha fatto una "*Service Publication*" con lo stesso servizio richiesto dal consumer, il broker inoltra al consumatore tutto ciò che gli serve per contattare il provider (*indirizzo e formato del messaggio*).

Questo meccanismo di registrazione dei provider aumenta la scalabilità del sistema e anche la sua flessibilità.

Servizi Web

Il termine "*servizio web*" si riferisce ad una applicazione modulare basata su servizi progettati per essere usati da altri servizi. Tutto basato su tecnologia web: infatti, i servizi web sono l'implementazione SOA più comune.

World Wide Web – Cosa offre?

Il web è comodo perché possiede già dei meccanismi standardizzati per:

- Identificazione standardizzata delle risorse (URI)
- Meccanismo di trasferimento dati (HTTP & HTTPS).
- Linguaggio di markup per codificare informazioni (XML).

Questi tre elementi sono la base per la creazione dei servizi web.

URI – Uniform Resource Identifier [RFC 2396]

Meccanismo per identificare una risorsa nel web, ossia definisce la sintassi dell'identificatore di una risorsa.

```
<uri> ::= <scheme> ":" <scheme-specific-part>
```

<http://www.iana.org/assignments/uri-schemes.html>

Questo meccanismo può essere usato anche per identificare dei servizi e non solo delle risorse.

HTTP – Hypertext Transfer Protocol

Protocollo usato per il trasferimento dati in internet. Lo conosciamo tutti, il client invia una richiesta e il server risponde, tutto usando socket TCP.

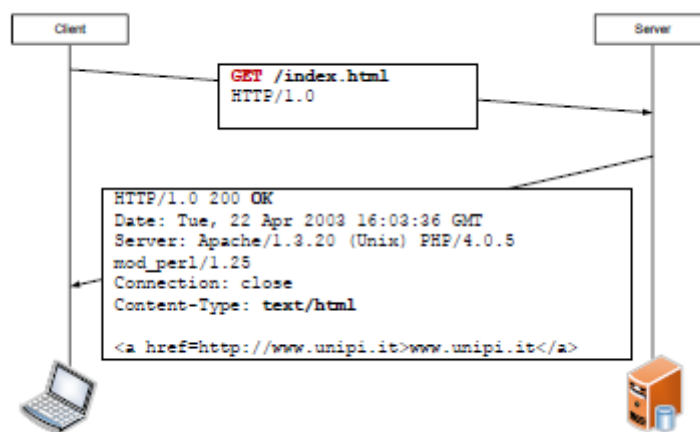


Figure 80 - Esempio di richiesta HTTP.

XML – eXtensible Markup Language

Linguaggio per codificare informazioni dentro messaggi HTTP. Il linguaggio usato per lo scopo inizialmente era XML: ossia un linguaggio text-based che permette di costruire documenti leggibili da una macchina ma anche da un essere umano (*human-readable*).

XML specifica la sintassi con cui scrivere le informazioni sul documento.

Il linguaggio è detto “*extensible*” perché il set di tag da usare nel documento non è specificato a priori e quindi ciò lo rende molto flessibile.

Sintassi di un documento XML

XML è costruito in modo gerarchico; ogni elemento è un componente logico che inizia con uno start-tag e finisce con un end-tag.

I tag possono contenere attributi.

Validazione di un documento XML

XML dà l'opportunità di validare un documento, ossia controllare se il documento è ben fatto:

- Tutti i tag sono aperti e chiusi correttamente.
- ...

Anche se un documento è ben fatto può comunque non avere senso...cioè, supponiamo che un documento XML debba fornire delle coordinate, se le elimino dal documento quest'ultimo continua ad essere ben fatto ma non avrà senso.

Per questo motivo, per controllare se il documento abbia senso si usa lo **XML Schema**, ossia un altro documento che specifica tutte le regole per verificare che un altro documento XML è valido (*ossia che contenga tutte le informazioni necessarie*).

Web Service Description Language - WSDL

WSDL è usato dai moduli consumatori e produttori per comunicare con il broker: Usato dal provider del servizio per interagire con il broker, ossia per registrarsi nel registro dei provider; in modo analogo è usato dai consumer cercare i servizi a lui necessari dal broker.

In WSDL i client possono capire automaticamente:

- Cosa può fare un servizio;
- Dove si trova un servizio;
- Come invocare un servizio.

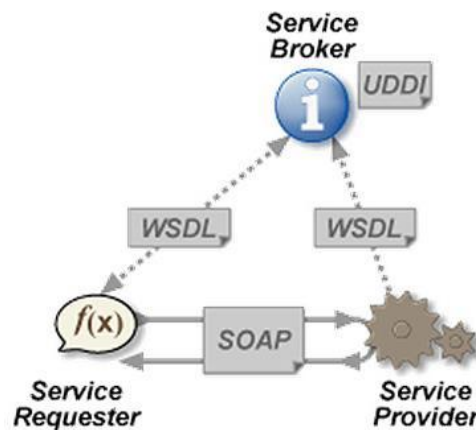


Figure 81 - Come è utilizzato WSDL e SOAP.

Un messaggio WSDL è un file XML composto da due sezioni:

Service Interface Definition (abstract)

Descrizione astratta del servizio; comprende:

- **Type**: Il tipo di dato, serve per definire lo scopo del messaggio.
- **Messages**: Il messaggio.
- **portType**: il tipo di porta.

Service Implementation Definition (concrete)

Definisce in modo più concreto riguardo come il servizio deve essere invocato; comprende:

- **Binding**: Come il messaggio deve essere trasferito.
- **Service**: Come invocare il messaggio.

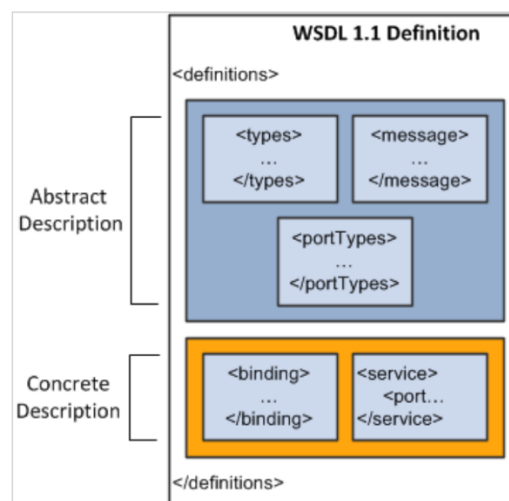


Figure 512 - Struttura di un messaggio di WDSL.

WSDL Type

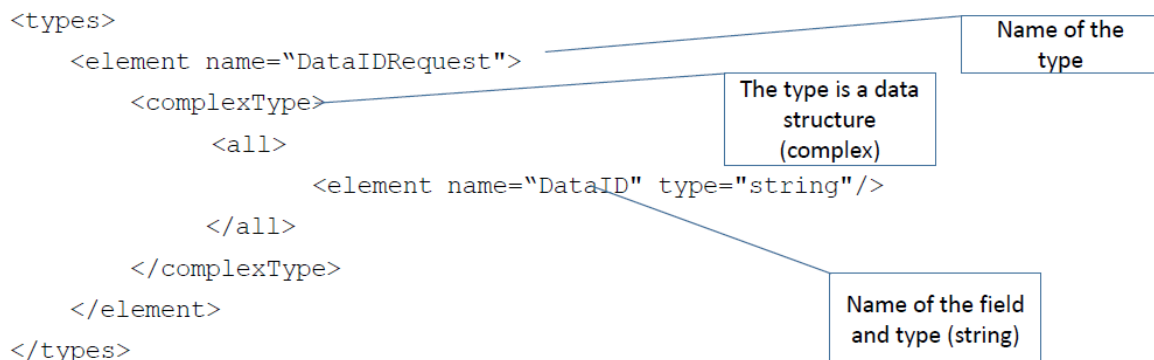


Figure 523 - Struttura WSDL type.

WSDL Messages

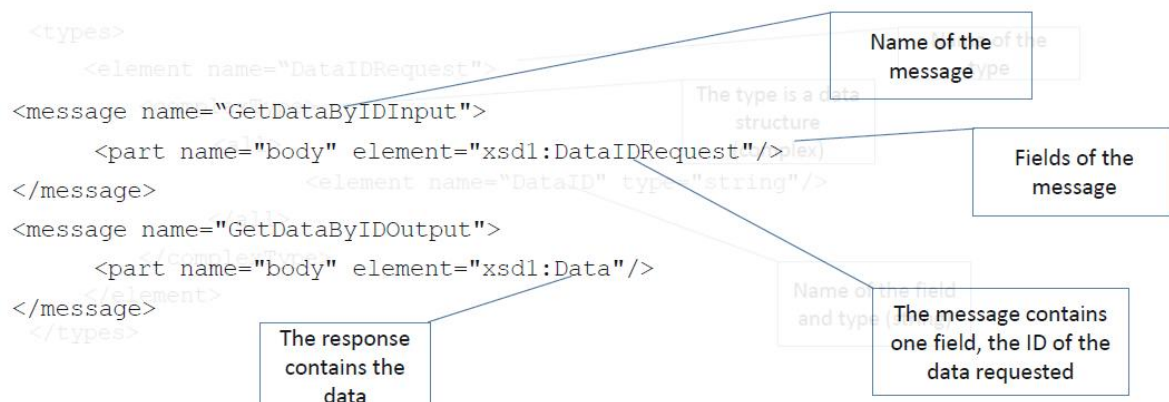


Figure 534 - Struttura WSDL message.

WSDL PortType

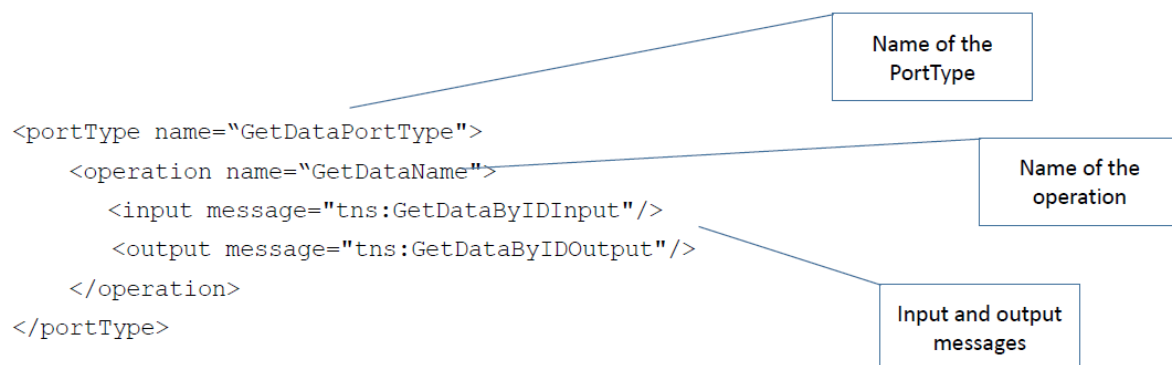


Figure 545 - Struttura WSDL portType.

WSDL Binding



Figure 556 - Struttura WSDL binding.

Simple Object Access Protocol - SOAP

SOAP è usato dai moduli consumatori e produttori per comunicare tra loro: SOAP è un protocollo che definisce una standard packaging structure, ossia definisce come il produttore ed il consumatore si scoprono a vicenda.

La loro interazione avviene scambiandosi documenti XML tramite protocollo HTTP (**binding**), infatti SOAP specifica solo la struttura dei messaggi scambiati tra producer e consumer. SOAP può utilizzare i metodi GET e POST.

Un messaggio SOAP è composto in un elemento root chiamato **envelope**, il quale contiene un header ed un body. L'header contiene informazioni aggiuntive e di sicurezza, il body invece

contiene il payload (include informazioni risposta/richiesta) e una sezione che include errori ed eccezioni.

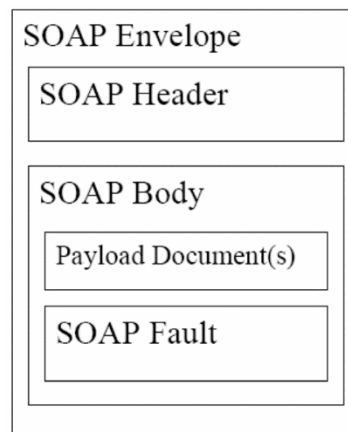


Figure 567 - Struttura di un messaggio SOAP.

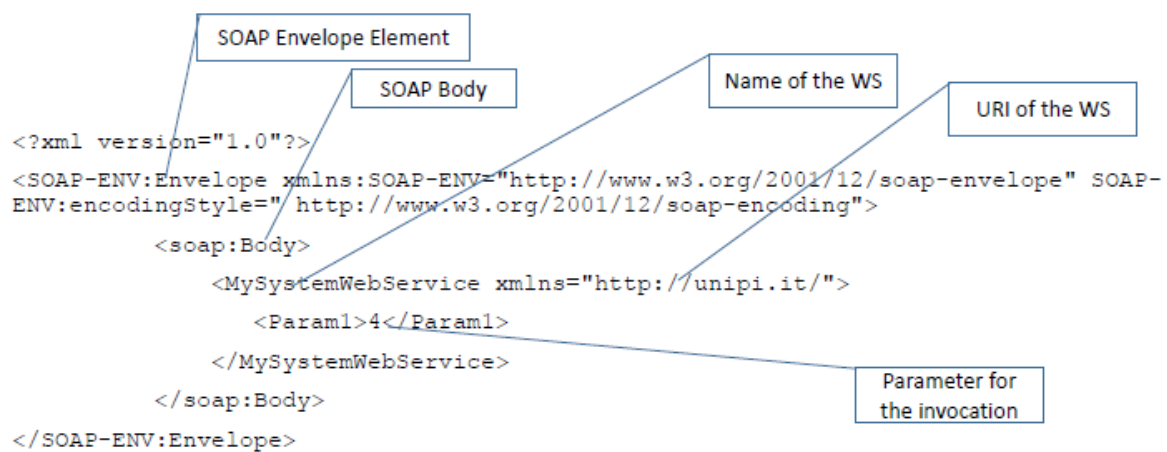


Figure 88 - Struttura di un messaggio SOAP nel particolare.

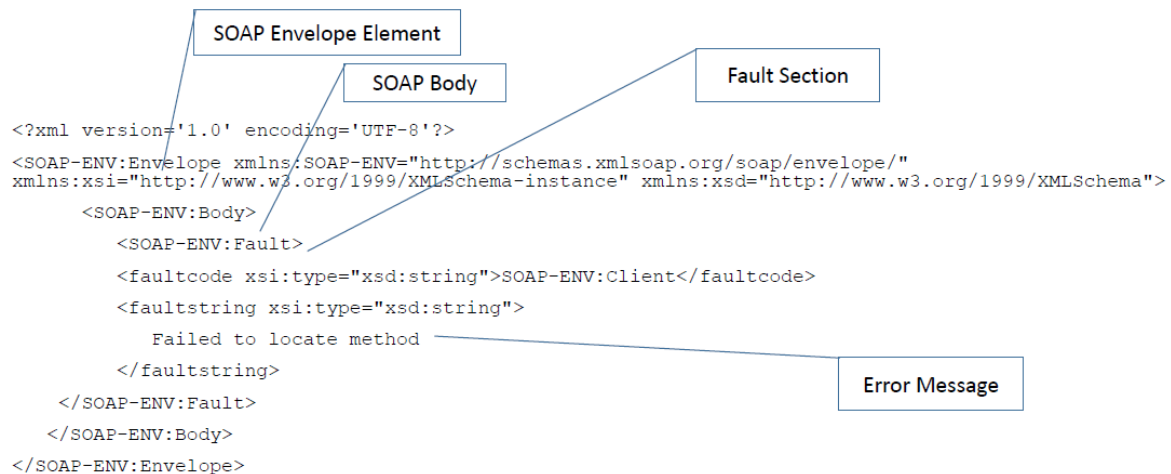


Figure 89 - Struttura di un messaggio SOAP nel particolare.

REST – Representational State Transfer

Ogni servizio è implementato in moduli e ogni modulo mostra un'interfaccia verso l'esterno. REST sfrutta un sistema molto simile a quello di SOAP ma pesantemente semplificato, non c'è uno schema XML associato ad un messaggio come in SOAP (*comunque sfruttano entrambi http*).

SOAP è ancora utilizzato in grandi progetti, mentre per progetti medio piccoli REST è la soluzione migliore.

REST fornisce un modello per l'interazione tra i servizi.

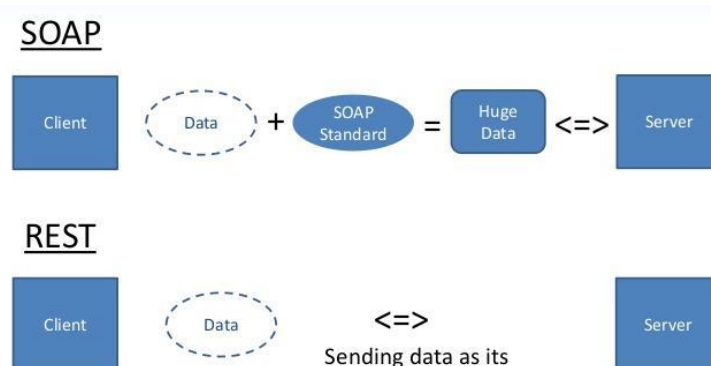


Figure 90 - Differenza tra SOAP e REST in termini di "cosa da fare".

In un sistema **RESTful**:

1. Il client invia una richiesta HTTP al server (*GET o POST*).
2. Il server invia la risposta (*documento XML o JSON*) sempre usando il protocollo HTTP.

Il vantaggio rispetto a SOAP è che abbiamo meno overhead in quanto un sistema REST è molto più semplice. REST non include un preciso set di specifiche o standard, ciò lo rende facile da implementare e molto flessibile. In REST un client invia una richiesta HTTP sfruttando i metodi HTTP standard (PUT, GET, POST e DELETE).

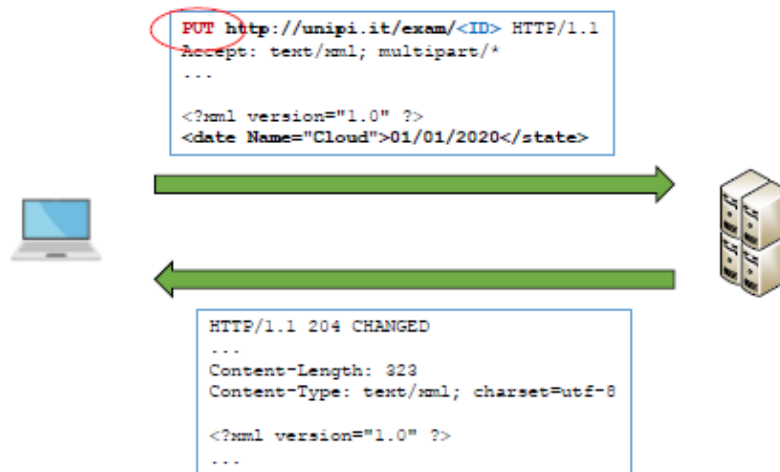


Figure 91 - Esempio di una chiamata REST con XML.

JSON – JavaScript Object Notation

Metodo per codificare le informazioni in modo che anche un essere umano possa leggerle. Sostituì XML, in quanto molto più semplice da usare soprattutto con linguaggi ad alto livello come Javascript o Python.

In REST viene usato molto spesso il formato JSON per codificare le informazioni delle risposte del server.

JSON	XML
<pre>{ "menu": { "id": "file", "value": "File", "popup": { "menuitem": [{ "value": "New", "onclick": "CreateNewDoc()" }, { "value": "Open", "onclick": "OpenDoc()" }, { "value": "Close", "onclick": "CloseDoc()" }] } } }</pre>	<pre><menu id="file" value="File"> <popup> <menuitem value="New" onclick="CreateNewDoc()" /> <menuitem value="Open" onclick="OpenDoc()" /> <menuitem value="Close" onclick="CloseDoc()" /> </popup> </menu></pre>

Figure 92 - Differenze tra JSON e XML.

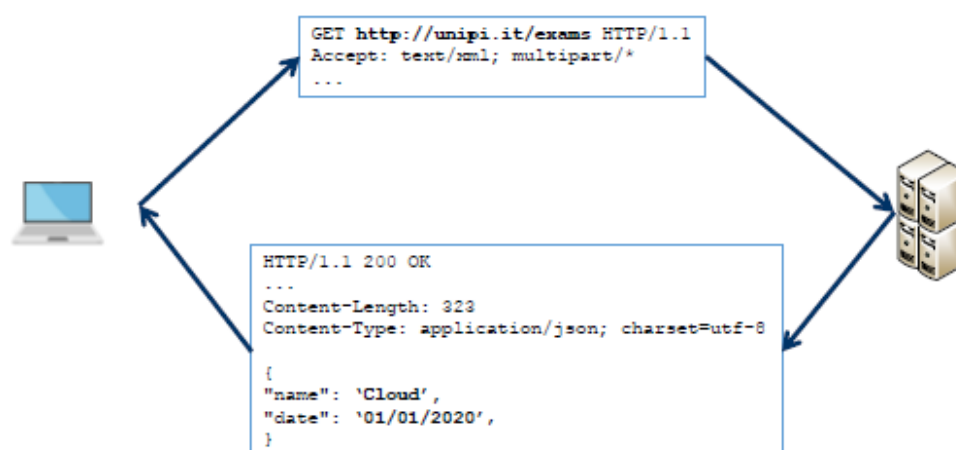


Figure 93 - Esempio di una chiamata REST con JSON.

CRUD Operations – Operazioni di tipo Create, Read, Update & Delete

Queste quattro operazioni basilari sono totalmente coperte dalle operazioni base di http; ossia ognuna delle CRUD ha un corrispettivo in http.

Route name	URL	HTTP Verb	Description
Index	/blogs	GET	Display a list of all blogs
New	/blog/new	GET	Show form to make new blogs
Create	/blogs	POST	Add new blog to database, then redirect
Show	/blogs/:id	GET	Show info about one blogs
Edit	/blogs/:id/edit	GET	Show edit form of one blog
Update	/blogs/:id	PUT	Update a particular blog, then redirect
Destroy	/blogs/:id	DELETE	Delete a particular blog, then redirect

Figure 94 - Operazioni REST e relativo metodo HTTP da chiamare.

Compute Cluster

Le macchine virtuali che compongono un livello sono chiamate “*compute cluster*” e ognuna di esse implementa delle funzionalità relative a quel livello.

Lo scopo di un compute cluster è quello di eseguire un unico grande job oppure tante piccole richieste (*in genere operazioni di tipo analitico sui dati e di tipo I/O per trasferire i dati tra i nodi del cluster*).

In ogni caso le macchine virtuali che compongono il cluster dovranno comunicare e soprattutto collaborare.

Nel caso in cui si debba lavorare su un unico grande job; una buona pratica è quella di fare in modo che il job sia divisibile in tanti piccoli task, in modo che ogni task è assegnato ad una VM e che sia anche duplicabile su più macchine virtuali (*per avere più disponibilità e scalabilità*).

High Availability Cluster – HAC

L’obiettivo è quello di rendere il sistema resistente ai guasti e infatti ed evitare single point of failure, è solitamente adottato nelle applicazioni che devono lavorare costantemente e che non tollerano neanche un secondo di inattività.

Le funzioni di ogni livello sono organizzate in modo da poter essere replicate su più macchine virtuali dello stesso livello, poiché la ridondanza porta la disponibilità. Quindi ogni macchina virtuale ha una o più repliche (*in un altro compute cluster del livello*) pronte a sostituirla in ogni momento.

L’esempio più semplice di una HAC è quello di un server web e di una sua replica su un altro nodo. Un modo semplice per organizzare la replicazione è una configurazione Master-Slave dove il server web è replicato molteplici volte nelle macchine slave.

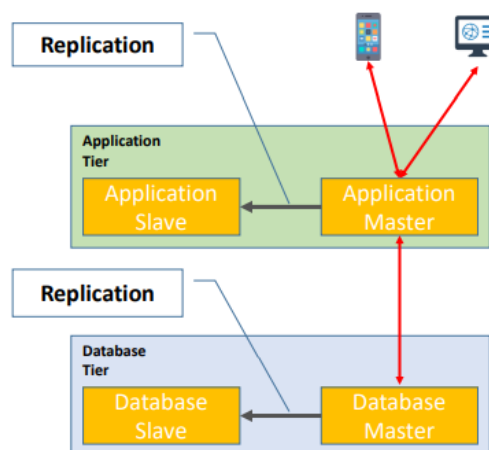


Figure 95 – Diagramma che mostra un’architettura HAC, implementazione Master-Slave.

Il master è quello che riceve e processa le richieste dei client (e *nel caso aggiorna le repliche negli slave*).

Gli slave rimangono in attesa che il master fallisca e nel mentre ricevono gli update dal master per rimanere consistenti.

Appena il master fallisce uno degli slave diventa il nuovo master secondo una di queste procedure:

- **Ordine Statico:** Deciso a priori dal progettista e che tutti gli slave conoscono.
- **Elezione:** Sistema di elezione tra gli slave, secondo un certo protocollo.

Anche il database va replicato e mantenuto consistente:

- **Sincronizzato:** Lo slave è sincronizzato con lo stato del master.
- **Non sincronizzato:** Lo slave non è sincronizzato con lo stato del master.

Gli svantaggi di questa architettura sono:

- Gli slave aspettano molto e quindi sprecono risorse.
- Il master è un single point of failure.

Load-Balancing Cluster - LBC

L'obiettivo è quello di aumentare l'utilizzazione delle risorse e la scalabilità orizzontale del sistema senza influenzare l'esperienza utente.

Le richieste vengono distribuite tra tutte le macchine virtuali applicative del compute cluster in modo da bilanciare il carico su ognuno: il risultato è che il sistema può servire tanti utenti quante le risorse assegnate al cluster (*non si sprecono risorse poiché tutte sono utilizzate in egual misura*).

Il risultato è un carico bilanciato tra varie VM che porta ad una maggiore utilizzazione delle risorse e prestazioni migliori come il tempo di risposta.

Load Balancer Tier

Per fare ciò si introduce un livello (*tier*) aggiuntivo tra l'utente e l'applicazione che si prende la responsabilità di distribuire le richieste alle entità del livello sottostante, dove si trovano le macchine virtuali sulle quali girano le applicazioni.

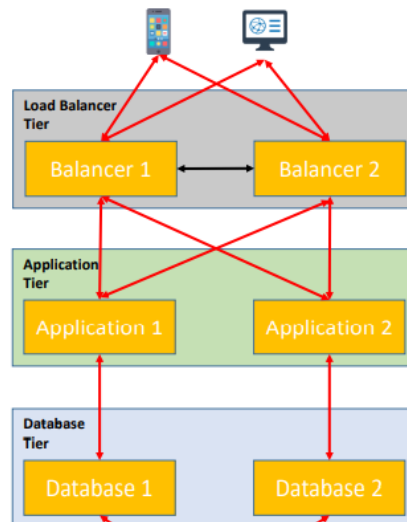


Figure 96 – Diagramma che mostra un'architettura LBC.

Questo livello (*implementato in hardware o tramite macchine virtuali*) ha il compito di ricevere le richieste degli utenti e smistarle verso le istanze applicative secondo una certa politica di bilanciamento del carico.

Le macchine virtuali applicative non si accorgono del livello addizionale che si occupa del bilanciamento del carico.

Le applicazioni invece inoltrano le richieste al livello database in modo semplice, ad ogni applicazione è assegnata un'istanza del livello database.

È importante la sincronizzazione dei dati tra le istanze dello stesso livello poiché ho molte macchine virtuali che gestiscono delle richieste contemporaneamente.

LBC e il problema della sincronizzazione

In una architettura master-slave ho il master che gestisce la replicazione sulle altre macchine, ma in un'architettura LBC invece ho bisogno di un sistema di sincronizzazione che permette ad ogni macchina virtuale di produrre risultati coerenti (*lo stesso input produce lo stesso output*).

Dato che ogni istanza di applicazione può ricevere le richieste, dobbiamo trovare un metodo per propagare gli aggiornamenti.

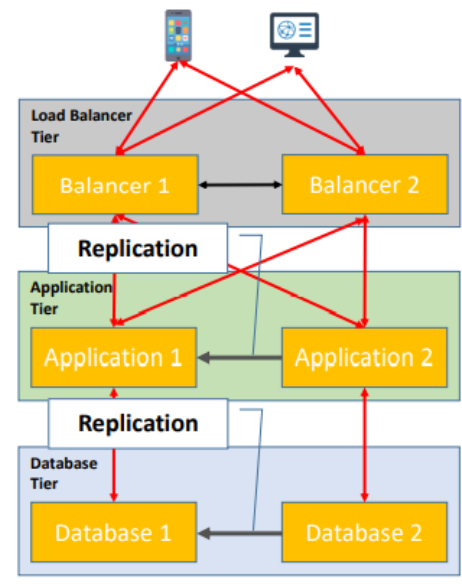


Figure 97 - LBC + Sincronizzazione tra macchine dello stesso livello.

I nodi porteranno avanti anche il compito di sincronizzare i nodi dello stesso cluster. Vogliamo che le richieste vengano gestite alla stessa maniera dalle VM, una possibilità è di implementare replicazione, tutte le VM che appartengono allo stesso cluster condividono lo stato attuale.

Storage tier

Una soluzione per evitare la replicazione e le procedure relative alla sincronizzazione consiste nell'avere un unico livello chiamato "storage tier".

Ovvero un tier in cui i dati vengono memorizzati utilizzando una tecnologia cloud (*quindi una tecnologia che minimizza i tempi di attesa e che è in grado di gestire moltissime richieste, insomma, non è un server comune*).

Questo tier è dove effettivamente i dati vengono memorizzati, quindi tutte le istanze del livello database prelevano i dati dalla stessa identica fonte, quindi sono per forza sincronizzati.

Capire quale metodo di sincronizzazione scegliere, dipende dal caso d'uso e dalla quantità di dati da memorizzare.

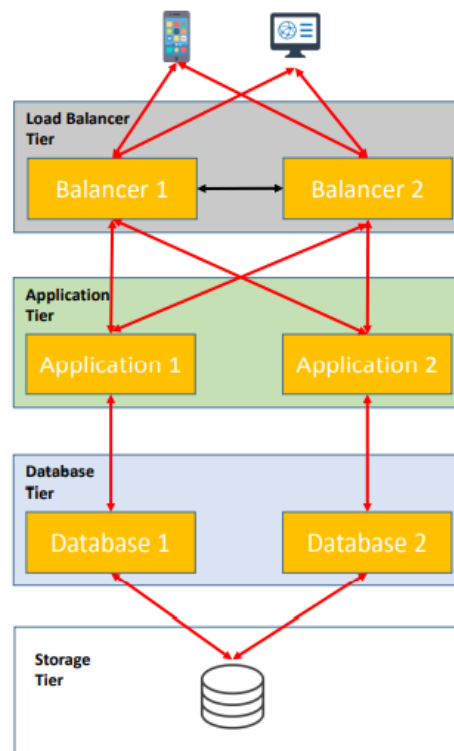


Figure 98 - Architettura finale di LBC.

CIC - Compute intensive cluster

Sono cluster progettati per analizzare e gestire una grande quantità di dati, considerando che i dati sono molti, l'analisi in un solo server (o VM) richiederebbe troppo tempo.

Il livello più alto ha il **load balancer tier** che serve a bilanciare il traffico proveniente dai client tra le varie istanze del livello applicativo.

Al centro abbiamo un **application tier**, il quale ha multiple istanze che gestiscono le richieste degli utenti. Ad ogni richiesta viene associato un job.

Ad ogni job sono associati dei dati di input i quali vengono divisi in chunk e ogni chunk è eseguito da un task.

Ogni task viene assegnato ad un nodo **worker**, il quale esegue il task sul chunk che ha ricevuto. I task sono eseguibili in parallelo.

Ogni worker è eseguito in una macchina virtuale/container separato. Il set di tutti i worker è quindi eseguito in una infrastruttura di processazione distribuita (tipo *OpenStack*).

Non è fattibile sincronizzare questa grande quantità di dati tra i vari worker. Perciò, si aggiunge uno **storage tier**, il quale è condiviso con tutti gli altri tier (*application e worker*). In questo modo non c'è bisogno di spostare dati.

I risultati di ogni task vengono inseriti in un **collector**, il quale unisce i risultati calcolati.

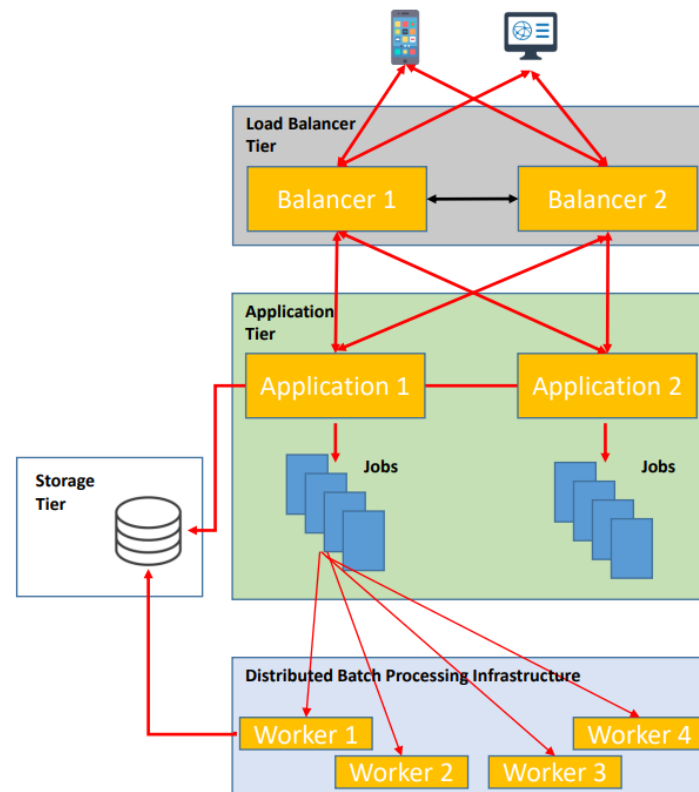


Figure 99 - Compute intensive cluster con storage tier condiviso tra livello applicativo e workers.

Il vantaggio della virtualizzazione è che in caso di aumento del carico possiamo facilmente scalare verticalmente oppure orizzontalmente in modo da migliorare il throughput (*numero di richieste soddisfatte al secondo*).

Load Balance cluster e Compute intensive cluster scalano orizzontalmente, non appena il traffico aumenta, nuove VM vengono create e incluse nel tier, il loro carico viene ridotto, non appena il carico decresce, le VM possono essere deallocate, le risorse non vengono sprecate.

High availability cluster sfrutta la scalabilità orizzontale per migliorare ridondanza o ribilanciare il tier in caso di fallimento.

Modulo di frontend e di backend

In genere possiamo sempre dividere un l'architettura di un'applicazione cloud in due grandi livelli:

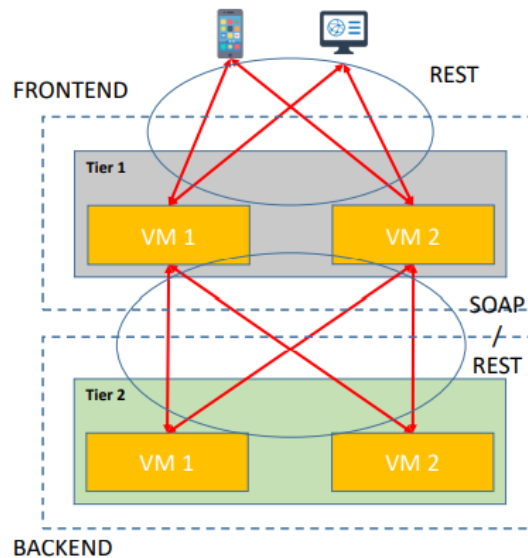


Figure 100 – Applicazione cloud divisa in frontend e backend

Il livello di **frontend** è quello che interagisce con i client e le loro richieste; quindi, accoglie le richieste e invia le risposte ricevute dal modulo di backend.

Il livello di **backend** invece è quello che effettivamente implementa la logica dell'applicazione; esso riceve le richieste dal modulo di frontend e invia le risposte al modulo di frontend.

Ogni macchina virtuale all'interno di questi due livelli ospita uno o più componenti software che chiameremo “servizi”. I servizi di diverse macchine virtuali e di diversi livelli interagiscono tra loro tramite scambio di messaggi, e questa comunicazione è un punto cruciale di questa parte.

Comunicazione diretta

Abbiamo visto come si possano usare il protocollo SOAP o una interfaccia REST per scambiarsi informazioni tra moduli, ma entrambi hanno un problema: sia SOAP che REST sono legati ad un protocollo richiesta-risposta ossia un protocollo di comunicazione con tre caratteristiche fondamentali:

- **Sincrono** Il mittente si blocca fino all'arrivo della risposta del destinatario (*ciò causa il time coupling*).
- **Affidabile**: Viene sempre inviata una risposta a conferma della ricezione del messaggio (*questa cosa va sempre bene*).

- **Diretto**: Il mittente e il destinatario comunicano direttamente senza intermediari (cioè causa lo space coupling).

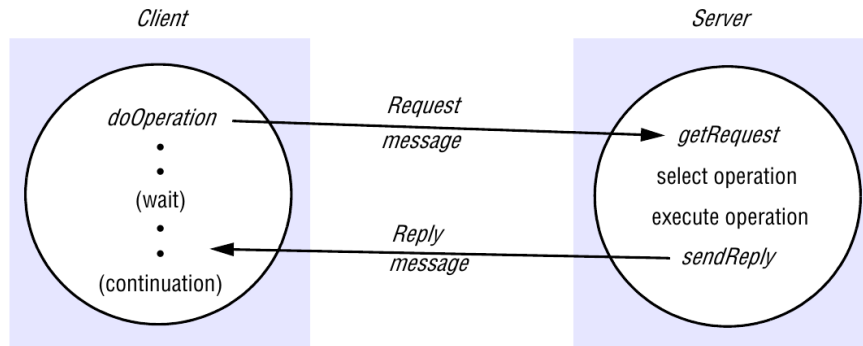


Figure 101 - Comunicazione sincrona, quindi con l'invio del messaggio e l'attesa della risposta.

Il paradigma richiesta-risposta soddisfa i requisiti per un interfaccia di comunicazione tra client e frontend: è semplice, affidabile e le applicazioni client possono sopportare un pò di delay.

Ma in caso di migliaia di richieste al secondo (come la comunicazione tra frontend e backend) il coupling spaziale e il coupling temporale sono un problema che va a compromettere la scalabilità orizzontale del sistema (il punto cruciale di una applicazione cloud).

Coupling temporale

Quando viene inviato un messaggio, il mittente deve attendere la risposta del server. Nel mentre il mittente non fa niente e quindi perde tempo.

Coupling spaziale

Il mittente conosce l'identificatore del destinatario (ad esempio l'indirizzo IP o lo URL) prima di inviare la richiesta.

In una comunicazione di tipo cloud (con molta scalabilità orizzontale e quindi macchine virtuali che vanno e vengono di continuo); non va bene perché non permette di sfruttare al meglio la scalabilità orizzontale. Il protocollo deve gestire l'elevatissima scalabilità dell'infrastruttura, con macchine virtuali che mutano, nascono, muoiono o che comunque cambiano spesso indirizzo IP (o comunque l'identificatore).

Comunicazione indiretta

In questo tipo di comunicazione si punta ad eliminare il problema dello space coupling e del time coupling.

Cosa dobbiamo ottenere teoricamente per eliminare entrambi i problemi?

- **Ridurre il coupling spaziale:** Il mittente deve poter non conoscere l'identificatore univoco dei destinatari. Se questa cosa vale il mittente e i destinatari possono essere sostituiti a piacere poiché gli uni sono nascosti dagli altri.
- **Ridurre il coupling temporale:** Il destinatario deve non poter essere obbligato ad esistere nello stesso momento del mittente per comunicare con lui. Se questa cosa vale il mittente e il destinatario possono essere in esecuzione in due momenti diversi e comunque comunicare.

Entrambi i problemi si eliminano con l'aggiunta di un intermediario (*Man in The Middle* o chiamato anche *Message Broker*).

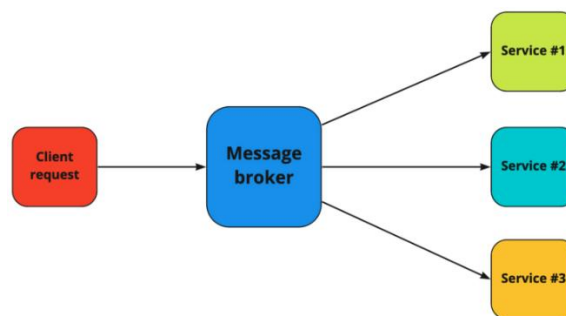


Figure 102 - Funzionamento semplificato del message broker in una comunicazione indiretta.

Risolve il coupling spaziale

Il message broker si interpone tra i due interlocutori (*processi, servizi o macchine virtuali*) e mostra un comportamento simile a quello di uno switch di una LAN: Il mittente invia il messaggio al broker e quest'ultimo inoltra il messaggio ai destinatari.

Il mittente non conosce il destinatario (*basta solo che si conosca il broker*) e viceversa poiché il primo invia messaggi al broker e il secondo riceve messaggi dal broker.

Risolve il coupling temporale

1. Il mittente lascia il messaggio nel broker (*il broker è sempre in esecuzione*).

Il mittente a questo punto può anche terminare o sospendersi.

2. Il destinatario recupererà il messaggio quando vuole lui dal broker.
3. Il destinatario lascia la risposta nel broker (*il broker è sempre in esecuzione*).

Il destinatario a questo punto può anche terminare o sospendersi.

4. Il mittente recupererà la risposta quando vuole lui dal broker.

Come si può notare, il mittente e il destinatario non devono esistere contemporaneamente per funzionare.

Non tutto è oro ciò che luccica! aggiungendo il broker abbiamo aggiunto un elemento di centralizzazione, il quale può diventare un collo di bottiglia e anche un single point of failure; quindi, deve essere implementato in modo opportuno.

A seguire verranno illustrati due metodi per implementare il message broker.

Sistema Publish-Subscribe

Sistema di comunicazione **one-to-many**, quindi per messaggi che devono essere inoltrati a tanti destinatari.

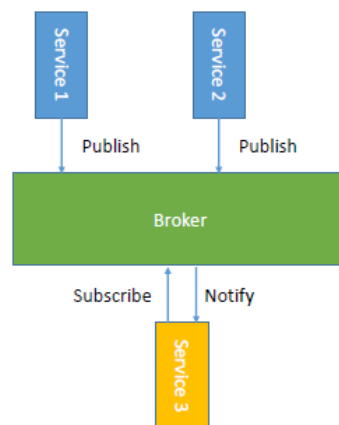


Figure 103 - Sistema di Publish-Subscribe.

I subscribers (*destinatari*) all'inizio della loro vita si iscrivono ad alcuni tipi (*topic*) di messaggi.

1. I publisher (*mittenti*) incapsulano i messaggi e li inviano al broker.

Il publisher non conosce quali e quanti subscriber riceveranno il messaggio (*risolto il coupling spaziale*).

Da questo punto in poi il publisher può anche scollegarsi (*risolto il coupling temporale*).

2. All'arrivo di un messaggio di uno specifico topic, il broker notifica tutti i subscribers iscritti a quel topic.

Ogni messaggio ha delle proprietà (*topic, canale, evento, etc...*) che identificano il tipo di dato contenuto nel messaggio.

Tipologia di iscrizioni

Un'iscrizione può avere varie tipologie:

- **Channel-Based:** L'iscrizione è legata al canale, quindi il subscriber riceverà tutti i messaggi del canale scelto.
- **Topic-Based:** L'iscrizione è legata al topic, quindi il subscriber riceverà tutti i messaggi del topic scelto. Ogni dato ha un insieme di campi, ognuno di essi denota il topic del dato. Questo metodo è equivalente al channel-based con la differenza che i topic sono definiti implicitamente nel caso channel-based ma dichiarati esplicitamente come uno dei campi nell'approccio topic-based.
- **Type Based:** L'iscrizione simile a quella per topic, solo che questa è relativa al tipo di evento legato al messaggio. C'è un collegamento tra il topic di un messaggio e un evento, alla configurazione del sistema c'è un insieme di eventi sui quali il produttore può postare un messaggio, l'evento è relativo ad un evento fisico. Quando un evento viene generato, il messaggio associato viene inviato al broker, il broker invia il messaggio ai subscriber iscritti a quell'evento.

Sistema a coda di messaggi

Sistema di comunicazione **one-to-one**, quindi per messaggi che devono essere inoltrati a solo un destinatario; in questo caso non importa a quale destinatario di preciso verrà inviato.

Il sistema è one-to-one perché, quando il consumatore lo preleva viene anche rimosso dalla coda.

In questo caso il broker è un sistema di diverse code:

1. Il mittente (*producer*) invia un messaggio (*operazione di send*), il quale viene accodato in una delle code del sistema.

Nota che la coda in cui verrà messo il messaggio è scelta dal mittente.

Il producer non sa quale consumer riceverà il messaggio (*risolto il coupling spaziale*).

Da questo punto in poi il producer può anche scollegarsi (*risolto il coupling temporale*).

2. Il consumatore può ricevere un messaggio dalla coda scelta, estraendolo (*con una operazione bloccante o non bloccante*) secondo una **politica FIFO**.

A meno che non ci sia scritto nel messaggio, il consumer non sa chi ha inviato il messaggio (*risolto il coupling spaziale*).

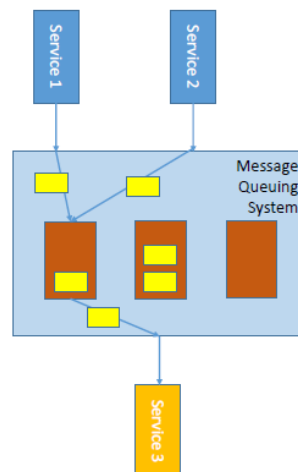


Figure 104 - Sistema a coda di messaggi.

Estrazione bloccante

Blocca il consumatore fino all'arrivo di un messaggio nella coda voluta.

Estrazione non bloccante

Chiamata anche “*notify*” ed è anche la soluzione usata nel sistema Publish-Subscribe: il broker notifica tutti i consumatori iscritti a quella coda nel caso venga ricevuto un messaggio nella coda voluta.

Il primo consumer che effettuerà l'estrazione (*eseguita in modo atomico ed esclusivo*) riceverà il messaggio, per gli altri non cambierà nulla, rimarranno in attesa della prossima notifica.

Struttura di un messaggio

Ogni messaggio ha una struttura ben precisa ma che deve comunque essere abbastanza generica in modo da poter essere facilmente scalabile:

- **Destinazione:** Identificatore univoco della coda di destinazione.
- **Metadati:** Vari campi come la priorità o la modalità di spedizione.
- **Body:** Il messaggio vero e proprio; di solito è considerato opaco (*ossia non viene mai toccato o letto dal broker*).

Sistemi avanzati sono in grado di analizzare il body per effettuare scansioni (*con scopi di sicurezza o per servizi di complessità maggiori*).

Questa cosa è anche in grado di semplificare il ricevitore e quindi di permettere una più facile sostituzione o aggiornamento dell'implementazione di quest'ultimo.

I messaggi sono salvati in modo persistente

I messaggi depositati nella coda vengono salvati in modo persistente da qualche parte nella macchina che ospita il message broker.

In questo modo in caso di crash del message broker i messaggi vengano salvati e riproposti nel momento in cui il message broker ritorna attivo.

Scalabilità del sistema dei broker e delle code di messaggi

Il sistema di code di messaggi può essere un collo di bottiglia se implementato malamente o anche un single point of failure.

Per questo i broker sono anche implementati in modo distribuito. In caso di broker centralizzato si può utilizzare il broker o il sistema di code, ma in caso di broker distribuito si può usare il sistema di code.

Le code di messaggi sono perfette per la comunicazione nel backend delle applicazioni cloud, poiché possono gestire la dinamicità che caratterizza quell'ambiente

Ad esempio, possono gestire il numero variabile di VM in un livello per scalare il sistema: le VM che compongono il cluster di un livello che implementa le stesse funzionalità possono ricevere dati dalla stessa coda in base a una politica (ad esempio bilanciamento del carico o disponibilità)

Le situazioni dinamiche in cui le VM vengono create dinamicamente in fase di esecuzione vengono gestite, poiché il consumatore non deve esistere quando viene generato il messaggio

I produttori non devono attendere lo scambio di messaggi di richiesta-risposta bloccanti.

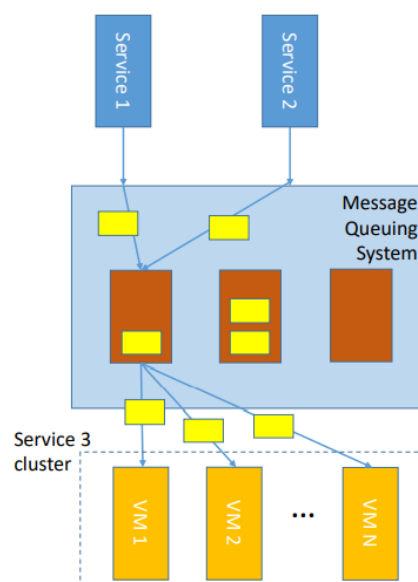


Figure 105 - Esempio in cui una coda diventa un collo di bottiglia.

AMQP - Advanced Message Queieing Protocol

AMQP è un'implementazione **standard** (e quindi anche pubblica) di un sistema di code di messaggi distribuito.

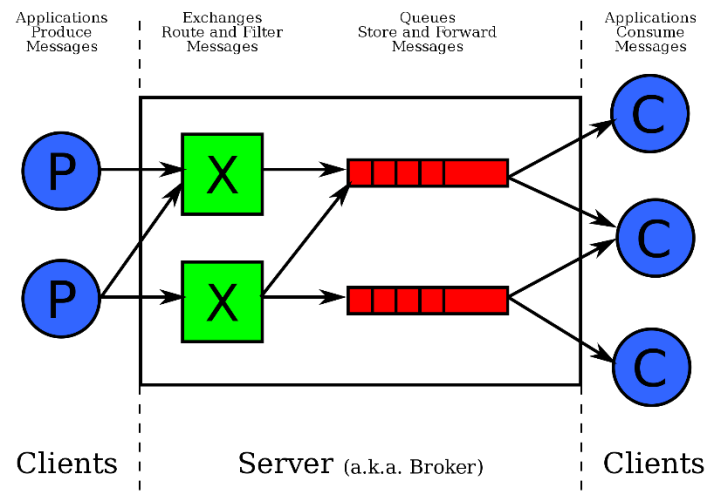


Figure 576 - Diagramma che mostra una AMQP.

È progettato per essere affidabile, scalabile e interoperabile tra diverse piattaforme e linguaggi di programmazione.

Produttore di messaggi

Invia messaggi a un exchange (*scambio*). L'exchange è responsabile di ricevere i messaggi dai produttori e instradarli verso le code appropriate in base a determinate regole definite.

Exchange

Funziona come un intermediario che riceve i messaggi dai produttori e li instrada verso le code corrette in base a criteri definiti.

Code di messaggi

I messaggi inviati all'exchange vengono quindi instradati verso una o più code, in base alle regole configurate.

Consumatore di messaggi

Un'applicazione che desidera ricevere e elaborare i messaggi si connette a una coda di messaggi e inizia a consumare i messaggi disponibili. Dopo aver elaborato un messaggio, il consumatore può confermarne l'elaborazione al server AMQP, che provvederà quindi a rimuoverlo dalla coda.

Broker AMQP

Il broker AMQP è il server che implementa il protocollo AMQP e gestisce lo scambio, l'instradamento e lo stoccaggio dei messaggi. Il broker gestisce le connessioni dei produttori e dei consumatori, garantendo la consegna affidabile dei messaggi e la scalabilità del sistema.

LAB - RabbitMQ

Implementazione open-source di un message broker che utilizza il protocollo AMQP.

È progettato per facilitare la comunicazione tra applicazioni distribuite, fornendo un'infrastruttura robusta e affidabile per lo scambio di messaggi tra diversi componenti software.

Fornisce un'applicazione produttore e un'applicazione consumatore, mentre il broker inoltra i messaggi tra esse.



Figure 587 - Funzionamento di RabbitMQ.

Oltre a AMQP, RabbitMQ supporta anche altri protocolli di messaggistica come MQTT, STOMP e HTTP, rendendolo flessibile e adatto a diverse esigenze di comunicazione.

RabbitMQ supporta la configurazione di cluster per garantire alta disponibilità e scalabilità. I messaggi possono essere replicati tra diversi nodi del cluster per assicurare che non vadano persi in caso di guasti.

RabbitMQ è ampiamente utilizzato nel contesto del cloud computing per facilitare la comunicazione tra servizi distribuiti, gestire carichi di lavoro, scalare applicazioni e garantire la resilienza del sistema.

Workflow di RabbitMQ

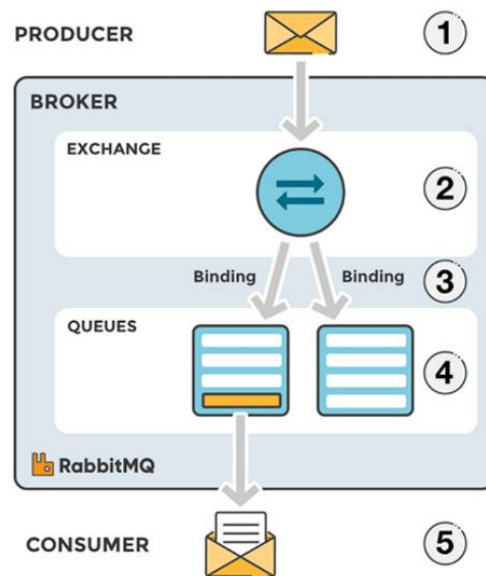


Figure 598 – Workflow dell'accodamento di un messaggio in RabbitMQ.

1. Il produttore pubblica un messaggio e per farlo lo invia all'exchange di RabbitMQ.
2. L'exchange riceve il messaggio e lo inoltra verso una delle code secondo una certa politica (*spiegata dopo*).

Tra le code e l'exchange ci sono delle strutture dette *binding*, contengono regole che legano un exchange con una delle code.

3. Il messaggio viene inserito nella coda.
4. Il messaggio rimane nella coda fino a che un consumatore non lo preleva.
5. Un consumatore estrae il messaggio secondo una politica FIFO.

Tipologie di exchange

Ogni coda ha associata una *routing key*, in base alle *routing key* e ai *binding* decidere dove inoltrare il messaggio può basarsi su tre tipologie:

- **Direct**: Il messaggio è inoltrato nelle code le quali *routing key* sono esattamente uguali a quella del messaggio.
- **Fanout**: Il messaggio è inviato a tutte le code a cui l'exchange è collegato.
- **Topic**: Dalla *routing key* vengono estratte delle parole ed esse sono usate (*tipo hashtag*) per indirizzare il messaggio della coda giusta (in base ai match delle parole con il binding). I messaggi inoltrati non sono basati su *routing key*, sono basati su una lista di parole separate da un punto. È possibile utilizzare *** per sostituire una parola oppure *#* per sostituire 0 o più parole.

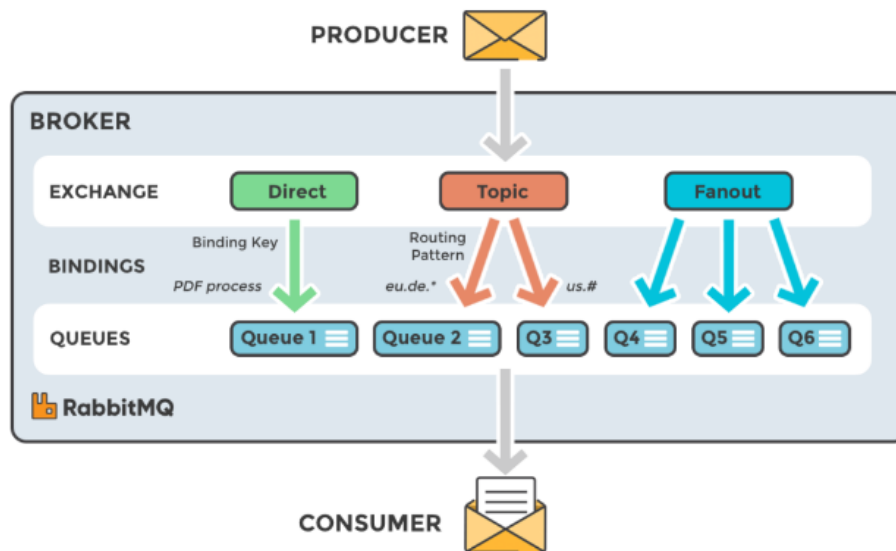


Figure 609 - Comportamento a seconda del tipo di exchange.

In particolare, topic permette di instradare i messaggi basandosi sulle parole (come dei tag in un social network) contenute nella routing key.

`routing_key1 = "lazy.green.rabbit" → [ "lazy" , "green" , "rabbit" ]`

Questa routing key ha tre parole diverse separate dal punto.

Laboratorio RabbitMQ

- Installiamo rabbitMQ e lo implementiamo all'interno di un container.
- Si controlla l'indirizzo IP del container rabbit con (*docker network inspect bridge*) e si ottiene l'indirizzo, ad esempio `172.17.0.3/16`.
- Le credenziali sono `guest/guest`.
- Adesso si crea una cartella *producer* e una cartella *consumer*, all'interno di queste si crea un DockerFile inserendo il codice in figura, cambiando il nome del programma python nella terzultima e nell'ultima riga:

• Dockerfile

```
FROM python:3-alpine
RUN mkdir -p /usr/src/app
WORKDIR /usr/src/app
# Install the PIKA library
RUN pip3 install pika
# This variable forces pika to print something out
ENV PYTHONUNBUFFERED=1
COPY producer.py /usr/src/app
ENTRYPOINT ["python3"]
CMD ["producer.py"]
```

- Si crea il programma *producer.py* e si inserisce il seguente codice cambiando nella quarta riga l'indirizzo del container rabbitMQ (172.17.0.3).

```
import pika

# Connect to RabbitMQ
connection =
pika.BlockingConnection(pika.ConnectionParameters('172.17.0.5'))
channel = connection.channel()

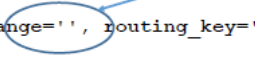
# Create a queue
channel.queue_declare(queue='hello')

# Send the message
channel.basic_publish(exchange='', routing_key='hello', body='Hello World!')

# Close the channel
connection.close()
```

Default exchange type
is used (direct)

Default exchange will
be used



- Si crea nella cartella *consumer*, il programma *consumer.py* inserendo il seguente codice importando la libreria all'inizio.

```
# Connect to RabbitMQ
connection = pika.BlockingConnection(pika.ConnectionParameters(host='172.17.0.5'))
channel = connection.channel()

# Connect to a queue
channel.queue_declare(queue='hello')

# Define a callback invoked every time a message is received
def callback(ch, method, properties, body):
    print(" [x] Received %r" % body)

# Subscribe to the queue and assign the callback
channel.basic_consume(queue='hello', on_message_callback=callback, auto_ack=True)

print(' [*] Waiting for messages. To exit press CTRL+C')
channel.start_consuming()
```

- Eseguire build e run del produttore e del consumatore.
- Controlliamo il log, ovvero mostriamo l'output del programma con i seguenti comandi.

```
docker logs consumer
docker logs producer
docker logs some-rabbit
```

```
root@IAJ1VX80PD7M5Q0:~/rabbitmq-consumer# docker logs consumer
[*] Waiting for messages. To exit press CTRL+C
[x] Received b'Hello World!'
[x] Received b'Hello World!'
root@IAJ1VX80PD7M5Q0:~/rabbitmq-consumer# docker logs producer
[x] Sent 'Hello World!'
```

```
2020-02-19 17:47:07.657 [info] <0.597.0> started TCP listener on [::]:5672
2020-02-19 17:47:07.657 [info] <0.267.0> Running boot step cluster_name defined by app rabbit
2020-02-19 17:47:07.657 [info] <0.267.0> Running boot step direct_client defined by app rabbit
2020-02-19 17:47:07.792 [info] <0.0.0> Server startup complete; 0 plugins started.
    completed with 0 plugins.
2020-02-20 12:18:50.209 [info] <0.2650.2> accepting AMQP connection <0.2650.2> (172.17.0.6:34150 -> 172.17.0.5:5672)
2020-02-20 12:18:58.227 [info] <0.2650.2> connection <0.2650.2> (172.17.0.6:34150 -> 172.17.0.5:5672): user 'guest' authenticated and granted access to vhost '/'
2020-02-20 12:18:58.232 [info] <0.2650.2> closing AMQP connection <0.2650.2> (172.17.0.6:34150 -> 172.17.0.5:5672, vhost: '/', user: 'guest')
2020-02-20 12:19:56.159 [info] <0.2728.2> accepting AMQP connection <0.2728.2> (172.17.0.6:34154 -> 172.17.0.5:5672)
2020-02-20 12:19:56.161 [info] <0.2728.2> connection <0.2728.2> (172.17.0.6:34154 -> 172.17.0.5:5672): user 'guest' authenticated and granted access to vhost '/'
2020-02-20 12:20:30.676 [info] <0.2780.2> accepting AMQP connection <0.2780.2> (172.17.0.7:49886 -> 172.17.0.5:5672)
2020-02-20 12:20:30.678 [info] <0.2780.2> connection <0.2780.2> (172.17.0.7:49886 -> 172.17.0.5:5672): user 'guest' authenticated and granted access to vhost '/'
2020-02-20 12:20:30.682 [info] <0.2780.2> closing AMQP connection <0.2780.2> (172.17.0.7:49886 -> 172.17.0.5:5672, vhost: '/', user: 'guest')
```

- Per stoppare producer e consumer inserire *docker stop producer* e *docker stop consumer*.
- Adesso si modificano producer e consumer in questo modo, ovvero abbiamo un exchange legato a due code, i messaggi vengono inoltrati alla coda corretta in base al tipo di messaggio.

```
# Message producer
...
# There is no need for queue declaration (I use the exchange)
channel.exchange_declare(exchange='exchange_name',
exchange_type='direct')

channel.basic_publish(exchange='exchange_name',
routing_key='name1', body='hello1')
channel.basic_publish(exchange='exchange_name',
routing_key='name2', body='hello2')
```

```
connection.close()
```

```
# Message consumer
...
channel.exchange_declare(exchange='exchange_name', exchange_type='direct')

# I let the system to create the queue name
result = channel.queue_declare(queue='', exclusive=True)
queue_name = result.method.queue

# Bind the queue to one or more keys/exchanges (it can be done at runtime)
channel.queue_bind(exchange='exchange_name', queue=queue_name, routing_key='name1')
channel.queue_bind(exchange='exchange_name', queue=queue_name, routing_key='name2')

channel.basic_consume(
    queue=queue_name, on_message_callback=callback, auto_ack=True)
channel.start_consuming()
```

```
root@IAJ1VX80PD7M5Q0:~/rabbitmq-producer# docker logs producer
[x] Sent 'Hello World!'
root@IAJ1VX80PD7M5Q0:~/rabbitmq-producer# docker logs consumer
[*] Waiting for messages. To exit press CTRL+C
[x] Received b'Hello1'
[x] Received b'Hello2'
```

- Adesso cambiamo producer e consumer in modo che la comunicazione sia **topic**.

Producer - Topic

```
channel.exchange_declare(exchange='topics', exchange_type='topic')

routing_key1 = 'lazy.green.rabbit'
routing_key2 = 'fast.yellow.zebra'

message = 'Hello World!'
channel.basic_publish(exchange='topics', routing_key=routing_key1,
body=message)
channel.basic_publish(exchange='topics', routing_key=routing_key2,
body=message)

connection.close()
```

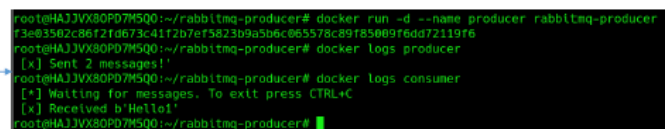
Consumer - Topic

```
channel.exchange_declare(exchange='topics', exchange_type='topic')

result = channel.queue_declare('', exclusive=True)
queue_name = result.method.queue

channel.queue_bind(exchange='topics', queue=queue_name,
routing_key='*. *.rabbit')
channel.queue_bind(exchange='topics', queue=queue_name,
routing_key='lazy.#')
```

Only the first message is
received!



```
root@HAJJ3VX80P07M5Q0:~/rabbitmq-producer# docker run -d --name producer rabbitmq-producer
f3e83502c86f21d673c41f2b7ef3823b9a5bdc065578c89f85009f6dd72119f6
root@HAJJ3VX80P07M5Q0:~/rabbitmq-producer# docker logs producer
[x] Sent 2 messages!
root@HAJJ3VX80P07M5Q0:~/rabbitmq-producer# docker logs consumer
[*] Waiting for messages. To exit press CTRL+C
[x] Received b'Hello!'
root@HAJJ3VX80P07M5Q0:~/rabbitmq-producer#
```

Replicazione dei dati

Il meccanismo della replicazione nasce prima dei servizi cloud, nasce con i sistemi distribuiti.

La replicazione dei dati tra macchine virtuali di uno stesso livello è fondamentale per garantire che tutte le macchine virtuali lavorino sugli stessi dati e che quindi il sistema restituisca sempre un risultato consistente.

Questo meccanismo di replicazione dei dati è implementato tramite scambio di messaggi seguendo un certo protocollo.

Di solito quando si ha un dataset molto grande, si adotta la strategia dello storage tier, dove utilizzando tecnologie cloud (*quindi altamente affidabili e veloci*) si crea un dataset condiviso tra tutte le macchine virtuali.

In ogni caso, le macchine virtuali devono coordinarsi tra loro e perciò abbiamo i seguenti requisiti:

Trasparenza della replicazione

Gli utenti non devono accorgersi di nulla riguardo al fatto che possono accedere a repliche diverse degli stessi dati.

Consistenza

Le repliche devono essere consistenti tra loro.

Ciò garantisce che ogni istanza di macchina virtuale dia lo stesso risultato di altre istanze partendo dallo stesso input.

Resilienza

Il meccanismo di replicazione deve resistere al fallimento di macchine virtuale (*dal punto di vista di una macchina virtuale*).

Cioè, se una macchina virtuale ha un problema i suoi dati devono essere al sicuro e il meccanismo di replicazione non deve avere problemi.

Questa cosa non vale nel caso di network partitions (*ossia macchine virtuali della stessa rete ma di diverse subnet non possono comunicare ed è giusto che non possano*).

Modello del sistema

I dati in un sistema possono essere una collezione di record, files oppure oggetti; qualunque cosa siano essi sono considerati come una collezione di oggetti logici.

Ogni oggetto logico è composto da una collezione di copie fisiche, chiamate repliche.

Ogni replica è memorizzata in un diverso Replica Manager (*non posso avere due repliche della stessa cosa nello stesso replica manager*), ossia un componente che memorizza un'istanza di una replica.

Nel nostro caso, i replica manager sono macchine virtuali appartenenti allo stesso livello. In ogni livello ci sono diversi RM.

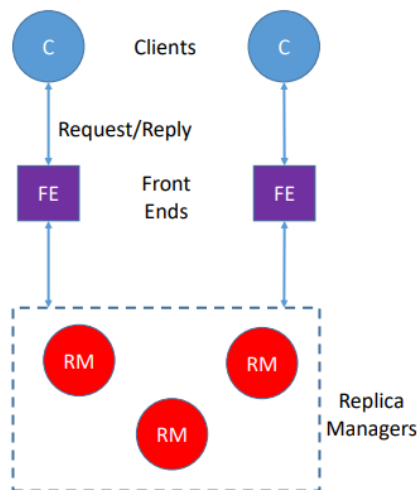


Figure 6110 - Diagramma di un sistema in cui vi è un sistema di replicazione.

Per assicurare la consistenza tutti i replica manager è a loro volta replicati. I dati delle repliche devono essere coerenti.

Gestione delle richieste

Una richiesta può essere:

- Una **query (lettura)**, la quale restituisce lo stato dell'oggetto senza modificarlo.
- Un **aggiornamento (modifica)**, la quale modifica lo stato di un oggetto.

In generale le richieste del client inviate ad un sistema di repliche vengono gestite in questo modo, seguendo cinque fasi generali, le quali cambiano a seconda della strategia.

Request

1. Il client invia una richiesta al modulo di frontend.
2. Il modulo di frontend verifica la correttezza della richiesta.
3. Il modulo di frontend inoltra la richiesta ad una o più istanze del replica manager.
4. Il replica manager riceve la richiesta.

Coordination

5. Si coordina con gli altri replica manager per decidere se va eseguita subito o rimandata (*deferred*) per mantenere la consistenza.

Al momento dell'esecuzione si coordina con gli altri replica manager per preparare l'esecuzione della query.

Execution

6. I replica manager eseguono la richiesta (viene eseguita nel backend).

In caso di fallimento si eseguono vari tentativi, in maniera che possa anche essere ritentata in futuro.

Aggrement

7. Dopo l'esecuzione ogni RM ha un risultato, in questa fase avviene una seconda coordinazione per determinare se un'inconsistenza è avvenuta. Quando c'è inconsistenza significa che ci sono RM i cui risultati non sono gli stessi.

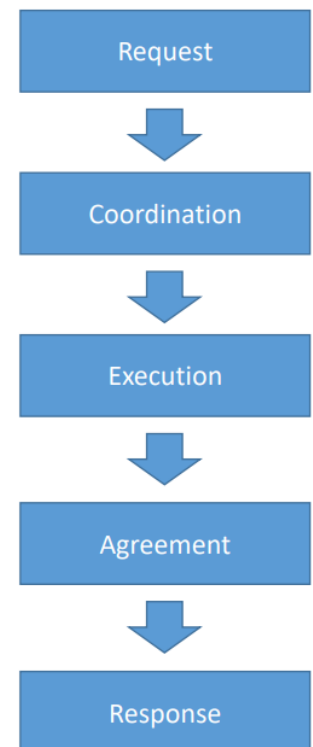


Figure 6211 - Le cinque fasi della gestione di una richiesta in un sistema con replicazione.

8. I replica manager si mettono d'accordo (solo se necessario a causa di inconsistenza) sull'effetto della query, ovvero su quale risultato sia quello corretto.
9. Se raggiungono un accordo avviene il commit e se ci sono aggiornamenti, questi vengono salvati in modo permanente.

Response

10. Uno o più replica manager rispondono al modulo frontend.
11. Il modulo frontend invia una delle risposte al client.

Comunicazione multicast

Funzionalità importante per l'implementazione delle funzionalità di agreement e coordination tra i replica manager.

Ci sono varie strategie per fare questa cosa e vanno scelte in base ai nostri requisiti e ognuna di queste provvede alla sincronizzazione dei dati tra diversi replica manager. Una di queste strategie è la comunicazione multicast, dove un RM invia in multicast o in broadcast una certa informazione ad un insieme di RM.

Il contro di una comunicazione multicast è l'aumento di complessità del sistema.

Il requisito è sempre lo stesso: la rete non deve essere partizionata; infatti, queste strategie hanno senso dentro un datacenter.

Reliable/Atomic multicast

Garantisce che ogni messaggio venga consegnato a tutti i membri del gruppo di destinazione oppure a nessuno.

Eventuali perdite nella comunicazione sono gestite e duplicati scartati. Non ci sono garanzie riguardo l'ordine di arrivo dei messaggi.

Ovvero se ci sono 3 messaggi inviati in questo ordine "A, B, C", un replica manager potrebbe ricevere prima il messaggio "B", dopo il messaggio "A" e infine il messaggio "C".

Total/Casual order multicast

Forma di comunicazione multicast che garantisce che tutti i messaggi inviati a un gruppo di destinatari siano ricevuti da tutti nello stesso ordine: se il messaggio "A" viene prima del messaggio "B", allora "A" deve sempre arrivare prima di "B" in tutti i destinatari.

L'ordine assoluto (*l'ordine seguendo il tempo assoluto di ricezione*) tra diverse repliche non è garantito. Ciò significa che nell'istante in cui un nodo ha già ricevuto "A" e "B" un altro nodo ha ricevuto solo "A".

View-Synchronous communication

È il modello di comunicazione multicast più costoso ma anche quello con più garanzie. se il messaggio “A” viene prima del messaggio “B”, allora quando un replica manager riceve “B” significa che tutti i replica manager hanno già ricevuto “A”.

Primary-Backup replication

La più semplice strategia di replicazione che ci sia: è usata per garantire la **resistenza ai guasti e ai fallimenti**.

All’arrivo di una richiesta da parte del modulo di frontend viene selezionato un replica manager come “*primario*” (il *replica manager primario* può anche essere “fisso”).

Quando un manager è selezionato come primario (*quello che processa la richiesta del client*) gli altri manager si comportano come backups (*slaves*) in modo da mantenere i propri dati aggiornati.

Se il manager primario fallisce, uno degli slaves (*secondo una certa politica*) prende il suo posto nell’esecuzione della richiesta in quanto è perfettamente aggiornato con il primario.

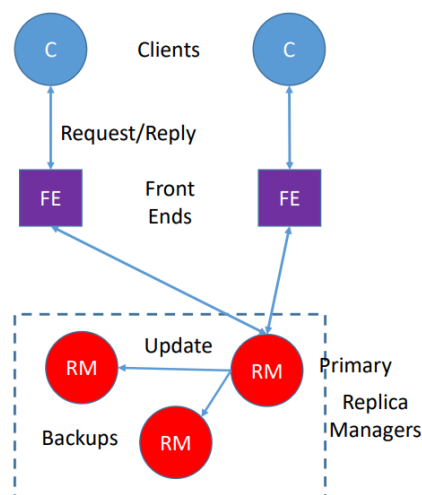


Figure 6312 – Diagramma che mostra la replicazione primary-backup.

Questa strategia è adatta ad un sistema ad alta disponibilità.

1. **Request:** Il frontend riceve una richiesta e gli appende un ID unico, che identifica la richiesta.

L’identificatore è necessario al manager primario per capire se quella richiesta è già stata ricevuta durante la fase di coordinazione. Se è già stata ricevuta la rinvia al modulo frontend.

2. **Coordinazione:** Il manager primario gestisce le richieste in autonomia nell’ordine in cui le ha ricevute. Processa le richieste una per una e non in parallelo per evitare inconsistenze.

3. **Esecuzione:** Il manager primario esegue la richiesta e memorizza la risposta.
4. **Agreement:**
 - a. Se la richiesta è un update: Il primario invia l'update agli slaves e attende le conferme (*per questa cosa una atomic communication è sufficiente*).
 - b. Se la richiesta è una query di lettura: Non succede nulla.
5. **Response:** Il primario risponde al modulo di frontend.

Per questo tipo di approccio, la comunicazione reliable/atomic multicast è sufficiente perchè l'unica garanzia che ci serve è che i backup ricevano tutti gli aggiornamenti.

Active replication

Un'altra strategia per garantire **high availability** ed è comparabile con la strategia load balancing.

Ogni replica del servizio esegue tutte le operazioni in modo identico, mantenendo così uno stato coerente tra tutte le repliche. Il modulo di frontend invia la richiesta a tutti i replica manager che appartengono al tier di backend oppure ad un sottogruppo.

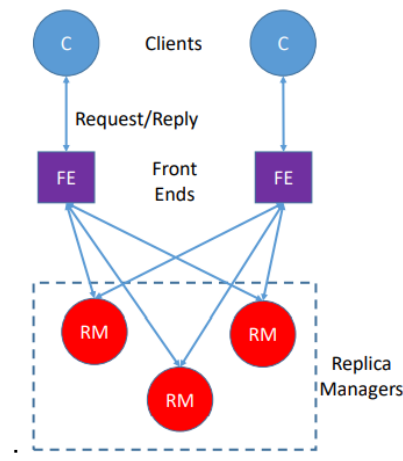


Figure 6413 – Diagramma che mostra la active replication.

Alla terminazione delle computazioni, il modulo di frontend sceglierà una delle risposte. Se un replica manager di un gruppo fallisce non importa perchè gli altri stanno eseguendo la stessa richiesta.

Questa soluzione ha un evidente problema di spreco di risorse: molti manager lavorano per nulla. Nonostante ciò, abbiamo un vantaggio, non è necessario uno scambio di messaggi di update.

1. **Request:** Il frontend riceve una richiesta e gli appende un ID unico, che identifica la richiesta e la invia a tutti i replica manager.

L'identificatore è necessario al manager primario per capire se quella richiesta è già stata ricevuta durante la fase di coordinazione.

1. **Coordinazione:** Non serve una coordinazione in questa fase visto che utilizziamo un total/order multicast.

Il sistema multicast (*bisogna usare il total order*) invierà la richiesta a tutti i replica manager in broadcast. In modo che tutti i replica managers ricevano gli update nello stesso ordine.

2. **Execution:** Ogni replica manager all'interno del gruppo esegue la stessa richiesta. Se non ci sono errori, tutti i replica manager calcoleranno lo stesso risultato.
3. **Agreement:** Non serve visto che utilizziamo un total/order multicast, tutti i replica manager eseguono la stessa richiesta.
4. **Response:** Ogni replica manager invia la propria risposta al frontend; il quale sceglierà quale inviare al client (*di solito invia la prima che riceve*).

In questo approccio serve un **total/casual order multicast** perchè non è necessario attendere che tutti i secondary RM ricevano la richiesta dal primary perchè tutti i RM le ricevono, inoltre è necessario garantire l'ordine delle richieste perchè cambiando l'ordine delle richieste, potrebbe cambiare anche il risultato.

Varianti della active replication

Passive: Nel gruppo di repliche del servizio c'è una replica primaria, la quale è l'unica ad eseguire effettivamente la richiesta inviata dal modulo di frontend, le altre repliche fanno da backup.

Active: i replica manager lavorano, però fanno lo stesso lavoro e quindi si spreca tempo e risorse.

Un approccio differente consiste nell'avere differenti replica manager che gestiscono richieste dal frontend in parallelo, massimizzando l'utilizzazione.

Inoltre, nella soluzione primary-backup un RM esegue la richiesta ed invia l'update anche prima di inviare la risposta al frontend, questo comporta un tempo di risposta elevato dal punto di vista del client, vogliamo evitare ciò.

Per risolvere questo problema, vogliamo un meccanismo in cui gli update tra i RM vengano inviati in **lazy fashion**, questo significa che i RM non trascorrono molto tempo ad inviare update, la maggior parte del tempo lo trascorrono ad eseguire richieste. Quindi le richieste vengono ricevute dai RM, processate, vengono inviate le risposte al frontend ed infine i RM si inviano gli update.

Architettura gossip

I replica manager si scambiano degli updates con tutti gli altri replica managers (*messaggi di gossip*) in modo periodico in modo da diffondere gli update ricevuti dal client.

Gli update vengono raggruppati assieme in messaggi singoli (riducendo l'overhead) e non sono inviati subito dopo l'esecuzione, ma vengono inviati periodicamente.

Evita che i RM siano occupati ad inviare update invece che gestire richieste.

Ogni replica manager può processare diverse richieste in parallelo (*massima utilizzazione delle risorse*) lo stato interno è aggiornato ogniqualevolta se ne riceve.

Il messaggio di update è inviato in modo periodico o comunque in momenti opportuni (*più update assieme*).

Potrebbe accadere che un RM riceva una richiesta su un dato che ha un update pendente, in attesa di essere trasmesso, questo perchè un altro client ha effettuato una modifica su quella struttura dati qualche secondo fa. In questo caso si verifica un'inconsistenza, quindi occorre un meccanismo per evitare che ciò avvenga.

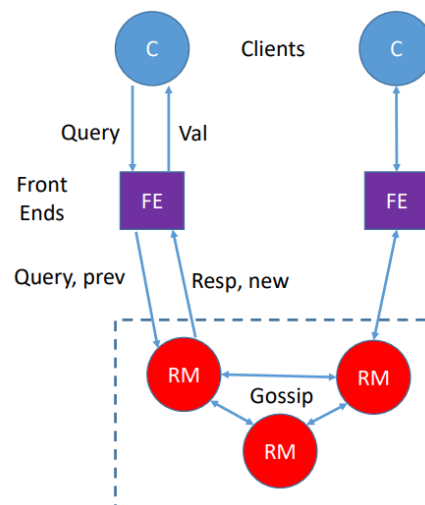


Figure 6514 - Architettura gossip.

I client sono assegnati ad una specifica istanza del modulo di frontend (*assumiamo che siano affidabili e che non falliscano mai*).

1. **Request:** Quando l'istanza del modulo di frontend riceve una richiesta da un client la inoltra ad un unico replica manager alla volta. Quindi abbiamo l'esecuzione di richieste multiple da diversi RM, però una richiesta è eseguita solo da un RM.
2. **Coordination:** Il replica manager che riceve la richiesta non la processa finché non può soddisfare i requisiti di ordinamento (sequential consistency).

Per capire quando può eseguire la suddetta richiesta possono essere necessari svariati scambi di messaggi di gossip che eseguono controlli sul timestamp (*vedere dopo*).

In modo da garantire l'ordinamento delle richieste vengono usati dei timestamp.

3. **Execution:** Il replica manager esegue la richiesta in modo regolare.
4. **Agreement:** Il replica manager che ha eseguito la richiesta, per inviare l'aggiornamento, aggiorna il messaggio di gossip.

Periodicamente, ogni replica manager invia un messaggio di gossip (in lazy fashion). L'architettura gossip punta a fare in modo che i replica managers si aggiornino a vicenda con le richieste dei client che ricevono.

5. Response:

- a. Se la richiesta era una query il replica manager risponde subito dopo l'esecuzione.
- b. Se la richiesta è un update, il replica manager risponde subito dopo averla ricevuta.

Consistent service over time

L'architettura gossip garantisce che ogni volta che un client richiede una query il replica manager che risponderà invierà sempre il dato che riflette l'ultimo aggiornamento fatto dal client stesso.

Sequential consistency - Consistenza tra repliche

La consistenza tra repliche è "*rilassata*": ossia dipendentemente dalla frequenza di invio dei messaggi di gossip, la probabilità che un client riceva un risultato vecchio (*ma consistente*) non è nulla. Però siamo certi che prima o poi tutte le repliche saranno consistenti.

Un client riceve un dato non aggiornato se sia il modulo di frontend che il replica manager contattato hanno un dato non aggiornato.

Due client differenti possono ricevere lo stesso dato, lo stato di quel dato può essere inconsistente tra i due client, però prima o poi i client otterranno il valore aggiornato, inoltre i dati che i client ricevono sono sempre in stato consistente e mai inconsistente.

Le richieste di un singolo client vengono ricevute da diversi RM e non sempre dal solito; quindi, il RM potrebbe inviare una risposta non aggiornata al client, bisogna evitare che ciò avvenga utilizzando dei timestamp.

Utilizzo dei timestamps - Processazione di una query

Si usano dei timestamp, quindi le istanze del modulo di frontend necessitano di avere un timer sincronizzato.

Nello specifico: Ogni modulo di frontend possiede un vettore di timestamp che riflette le ultime versioni dei dati acceduti.

1. Questo timestamp è inviato al replica manager nel campo "*prev*" insieme alla query.
2. Il replica manager al momento di inviare la risposta genera un nuovo timestamp e lo allega alla risposta nel campo "*new*".

3. Nel modulo di frontend il timestamp nel campo “*new*” viene unito al timestamp del campo “*prev*”.

In breve, “*prev*” serve per notificare il replica manager (*che riceverà la richiesta*) dell'ultimo momento in cui il dato è stato letto o aggiornato da un altro replica manager.

Il timestamp associato alla richiesta di query riflette l'ultima versione del valore che il frontend ha letto o inviato come update.

Questo obbliga il replica manager, ogni volta che riceve una query su quell'oggetto, di ritornare un valore uguale o più aggiornato rispetto al campo “*prev*” fornito con la query.

Per questo motivo, se il timestamp dell'ultimo update della replica locale è più vecchio di “*prev*” ($\text{valueTS} < \text{prev}$) allora la richiesta è posticipata (*coordination*) e mantenuta in un elenco di operazioni in sospeso in modo da permettere al replica manager di ricevere gli update mancanti.

Dopo che la richiesta è stata eseguita, il replica manager inserisce nel campo “*new*” il timestamp dell'ultimo update della replica locale.

Infine, invia “*new*” al modulo di frontend che andrà a sostituire “*prev*”.

Processazione di un update (*scrittura*)

Quando un replica manager riceve una richiesta di update da un modulo di frontend:

1. **Duplicato?** Analizza l'ID della richiesta in modo da scoprire se è già stata processata o meno.
2. **Creazione timestamp** Se la richiesta non è duplicata, replica manager crea un timestamp unico incrementando quello ricevuto nel campo “*prev*”.
3. **Invio Risposta** Il replica manager invia la risposta al modulo di frontend con il nuovo campo “*new*”.
4. Aggiunge anche una nuova voce al record del log locale delle operazioni di aggiornamento in sospeso.
5. Se la condizione sul timestamp della richiesta è soddisfatta (*ossia l'aggiornamento della replica è più nuovo di “prev”, $\text{prev} \leq \text{valueTS}$*) allora la richiesta viene portata a compimento, altrimenti rimandata finché non viene soddisfatta (ogni volta che viene ricevuto un successivo aggiornamento gossip, la condizione viene verificata).

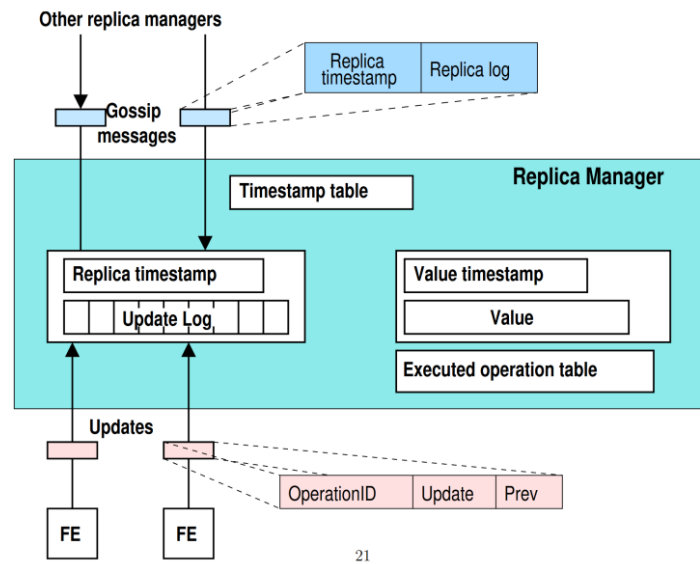


Figure 6615 – Struttura di un replica manager.

Capitolo 10 - Bilanciamento globale del carico

Applicazioni che lavorano su diversi datacenters. Le applicazioni su scala globale sono disponibili in tutto il mondo e devono poter essere usate da milioni di utenti allo stesso tempo.

Per fare ciò occorre dividere le funzionalità tra diversi datacenter in modo da servire in client con il datacenter più vicino.

Di solito il bilanciamento è fatto su due livelli (*due decisioni*):

- **Globale (*Global Load Balancing*)**: Distribuisco il carico su vari datacenters.
- **Locale (*Local Load Balancing*)**: Distribuisco il carico su varie macchine virtuali in un datacenter; questo tipo di bilanciamento punta ad una maggiore scalabilità e ad una maggiore fairness.

A livello locale, la politica di load balancing utilizzata punta a bilanciare il carico per la scalabilità.

A livello globale abbiamo due politiche implementabili:

- Richiesta viene inoltrata al datacenter disponibile più vicino per minimizzare la latenza della risposta (servizio di ricerca).
- Richiesta viene inoltrata al datacenter meno utilizzato per massimizzare il throughput (servizio di streaming).

Bilanciamento globale con DNS

L'idea è molto semplice, sfruttare il servizio DNS per effettuare un bilanciamento di tipo globale.

Il client prima di poter inviare una richiesta deve conoscere l'indirizzo IP del servizio a cui vuole inviare la richiesta.

Si restituisce al client una lista di indirizzi IP (*o anche solo uno*) relativi allo stesso servizio ma localizzati in posti diversi, il client invierà la richiesta all'indirizzo IP scelto.

Il problema di questo approccio è che il client non sa quale è l'opzione più vantaggiosa, poiché non ha modo di conoscere le metriche (*distanza, latenza, carico, etc*), quindi sceglie l'indirizzo randomicamente.

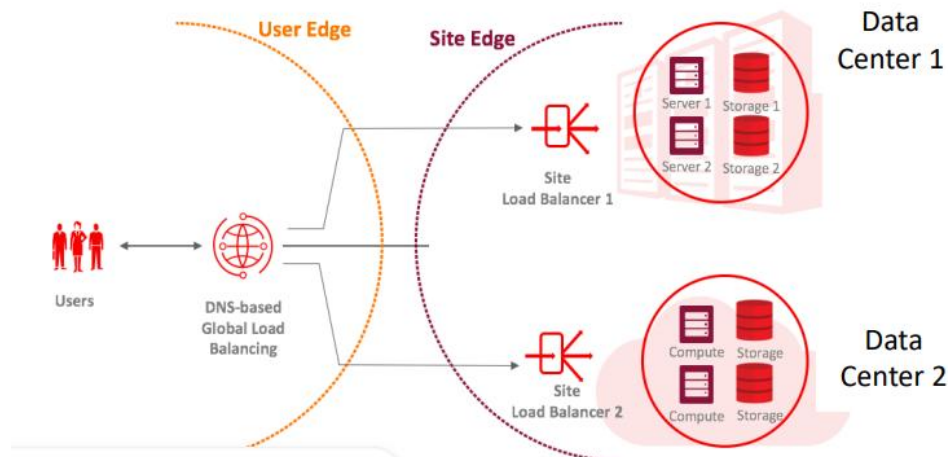


Figure 116 - Bilanciamento con DNS, da notare che poi all'interno del datacenter c'è il load balancer globale.

L'opzione migliore sarebbe quella di avere una politica di bilanciamento dinamica e che sia conscia del carico attuale di ogni datacenter

DNS localizzato

Un client in una data posizione invierà la richiesta DNS sempre allo stesso server, in questo modo il risultato del DNS dipende da dove il client ha inviato la richiesta. Questo perché il client invia la richiesta al suo server DNS locale (*di solito nella stessa nazione o continente*).

1. Quando un client fa una richiesta DNS per un nome di dominio la invia al suo server DNS locale.
2. Il DNS locale (*se non ha la risposta al suo interno già memorizzata*) la inoltra al server DNS globale.
3. Il DNS globale (*global load balancer*) risponde con uno degli indirizzi IP associati a quel dominio.

Questo indirizzo IP punta a uno dei server disponibili. La scelta dell'indirizzo IP può essere basata su vari algoritmi, come round-robin, weighted round-robin, least connections e anche da quale DNS locale è stata inviata la richiesta.

Il problema è che il record DNS non può essere aggiornato in tempo reale.

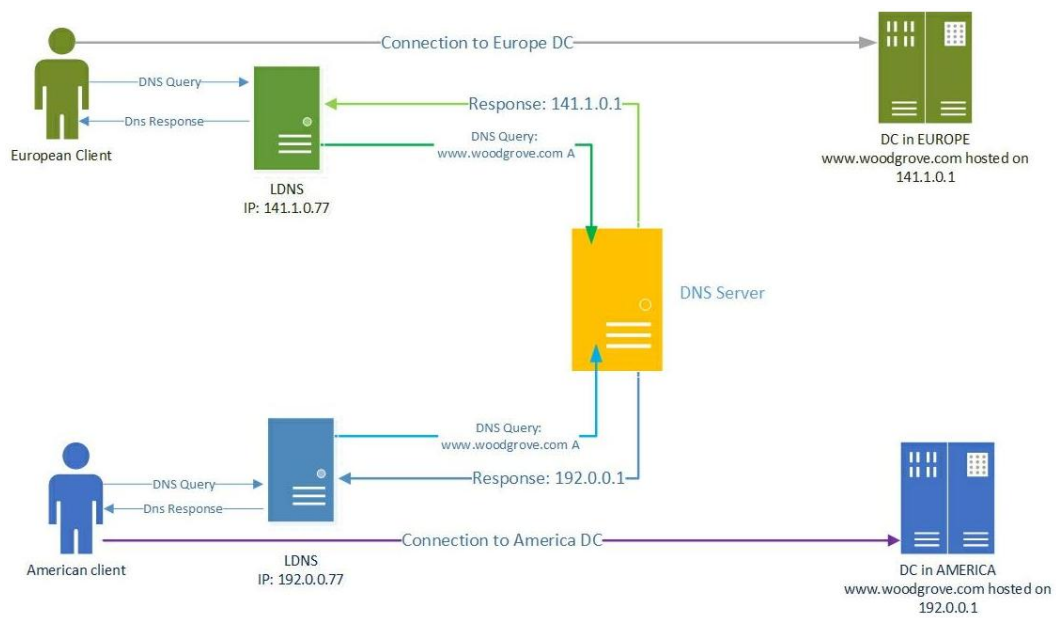


Figure 677 - Funzionamento del DNS localizzato.

VIP - Indirizzi IP virtuali (*Any Cast address*)

Metodo per migliorare le performance del DNS localizzato. Gli indirizzi IP virtuali sono indirizzi IP condivisi da un gruppo di server invece che a un singolo server o ad una certa interfaccia di rete.

Dal punto di vista dell'utente appaiono come indirizzi IP normali.

I VIP fungono da strato di astrazione tra i client e i server fisici. I client si connettono utilizzando il VIP, ma il traffico viene deviato verso uno dei server fisici nel pool secondo una certa politica applicata dal load balancer (*trasparente al client*).

Questo permette di aggiungere o rimuovere server dal pool senza influenzare gli indirizzi IP a cui i client si connettono.

Dal punto di vista dell'utente, l'indirizzo IP virtuale appare come “*normale*” e quindi nasconde tutti i dettagli implementativi alle spalle.

Funzionamento dei VIP

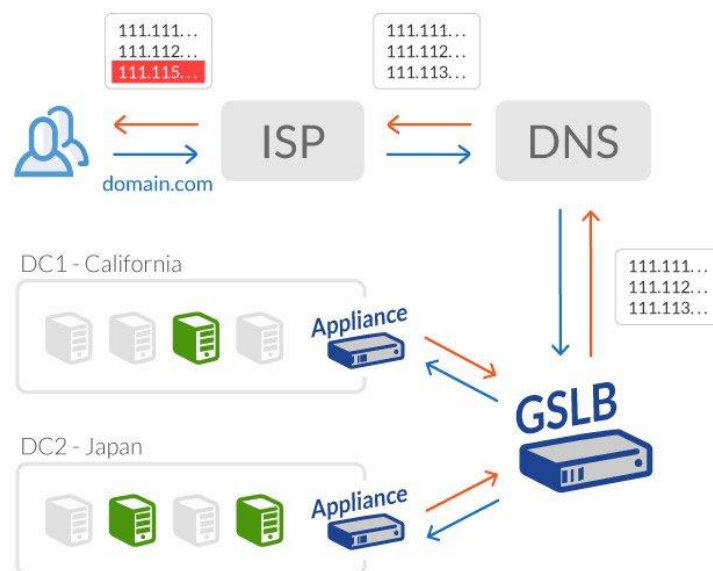


Figure 688 - Load balancing usando i VIP.

Ad una applicazione o servizio verrà assegnato un set di VIP.

Dividiamo il globo in varie regioni (*ad esempio una regione può essere un continente*).

Ad ogni regione verrà assegnato un sottogruppo di questi VIP (*una regione molto grande può essere divisa in sotto regioni e così via*).

In ogni regione possono esserci più datacenter.

Il DNS di quella regione è configurato con i VIP assegnati a quella determinata regione.

Per ogni regione, si ha uno o più load balancer globali (*di solito più di uno per garantire availability in caso di guasto*).

Ogni load balancer globale ha un solo VIP associato al servizio per quella determinata regione gestita dal load balancer globale.

1. **Richiesta DNS:** Un client invia una richiesta DNS per un dominio, come `www.example.com`.
2. **Risposta DNS:** Il DNS server, configurato per il load balancing globale, risponde con un indirizzo IP virtuale relativo al global load balancer per quel servizio.
3. **Connessione Client:** Il client invia la richiesta all'indirizzo IP virtuale.
4. **Instradamento del Traffico:** Il load balancer globale di quella regione riceve la richiesta sull'indirizzo IP virtuale e lo instrada ad uno dei load balancer locali.
5. **Scelta del Server Fisico:** il load balancer locale riceve la richiesta e la inoltra ad uno dei server fisici nel pool.

Il load balancer locale monitora lo stato e il carico dei server fisici, adattando dinamicamente la distribuzione del traffico in base alle regole configurate (ad es. round-robin, least connections).

GRE - Incapsulamento dei pacchetti IP

Il load balancer globale ha il compito di ricevere i messaggi e di inoltrarli al datacenter più conveniente in quel momento (*politica dinamica*).

Ma in tutto questo, l'indirizzo IP di destinazione e della sorgente del pacchetto dovrebbe cambiare perché il global load balancer deve inoltrare il pacchetto al local load balancer, se non cambiasse indirizzo di destinazione il pacchetto verrebbe scartato, oppure verrebbe inoltrato a se stesso o ad un altro global load balancer di tale regione (visto che i global load balancer in una regione hanno lo stesso VIP).

Per questo viene fatta una incapsulazione: trasmetto il pacchetto IP iniziale (*incapsulato*) come payload di un altro pacchetto IP (*incapsulatore*) con diverso indirizzo IP di destinazione (*indirizzo IP del load balancer del datacenter di destinazione*) e sorgente (*indirizzo IP del global load balancer*).

GRE è utilizzato per stabilire un tunnel tra il global load balancer e il datacenter.

Questa cosa viene fatta ad ogni inoltro: all'arrivo, il pacchetto viene privato dei campi di GRE e quindi viene ottenuto il pacchetto IP originale.

Queste operazioni sono effettuate dal local load balancer del datacenter.

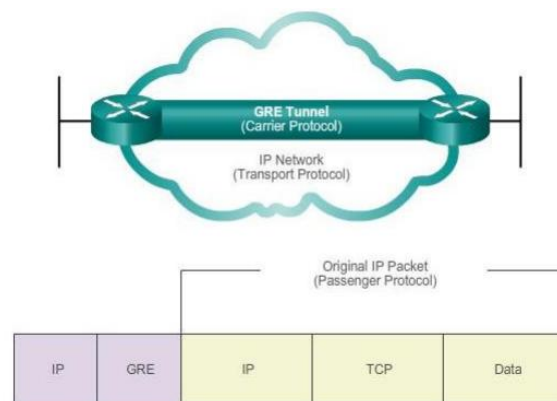


Figure 699 - Protocollo GRE

Capitolo 11 - Sistemi geogr. distribuiti

Un sistema geograficamente distribuito necessita di un meccanismo per replicare e sincronizzare i dati su larga scala.

Dobbiamo quindi considerare fenomeni più complicati rispetto ai fenomeni che accadono all'interno di un datacenter.

Le sfide sulle reti su larga scala

Su larga scala, ci sono problemi che prima potevano essere trascurati e nuovi colli di bottiglia. Quindi i problemi che dobbiamo risolvere sono:

Poco controllo sulle azioni dei client

I client sono sparsi per tutto il mondo e quindi per la teoria dei grandi numeri aumenta la possibilità che ci siano client malevoli.

Larghezza di banda

La banda dentro un datacenter è estremamente superiore a quella della rete pubblica; Quindi, un servizio su larga scala è soggetto ad una grande latenza e quindi oltre al degrado del servizio, aumenta anche il tempo richiesto per la sincronizzazione.

Partizionamento della rete

Problema minore ma presente, in una LAN si assume che sia molto affidabile e sempre disponibile, ma spesso i link subiscono momenti di disturbo che creano partizione della rete.

Questa cosa può accadere (*anzi accade anche più spesso*) anche in reti globali in cui c'è minore affidabilità della comunicazione: Ad esempio un datacenter in Australia potrebbe non

poter comunicare con il resto di internet...il servizio deve comunque poter andare avanti; Questa cosa si risolve con una strategia di replicazione che tollera la partizione della rete.

Il partizionamento può accadere anche in caso di guasti, dispositivi configurati male, etc...

Le strategie per replicare i dati nelle reti tradizionali non sono adatte per sistemi geograficamente distribuiti perchè assumono bassa latenza e affidabilità della rete.

Teorema CAP

Un sistema non può essere contemporaneamente tutte e tre (*al più due complete e una "rilassata"*):

- **Consistency**: Sempre consistente: ogni replica è sempre consistente in ogni momento; quindi, tutti i nodi hanno la stessa visione dei dati in ogni momento.
- **Availability**: Sempre disponibile: In ogni momento il servizio può accettare nuove richieste.
- **Partiton**: Resistente al partizionamento della rete: In caso di partizionamento della rete il sistema continua a funzionare normalmente.

Per questo motivo dobbiamo progettare una soluzione che trovi un buon compromesso (*dobbiamo rilassare la terza opzione*).

CP (Consistency – Partition)

La disponibilità non è sempre garantita. A questa categoria appartengono:

- ZooKeeper.
- MongoDB.

AP(Availability – Partition)

Può avere inconsistenze ma sarà sempre affidabile e resistente alle partizioni. A questa categoria appartengono:

- Servizio DNS.
- Web Cache.
- Tutti i sistemi **BASE**.

CA(Consistency – Availability)

Non è garantito che resista alle partizioni ma sarà sempre disponibile e consistente. A questa categoria appartengono:

- RDBMS.
- Tutti i sistemi **ACID**.

Sistemi BASE

Sistemi con una consistenza di tipo “*best effort*”. Ci può essere un periodo in cui la consistenza non è rispettata ma sono sicuro che prima o poi verrà rispettata.

Sistemi ACID

Sistemi che sono sempre consistenti in ogni momento. Secondariamente, il sistema deve essere sempre attivo ma se il sistema non è consistente non può neanche essere disponibile.

In questi sistemi si presuppone che le partizioni di rete e i guasti siano molto rari; per questo motivo su larga scala non vanno bene, perché queste assunzioni non sono più valide.

Sistema BASE

Sistemi che devono gestire una quantità di dati molto grande e impossibile da gestire per un sistema ACID. Puntano ad essere **sempre disponibili (A)** e **resistenti alle partizioni di rete (P)** ma implementano una **consistenza eventuale (C)** (*prima o poi sarà consistente*).

Ci saranno momenti di inconsistenza ma che saranno risolti dopo un tempo di transizione (*la consistenza non è garantita subito dopo un update*).

Nota che questo approccio non va bene per tutte le applicazioni: ad esempio un sistema bancario necessita di un sistema diverso che garantisce la massima consistenza.

Query/update in un sistema BASE

In un sistema BASE gli update sono replicati in modo asincrono in background (*contemporaneamente all'attività dell'utente*). L'update viene eseguito nella replica locale che ha ricevuto la richiesta e la risposta viene subito inviata al client, la replicazione su tutte le altre repliche avverrà in modo asincrono dopo un po' di tempo.

Vantaggi & svantaggi di un sistema BASE

- Può gestire un partizionamento della rete anche su larga scala: Perché nel momento in cui la rete ritorna ad essere connessa le repliche verranno aggiornate (*eventual consistency*).
- Il delay di risposta è ridotto poiché il client non deve aspettare che tutte le repliche siano aggiornate.
- Ci possono essere inconsistenze temporanee, a causa del tempo richiesto alla propagazione degli aggiornamenti (*come nell'architettura gossip*). Non è un sistema utilizzabile in applicazioni che non richiedono conflitti.
- Un sistema BASE gestisce molto meglio le network partition: se una certa regione diventa isolata, le altre possono comunque lavorare.

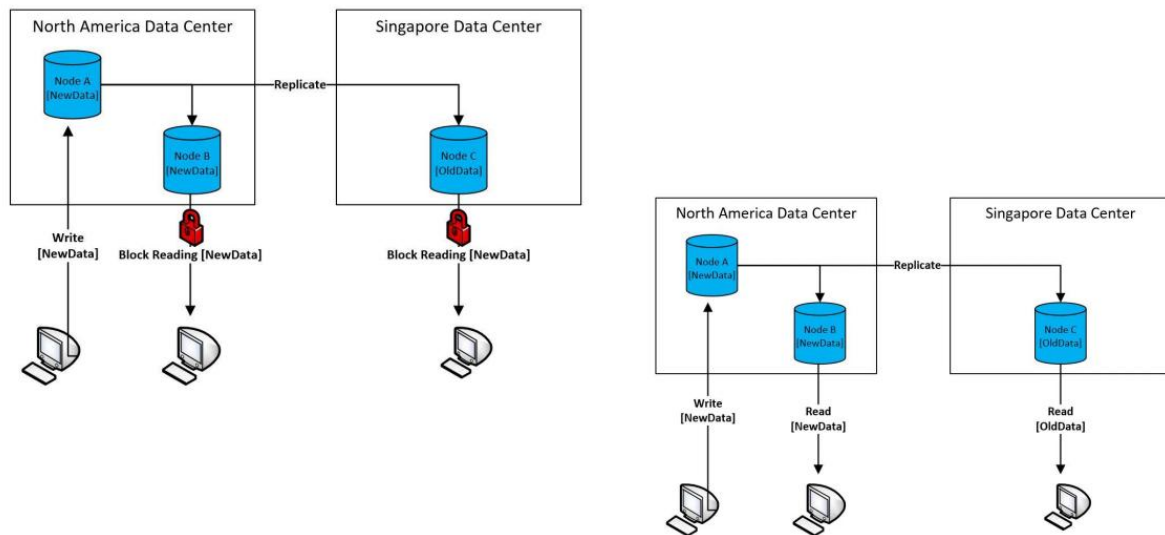


Figure 7020 - Differenza tra ACID (sinistra) e BASE (destra).

Architettura “Bayou”

Architettura eventual consistent, progettata per gestire la replicazione e la sincronizzazione dei dati in ambienti mobili e distribuiti, dove le connessioni di rete possono essere intermittenti e inaffidabili.

È stata sviluppata per affrontare le sfide specifiche legate alla coerenza dei dati in scenari in cui dispositivi diversi possono aggiornare indipendentemente copie locali dei dati.

Questo sistema teorico garantisce un’alta disponibilità, una buona tolleranza alle partizioni ma con una scarsa garanzia sulla consistenza sequenziale.

L’idea di base è: Si hanno molteplici replica manager (*ogni manager è un datacenter*) e ogni manager scambia gli update applicando un protocollo anti-entropia che fornisce gli update in gruppo.

L’architettura Bayou include anche una risoluzione dei conflitti di dominio.

Assicura che tramite il continuo scambio dei messaggi, i dati su ogni replica manager è lo stesso.

Replicazione dei dati

Ogni nodo nel sistema Bayou mantiene una copia locale dei dati. Questo consente ai nodi di funzionare autonomamente anche quando non sono connessi alla rete. Le modifiche ai dati possono essere effettuate localmente e vengono salvate in un registro di log locale.

Tentative

Quando si riceve un update da un utente l’aggiornamento è registrato in locale nel log ma è segnato come “*non definitivo*”. L’aggiornamento viene propagato ai nodi vicini. I nodi che ricevono l’aggiornamento lo registrano anch’essi nel loro log in uno stato non confermato.

Se un update è “*non definitivo*”, il sistema potrebbe non confermarlo oppure riapplicarlo se è necessario per raggiungere uno stato consistente. Se un update è “*committed*”, allora rimane salvato.

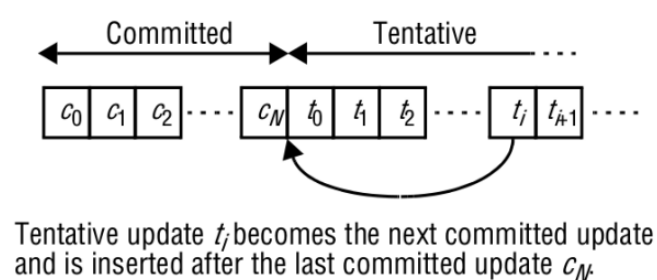


Figure 7121 - Una Bayou operation.

Il sistema in futuro potrebbe confermare (*commit*) o ritirare l'update non definitivo.

Bayou utilizza un meccanismo di risoluzione dei conflitti basato su regole definite dagli utenti. Quando un nodo riceve un aggiornamento, applica queste regole per determinare se e come l'aggiornamento dovrebbe essere integrato con i dati esistenti.

In ogni momento lo stato del sistema è composto dagli aggiornamenti confermati (*ordinati per il tempo di conferma*) e seguiti da quelli non definitivi (*ordinati in base all'istante di ricezione dalla replica locale*).

Gli aggiornamenti confermati sono ordinati in ordine di conferma. I tentativi sono ordinati in ordine di ricezione.

Dopo che tutti i nodi hanno ricevuto il tentativo. Uno dei nodi viene designato come nodo primario (*primary replica*). Questo nodo è responsabile della serializzazione degli aggiornamenti e della loro conferma finale. La scelta del nodo primario può essere dinamica e cambiare nel tempo.

Il nodo primario riceve gli aggiornamenti dai vari nodi e li ordina in una sequenza globale. Questo ordinamento garantisce una visione coerente degli aggiornamenti attraverso il sistema.

Lista degli aggiornamenti committed è scambiata tramite un protocollo di anti-entropia, il primary replica manager agisce come gli altri e in più decide quando effettuare il commit di un update.

Il protocollo di commit può essere qualsiasi politica che stabilisce l'ordine in cui gli aggiornamenti sono applicati e può essere definito specificatamente per soddisfare i requisiti dell'applicazione.

Se il primario è temporaneamente indisponibile non previene che le richieste vengano gestite dagli altri replica manager:

- Update sono accettati come non confermati nell'ordine in cui sono ricevuti.
- Query ricevono risposte utilizzando lo stato corrente del sistema.

Quando il replica manager viene riconnesso, effettua il commit degli update.

Risoluzione dei conflitti

I conflitti possono avvenire quando un update è applicato.

Bayou adotta due metodi per risolvere conflitti:

Controllo sulle dipendenze: Ogni transazione include sia le operazioni da eseguire che le condizioni di preconferma e postconferma.

Le condizioni di preconferma (query e relativo risultato) verificano che i dati siano in uno stato specifico prima di applicare la transazione (il replica manager esegue la query prima di applicare l'aggiornamento), mentre le condizioni di postconferma garantiscono che lo stato finale dei dati sia corretto. Se una condizione di preconferma non è soddisfatta (avviene un conflitto), la transazione viene messa in attesa fino a quando le condizioni non sono soddisfatte.

Una volta che un aggiornamento viene inserito nella sequenza globale dal nodo primario, viene considerato confermato (*committed*).

Il nodo primario invia una notifica di conferma agli altri nodi nel sistema.

Ogni nodo aggiorna il proprio log locale per riflettere lo stato confermato dell'aggiornamento.

Gli aggiornamenti confermati vengono applicati ai dati permanenti (*stato coerente del database*). Questo passaggio assicura che tutti i nodi alla fine convergano verso uno stato consistente.

Procedura di merge: procedura eseguita quando un conflitto viene rilevato, queste procedure sono fornite dagli sviluppatori.

Questi due metodi sono deterministici, quindi ogni replica manager risolve lo stesso conflitto allo stesso modo degli altri. Quindi la storia dell'esecuzione e la risoluzione dei conflitti sono gli stessi in tutti i replica manager.

Performance di Bayou

Si può provare però che in caso di un buon algoritmo di anti-entropia e con la giusta frequenza di update, il sistema può comunque dare un certo grado di sicurezza e consistenza. Il tempo può essere utilizzato come metrica per misurare l'eventual consistency, include finestra di inconsistenza e il tempo richiesto per avere un'informazione visibile nel sistema.

Dato un certo rate di anti-entropia si può calcolare il tempo di consistenza necessario per avere una certa PBS.

Probabilistically Bounded Staleness (PBS)

Concetto che si riferisce alla garanzia di coerenza nei sistemi di archiviazione distribuiti, particolarmente nei contesti di coerenza eventuale. Questo modello fornisce un modo per quantificare la "*staleness*" (obsolescenza) dei dati in termini probabilistici. In altre parole, permette di esprimere e capire quanto spesso i dati recuperati dal sistema possono essere obsoleti e di quanto tempo.

Capitolo 12 - Piattaforme Cloud

Nei precedenti capitoli abbiamo dato un'occhiata al basso livello di una infrastruttura cloud. Ad oggi un “cloud” è un set di server (*infrastruttura fisica*) sui quali è installato un hypervisor per generare il livello virtuale.

Recap

Il cloud è un'infrastruttura che offre la possibilità di creare delle macchine virtuali.

Su ogni macchina virtuale è schierata (*dal cloud consumer*) una o più applicazioni cloud destinate agli utenti finali.

Una macchina virtuale è creata utilizzando tecnologie di virtualizzazione. Ma come facciamo a creare un'infrastruttura cloud? Vogliamo rimuovere l'intervento umano e quindi poter gestire automaticamente gli Hypervisor installati nei vari server.

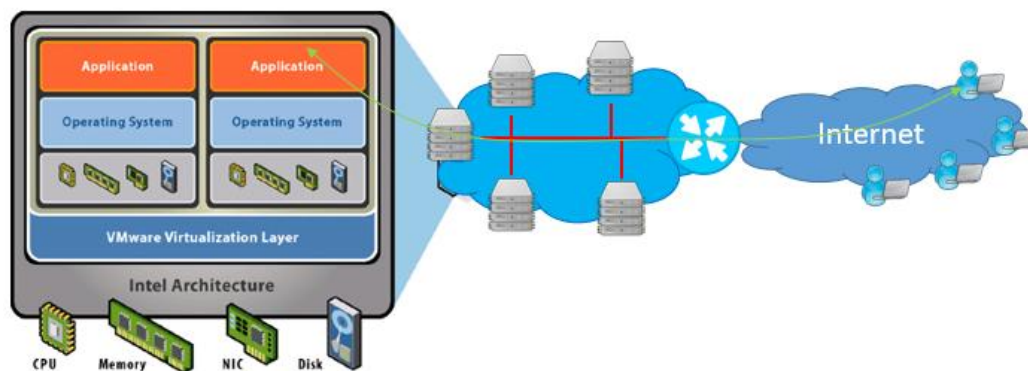


Figure 7222 - Infrastruttura cloud.

Piattaforma cloud

La risposta è una **piattaforma cloud**: Ossia un software installato su tutte le macchine fisiche dell'infrastruttura con lo scopo di automatizzare tutti i processi che riguardano le macchine virtuali (*creazione, distruzione e gestione delle macchine virtuali*).

Una piattaforma cloud è in realtà un sistema distribuito e ogni server ha una istanza della piattaforma cloud installata in esso; possiamo considerarla come un'applicazione cloud speciale con tutti i requisiti e caratteristiche dette nei capitoli precedenti.

La piattaforma cloud, nello specifico, controlla gli hypervisor locali installati nei server fisici su cui è installata tramite un'interfaccia fornita dall'hypervisor stesso; la piattaforma reagisce alle richieste e alle situazioni per gestire gli hypervisor.

Architettura di una piattaforma cloud

Una piattaforma cloud è composta da un gruppo di server connessi tramite una LAN veloce. Ogni server ha installato un sistema operativo sul quale è installato un hypervisor; oppure è fornito di un hypervisor “bare metal” (*in entrambi i casi il sistema è lo stesso*).

La piattaforma cloud espone una interfaccia che permette di vedere l'infrastruttura cloud come un'unica grande macchina che possiamo gestire a piacimento; questa interfaccia è utilizzabile dagli esseri umani o da software.

Ogni server ha installato un software per controllare le risorse locali virtualizzate dall'hypervisor, questo software è detto **cloud platform agent**.

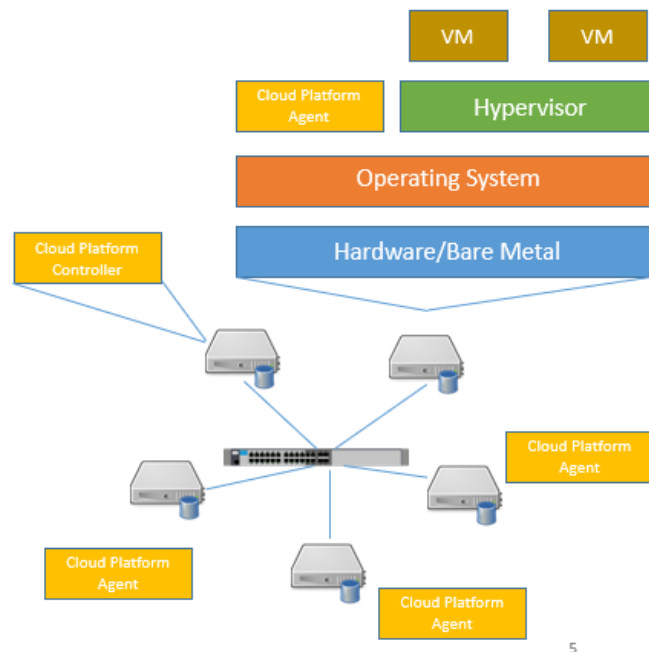


Figure 7323 - Architettura generale di una piattaforma cloud.

Compute nodes

In aggiunta all'hypervisor (*allo stesso livello*) è presente la piattaforma cloud, chiamata anche “Cloud Platform Agent”.

I server di questo tipo vengono detti “compute nodes”, ossia quelli che effettivamente lavorano; ossia che offrono le loro capacità di memorizzazione e di elaborazione per la creazione di macchine virtuali.

Controllers

Nella infrastruttura della piattaforma cloud ci sarà uno o più server speciali che hanno installata un tipo di istanza diversa della piattaforma cloud.

I server di questo tipo vengono detti “*controllers*” e gestiscono i compute nodes, offrono funzionalità necessarie per gestire l'intera infrastruttura.

OpenStack

Progetto open source che ha guadagnato popolarità negli ultimi anni. Ha l'obiettivo di fornire una piattaforma cloud gratuitamente e quindi evitando costose licenze per IaaS cloud. È la soluzione principale utilizzata per il private cloud.

Requisiti di Open Stack

OpenStack è studiato per girare su hardware comune (*commodity hardware*); l'unico requisito è che il server fisico abbia installato un sistema operativo basato su Linux. Diversamente dal sistema operativo, l'hypervisor (*nel caso non sia bare metal*) installato sui server può variare, basta che sia compatibile con Open Stack.

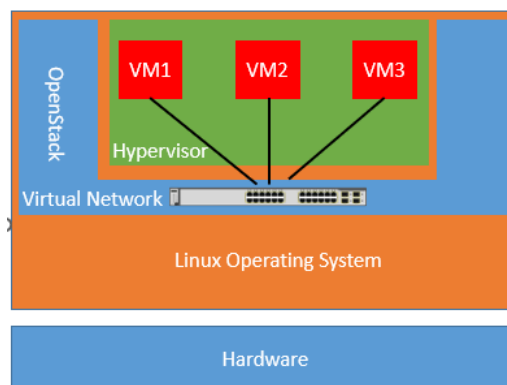


Figure 7424 - Architettura di OpenStack su un compute node.

Open Stack deve essere installato in ogni server, in questo caso il server è chiamato “*Open Stack Node*”.

Open Stack si occupa non solo della creazione/distruzione di VM, ma anche di ogni altro aspetto dalla creazione e configurazione di una rete virtuale su cui le VM comunicano alla gestione dei virtual hard drive per la memorizzazione ecc.

Open Stack Virtual Network

Open Stack crea una rete virtuale nella quale tutte le macchine virtuali comunicano; tutto ciò deve ovviamente avere una vera rete fisica al di sotto che connette fisicamente i nodi.

Open Stack assume che ogni nodo sia collegato alla stessa LAN e che ogni nodo sia nello stesso datacenter.

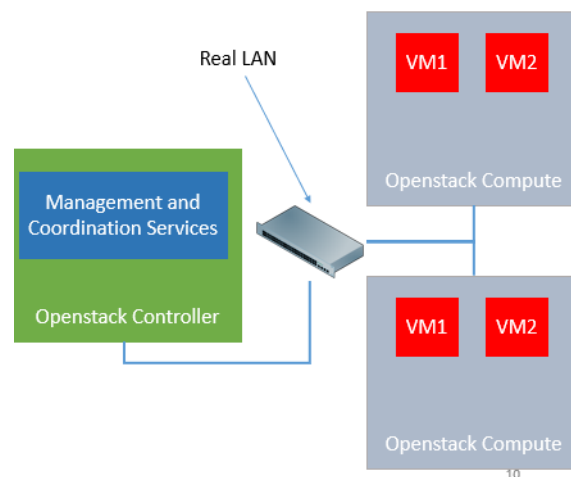


Figure 7525 - rete virtuale creata da OpenStack.

Open Stack espone di default un'interfaccia grafica per consentire agli utenti di interagire con la piattaforma, in alternativa fornisce anche un'interfaccia a linea di comando.

Moduli

Open Stack si basa su uno schema **modulare**: tutte le funzionalità sono raggruppate in moduli (*i quali hanno diversa importanza e possono anche essere installati separatamente, poiché tra loro quasi totalmente indipendenti*).

Ogni modulo lavora in modo (*quasi*) indipendente dagli altri e ha la sua personale interfaccia REST per comunicare con altri moduli; questa indipendenza tra i moduli permette di riutilizzare o sostituire i moduli.

Interfaccia REST

I moduli non sono totalmente indipendenti, infatti ogni modulo può anche richiamare le funzionalità di altri moduli utilizzando la loro interfaccia REST. L'interfaccia nasconde l'implementazione interna dei moduli, presentando invece un'interfaccia standard.

La comunicazione tra moduli può avvenire anche tramite scambio di messaggi oltre che tramite l'interfaccia REST.

Installazione di OpenStack

I servizi scelti durante l'installazione sono installati in parte nel controller e in parte nei compute node e alcuni in entrambi le tipologie di nodi. I servizi in Open Stack sono divisi in due gruppi principali: "servizi essenziali" e "servizi opzionali".

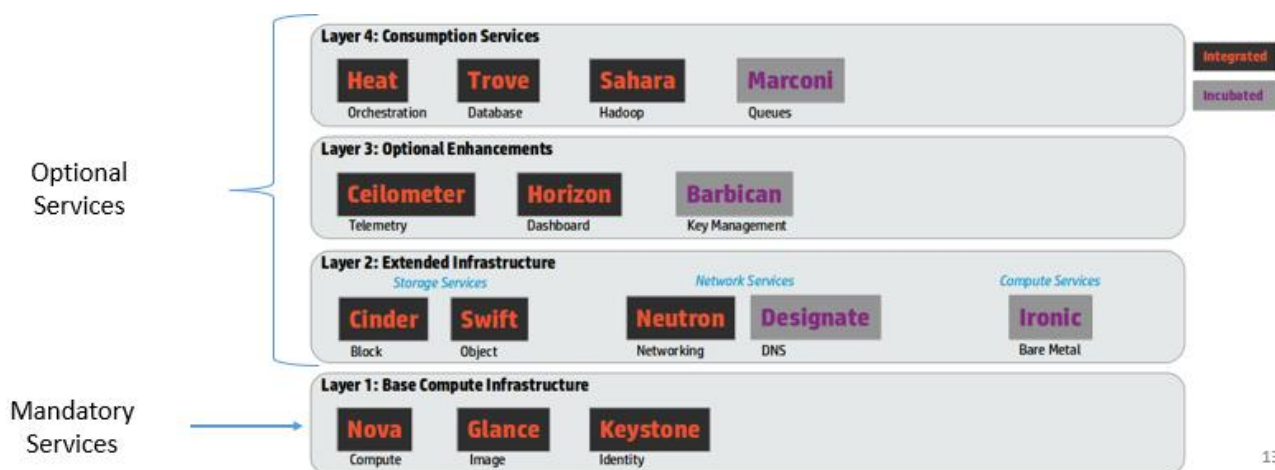


Figure 766 - I moduli di OpenStack suddivisi in base all'importanza.

Servizi essenziali

Sono moduli strettamente necessari per costruire la piattaforma cloud e infatti sono sempre inclusi nell'installazione poiché indispensabili per mandare avanti le funzionalità di base.

Compute module (Nova)

Responsabile di creare e distruggere le macchine virtuali.

Image module (Glance)

Responsabile di memorizzare le immagini usate per creare le macchine virtuali.

Identity module (Keystone)

Responsabile di regolare l'accesso degli utenti all'infrastruttura di Open Stack.

Servizi Opzionali

A loro volta divisibili in altri tre sottogruppi in base all'importanza.

Gruppo 2

Servizi non essenziali ma solitamente installati poiché molto importanti:

- **Cinder**: fornisce **Block Storage**.
- **Neutron**: modulo per il networking, permette alle macchine virtuali di comunicare tramite una infrastruttura di rete virtuale.

Gruppo 3

Servizi non essenziali e non sempre installati poiché meno importanti:

- **Dashboard:** Permette una interazione più amichevole con l'utente (*invece che la linea di comando*).
- **Telemetry:** Per misurare le performance della piattaforma.

Gruppo 4

Servizi non essenziali e avanzati, spesso installati.

- **Orchestration:** Permette la creazione on-demand dei servizi.
- **Hadoop.**

Supporting Services

In ogni istanza controller di Open Stack ci sono due servizi essenziali che forniscono le loro funzionalità, queste funzionalità sono spesso usati da altri moduli:

Database (Controller)

Istanza di MySQL che si occupa di memorizzare lo stato, la configurazione e altre informazioni necessarie alle macchine virtuali.

Message Broker (Controller)

È RabbitMQ, si occupa di gestire lo scambio di messaggi tra i moduli.

Suddivisione dei moduli

In figura sotto è possibile vedere una possibile ripartizione dei servizi tra le istanze di compute node e il controller.

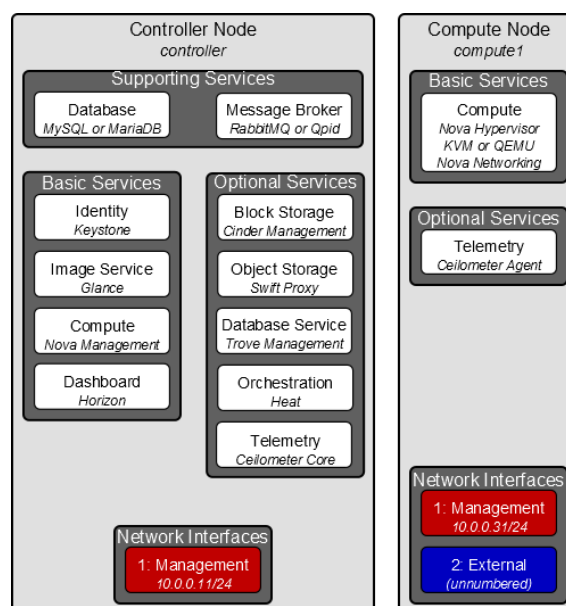


Figure 777 - Suddivisione dei moduli tra i compute node e il controller.

Comunicazione tra moduli

Come possono interagire i moduli tra loro? Invece come possono interagire con terze parti (*dispositivi esterni alla rete*)?

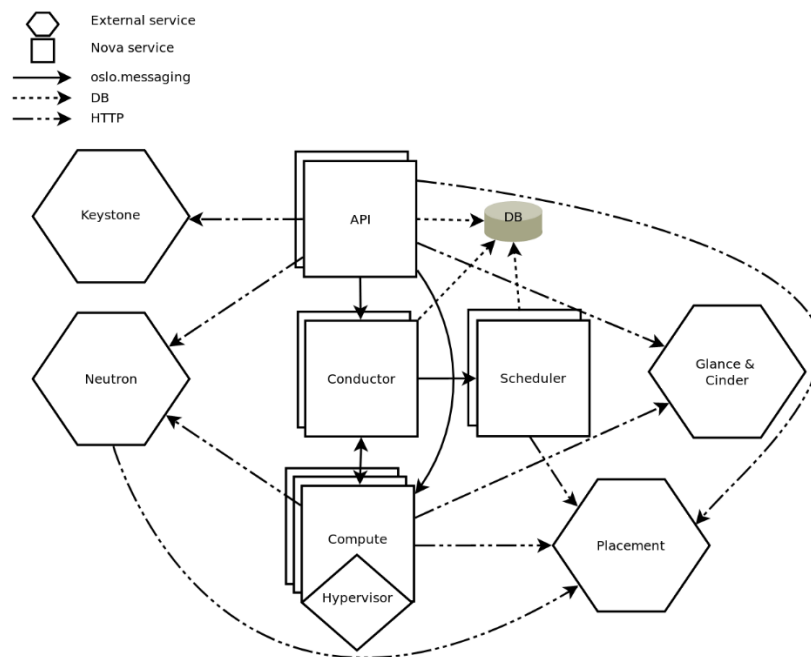


Figure 788 - Comunicazione tra moduli.

Possono farlo in tre modi: La prima modalità è una capacità intrinseca di ogni modulo mentre le ultime due sono utilizzabili solo tramite un modulo installato (*di solito*) nel controller ma a volte sono anche installati su altri nodi per fare bilanciamento.

Interfaccia REST

Ogni modulo espone una sua interfaccia REST; tramite essa ogni modulo espone le proprie funzionalità a chiunque sia autorizzato ad utilizzarli.

Message Broker

Si utilizza un classico sistema basato su un **message broker** installato nel controller.

Database

Si utilizza un'istanza di database dove si inserisce lo stato di ogni macchina virtuale, nodo, oggetto, etc.

Le informazioni sono scambiate tramite questo database accessibile da tutti i moduli autorizzati tramite una connessione TCP/IP.

Keystone

Componente obbligatorio da installare in OpenStack. Solitamente installato nel controller si occupa dell'autenticazione degli utenti. Adotta una strategia basata sulla coppia username & password e su un token di accesso generato partendo dallo username e dalla password.

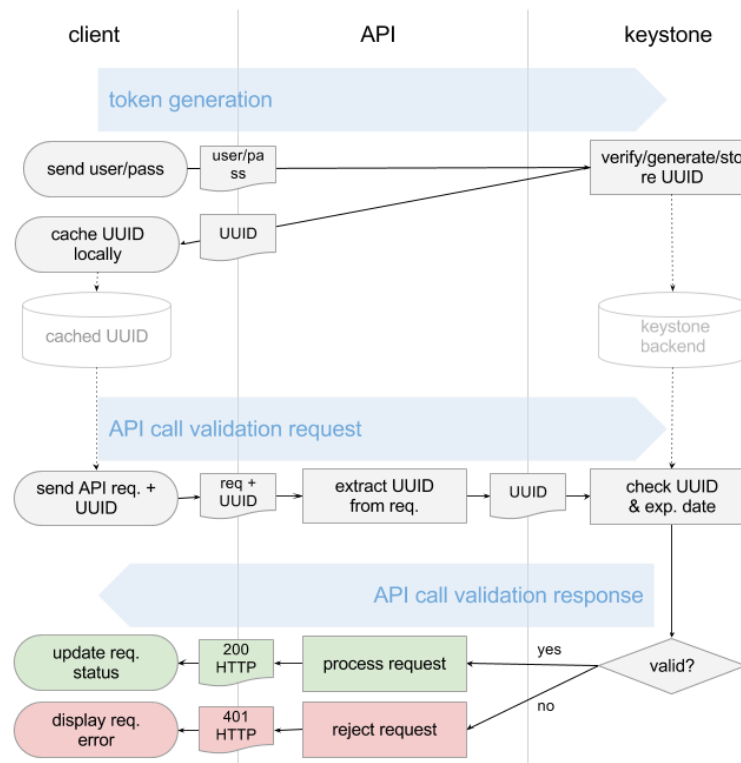


Figure 799 - workflow di un job di Keystone.

UUID

Lo UUID è un token di accesso che può essere usato per accedere alle funzionalità dei moduli; ogni modulo verificherà (*inviandolo al modulo keystone*) che il token sia corretto o meno. Se il token è valido, il modulo controlla se il token ha il livello di privilegio sufficiente o in generale i diritti per accedere alla funzionalità che richiede.

In Open Stack gli oggetti (VM, virtual hard drive e virtual network) sono raggruppati in progetti.

Nova

Modulo responsabile di instanziare e gestire le macchine virtuali; non è un hypervisor, semplicemente comunica con l'hypervisor installato nella macchina locale per fargli gestire le macchine virtuali che verranno tenute in quella specifica macchina. Nova si adatta perfettamente all'hypervisor installato nella macchina, basta che sia compatibile con OpenStack.

Nova ha due componenti: uno installato nel controller e uno in un compute node.

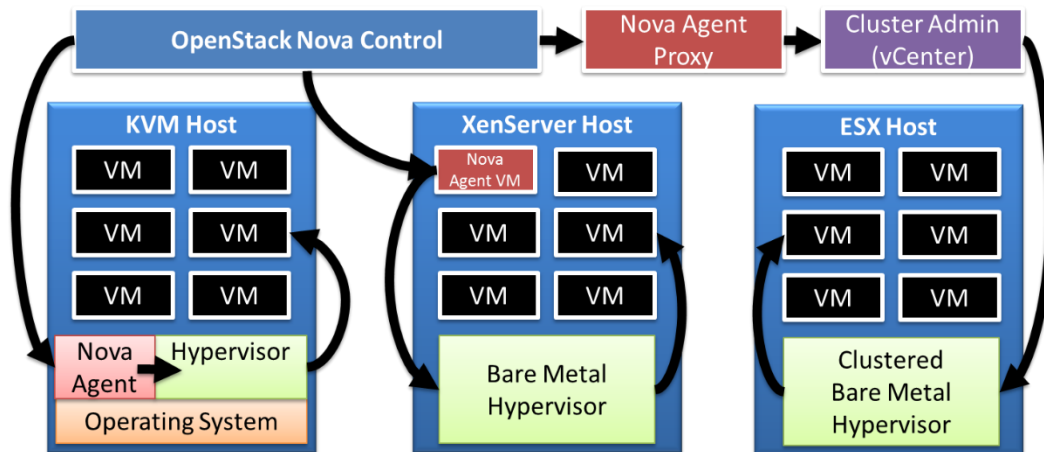


Figure 8030 - Workflow di Nova in 3 casi di hypervisor diversi.

Istanza di Nova nel Controller

Questa istanza di Nova è responsabile di gestire le risorse dei compute node: riceve le richieste di creazione/distruzione di macchine virtuali grazie alla propria interfaccia REST. La parte controller di Nova è divisibile in altri sotto moduli:

- **API:** Espone e gestisce l'interfaccia REST con cui richiedere i servizi di Nova.
- **Conductor:** Effettivamente quello che processa le richieste ricevute.
- **Scheduler:** Implementa una **politica di scheduling** per la gestione delle risorse; ossia decide dove (*in quale server fisico*) una macchina virtuale deve essere creata (istanziata).
- **Network:** Implementa i servizi base di networking, anche se il networking è implementato in altri componenti.
- **Consoleauth e Novncproxy:** Espone una console per l'accesso alle macchine virtuali.

Tutti questi componenti interagiscono tra loro grazie ad un sistema a coda di messaggi.

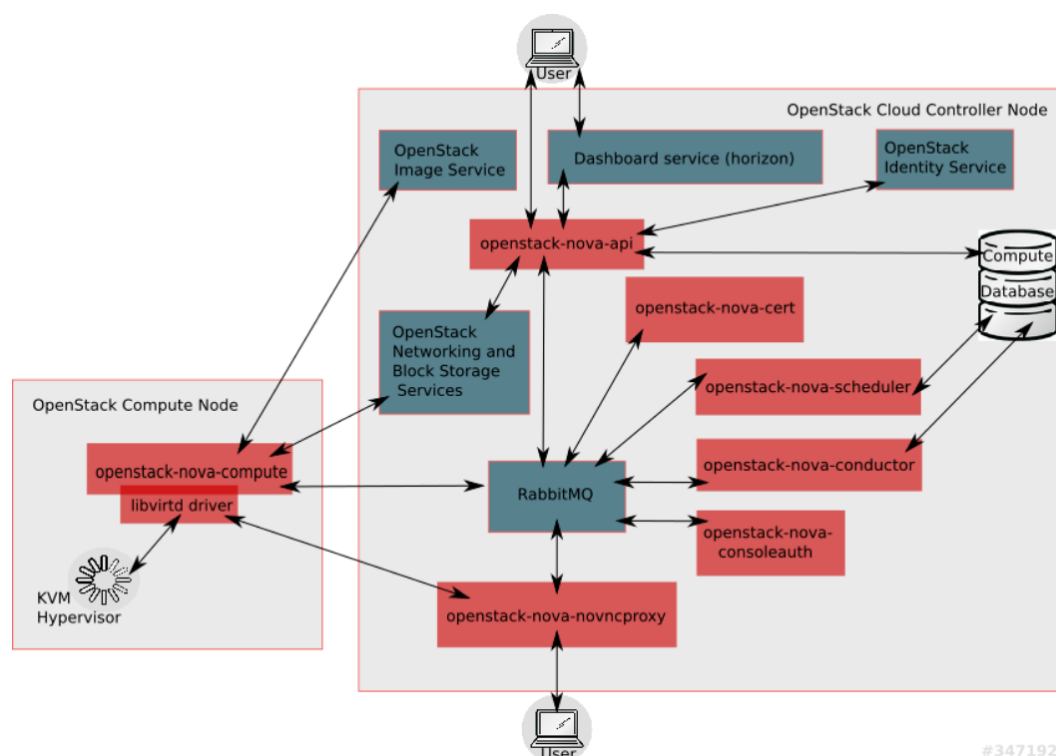


Figure 8131 – Interazione tra le due istanze di NOVA.

Istanza di Nova nel compute node

Deve ricevere i comandi dalla istanza di Nova installata nel controller e deve tradurli in comandi compatibili all'hypervisor locale installato nel compute node.

- **Compute service:** è un sottocomponente, un driver configurato specificatamente per l'hypervisor locale e serve per appunto tradurre i comandi ricevuti dal controller.

Nova - Creazione di una macchina virtuale

1. Si riceve una richiesta tramite l'interfaccia REST di Nova e il modulo API di Nova la riceve.
2. Verifica la validità dei parametri e dei requisiti di autenticazione (*il token di keystone*).
3. La richiesta viene inoltrata al modulo Conductor di Nova (*installato nel controller*).
4. **Macchine Fisiche papabili:** Il conductor invia la richiesta al modulo scheduler di Nova, il quale invia una lista di macchine fisiche sulle quali è possibile installare la nuova macchina virtuale.
 - a. Lo scheduler ha una lista dove sono elencate tutte le macchine su cui è installata una istanza di OpenStack.
 - b. **Filtering:** Scorre questa lista e in base alle specifiche della richiesta si rimuovono gli host che non possono ricevere altre macchine virtuali, che non possono riceverne affatto oppure se hanno caratteristiche speciali (*ad esempio la creazione di una VM con GPU, quindi la macchina fisica deve avere una GPU configurata a dovere*);

- c. **Weighting:** Infine le ordina mettendo in alto quelle su cui è più consigliabile inserire la macchina virtuale.
5. **Scelta della Configurazione di Rete:** Se lo scheduler consente la creazione della VM, il Conductor invia la richiesta al modulo Nova-Network in modo da ricevere una configurazione di rete opportuna per la nuova macchina virtuale.
6. **Scelta della Macchina Fisica:** Il modulo conductor sceglie una macchina fisica tra quelle proposte dallo scheduler.

La scelta viene anche comunicata allo scheduler in modo che possa aggiornare i suoi dati.

7. La richiesta con tutte le caratteristiche viene inviata alla istanza di Nova nel compute node scelto.
8. L'istanza di Nova nel compute node interagisce con l'hypervisor locale della macchina fisica scelta per creare la nuova macchina virtuale voluta.

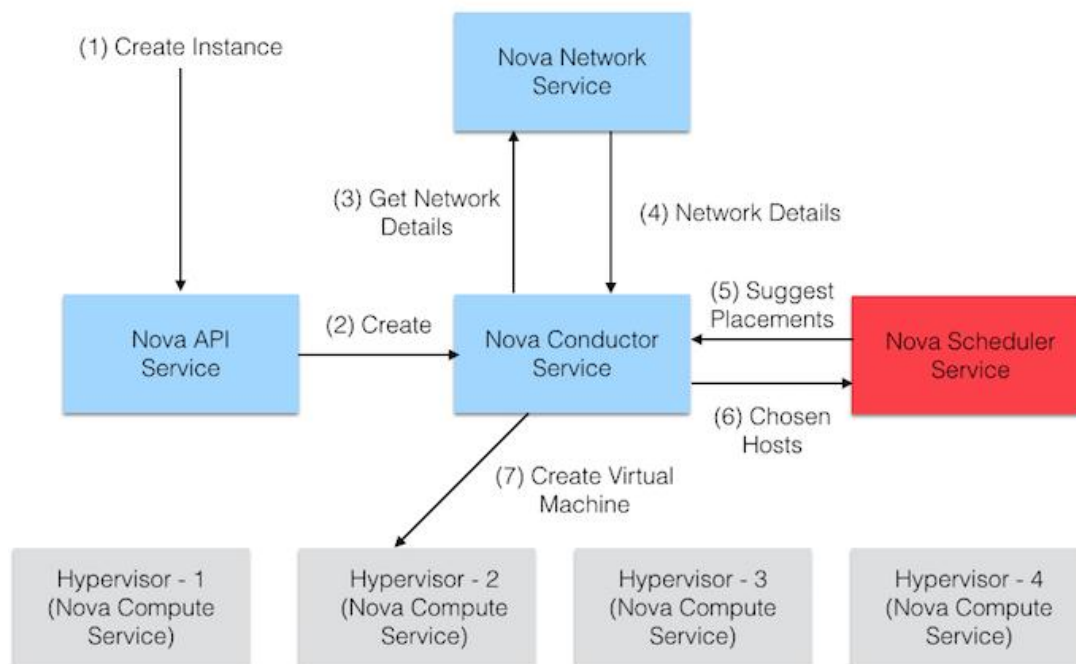


Figure 8232 - Creazione di una macchina virtuale con Nova.

Over commitment

Come funziona lo scheduler?

Open Stack è modulare, la politica utilizzata per decidere in quale nodo creare una VM è la seguente:

- Lo scheduler ha una lista di host.
- Gli host che non hanno risorse disponibili/sufficienti vengono rimossi dalla lista (*filter*).
- Gli host rimasti vengono ordinati in base all'idoneità (*weighting*).

Per misurare quanto un compute node è “*affaticato*” si usa un rateo chiamato **over commitment**: questo rateo esprime quante risorse virtuali vengono create partendo dal set di risorse fisiche del compute node.

- Un over commitment pari a 1 (100%) significa che una risorsa fisica è venduta solo una volta; ossia 16 GB di RAM virtuale sono creati da 16 GB di RAM fisica.
- Un over commitment pari a 2 (200%) significa che una risorsa fisica è venduta due volte; ossia 16 GB di RAM virtuale sono creati da 8 GB di RAM fisica.

Un over commitment superiore a 1 è accettato quando si parla della CPU, ma con la RAM molto meno, perché il degrado di prestazione aumenta molto con l'aumento dell'over commitment della RAM.

Lo scheduler di Nova ragiona principalmente usando questo rateo in modo da bilanciare il carico.

Glance

Glance è composto da due sotto-moduli **glance** (implementa la logica per la gestione delle immagini) e **glance storage**.

Glance

Sotto-modulo responsabile di gestire le immagini (*in questo contesto chiamate “VM Templates”*) da usare nella creazione delle macchine virtuali. Glance ha il compito di esporre una interfaccia REST per ricevere delle richieste del tipo: “*memorizza questa nuova immagine*”, “*cancella questa immagine*”, etc. Immagine

Ogni macchina virtuale è creata partendo da un'immagine che di solito comprende:

- Sistema operativo preconfigurato.
- Set di software necessari.
- Piattaforma cloud preconfigurata.
- Hypervisor preconfigurato.
- Set driver.

Glance Store

Glance ha un sotto modulo (*Glance Store*) che ha la responsabilità di memorizzare queste immagini da qualche parte come ad esempio:

- Sul file system locale del compute node.
- Su un'istanza di database.
- Cloud file system.
- Cloud storage.

In tutti questi casi (*eccetto il terzo*), le immagini sono memorizzate nella memoria persistente del nodo sul quale è installato anche Glance.

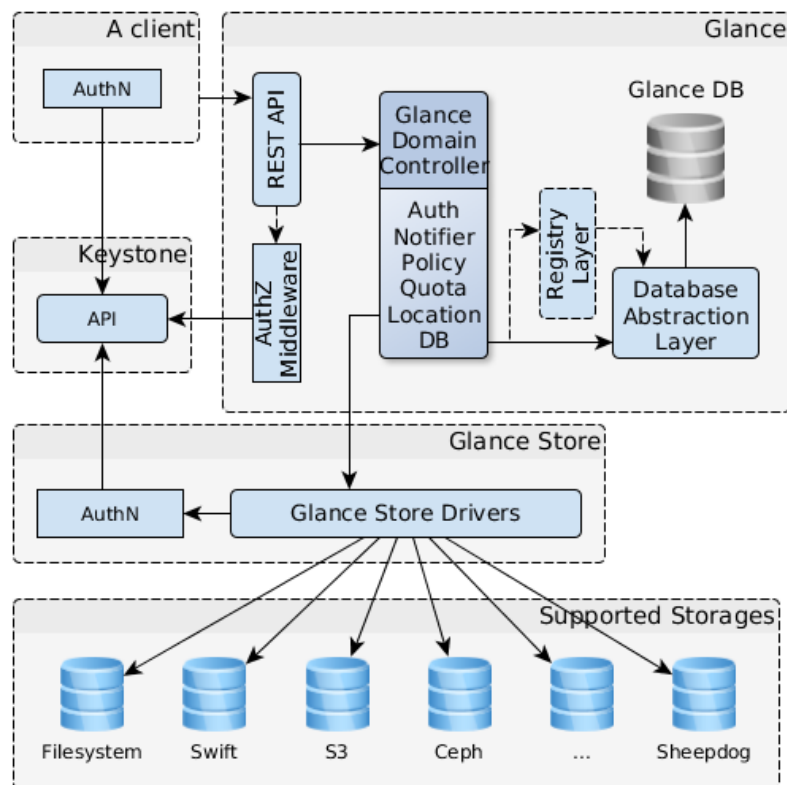


Figure 8333 - Workflow di un job di Glance, si parte dall'utente che richiede il servizio (quindi prima si autentica) e poi Glance preleva l'immagine.

Neutron

Neutron ha la responsabilità di creare e gestire i componenti di rete; ossia permette la creazione di una rete virtuale sulla quale possono comunicare le macchine virtuali. Per creare questa rete virtuale ovviamente si necessita di una infrastruttura di rete fisica.

La parte più difficile consiste nel far comunicare due VM situate in due compute node differenti. Inoltre, si vuole garantire anche isolamento, ad esempio VM1 e VM4 in figura appartengono alla VN2, esse devono comunicare solo tra loro e non anche con VM nella rete VN1.

Per virtualizzare una rete vengono utilizzati metodi esistenti prima dei cloud.

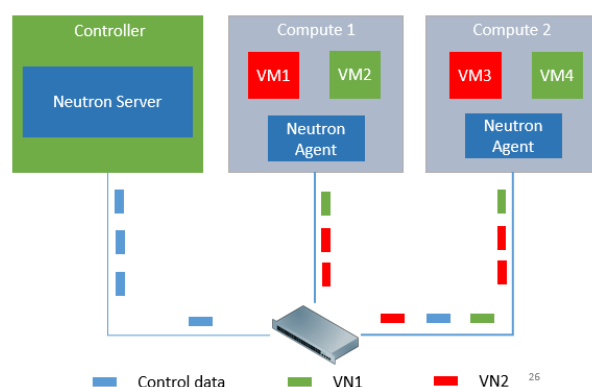


Figure 8434 - Rete virtuale creata con Neutron, in questo caso è possibile vedere due VLAN: quella verde e quella rossa.

Neutron si divide in altri due sottocomponenti:

Neutron-Server (controller)

Installato nel controller e gestisce la rete virtuale coordinando i Neutron-Agent. Espone una interfaccia REST per la gestione della rete virtuale.

Neutron-Agent (compute node)

Installato nel compute node e gestisce il traffico che va e parte dalle macchine virtuali installate sul compute node; questa cosa la fa emulando il funzionamento di una rete reale. gestisce l'isolamento tra diverse reti virtuali.

Indirizzamento IP

Neutron si prende cura dell'inoltro dei pacchetti tra due tenant nella stessa macchina fisica. Supponiamo che tutte le macchine virtuali di un tenant siano nella stessa rete virtuale.

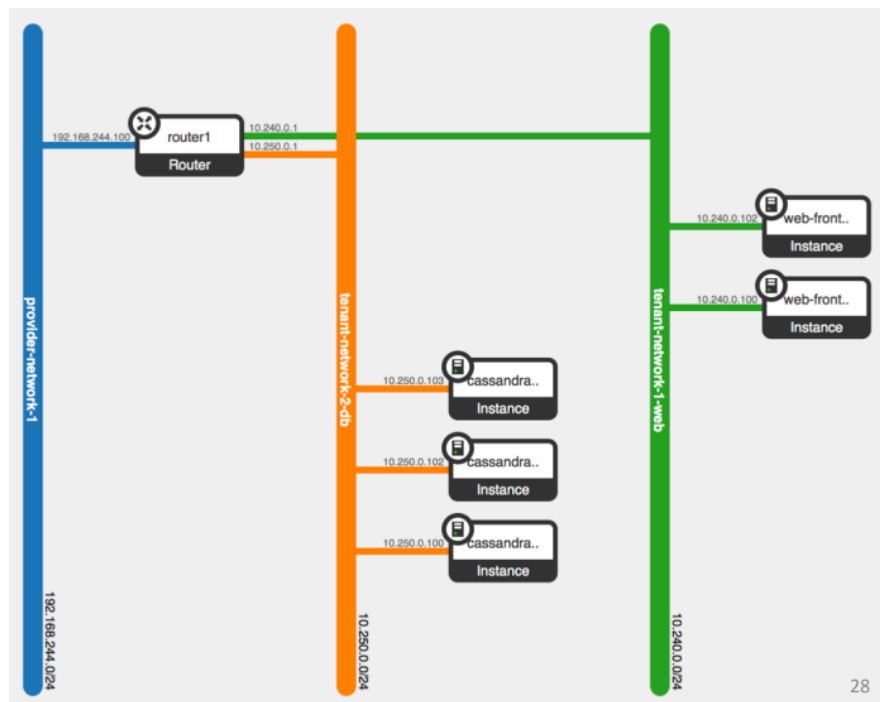


Figure 8535 - Indirizzamento IP tra due tenant diversi.

Network Node

Una rete virtuale è di solito privata, quindi si utilizzano indirizzi IP privati; Neutron però può permettere alle macchine virtuali di comunicare nella rete pubblica e viceversa. Questa cosa è possibile solo se si ha un Network Node.

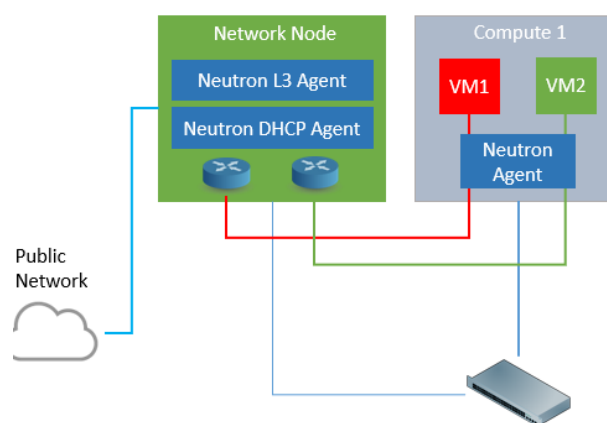


Figure 8636 - Come la rete virtuale comunica con l'esterno.

Il network node è di solito installato sul controller ed è connesso alla rete esterna oltre che alla rete in cui sono i compute node. Il network node è l'equivalente di un router di una rete tradizionale. In genere c'è un controller che svolge solo le funzioni del network node.

Si può dire che funge da router virtuale per le reti virtuali del cluster.

Neutron L3-Agent

Il network node ha installato un agente particolare chiamato Neutron-L3-Agent che è responsabile di gestire il traffico tra la rete privata e la rete pubblica, ovvero il traffico di livello 3. In aggiunta il network node ha anche le funzionalità di DHCP per assegnare gli indirizzi IP alle macchine virtuali.

Neutron Agent

Ha una istanza in esecuzione su ogni compute node e dispone di svariate tecnologie per lo smistamento del traffico.

Una delle ultime versioni possiede anche OpenVSwitch, ossia un software che include molte funzionalità di processazione e smistamento del traffico tra i compute node e il network node.

Ha il compito di smistare il traffico tra le macchine virtuali di un compute node; equivalente ad uno switch.

Neutron quindi consente di creare reti virtuali private, connettere queste reti con la rete pubblica, consente anche di assegnare dinamicamente gli indirizzi IP pubblici (*floating IP address*).

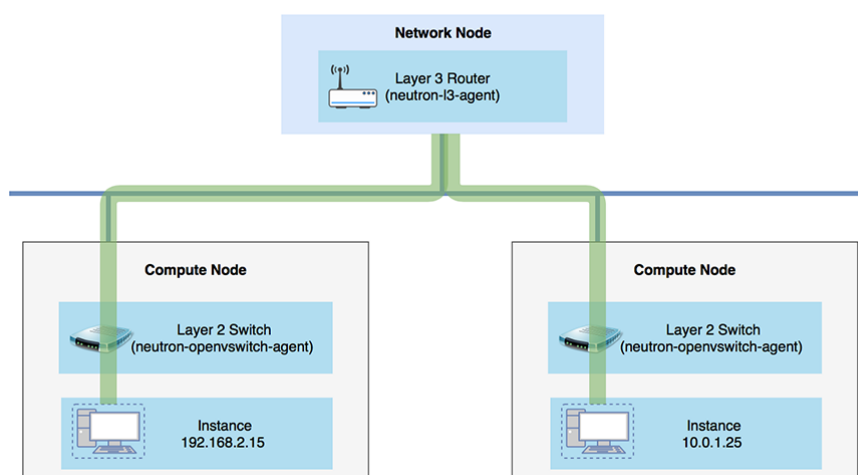


Figure 137 - Neutron Agent.

Physical Network Infrastructure

Per distribuire un'installazione di Open Stack abbiamo bisogno di definire almeno 3 reti fisiche. Serve una rete esterna connessa ad internet, questa rete è connessa solitamente al network node. Ci serve una seconda rete, **internal network** che collega tutti i nodi che fanno parte di un'istanza Open Stack (i controller e i compute node).

La terza rete è opzionale, però solitamente è inclusa, **management network** è isolata dalla altre ed è utilizzata per gestire il servizio, è utilizzata dagli amministratori del servizio per gestire le configurazioni dei server.

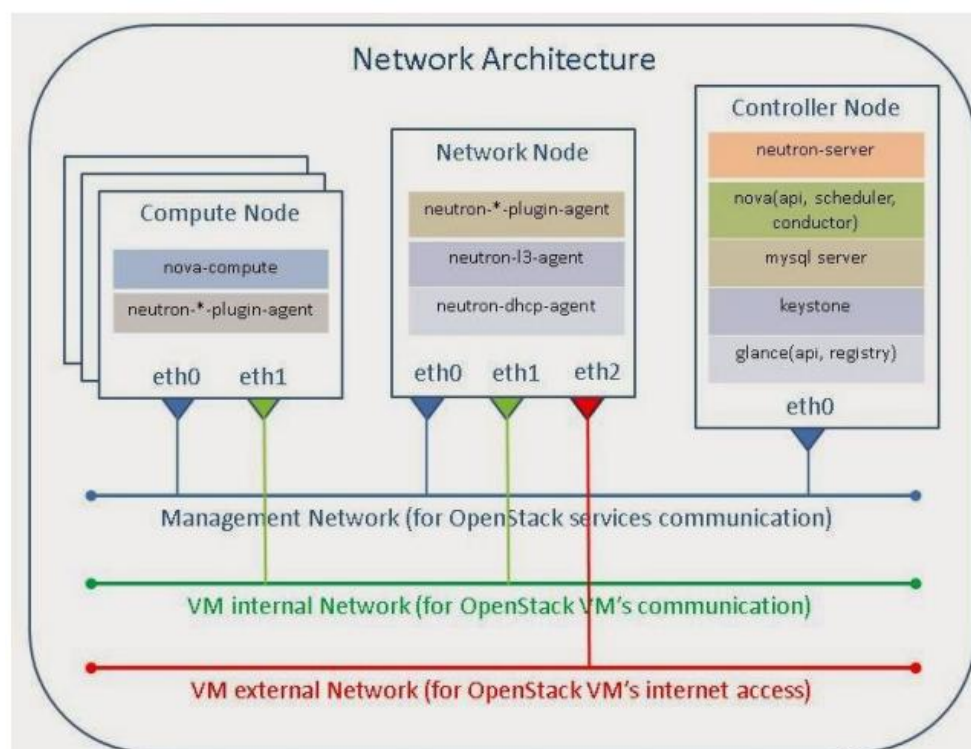


Figure 138 - Come appare l'architettura di rete.

OpenVSwitch - OVS

Implementazione open-source di uno switch virtuale distribuito. È stato progettato proprio per fornire un dispositivo di smistamento del traffico di rete in un ambiente completamente virtualizzato; quindi, supporta funzionalità di: filtraggio dei pacchetti, routing, switching, manipolazione dei pacchetti e molte altre ancora.

Quindi in realtà OVS virtualizza un dispositivo che non è esclusivamente relegato a livello due, ma spazia in tutti i livelli.

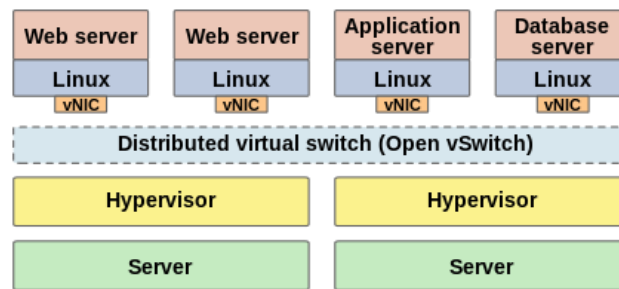


Figure 879 - Visione modulare di OVS.

OVS adotta un approccio atipico nel mondo delle reti, ossia un approccio centralizzato (*SDN*): ogni nodo ha un'istanza attiva di OVS ma tutte queste istanze sono controllate da un unico controller (*controller SDN*).

Software Defined Networks – SDN

SDN è un paradigma che definisce il comportamento della rete: Le regole di rete sono configurate dinamicamente su ogni istanza basandosi sulle direttive del controller centrale. Introdotto per semplificare la configurazione complessa delle reti tradizionali, inoltre essendo che le reti possono cambiare dinamicamente richiedono spesso riconfigurazioni; quindi, è stato introdotto un metodo per configurare le reti in breve tempo.

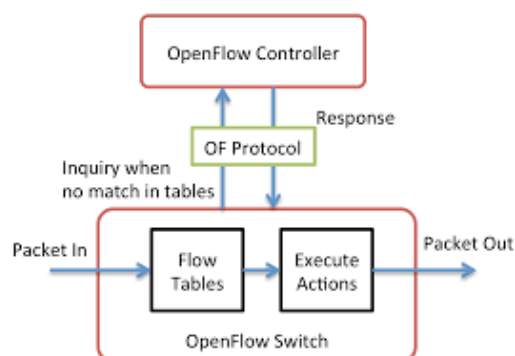


Figure 8840 - Applicazione della flow table.

Queste regole sono contenute in una tabella (*flow table*) gestita dalla singola istanza di OVVS all'interno del compute node; le regole sono molto simili a quelle che possiamo trovare su un firewall di alto livello; quindi, possono riguardare qualsiasi aspetto del pacchetto.

Quando un pacchetto viene ricevuto:

1. L'istanza di OVS scorre tutte le regole.
2. Se una regola fa match viene applicata e si esegue l'azione corrispondente.
3. Se non si trova nessuna regola che fa match:
 - a. Si chiede istruzioni al controller per un'azione specifica.
 - b. Oppure si scarta il pacchetto.

Il controller gestisce e riempie le flow table dei vari nodi.

Le istanze di OVS comunicano tra loro tramite il protocollo OpenFlow. In ogni Neutron Agent è presente un componente OpenVSwitch che implementa l'approccio SDN.

Comunicazione tra istanze di OVS

Ogni VM ha una NIC virtuale (vNIC), il quale hardware è emulato dall'hypervisor installato sulla macchina locale.

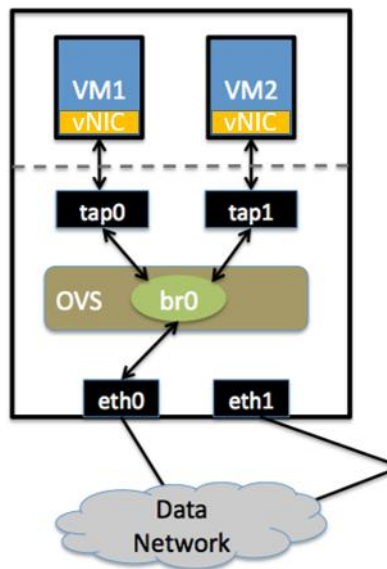


Figure 8941 - Come OVS fa comunicare le macchine virtuali con l'esterno attraverso la NIC fisica.

In modo da connettersi con l'istanza di OVS del compute node, la NIC virtuale deve essere collegata ad un'altra NIC virtuale appartenente al sistema operativo host (dall'immagine sopra, *tap1* e *tap0*).

Tap

Una “*tap*” è una NIC virtuale che può essere connessa via software; per “*connessa*” si intende che l'hypervisor è in grado di ricevere o inviare pacchetti dalla NIC provenienti dal software. In questo contesto le tap sono usate per ricevere o inviare traffico dalle macchine virtuali.

Il traffico proveniente dalle interfacce tap è indirizzato verso il componente OVS, il quale riceve il traffico e decide (basandosi sulle direttive nelle tabelle provenienti dal controller) dove instradare quel pacchetto tramite la scheda di rete fisica.

Viceversa, quando un nodo host riceve un pacchetto da un altro compute node, questo pacchetto attraversa la scheda di rete fisica, viene ricevuto da OVS che controlla se quel pacchetto appartiene ad una VN ed in base ai comandi ricevuti, inoltra il pacchetto alle VM locali connesse a quella VN.

Network Node

Le istanze di OVS possono essere programmate dal Neutron Server in modo da indirizzare il traffico tra diverse macchine virtuali che sono attive nello stesso compute node o su diversi compute node. Il network node ha una scheda di rete con cui comunicare con la rete esterna.

Quando dei dati sono diretti verso la rete esterna, vengono inoltrati al network node. Il network node ha una istanza di OVS installata in esso, in modo da poter usare le funzionalità di routing. Implementa un router virtuale.

Grazie a queste funzionalità il network node è in grado di inoltrare i pacchetti fuori dalla rete virtuale verso la rete esterna o verso altre reti virtuali.

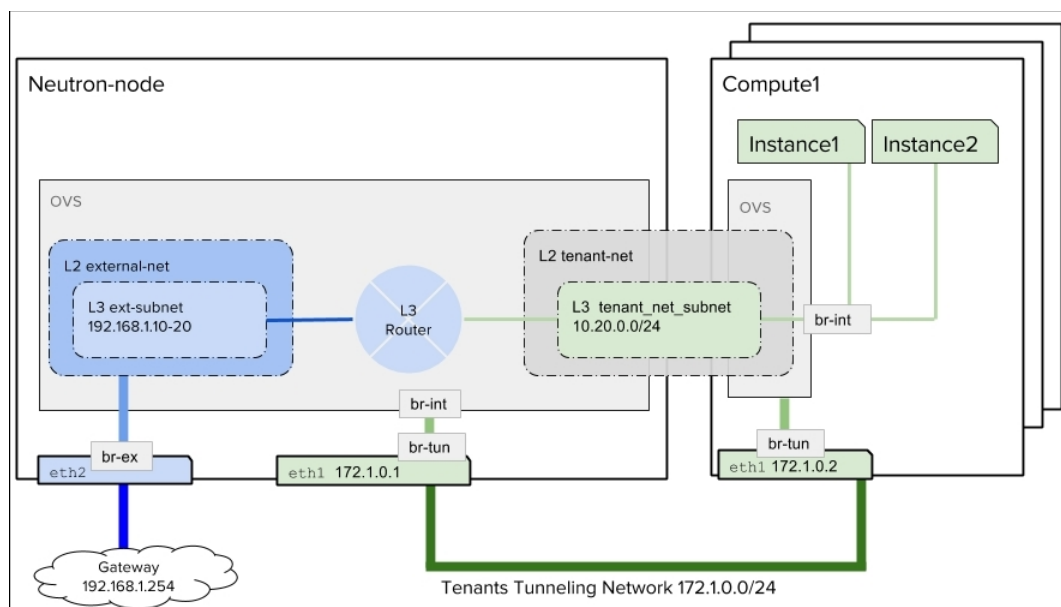


Figure 9042 - Schema logico di un network node che inoltra un pacchetto verso una rete esterna.

Virtualizzazione della rete

La virtualizzazione di una rete è un processo attraverso il quale si creano molteplici reti virtuali (VN – *virtual network*) per macchine virtuali al di sopra della medesima infrastruttura di rete.

Una virtual network è una rete di livello 2 (*Ethernet*). L'implementazione segue questa logica:

- **Traffico di livello 2:** Tra macchine virtuali della stessa rete virtuale.
- **Traffico di livello 3:** Tra macchine virtuali di diverse reti virtuali, per fare ciò si deve avere un router tra le due reti virtuali.

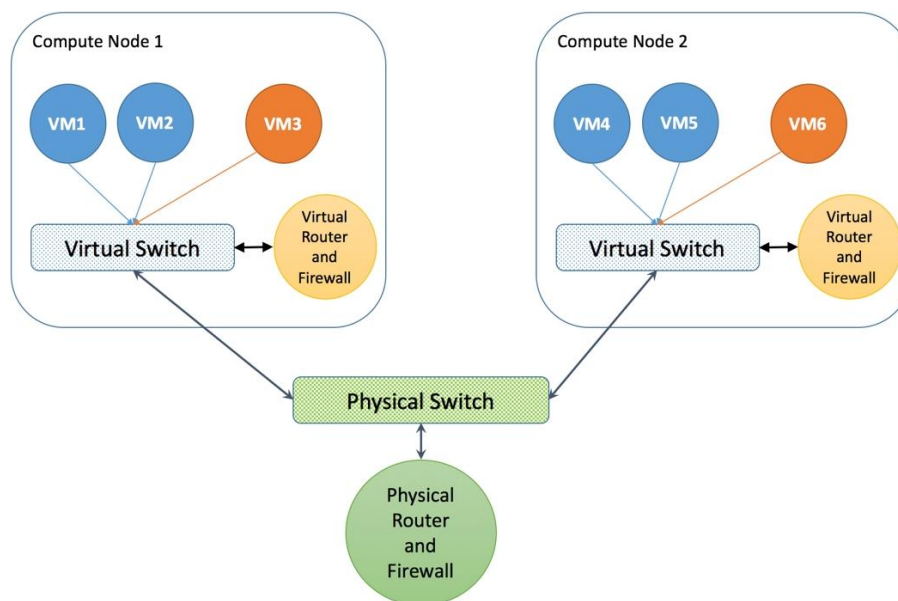


Figure 9143 - Schema logico di una rete virtualizzata.

In OpenStack la virtualizzazione della rete richiede che il traffico sia inoltrato dai compute node verso il network node seguendo l'esistenza delle reti virtuali: Se due compute node ospitano due macchine virtuali che appartengono alla stessa rete virtuale allora il traffico tra questi due compute node deve essere abilitato.

Per fare ciò si necessita che ogni pacchetto abbia un campo ID che esplicita l'appartenenza ad una VN oppure no oppure viene utilizzato il tunneling che consiste nel creare un tunnel solo per i pacchetti appartenenti ad una VN specifica.

Creazione di una rete virtuale - VLAN

Il modulo Neutron può utilizzare il meccanismo delle VLAN per creare delle VN.

Quando una VN viene creata per connettere due macchine virtuali (*a prescindere dai compute node nelle quali si trovano*) il Nova-Server crea una nuova VN e le assegna un “VID” il quale identifica in modo univoco la VN. VID è un campo contenuto nell’header del frame ethernet.

Quindi, quando due compute node ospitano due macchine virtuali che appartengono alla stessa rete virtuale, allora questi due compute node appartengono alla stessa VLAN.

Dall’esempio nell’immagine sotto: l’istanza di OVS nel compute node applicherà il tag relativo alla VN **blu** al traffico che arriva da una macchina virtuale appartenente alla VN **blu**.

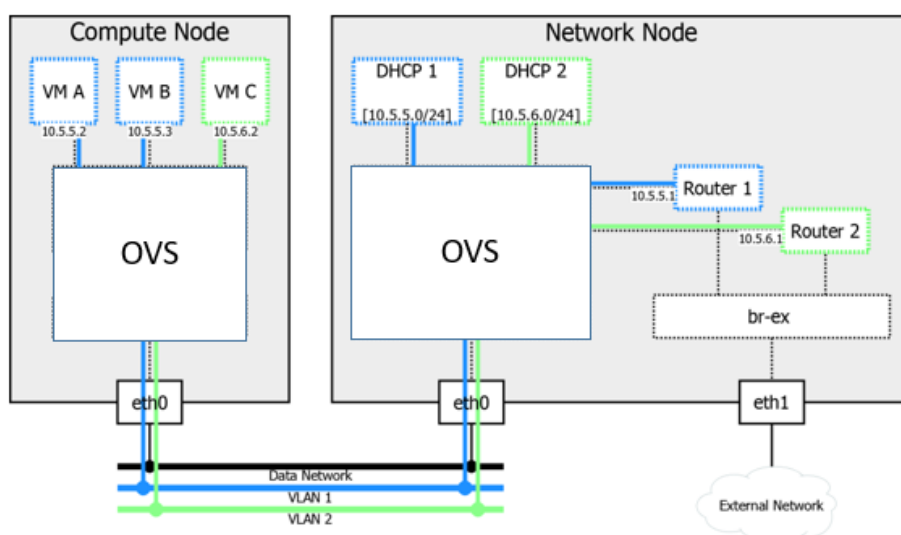


Figure 9244 - Funzionamento delle VN create tramite VLAN, sia nel caso interno che nel caso esterno.

Quando una VN è creata per connettere due o più VM, un VID è assegnato alla VN dal Nova server, quest’ultimo invia a tutti gli OVS in esecuzione nei compute nodes, tutti i comandi per implementare i seguenti passaggi.

Il funzionamento è lo stesso delle classiche VLAN: il pacchetto viene inviato e automaticamente (*tramite il funzionamento delle VLAN*) il pacchetto sarà ricevuto solo dai compute node coinvolti (*ossia i compute node che ospitano almeno una macchina virtuale appartenente a quella VN*).

Una volta ricevuto dal compute node l’istanza di OVS nel compute node stesso farà arrivare il pacchetto solo alle macchine virtuali che appartengono a quella VN.

Nello specifico: quando un pacchetto viene ricevuto da un’istanza di OVS in un compute node, il tag della VLAN viene rimosso e il pacchetto viene regolarmente inoltrato alla macchina virtuale destinataria.

Nel caso in cui ci siano compute node coinvolti in una rete esterna (*si prevede che i pacchetti di quella VN vadano all'esterno*) allora il Neutron-Server configura anche l'istanza di OVS nel network node (*ossia configura un virtual router*) in modo da eseguire la stessa procedura per i pacchetti che sono diretti o provenienti dal router virtuale assegnato a quella VN.

Creazione di una rete virtuale - GRE Tunneling

Un'altra tecnica per creare una rete virtuale si basa su un protocollo di incapsulamento dei pacchetti IP, ossia il protocollo GRE.

Se abbiamo tre compute node e due reti virtuali (VN1 e VN2), a queste due reti vengono assegnati i GRE ID 1 e 2 rispettivamente, i neutron agent di ogni compute node creano tunnel con un ID specifico con tutti gli altri compute node.

Ad esempio, viene creato un tunnel tra compute node 1 e 2, tra 1 e 3, infine tra 2 e 3, se questi tunnel hanno 1 come ID, il traffico che scambiano è relativo solo alla rete virtuale 1.

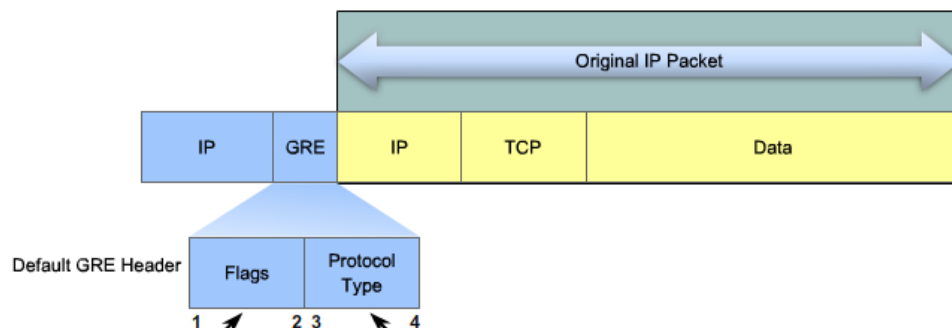


Figure 9345 - Incapsulamento di GRE, si può notare come venga aggiunta una nuova intestazione IP.

Tramite GRE:

- Si crea un “*tunnel*” tra ogni compute node e il network node.
- Si crea un “*tunnel*” per ogni coppia di compute node che ospitano due macchine virtuali che appartengono alla stessa rete virtuale.

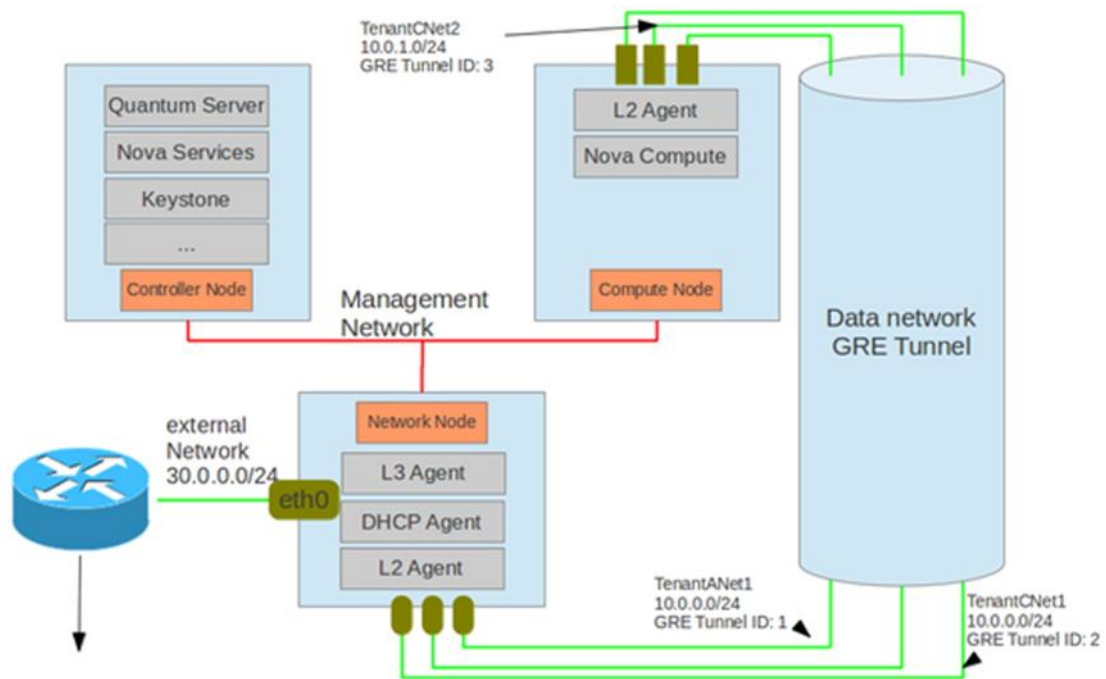


Figure 9446 - Utilizzo di GRE per creare una rete virtuale, si può notare come ogni compute node ha un tunnel (Data network GRE tunnel) con il network node.

In ogni compute node c'è un'istanza di OVS, ciascuna istanza riceve i comandi dal controller, questi comandi sono richieste di creazione di tunnel e inoltre dei dati in questi tunnel.

Ogni rete virtuale ha un “*GRE ID*” univoco il quale è memorizzato nel Nova Server; quest'ultimo istruisce le istanze di OVS al fine di creare un'interfaccia GRE-1 al di sopra dell'interfaccia di rete fisica.

Questa interfaccia è il terminale per utilizzare il GRE tunnel.

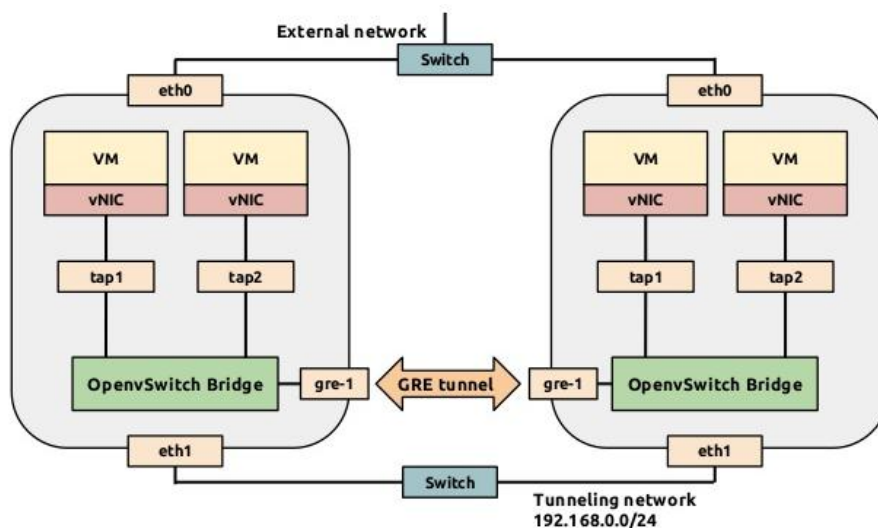


Figure 9547 - Creazione delle interfacce GRE-1.

Questa interfaccia GRE-1 è responsabile di creare il tunnel tra i compute node coinvolti nella comunicazione e di incapsulare/de-incapsulare il traffico coinvolto.

VXLAN – Virtual eXtensible LAN

Terza tecnologia per creare le reti virtuali, migliora la soluzione delle VLAN.

Le VLAN hanno dei limiti dati dal fatto che non sono state progettate per questi scopi: ad esempio l'identificatore da mettere nell'intestazione del pacchetto è a soli 12 bit; quindi, posso avere al più 4094 VLAN; questo limite è un problema in una infrastruttura cloud dal punto di vista della scalabilità.

Per questo sono state introdotte le VXLAN, ossia una versione più aggiornata delle VLAN che permettono più di 16 milioni di VLAN (*identificatore a 24 bit*) e in più posseggono migliorie dal punto di vista della scalabilità e della flessibilità.

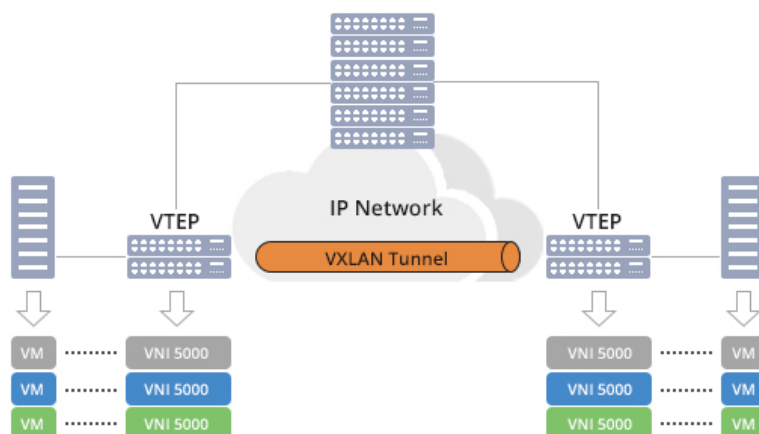


Figure 9648 - Utilizzo delle VXLAN.

VTEP – Virtual Tunnel Endpoints Protocol

VXLAN usa un protocollo di incapsulamento particolare chiamato VTEP: In questo protocollo i frame di livello 2 (MAC) sono incapsulati in pacchetti di tipo UDP (quindi di livello 4 e che quindi estendono anche il livello 3).

Invece in GRE i pacchetti di livello 3 venivano incapsulati in altri pacchetti di livello 3.

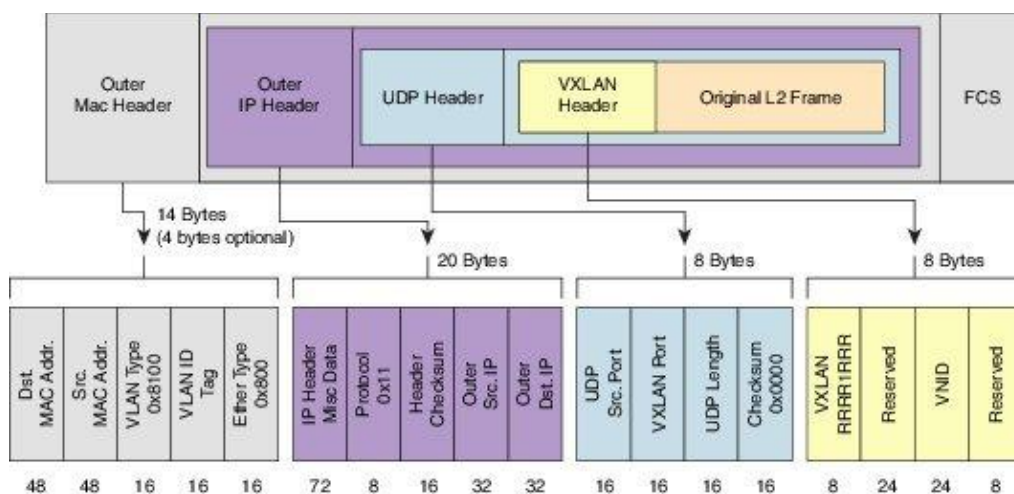


Figure 979 - Formato di VTE.

Un VTE (*Virtual Tunnel Endpoint*) può essere un qualsiasi dispositivo di rete come un router, switch, una macchina virtuale.

Il pacchetto incapsulato può essere inviato anche in una rete di livello 3 in quanto possiede anche quella parte di intestazione (*l'intestazione di VTEP arriva fino a livello 4*); ergo può essere inoltrato da un router verso un'altra rete.

Ogni host ha un'istanza di OVS che riceve comandi dal controller per stabilire tunnel tra due compute node, ogni tunnel ha un ID associato ad una VN.

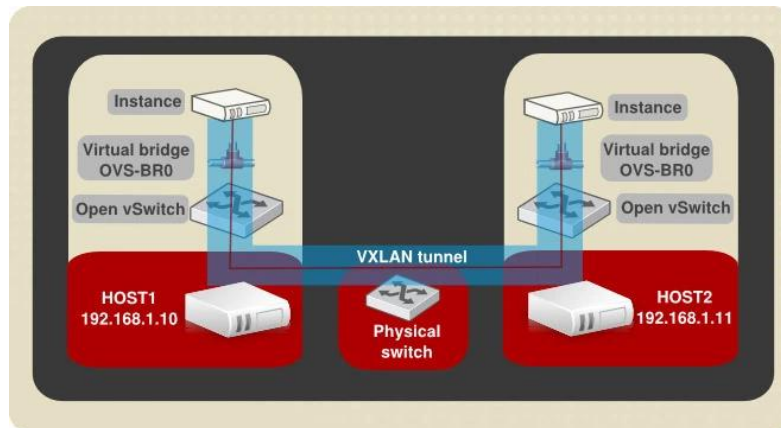


Figure 9850 - Visione semplificata del funzionamento di VTEP.

Cinder

Modulo di OpenStack con lo scopo di gestire i volumi. I volumi sono hard drive secondari per memorizzare dati.

Ogni macchina virtuale possiede un hard drive virtuale dove memorizza il sistema operativo. Se una macchina virtuale ha bisogno di spazio extra, deve ampliare il proprio virtual hard drive. I volumi sono visti dalle VM come hard drive virtuali, un hard drive virtuale è memorizzato nel file system come immagine (come file o oggetto).

Un hard drive virtuale è esposto dall'hypervisor il quale accede all'hard drive fisico tramite il protocollo iSCSI. Questo protocollo fornisce accesso block-level allo storage.

Questi hard drive virtuali possono essere installati in una macchina fisica diversa da quella in cui è in esecuzione la VM.

Le immagini dei volumi sono memorizzate nel file system locale del nodo in cui il servizio è installato o in un cloud file system.

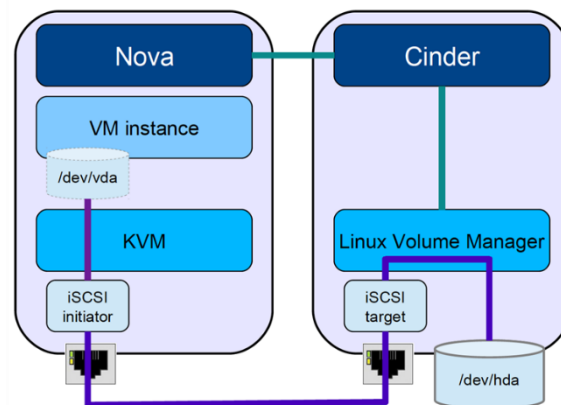


Figure 9951 – Struttura Cinder.

Cinder è composto da diversi componenti:

- **Cinder-API:** espongono un'interfaccia che i client esterni possono utilizzare per controllare le operazioni di Cinder.
- **Cinder-Volume:** responsabile di gestire le richieste che provengono dall'interfaccia rest.
- **Cinder-Scheduler:** responsabile per selezionare lo storage corretto, per memorizzare nuovi driver.
- **Cinder-Backup:** responsabile per creare il backup di volumi esistenti quando richiesto dagli utenti.

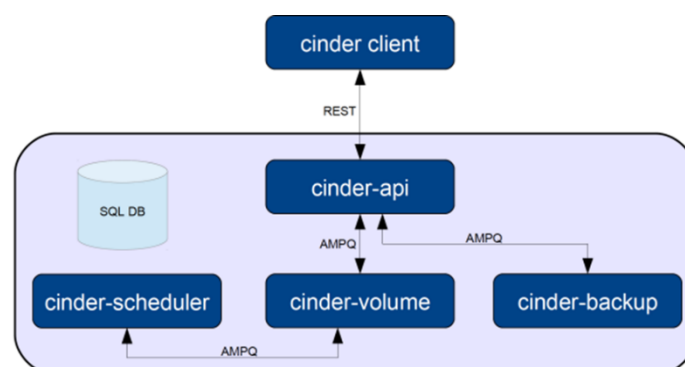


Figure 10052 – Componenti Cinder.

Ceilometer

Componente di telemetria, è opzionale, però può essere utilizzato per monitorare (tramite interfaccia grafica) le prestazioni della piattaforma come, ad esempio, quante VM ci sono nella piattaforma, qual è il carico medio, qual è il rate di operazioni di I/O ecc.

Inoltre, può anche essere utilizzata per collezionare dati utili per stabilire quanto gli utenti devono pagare.

Ceilometer architecture

In ogni compute node è presente il **ceilometer agent**, il quale interagisce con i componenti locali in ogni compute node (hypervisor, cinder ecc) per raccogliere statistiche.

Invia le statistiche al **ceilometer collector** che inserisce le informazioni in un database, inoltre espone delle API che consentono di ricavare i dati dal database.

Horizon

Interfaccia web esposta dal controller che gestisce tutte le istanze e fornisce anche uno strumento command line per la gestione del backend. Consente all'utente di creare/distuggere macchine virtuali e anche altro.

Per comunicare con ogni componente non ci sono solo dashboard e command line, ogni servizio Open Stack espone un set di API, tutte le API utilizzano una comunicazione REST. Queste API possono essere utilizzate dai moduli per interagire ma anche dall'amministratore che può scrivere programmi esterni per automatizzare alcuni processi.

Swift

Object storage, è uno storage per memorizzare i dati basato su oggetti, gli oggetti memorizzati hanno delle caratteristiche specifiche, una è che questi oggetti sono immutabili. Swift consente di creare un object storage nel cloud. Questo componente espone delle rest API, tramite queste i client possono richiedere la creazione di un oggetto, la rimozione ecc. Questo componente gestisce tutte le funzionalità richieste per memorizzare dati.

Heat

Orchestratore, il quale gestisce in modo automatizzato la creazione e distruzione delle VM secondo alcune politiche, automaticamente può monitorare lo stato corrente delle VM, le VM possono essere create in modo da distribuire il carico tra le varie VM. Se le VM sono sovraccaricate, nuove VM vengono create in modo da effettuare load balancing del carico, mentre se alcune VM non sono utilizzate, vengono distrutte e rimangono solo le VM necessarie.

High Availability (HA)

OpenStack è organizzato come sistema distribuito, se c'è solo un'istanza di un componente (ad esempio Nova), e quel componente crasha o il controller crasha, il servizio fornito dal

cloud non è più disponibile; quindi, deve essere introdotta la ridondanza dei componenti per garantire data synchronization, data replication e anche il load balance.

Una installazione Open Stack con un solo controller non è HA, se il controller fallisce, fallirà anche l'istanza Open Stack. Perché diventi HA bisogna implementare replicazione del controller, in modo che se fallisce, l'infrastruttura continui a funzionare perché uno degli host che implementano il ruolo del controller sostituisce il controller primario.

Un'opzione è quella di utilizzare l'approccio master-slave, un'istanza del controller è il master, il quale è in funzione e riceve tutte le richieste di creazione/distruzione di VM, le altre istanze del controller sono in standby, ogni volta che il master effettua un'operazione aggiorna gli slave, se il master fallisce, uno degli slave lo sostituisce.

Un'altra soluzione è quella della **ridondanza semplice**, serve un load balancer prima dei controller, le richieste vengono inoltrate ad uno dei controller in modo da bilanciare il carico. Un componente utilizzato per implementare il load balancer distribuito è il HA proxy. Le funzionalità dell'HA proxy sono implementate nel controller, uno di essi attivo, mentre gli altri rimangono inattivi. Le istanze del HA proxy devono coordinarsi per rilevare fallimenti e triggerare la configurazione se necessario. Lo stato dello storage deve essere replicato tra i nodi controller.

Lab - OpenStack

Sono state fatte due installazioni di OpenStack, una usando JuJu e una più *“faticosa”*.

Inizialmente utilizzeremo una configurazione **all-in-one**, ovvero tutti moduli di OpenStack viene installato in una singola macchina, utilizzeremo Microstack per automatizzare l'installazione.

- Per prima cosa installiamo *snapt* che è un package distribution system.
- Utilizziamo *snapt* per installare *Microstack*.

Installation

```
apt install snapd
snap install microstack --beta
```

```
root@cloud2023:~# snap install microstack --beta
2023-04-15T12:41:10Z INFO Waiting for automatic snapd restart...
microstack (beta) ussuri from Canonical✓ installed
root@cloud2023:~#
```

[illegible]

- Successivamente bisogna eseguire il comando per l'inizializzazione.

- Check that everything went fine

```
snap list microstack
```

- Initialize the installation

```
microstack init --auto --control
```

In case of errors, re-run the command

```
root@cloud2023:~# snap list microstack
```

Name	Version	Rev	Tracking	Publisher	Notes
microstack	ussuri	245	latest/beta	canonical✓	-

[illegible]

- Per accedere alla piattaforma OpenStack si utilizza il seguente comando per recuperare la password dell'admin. Per accedere alla piattaforma inserire lo **username: admin** e la password appena ricavata.

Access the platform

- Retrieve admin password

```
snap get microstack config.credentials.keystone-password
```

```
root@load2023:/var/log# snap get microstack config.credentials.keystone-password
LSBC41REEDM7MCD1N9e9V5oaQ0BnX99Q
root@load2023:/var/log#
```

- <https://10.1.1.X/admin/>



Un'installazione OpenStack richiede almeno 4 macchine fisiche, 1 per il controller e 3 per i compute node, questa configurazione non è corretta, perché abbiamo un solo controller, se fallisce il servizio non è garantito. Ci servono 3 compute node perché Ceph (cloud file system su cui si basa open stack) richiede come minimo 3 host.

OpenStack è un software distribuito composto da differenti componenti, è impossibile configurarli in modo statico, il processo deve essere automatizzato, solitamente le installazioni di OpenStack provengono da processi automatizzati, queste procedure di installazione sono eseguite in un sistema remoto (architecture manager).

L'architecture manager è l'unico host che viene acceduto durante un'installazione OpenStack, questo host esegue un framework che automatizza l'intero processo.

La configurazione è la seguente 5 VM, una di queste è l'architecture manager, una è il controller e tre sono i compute node, tutte le macchine sono interconnesse tramite una rete, il controller è connesso alla rete pubblica.

Vediamo un esempio su come installare ed utilizzare OpenStack come sarebbe nella realtà, si utilizza il framework **juju**, il quale può connettersi ad una macchina remota senza necessitare di una password ogni volta e gestisce il development.

Le configurazioni avvengono nella macchina remota tramite SSH, per questo bisogna creare un insieme di chiavi RSA nel manager e poi aggiungere queste chiavi a tutte le macchine.

- Per prima cosa installiamo juju nell'architecture manager.
- Il prossimo step è di iniziare la configurazione dell'architecture manager utilizzando il comando `add cloud` che mostra un insieme di opzioni per il tipo di cloud.
- Successivamente si va al bootstrap dell'architecture manager.
- Dopo questa fase si aggiungono le macchine, ovvero il controller ed i compute node tramite il comando `juju add-machine ssh:root@IP`.

- Il prossimo step consiste nella configurazione della rete per i container, alcuni dei componenti di OpenStack vengono forniti all'interno dei container, per questi componenti bisogna garantire una connessione in modo che possano comunicare.
- Con questo la fase di configurazione è conclusa e adesso c'è la fase di deployment per ogni modulo, uno per volta, utilizzando il comando *juju deploy*.
- Dopo questo step, la fase successiva consiste nello stabilire le relazioni tra i vari componenti, serve perché un componente potrebbe essere progettato per essere eseguito in diverse installazioni cloud, quindi non è detto che venga utilizzato in ogni caso.
- Il Nova Compute ha due componenti, uno che è eseguito nel controller e uno per ciascun compute node, creiamo un file yaml per configurarlo.
- L'altro componente è Neutron, il quale ha due componenti, il neutron-gateway che definisce l'interfaccia verso la rete esterna (Network node) ed il neutron-controller responsabile della configurazione di tutti i componenti virtuali e degli OVS.
- Con il comando *juju deploy*, l'opzione *-to* indica il container in cui si vuole fornire il controller.
- Per ogni compute node si effettua il deploy di ogni componente OVS, non c'è l'opzione *-to*, viene configurato direttamente dove questo componente viene fornito.
- Dopo aver installato questi componenti vanno eseguiti dei comandi per consentire al sistema di eseguire alcuni step di configurazione.
- Componente MySQL che contiene le informazioni permanenti, si installa Keystone e RabbitMQ, sia MySQL che RabbitMQ vengono installati in un container.
- Ceph è un modulo per creare un file system distribuito tra differenti macchine, per creare un cloud file system.
- NTP è un modulo che consente la sincronizzazione del tempo tra nodi differenti.
- A questo punto ci connettiamo alla dashboard tramite il browser.

Flavor

Le funzionalità principali fornite da OpenStack sono quelle di creare delle VM, una delle prime cose quando si effettua il login nell'interfaccia di OpenStack come admin, è quella di definire i flavor, i quali indicano l'insieme di risorse assegnate ad ogni VM.

L'admin è sia l'amministratore dell'istanza, del sistema, inoltre è anche un utente del sistema. Nella sezione *overview* vengono mostrate le risorse allocate, quindi VCPU, RAM, volumi ecc. La creazione dei flavor è compito degli admin in modo da consentire agli altri utenti di creare VM.

Create Flavor

Flavor Information

Flavor Access

Name *

basic

ID ⓘ

auto

VCPUs *

1

RAM (MB) *

64

Root Disk (GB) *

1

Ephemeral Disk (GB)

0

Swap Disk (MB)

0

RX/TX Factor

1

Flavors define the sizes for RAM, disk, number of cores, and other resources and can be selected when users deploy instances.

Cancel

Create Flavor

In figura vediamo i flavor pre-creati, è possibile accedervi andando su **Compute -> Flavors**.

openstack. admin

Admin / Compute / Flavors

Flavors

Filter

Create Flavor

Delete Flavors

Displaying 5 items

☐	Flavor Name	VCPUs	RAM	Root Disk	Ephemeral Disk	Swap Disk	RX/TX factor	ID	Public	Metadata	Actions
☐	m1.large	4	8GB	20GB	0GB	0MB	1.0	4	SI	No	Update Metadata
☐	m1.medium	2	4GB	20GB	0GB	0MB	1.0	3	SI	No	Update Metadata
☐	m1.small	1	2GB	20GB	0GB	0MB	1.0	2	SI	No	Update Metadata
☐	m1.tiny	1	512MB	1GB	0GB	0MB	1.0	1	SI	No	Update Metadata
☐	m1.xlarge	8	16GB	20GB	0GB	0MB	1.0	5	SI	No	Update Metadata

Displaying 5 items

Una volta creati i flavor, bisogna aggiungere le immagini.

Bisogna importare un'immagine dell'hard drive, scaricandola in rete e aggiungendola selezionando i seguenti campi: **Admin->Images->Create Image**.

Inserire il file installato nella sezione **file**, il formato deve essere quello in figura (QEMU è quello utilizzato nei lab), i campi *kernel* e *ramdisk* possiamo lasciarli vuoti, mentre nel campo *architecture* inseriamo amd64. Vanno assegnate anche la dimensione minima del disco in GB e della RAM in MB.

Una volta creata l'immagine, se impostiamo come pubblica, l'immagine è disponibile per tutti gli utenti nella piattaforma.

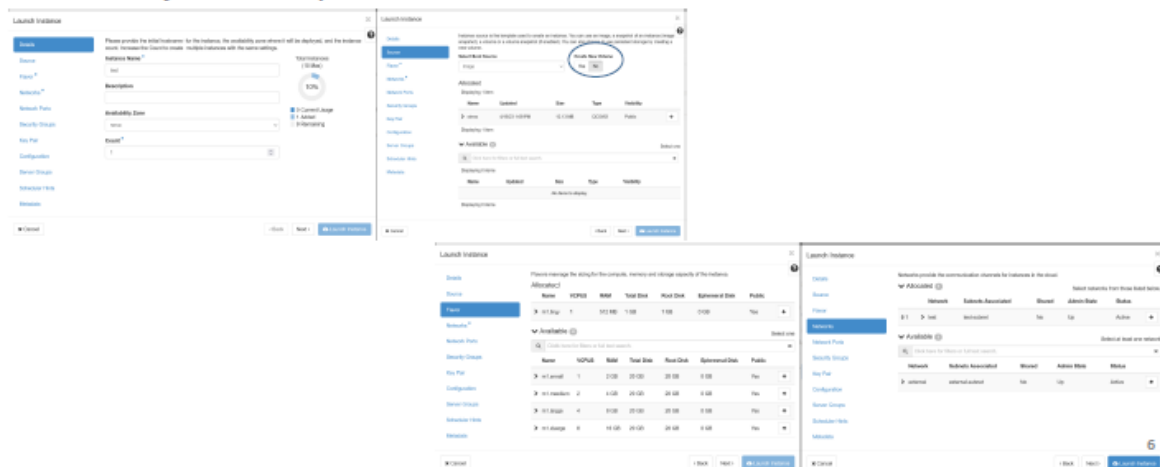
L'immagine compare nella sezione dedicata come in questo esempio.

Owner	Name	Type	Status	Visibility	Protected	Disk Format	Size
admin	cirros	Image	Active	Public	No	QCOW2	12.13 MB

Una volta che abbiamo un flavor e un'immagine possiamo creare una VM, per istanziare la prima VM andare su **Project->Compute->Instances->Launch Instance**.

Si seleziona l'immagine precedentemente creata, l'opzione **create new volume** indica un hard drive virtuale che ospiterà il SO. Se clicchiamo "yes" viene creato un volume, che è una funzionalità specifica di OpenStack per istanziare hard drive secondari che non sono allocati direttamente in una macchina fisica, ma nella macchina virtuale. Mentre se clicchiamo "no" viene creato un normale hard drive virtuale.

Se si crea il volume, viene istanziato in un sistema che potrebbe avere tecnologie di virtualizzazione completamente differenti. Nella configurazione vista finora, la creazione fallirebbe, per adesso non lo creiamo.

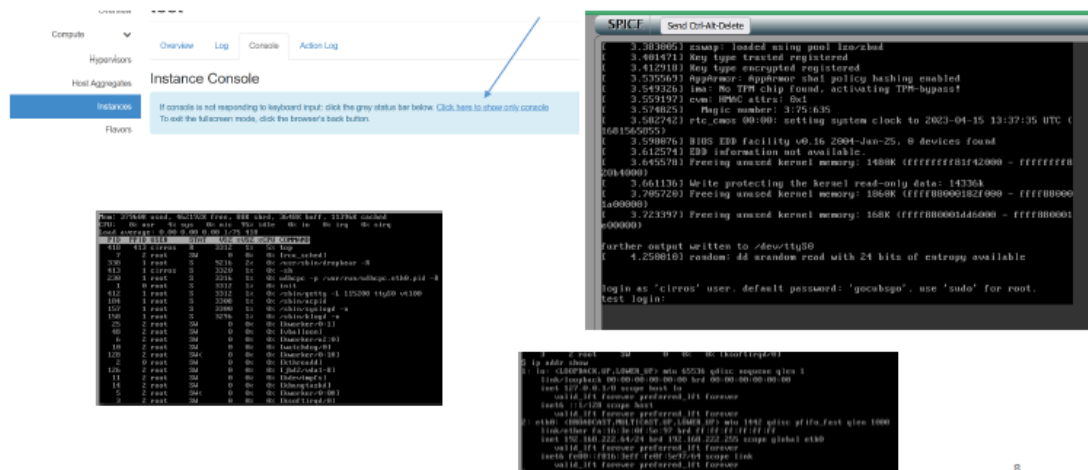


Quando creiamo una VM bisogna selezionare anche la rete cui vogliamo collegarla, una volta configurati gli hard drive, selezionati i flavor e la rete è possibile creare la VM, la piattaforma selezionerà in automatico la macchina fisica in cui allocare la VM.

Nel seguente modo è possibile osservare l'immagine in esecuzione: **Admin->Compute->Instances**.

Overview	Instances										
Compute											
Hypervisors											
Host Aggregates											
Instances											
Flavors											
Images											
Displaying 1 item											
☐	Project	Host	Name	Image Name	IP Address	Flavor	Status	Task	Power State	Age	Actions
☐	admin	cloud2023	test	cirros	192.168.222.64	m1.tiny	Active	None	Running	0 minuti	Rescue Instance

Una volta creata l'istanza (istanza è come viene chiamata una VM) è possibile aprire la **console**, è il terminale della VM, è il solito terminale della VM Cirros che abbiamo creato all'inizio dei lab.



Di default due reti vengono create, una interna ed una esterna, la rete interna (detta **test**) serve per connettere le VM che non sono connesse alla rete esterna, mentre la rete esterna serve a connettere le VM ad internet, è composta da indirizzi IP pubblici.

Le reti create dall'admin sono condivise con tutti gli utenti.

Displaying 2 items

☐	Name	Subnets Associated	Shared	External	Status	Admin State	Availability Zones	Actions
☐	external	external-subnet 10.20.20.0/24	No	SI	Active	UP	-	Edit Network
☐	test	test-subnet 192.168.222.0/24	No	No	Active	UP	-	Edit Network

Displaying 2 items



Per creare le reti occorre andare in queste sezioni: **Admin -> Network -> Networks -> Create Network**.

È possibile visualizzare la topologia della rete cliccando nella sezione appropriata.

Nella figura della rete, tra le due reti è presente il virtual router che consente l'interconnessione tra le VM che sono nella rete esterna e le VM che sono nella rete interna.

Per configurare la rete serve inserire il nome della rete, il nome della subnet, l'indirizzo IP della rete, la versione di IP ed il gateway.

Il gateway è l'indirizzo del router che connette le VM alla rete internet.

Nel campo **allocation pools** viene assegnato l'insieme d'indirizzi che possono essere assegnati alle VM. In seguito, configuriamo il DNS.

Successivamente si va nella tab dei router e si crea, si specifica la rete esterna, abilitando anche il NAT. Il router ha un'interfaccia connessa ad internet, quando vogliamo connettere una rete si aggiunge una nuova interfaccia al router.

Create a network

- Admin -> Network
-> Networks ->
Create Network

The image shows two screenshots of the 'Create Network' form in OpenStack. The left screenshot shows the 'Network' tab with fields for Name, Project, Provider Network Type, and Enable Admin State. The right screenshot shows the 'Subnet' tab with fields for Subnet Name, Network Address, IP Version, Gateway, and Enable Gateway.

Per connettere le VM alla rete basta andare nella seguente sezione: **Instances->Attach Interface**.

- Instances->Attach Interface

The image shows the 'Attach Interface' form in OpenStack. It includes a dropdown for 'The way to specify an interface' set to 'by Network (and IP address)', a dropdown for 'Network' set to 'prova (192.168.1.0/24)', and a field for 'Fixed IP Address'. Below the form is a table showing the instance 'test' with IP addresses 192.168.1.15 and 192.168.222.64.

Instance Name	Image Name	IP Address	Flavor	Key Pair	Status	Availability Zone	Task	Power State	Age	Actions
test	cirros	prova 192.168.1.15 test 192.168.222.64	m1.tiny	-	Active	nova	None	Running	39 minuti	Create Snapshot

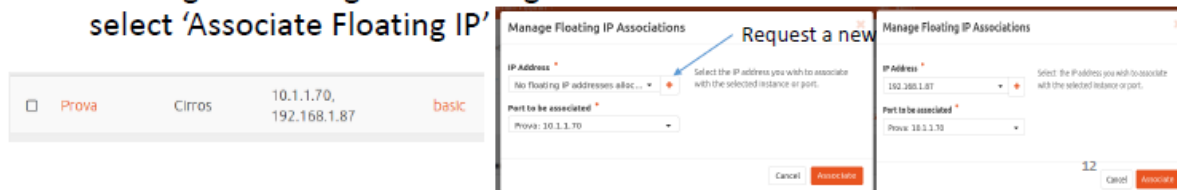
Floating IP

Alle VM può essere assegnato un IP pubblico per connetterle ad internet tramite il **floating IP**.

Un indirizzo floating IP è un IP pubblico assegnato ad una VM temporaneamente per accedere ad una rete esterna. Ovvero l'indirizzo viene assegnato dinamicamente ad una VM o ad un'altra.

Per assegnare un floating IP occorre andare nel menu dell'istanza e selezionare *Associate Floating IP*.

- VMs can be assigned a public IP address to connect to the internet via Floating IP addresses
- A Floating Ip is an IP address that is assigned to a VM to access an external network
- The IP address is not actually assigned to the VM, instead, NAT is performed by the router
- To assign a floating IP address go to the menu of the instance and select 'Associate Floating IP'



Di default il traffico in ingresso alle VM è disabilitato.

Per abilitarlo basta andare in: **Network->Security Groups->Manage Rules->Add Rule.**

Add Rule

Rule *

Custom TCP Rule

Description ?

Direction

Ingress

Open Port *

Port

Port * ?

Remote * ?

CIDR

CIDR * ?

0.0.0.0/0

Description:

Rules define which traffic is allowed to instances assigned to the security group. A security group rule consists of three main parts:

Rule:

You can specify the desired rule template or use custom rules, the options are Custom TCP Rule, Custom UDP Rule, or Custom ICMP Rule.

Open Port/Port Range:

For TCP and UDP rules you may choose to open either a single port or a range of ports. Selecting the "Port Range" option will provide you with space to provide both the starting and ending ports for the range. For ICMP rules you instead specify an ICMP type and code in the spaces provided.

Remote:

You must specify the source of the traffic to be allowed via this rule. You may do so either in the form of an IP address block (CIDR) or via a source group (Security Group). Selecting a security group as the source will allow any other instance in that security group access to any other instance via this rule.

Cancel

Add

Adesso viene creato il router virtuale o il gateway per connettere la rete virtuale con quella esterna, andando in: **Project -> Network -> Routers -> Create Router**.

Create Router

Router Name
internal_router

Description:
Creates a router with specified parameters.
Enable SNAT will only have an effect if an external network is set.

☒ **Enable Admin State** ⓘ

External Network
external

☒ **Enable SNAT**

Availability Zone Hints ⓘ
nova

Cancel Create Router

Il router necessita di essere connesso esplicitamente alla rete interna. Bisogna entrare nell'interfaccia tab della configurazione del router e successivamente aggiungere l'interfaccia.

Add Interface

Subnet *
Internal: 10.1.1.0/24 (internal)

IP Address (optional) ⓘ
10.1.1.1

Description:
You can connect a specified subnet to the router.
If you don't specify an IP address here, the gateway's IP address of the selected subnet will be used as the IP address of the newly created interface of the router. If the gateway's IP address is in use, you must use a different address which belongs to the selected subnet.

Cancel Submit

Project: network / Routers / internal_router

internal_router

Overview Interfaces Static Routers

+ Add Interface + Create Interface

Name	Fixed IP	Status	Type	Admin State	Actions
10.1.1.1	10.1.1.1	Active	External Gateway	UP	Configure Interface

Admin Identity

Un volume è un hard drive secondario che può essere connesso dinamicamente alla VM, bisogna andare in: **Volumes->Create volume**. Notare che il modulo del volume non è installato in Microstack.

Displaying 1 item

☐	Name	Description	Size	Status	Group	Type	Attached To	Availability Zone	Bootable	Encrypted	Actions
☐	test	-	1GiB	Error	-	__DEFAULT__	nova		No	No	Delete Volume

Displaying 1 item

Create Volume

Volume Name

Description

Volume Source

No source, empty volume

Type

__DEFAULT__

Size (GiB)

1

Availability Zone

No availability zones found

Group

No group

Description:

Volumes are block devices that can be attached to instances.

Volume Type Description:

__DEFAULT__

Default Volume Type

Volume Limits

Total Gibibytes

0 of 1,000 GiB Used

Number of Volumes

0 of 10 Used

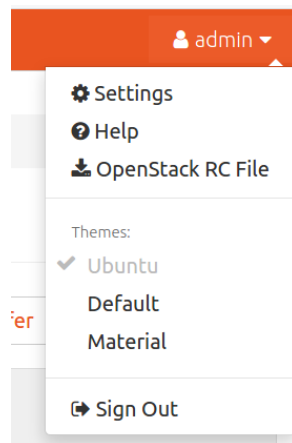
Cancel

Create Volume

OpenStack fornisce un SDK per creare applicazioni che interagiscono con la piattaforma, una versione SDK è disponibile per Python3.

Per installarlo, ad esempio nel nodo controller occorre digitare i seguenti comandi: **apt-get install python3-pip pip3 install openstacksdk**.

Scaricare il file di autenticazione e autenticare, successivamente upload del file nel controller e attivare: **source openstack.sh**.



OpenStack offre un'interfaccia a riga di comando alternativa all'interfaccia web.

Capitolo 13 - Software development

Lo sviluppo dei software di applicazioni cloud è basato sui **container** perchè in certe applicazioni sono meglio delle VM, sono meno complessi da gestire e da istanziare, introducono meno overhead, costituiscono un nuovo approccio per lo sviluppo dei software.

Approccio Monolitico

Le applicazioni software in passato erano solitamente sviluppate in modo **monolitico**: ogni funzionalità era nella stessa macchina e tutto ciò che veniva eseguito erano singoli processi o thread.

Questo approccio ha diversi problemi, uno è che questi software hanno **slow-release cycles**, ovvero per essere rilasciata un'applicazione richiedeva molto tempo di sviluppo, testing e correzione (*debugging*). In questo metodo è difficile effettuare cambiamenti, ci sono effetti a cascata, se viene modificato un modulo bisogna modificare anche altri moduli correlati affinché il sistema funzioni correttamente, questo richiede tempo.

Nell'esempio in figura sotto: un piccolo cambiamento nel cerchio genera degli effetti a cascata, creando grandi cambiamenti negli altri moduli.

Sistemi del genere sono detti **legacy system**, sono presenti ancora oggi, se è conveniente per i costi questi sistemi sono cloudificati, ovvero i server fisici sono rimpiazzati da VM.

Questo processo facilita il mantenimento dell'infrastruttura, però lo sviluppo e manutenzione del software sono ancora complicati.

Monolithic application

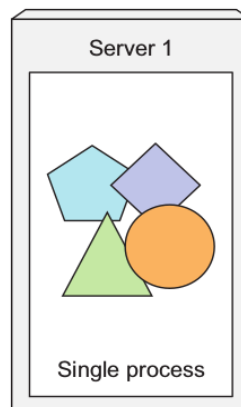


Figure 153 - Applicazione monolitica.

Approccio a servizi

Al giorno d'oggi lo sviluppo dei software non avviene più in modo monolitico, i componenti vengono separati in moduli indipendenti detti **servizi/microservizi**, uno dei vantaggi è che ognuno di essi è indipendente e viene disaccoppiato dagli altri e può essere sviluppato, distribuito, aggiornato e scalato individualmente.

Ogni modulo è gestito da un processo diverso che è eseguito in modo parallelo agli altri e comunica con interfacce ben definite.

Ogni servizio/microservizio, può essere sviluppato, aggiornato e scalato indipendentemente anche su server differenti, questo grazie alle interfacce esposte da ogni modulo, queste interfacce sono implementate tramite scambio di messaggi, possono essere scambiati internamente o tramite una LAN.

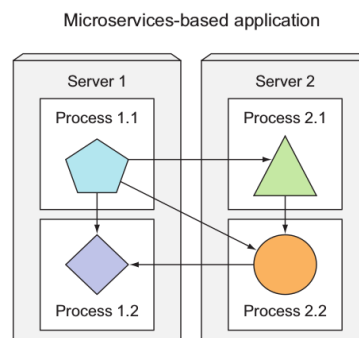


Figure 154 - Applicazione basata su microservizi.

Indipendenza

Ogni componente può essere distribuito o implementato fornito in un modo differente, ovvero ha il proprio ciclo di vita autonomo.

Isolamento

Cambiamenti in un modulo non provoca cambiamenti in altri moduli. Questo approccio ha cambiato l'intero ciclo di vita di un'applicazione, in passato il ciclo di vita dell'applicazione passava tra due team.

Team dev

Il primo era il **development team** incaricato di progettare l'applicazione e implementare i componenti.

Team ops

Il secondo era l'**operations team**, il quale era incaricato di mantenere un'infrastruttura in cui far girare il software, prendere i packages forniti dal development team e inserire i packages nell'infrastruttura (deployment).

Dopo il deployment bisogna verificare che l'applicazione funzioni correttamente.

DevOps

DevOps è una cultura e una serie di pratiche che integra i team di sviluppo (*Development*) e operativi (*Operations*) per migliorare la collaborazione dei processi di sviluppo e distribuzione del software.

Il team development avrà nuovi incarichi che richiedono anche il deployment e manutenzione dell'infrastruttura.

Ossia il team di development fa anche quello che prima faceva il team ops.

Il problema di questo metodo consiste nell'avere sviluppatori bravi non solo a programmare, ma che abbiano le competenze per creare e mantenere l'infrastruttura, non è fattibile nella realtà.

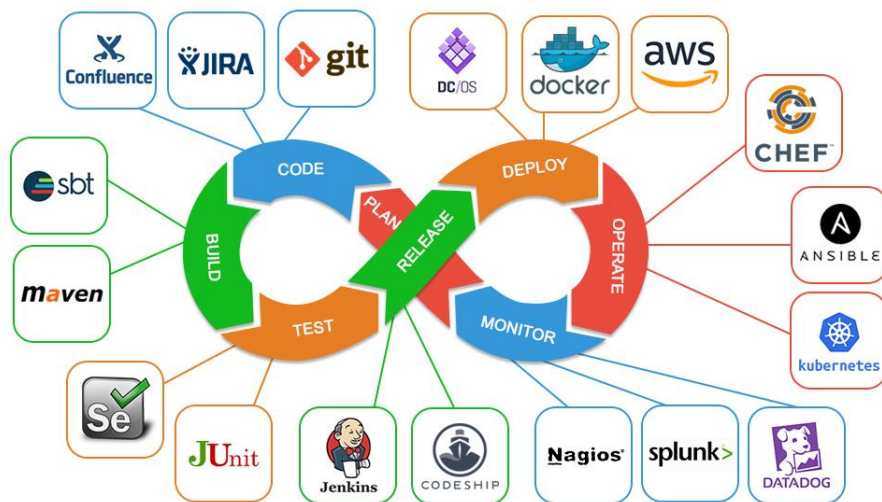


Figure 10155 - Esempi di software e frameworks usati in DevOps.

NoOps

NoOps è un'evoluzione di DevOps che mira ad eliminare la necessità di un team operativo attraverso l'automazione completa delle infrastrutture e dei processi operativi.

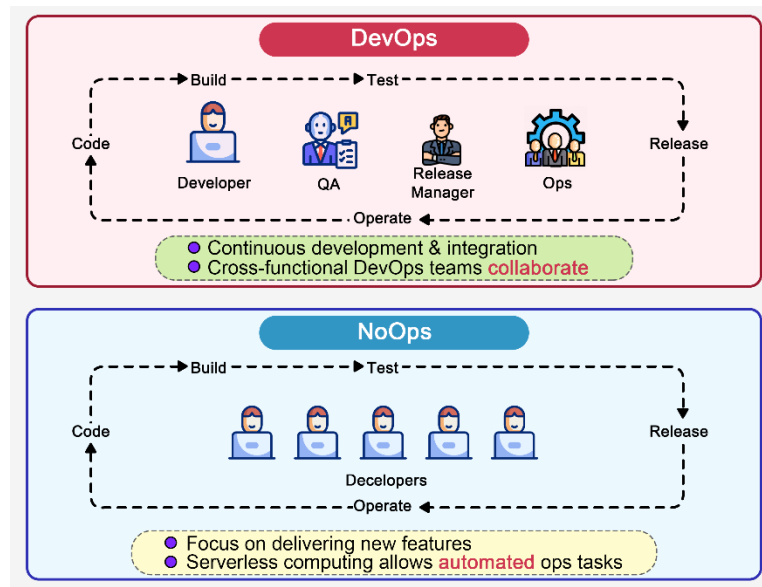


Figure 10256 - Differenza tra DevOps e NoOps.

Gli obiettivi principali del NoOps sono:

- **Automazione completa:** Utilizzo intensivo di strumenti di automazione per gestire l'infrastruttura, il deployment, la scalabilità e il monitoraggio senza intervento umano.
- **Serverless e cloud computing:** Sfruttamento di architetture serverless e servizi cloud per ridurre la necessità di gestire server fisici e configurazioni.
- **Auto-riparazione e auto-scalabilità:** Implementazione di sistemi che possano auto configurarsi, auto-ripararsi e auto-scalarsi in base alle esigenze delle applicazioni.
- **Focus sullo sviluppo:** Permettere agli sviluppatori di concentrarsi esclusivamente sullo sviluppo del codice, mentre l'infrastruttura e le operazioni vengono gestite automaticamente.

In NoOps il team dev ha molti meno incarichi, in quanto la gestione dell'infrastruttura è del tutto automatizzata (*Kubernetes*). L'operations team non è rimosso, adesso crea e mantiene l'infrastruttura non si concentrano più sul deployment, il quale è effettuato dal development team in modo automatizzato.

Service deployment and management

NoOps punta ad automatizzare il deployment delle applicazioni, in modo da migliorare il deployment stesso e riducendone il costo. Soluzione obbligata nei casi in cui servizi sono tanti. Automatizzare il deployment riduce anche gli errori migliorando la qualità del deployment.

Service dependencies

Per automatizzare il deployment bisogna risolvere il problema delle dipendenze, a questo scopo è necessario utilizzare la lightweight virtualization (*containers*).

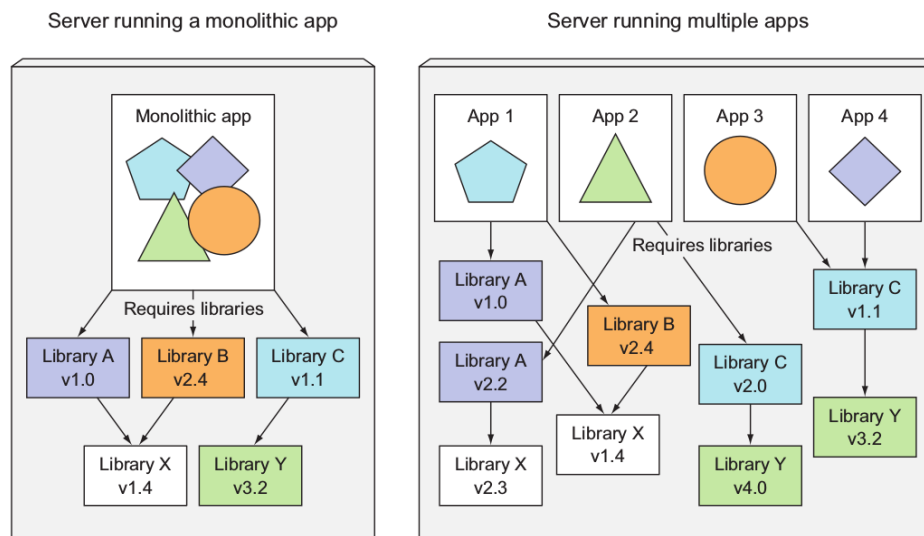


Figure 10357 – Gestione delle dipendenze tra una applicazione monolitica (sinistra) e una non monolitica (destra).

I moduli dell'applicazione utilizzeranno delle librerie per implementare delle funzionalità; queste librerie sono chiamate “dependencies”.

Le librerie hanno varie versioni che differiscono tra loro e due moduli diversi potrebbero usare due versioni diverse della stessa libreria, ed è impossibile avere due versioni diverse della stessa libreria installate nello stesso sistema, in questi casi ci sono dei conflitti.

Avere dei moduli separati (*a destra nella figura sopra*), ognuno con le sue dipendenze installate in modo separato dagli altri moduli elimina i conflitti; poiché ogni applicazione è eseguita in un ambiente virtualizzato separato.

Separazione in moduli risolve i conflitti delle librerie e aiuta il deployment del servizio utilizzando full virtualization, questo però ha un overhead significativo che viene moltiplicato con tutti i servizi dell'applicazione, inoltre ogni componente ha lo stesso OS con stesse configurazioni e versione, questo non è efficiente.

Questo problema è risolto con lightweight virtualization come i container, i quali garantiscono overhead minimo eliminando gli svantaggi delle VM.

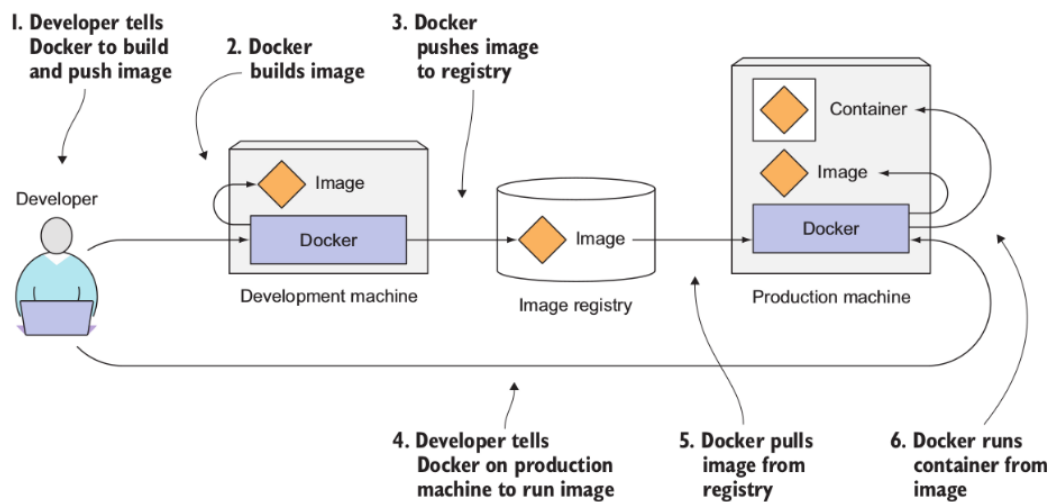


Figure 10458 - Utilizzo della lightweight virtualization in NoOps.

Kubernetes

Kubernetes è una piattaforma open source per l'orchestrazione dei container, inizialmente sviluppata da Google e ora mantenuta dalla Cloud Native Computing Foundation (CNCF).

Kubernetes automatizza il deployment, la gestione, la scalabilità e il networking delle applicazioni containerizzate.

Lo scopo è quello di offrire un'infrastruttura IaaS lightweight per effettuare il deployment di applicazioni in un modo semplice ed efficiente.

Kubernetes automatizza anche la fase di gestione dopo il deployment; quella in cui serve assicurarsi che l'applicazione e tutti i suoi servizi siano correttamente in esecuzione. Il ciclo di vita ed il tempo di rilascio dell'applicazione sono ridotti.

Utilizzo di Kubernetes

Un nodo Kubernetes è un software che può essere **master** o **worker**. In ogni caso il nodo è installato direttamente sul server fisico oppure può essere installato su una macchina virtuale creata tramite una piattaforma cloud come OpenStack (opzione più popolare).

Sopra Kubernetes è possibile creare applicazioni containerizzate.

Il master espone un'interfaccia agli sviluppatori con la quale è possibile richiedere di eseguire una certa applicazione.

Lo sviluppatore deve inserire un descrittore dell'applicazione (*app descriptor*) che include la lista delle componenti dell'applicazione e anche le configurazioni di ogni componente,

contiene anche il grado di replicazione di ogni componente, ovvero il numero delle istanze richieste (*per assicurare scalabilità*).

Ogni componente ha una configurazione specificata nel descrittore dell'applicazione.

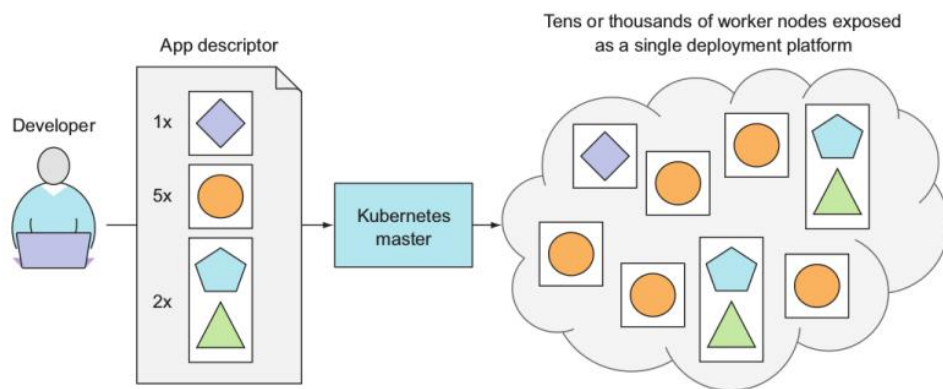


Figure 10559 - Deployment di un applicazione con Kubernetes.

Il nodo master è responsabile di ricevere l'ap descriptor e schierare i componenti su un nodo worker seguendo la descrizione data dallo sviluppatore in modo automatico.

I nodi worker (*installati su una macchina virtuale o su una macchina fisica*) ospiteranno le applicazioni in esecuzione.

Nell'esempio in figura sopra abbiamo un'applicazione composta da 4 componenti (*rombo, cerchio, pentagono, triangolo*) replicati rispettivamente 1, 5, 2, 2 volte.

Il master effettua il deployment delle istanze richieste. Dopo il deployment, il sistema deve monitorare ogni componente e vedere se uno di essi fallisce, se una componente fallisce allora quella replica viene distrutta e ne viene creata una nuova.

Se abbiamo un componente replicato n volte, il sistema gestirà il load balance per le richieste tra questi n componenti. A runtime il sistema monitora se un componente fallisce.

Vantaggi

- Gli sviluppatori non devono implementare funzionalità relative all'interfaccia nella loro applicazione.
- La piattaforma si occupa di implementare scalabilità, load-balancing, self-healing.
- I developer possono concentrarsi solo sullo sviluppo dell'applicazione senza perdere tempo cercando di integrare l'applicazione con l'infrastruttura.
- Il lavoro del team ops è alleggerito.
- Kubernetes consente di raggiungere un'utilizzazione migliore delle risorse.

Kubernetes Cluster Architecture

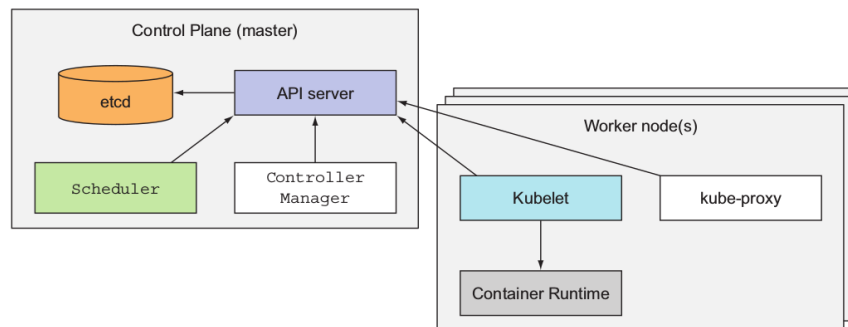


Figure 10660 – Diagramma che mostra l'architettura di un cluster con Kubernetes.

Master node

Il master node è responsabile della gestione dell'intera infrastruttura, se vogliamo avere una maggiore tolleranza ai fallimenti possiamo avere più master node replicati.

Il master node ha un processo chiamato “*server API*”, il quale espone le API agli sviluppatori; tramite queste API gli sviluppatori possono interagire con l'infrastruttura creata da Kubernetes.

Il master node possiede un modulo scheduler che è responsabile di scegliere i nodi worker per il deployment delle applicazioni. La politica di scheduling non è standard.

Infine, Etcd è una memoria secondaria distribuita ed affidabile.

Worker node

Abbiamo diversi worker node che ospitano i container (*Container Runtime*); di solito creati con Docker o altri ambienti di esecuzione dei container presenti nel mercato.

Il worker node ha un componente detto Kubelet, equivalente all'agent in OpenStack. Kubelet è il modulo che comunica con la API del master; quindi, riceve le richieste dal master ed applica i comandi ricevuti.

Un altro componente importante è il kube-proxy che è responsabile della gestione del traffico di rete tra i componenti delle applicazioni (*essenzialmente è uno switch di rete per i componenti delle applicazioni*), tiene anche conto del fatto che i container possono essere spostati tra i vari working node.

Kubernetes - Application Deployment

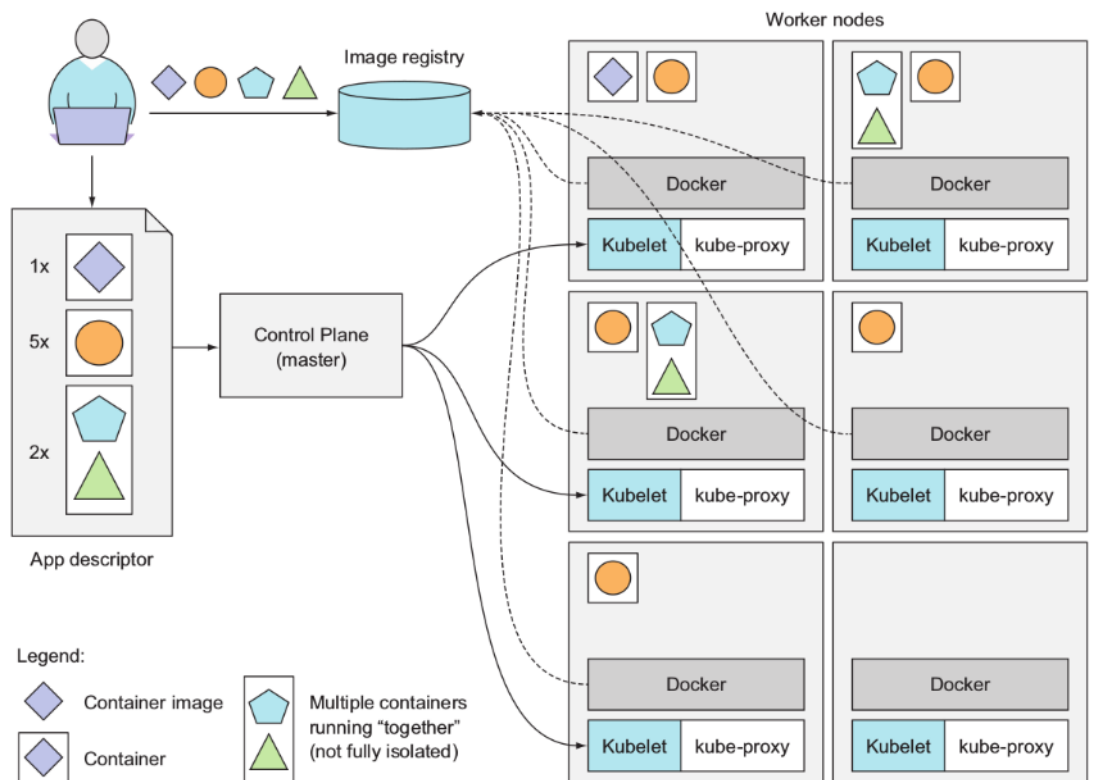


Figure 10761 – Diagramma che mostra il deployment di una applicazione con Kubernetes.

1. Il developer tramite la API invia l'app descriptor al master.
2. Lo scheduler decide su quali worker schierare il pod le sue repliche.
3. Il master traduce l'app descriptor in un insieme di comandi.
4. Ogni comando verrà inviato al componente Kubelet di ogni worker scelto.
5. Kubelet traduce i comandi in un insieme di comandi per il motore del container locale.

Le informazioni che riguardano il deployment di un pod sono contenute in un file yaml.

```

deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: helloworld
spec:
  selector:
    matchLabels:
      run: helloworld
  replicas: 2
  template:
    metadata:
      labels:
        run: helloworld
    spec:
      containers:
        - name: helloworld
          image: luksa/kubia
          ports:
            - containerPort: 8080

```

Number of replicas to be instantiated

Figure 10862 - Informazioni sul deployment.

Pod

All'interno dell'app descriptor i container possono essere raggruppati in pod, Kubernetes vuole l'app descriptor composta da una lista di pod. Un pod è un gruppo di container che devono essere per forza eseguiti nello stesso nodo worker. Un pod può anche essere costituito da un solo container.

Uno dei motivi per volere dei container nello stesso nodo sono le prestazioni, in questo caso i pod sono utilizzati per raggruppare container che necessitano di una comunicazione veloce tra loro. Avere i container che devono comunicare velocemente in nodi diversi non è efficiente.

Per ogni pod lo sviluppatore specifica la configurazione e il numero di repliche che devono essere schierate.

Non ci sono regole per raggruppare i pod, i container dovrebbero essere eseguiti in pod separati a meno che non ci siano ragioni per cui dovrebbero essere inseriti nello stesso pod, ad esempio nel caso in cui vogliamo avere comunicazioni veloci.

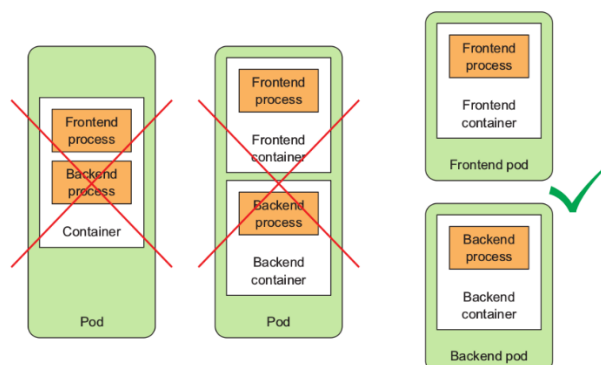
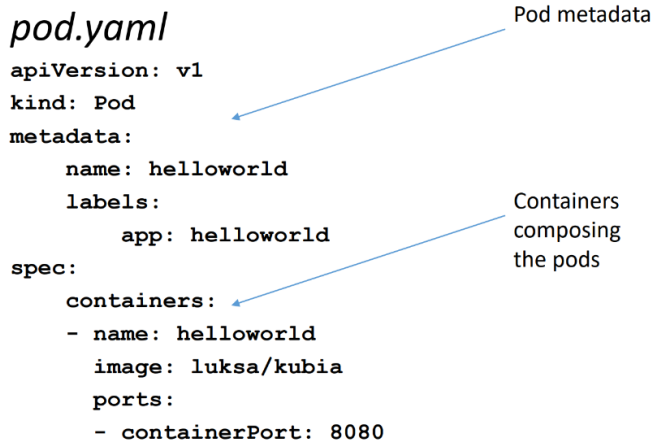


Figure 10963 - Regola generale per progettare i pod.

Un pod è descritto tramite un file yaml.



```

pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: helloworld
  labels:
    app: helloworld
spec:
  containers:
  - name: helloworld
    image: luksa/kubia
    ports:
    - containerPort: 8080
  
```

Figure 11064 - Esempio di pod descriptor.

App descriptor

L'app descriptor è una lista di componenti o, meglio, è una lista di container con il loro grado di replicazione.

Nella descrizione c'è anche l'immagine da prelevare e da mettere nel container.

Kubernetes - Gestione e manutenzione

Manutenzione

La manutenzione è estremamente semplificata, gli sviluppatori non devono conoscere niente dell'infrastruttura poiché Kubernetes aiuta ad automatizzare anche la fase di manutenzione: il master controlla continuamente se i container falliscono, in questo caso un nuovo container viene creato e quello non funzionante viene eliminato. Se un nodo fallisce, un insieme di container falliscono, quindi devono essere creati su un nuovo nodo.

Anche il lavoro dell'operations team è alleggerito, in quanto mantengono solo l'infrastruttura su cui Kubernetes viene eseguito.

Ottimizzazione delle risorse

In Kubernetes avviene anche l'ottimizzazione delle risorse, l'assegnazione di un pod ad uno worker si basa sulle risorse che il nodo worker ha disponibili. In casi particolari (se lo stato del carico del nodo cambia), è anche possibile inoltre spostare dei container tra uno worker ed un altro.

Gestione dei fallimenti

Se un nodo worker fallisce. Allora vengono create nuove istanze dei pod che erano schierati sul nodo fallito.

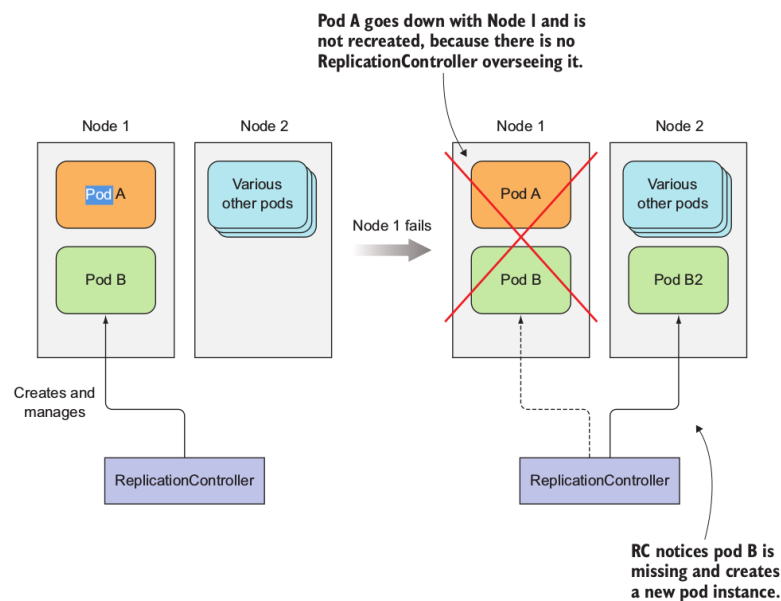


Figure 11165 - Gestione di un fallimento di un worker node in Kubernetes.

Networking tra container

Ogni pod ha il suo indirizzo IP, tale indirizzo è interno alla rete e non è accessibile dall'esterno. La rete locale privata consente la comunicazione tra container in esecuzione sullo stesso worker o su worker diversi.

Un pod è come una rete privata a parte, serve un servizio NAT per comunicarci dall'esterno.

Il pod può avere un unico indirizzo IP pubblico e gli utenti esterni possono interagirci, la seguente interazione è gestita dal componente kube-proxy del worker node il quale controlla la configurazione relativa al traffico consentito in quel pod. Il servizio inoltra traffico proveniente da un IP esterno verso un IP locale di una delle istanze pod associate.

Se abbiamo istanze multiple del servizio frontend viene effettuato anche il load balancing per bilanciare le richieste che arrivano dall'esterno verso il gruppo dei container che implementano il servizio di frontend.

Il Kube-proxy è incaricato quindi di ricevere il traffico diretto verso il container, applicare una politica di load balancing se necessario e consentire l'uscita del traffico dal container.

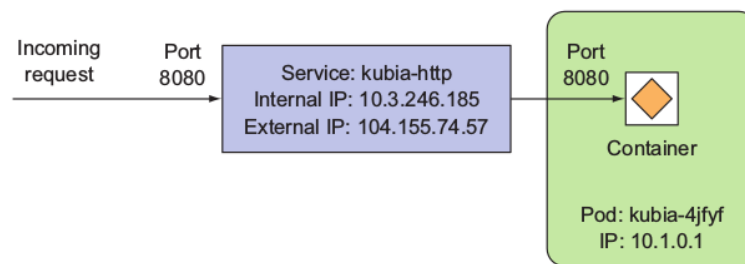


Figure 11266 - Funzionalità del Kube-proxy.

Load balancing

Supponendo che un container è replicato tre volte all'interno di un nodo worker; il mio obiettivo è smistare le richieste tra queste tre repliche in modo adeguato. Ogni volta che una richiesta arriva al nodo worker, viene inoltrata verso uno dei container in base al carico corrente.

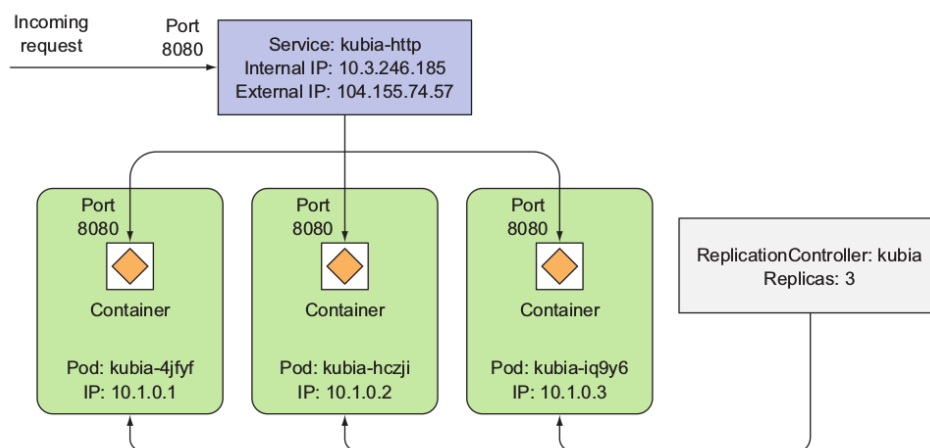


Figure 11367 - Load balancing del Kube-proxy.

Autoscale

L'autoscaling in Kubernetes è una funzionalità che permette di scalare automaticamente le risorse di un cluster Kubernetes, come i Pod, basandosi su metriche predefinite o condizioni specifiche. Ci sono tre principali tipi di autoscaling in Kubernetes:

Horizontal Pod Autoscaler (HPA)

Aggiunge o rimuove Pod in base alla necessità (*carico*). Utilizza metriche come l'utilizzo della CPU, memoria, o metriche personalizzate provenienti da applicazioni.

1. Definisci un oggetto HPA che specifica le soglie di utilizzo e la scala desiderata.
2. Il controller HPA monitora costantemente le metriche specificate.

3. Se le metriche superano le soglie predefinite (*ad esempio l'utilizzazione della CPU aumenta una certa soglia*), HPA incrementa il numero di Pod per distribuire meglio il carico di richieste.
4. Se le metriche scendono sotto le soglie predefinite, HPA decrementa il numero di Pod.

Vertical Pod Autoscaler (VPA)

Adatta le risorse assegnate ai singoli Pod (*CPU e memoria*) in base ai loro bisogni. Analizza l'utilizzo delle risorse storiche e attuali.

1. Definisci un oggetto VPA che specifica le raccomandazioni di risorse.
2. Il VPA controller monitora l'utilizzo delle risorse dei Pod.
3. Se un Pod ha bisogno di più risorse, VPA raccomanda una nuova configurazione di risorse.
4. Il Pod viene eventualmente riavviato con le nuove risorse configurate (*può causare downtime se non gestito correttamente*).

Cluster Autoscaler

Aggiunge o rimuove nodi nel cluster in base al numero di Pod in esecuzione e alla capacità del cluster. Analizza il numero di Pod in attesa di essere schedulati e l'utilizzo delle risorse dei nodi.

1. Monitora costantemente i Pod che non possono essere schedulati a causa della mancanza di risorse.
2. Se ci sono Pod in attesa, aggiunge nodi al cluster.
3. Se i nodi sono sottoutilizzati e possono essere rimossi senza impatto sui Pod esistenti, li rimuove.

LAB – Kubernetes

Per installare Kubernetes un modo è quello di installarlo tramite juju utilizzando i comandi sotto.

- Download a standard bundle and customize

```
wget https://github.com/charmed-kubernetes/bundle/blob/main/releases/1.27/bundle.yaml
```

- Deploy the bundle specifying where to install the controller node (node 10) and one worker (node 11)

```
juju deploy ./bundle.yaml --map-machines=existing,0=10,1=11
```

- If you have more than one worker extend the role to other machines (node 12 in this case)

```
juju add-unit --to 12 kubernetes-worker
```

Tutorial

<https://jaas.ai/kubernetes-core>

Controllare se docker funziona con il comando *docker ps*, se non funziona reinstallarlo:

- Install some pre-requisites

```
sudo apt install apt-transport-https ca-certificates curl  
software-properties-common
```

- Add the key of the official docker repository in the system

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg |  
sudo apt-key add -
```

- Add the Docker repository

```
sudo add-apt-repository "deb [arch=amd64]  
https://download.docker.com/linux/ubuntu jammy stable"
```

- Install Docker

```
sudo apt install docker-ce
```

Installare Minikube, il quale è un tool di test in cui un'installazione di Kubernetes è eseguita in un singolo ambiente:

- Download minikube

```
curl -Lo minikube  
https://storage.googleapis.com/minikube/releases/latest/minikube-  
linux-amd64 \ && chmod +x minikube
```

```
mkdir -p /usr/local/bin/  
install minikube /usr/local/bin/
```

Installare alcune dependency:

```

apt-get install conntrack

wget https://github.com/Mirantis/cni-dockerd/releases/download/v0.3.1/cni-dockerd_0.3.1.3-0.ubuntu-jammy_amd64.deb
dpkg -i cni-dockerd_0.3.1.3-0.ubuntu-jammy_amd64.deb

wget https://github.com/kubernetes-sigs/cni-tools/releases/download/v1.27.0/cni-v1.27.0-linux-amd64.tar.gz
tar xvfz cni-v1.27.0-linux-amd64.tar.gz
install cni /usr/local/bin/

wget https://github.com/containernetworking/plugins/releases/download/v1.2.0/cni-plugins-linux-amd64-v1.2.0.tgz
mkdir -p /opt/cni/bin
tar xvfz cni-plugins-linux-amd64-v1.2.0.tgz --directory /opt/cni/bin

```

Installare l'ambiente Minikube con il comando *minikube start --driver=none*.
Controllare lo stato di Minikube con *minikube status*.

Per interagire con il cluster eseguiamo i seguenti comandi:

- **Install kubectl**

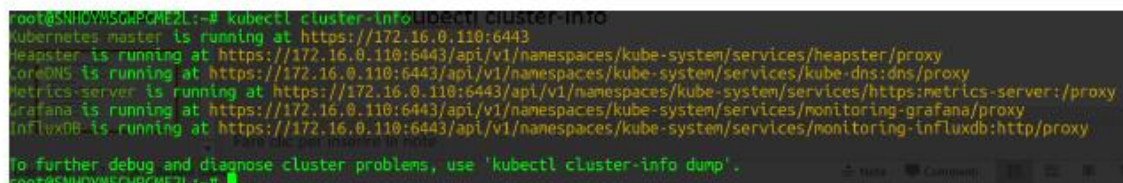
```

apt install snapd
snap install kubectl --classic

```

- **Query the cluster.**

```
kubectl cluster-info
```



```

root@SNHCYMSQPCME2L:~# kubectl cluster-info
Kubernetes master is running at https://172.16.0.110:6443
Heapster is running at https://172.16.0.110:6443/api/v1/namespaces/kube-system/services/heapster/proxy
CoreDNS is running at https://172.16.0.110:6443/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy
Metrics-server is running at https://172.16.0.110:6443/api/v1/namespaces/kube-system/services/https:metrics-server:/proxy
Grafana is running at https://172.16.0.110:6443/api/v1/namespaces/kube-system/services/monitoring-grafana/proxy
InfluxDB is running at https://172.16.0.110:6443/api/v1/namespaces/kube-system/services/monitoring-influxdb:http/proxy
To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.

```

Eeguire il comando *kubectl get nodes* per vedere i nodi.

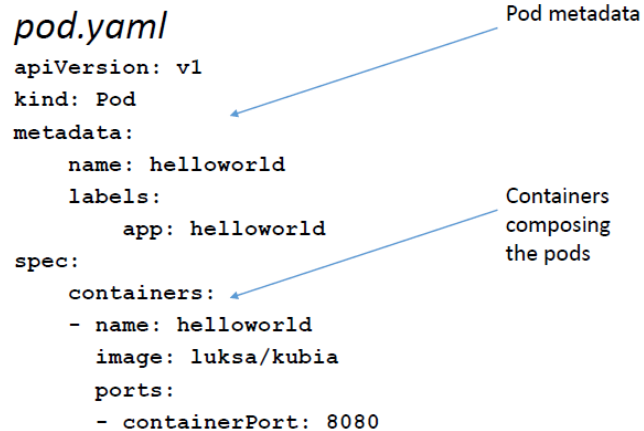
Attivare alcuni plugin:

- Comando per abilitare il collezionamento delle metriche/statistiche: *minikube addons enable metrics-server*.
- Comando per abilitare il load balancer: *minikube addons enable metallb*.

Configurare il metallb:

```
minikube addons configure metallb
```

- Adesso come prima cosa definiamo un Pod, il quale viene definito tramite un file di configurazione `.yaml` contenente i metadati e una lista dei container che compongono il pod.



The diagram shows a `pod.yaml` file with the following content:

```
pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: helloworld
  labels:
    app: helloworld
spec:
  containers:
  - name: helloworld
    image: luksa/kubia
    ports:
    - containerPort: 8080
```

Annotations with arrows:

- Pod metadata** points to the `metadata:` section.
- Containers composing the pods** points to the `containers:` list in the `spec:` section.

- Istanziamo il pod hello world con il comando `kubectl apply -f pod.yaml`.
- Controlliamo se il pod funziona con il comando `kubectl get pods`.
- Se vogliamo eliminare il pod utilizziamo il comando `kubectl delete pod helloworld`.

Il prossimo step riguarda il deployment di un'applicazione con repliche, un deployment è una lista di pod, ogni pod deve avere un certo numero di repliche in base al caso. Nel file `.yaml` abbiamo nella sezione `spec` la lista di pod con il numero di repliche per ognuno.

```

deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: helloworld
spec:
  selector:
    matchLabels:
      run: helloworld
  replicas: 2
  template:
    metadata:
      labels:
        run: helloworld
    spec:
      containers:
        - name: helloworld
          image: luksa/kubia
          ports:
            - containerPort: 8080

```

Number of replicas to be instantiated

- Istanziamo il deployment con il comando *kubectl apply -f deployment.yaml*.
- Controlliamo se il deployment funziona con il comando *kubectl get deployments*.
- Eseguendo nuovamente *kubectl get pods* vediamo che la replicazione funziona in automatico, adesso abbiamo due repliche del pod hello world.
- Possiamo provare ad eliminare uno dei due pod con il comando *kubectl delete pod helloworld-958544d5c-lrjww*, in questo caso il sistema rileva che una replica del pod è mancante; quindi, in automatico ne crea una nuova in un nuovo container con un nuovo nome.
- Nel prossimo step vogliamo accedere al container, abbiamo un esempio di un server che riceve una richiesta HTTP e restituisce una risposta con il nome del container che ha servito la richiesta.

```

1  const http = require('http');
2  const os = require('os');
3
4  console.log("Kubia server starting...");
5
6  var handler = function(request, response) {
7    console.log("Received request from " + request.connection.remoteAddress);
8    response.writeHead(200);
9    response.end("You've hit " + os.hostname() + "\n");
10 };
11
12 var www = http.createServer(handler);
13 www.listen(8080);

```

- il servizio di default è nascosto all'esterno, per esporlo bisogna eseguire il comando *kubectl expose deployment/helloworld* per esporre un servizio, un

servizio è un deployment che ha almeno un container abilitato a ricevere richieste dall'esterno.

- Con questo comando viene creato un nuovo servizio *kubectl get services*.
- Il servizio è accessibile tramite endpoint, per vedere l'endpoint assegnato al servizio eseguiamo *kubectl describe service helloworld*. Con questo comando il servizio è esposto tramite due endpoint, sono i due indirizzi IP assegnati dei due container istanziati per il pod hello world.

I due indirizzi IP possono essere acceduti nella VN che connette tutti i container insieme.

- Eseguendo il comando *curl* per entrambi gli endpoint, vengono restituiti i nomi dei due container.

```
root@SNUMUYKY62JPXH:~# curl http://10.1.43.28:8080
You've hit helloworld-958544d5c-m55fv
root@SNUMUYKY62JPXH:~# curl http://10.1.76.22:8080
You've hit helloworld-958544d5c-lhckq
root@SNUMUYKY62JPXH:~#
```

Il metodo ClusterIP crea un indirizzo IP virtuale per ogni pod, ogni tier dell'applicazione può avere un IP virtuale al quale le richieste possono essere dirette.

- Eseguendo il comando *curl <http://ClusterIP:8080>* è possibile osservare quale container soddisfa le richieste, ogni volta è diverso.

Kubernetes internamente ha anche un servizio DNS, invece di utilizzare l'indirizzo virtuale ClusterIP, i pod possono utilizzare i nomi DNS

- Per controllare se il DNS è attivo utilizziamo il comando *kubectl get services kube-dns --namespace=kube-system*.
- Per testare l'alias DNS utilizziamo il comando *kubectl run curl --image=radial/busyboxplus:curl -i --tty* per eseguire un nuovo pod.
- Controlliamo il DNS con *nslookup helloworld*.
- Per esporre il servizio nelle reti esterne ci sono due metodi, prima di vederli eliminiamo il servizio con *kubectl delete service helloworld*.

Bisogna eliminare il servizio perchè ogni pod può avere un solo tipo di servizio e un solo tipo di esposizione, dopo aver eliminato il servizio hello world, ne creiamo un altro di un altro tipo, *NodePort* che consente di esporre un servizio utilizzando l'indirizzo IP pubblico del controller dell'installazione Kubernetes.

Quindi le due soluzioni sono assegnare l'indirizzo IP pubblico direttamente al container, oppure sfruttare l'indirizzo IP pubblico che ha il controllore Kubernetes per instradare le richieste dalla rete esterna ad uno dei servizi che vogliamo esporre.

NodePort configura un port forwarding nel controller Kubernetes, quindi le richieste che arrivano ad una specifica porta del controller sono inoltrate ad uno degli endpoint d'istanze differenti della nostra applicazione esposta.

- Per esporre il servizio a sistemi esterni eseguiamo il seguente comando *kubectl expose deployment/ helloworld --type=NodePort*.
- Eseguendo il comando *kubectl get services* notiamo una porta che viene selezionata randomicamente, è quella configurata per il port forwarding dell'indirizzo IP pubblico del controller Kubernetes.
- Per testare bisogna effettuare il comando *curl* con l'indirizzo IP della VM e come porta quella selezionata randomicamente.

```
root@SNHOYM5GWPGME2L:~/kubia# curl http://172.16.0.110:31841
You've hit helloworld-958544d5c-lhckq
root@SNHOYM5GWPGME2L:~/kubia# curl http://172.16.0.110:31841
You've hit helloworld-958544d5c-lhckq
root@SNHOYM5GWPGME2L:~/kubia# curl http://172.16.0.110:31841
You've hit helloworld-958544d5c-lhckq
root@SNHOYM5GWPGME2L:~/kubia# curl http://172.16.0.109:31841
You've hit helloworld-958544d5c-m55fv
```

Come esponiamo la nostra applicazione con un indirizzo IP pubblico reale, abbiamo un terzo metodo per esporre un servizio, il load balancer.

- Per prima cosa eliminiamo la configurazione per esporre l'applicazione con *kubectl delete service helloworld*.
- Esponiamo nuovamente il deployment hello wordl, però questa volta il tipo è load balancer che vuole anche la porta che è aperta per rimuovere ogni firewall/regola per il traffico in ingresso, utilizziamo il comando *kubectl expose deploy helloworld --port 8080 --type LoadBalancer*.
- Questo comando assegna un indirizzo IP pubblico dal pool d'indirizzi che abbiamo assegnato con il comando *minikube addons configure metallb*.
- Quello che abbiamo adesso eseguendo il *get services* ed il *curl* è in figura.

```
root@SNHOYM5GWPGME2L:~# kubectl get services
NAME          TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
helloworld    LoadBalancer 10.152.183.145 172.16.1.1     8080:32699/TCP   10m
kubernetes    ClusterIP     10.152.183.1  <none>         443/TCP          13d
```

```
root@SNHOYM5GWPGME2L:~# curl http://172.16.1.1:8080
You've hit helloworld-958544d5c-m55fv
```

Adesso implementiamo l'autoscaling, opzione in Kubernetes che scala il sistema automaticamente.

- Inizialmente stoppiamo Minikube con il comando *minikube stop*.
- Eseguiamo il seguente comando *minikube start --extra-config=kubelet.housekeeping-interval=10s*.

- Modifichiamo il file `.yaml` per l'horizontal autoscaling per il nostro deployment hello world, in realtà non è adatto per il deployment hello world in quanto troppo leggero. Quindi creiamo un'altra applicazione in PHP che calcola la radice quadrata. Notiamo una nuova sezione `resources` in cui viene ridotta la quantità di risorse disponibili per l'esecuzione del container.

`autoscale.yaml`

```
apiVersion: apps/v1      spec:
kind: Deployment          containers:                kubectl apply -f autoscale.yaml
metadata:                 - name: php-apache
  name: php-apache        image: k8s.gcr.io/hpa-example
spec:                     ports:
  selector:               - containerPort: 80
    matchLabels:          resources:
      run: php-apache      limits:
replicas: 2               cpu: "500m"
template:                 requests:
  metadata:               cpu: "200m"
    labels:
      run: helloworld
```

```
<?php
$x = 0.0001;
for ($i = 0; $i <= 1000000; $i++) {
    $x += sqrt($x);
}
echo "OK!";
?>
```

- Effettuiamo il deployment dell'autoscaler e lo assegniamo all'applicazione php-apache con il comando `kubectl autoscale deployment php-apache --cpu-percent=50 --min=1 --max=10`. Il 50% per la CPU significa che se viene superato, vengono create nuove istanze, se siamo sotto, vengono distrutte istanze, min indica il numero minimo d'istanze che dobbiamo avere, mentre max è il massimo.
- Esponiamo il servizio con il comando `kubectl expose deployment/php-apache --type=ClusterIP`.
- Apriamo un nuovo terminale e lo connettiamo alla VM ed eseguiamo `kubectl get services` per ricavare il ClusterIP del nostro nuovo servizio.
- Creiamo un loop infinito in cui richiamiamo il servizio continuamente con il seguente comando in bash `while true; do wget -q -O- http://ClusterIP; done`.
- Nell'altro terminale osserviamo lo stato corrente che comprende CPU target, MINPODS, MAXPODS ed il numero corrente di repliche con il comando `kubectl get hpa`.

Capitolo 14 - Cloud Storage

Negli ultimi anni le tecnologie di memorizzazione si sono evolute in modo incredibilmente veloce sia in capacità che in velocità di trasferimento: in particolare le tecnologie di memorizzazione secondaria.

La transizione allo stato solido (*detti anche SSD*) ha permesso un aumento incredibile della velocità di accesso (*di circa dieci volte!*); ma con un aumento dei costi non indifferente.

Motivazioni

Il cloud storage è stato introdotto per le limitazioni dei compute nodes: affidarsi agli hard drive dei compute node non è una soluzione scalabile perché le macchine virtuali devono poter essere migrate da un compute node ad un altro nel modo più semplice (*live migration*); trasferire anche il contenuto dell'hard drive appesantisce molto il trasferimento, perché potrebbe richiedere diversi minuti. Ma un cloud storage accessibile a tutti i compute node permette di non trasferire il contenuto dell'hard drive e quindi rendere molto più leggera la migrazione.

Block storage

Il block storage è il metodo più semplice di memorizzazione permanente, inoltre è anche la strategia più usata nei personal computer. I byte sono visti come stringhe di bit e organizzati in blocchi (*detti anche record*).

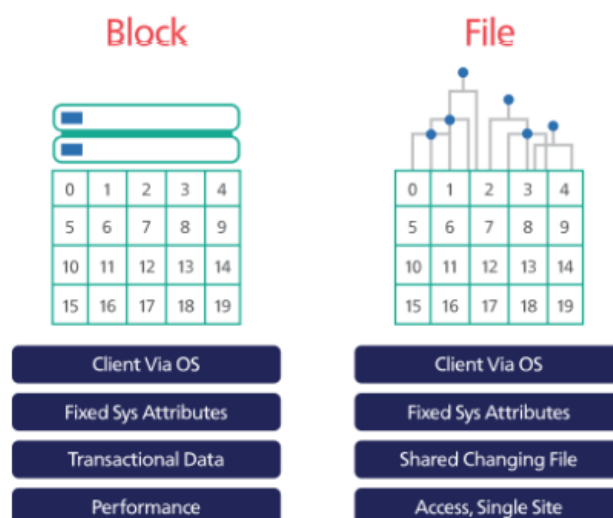


Figure 11468 - Differenze tra una visione a blocchi e una visione a file.

File System

Al di sopra di questi set di blocchi c'è un file system; ossia un software che crea un'astrazione più adatta all'utilizzo umano.

Il file system organizza questi dati in file e gestisce i metadata per memorizzare e recuperare file su diversi blocchi.

Object storage

Tecnica introdotta di recente e molto adottata dalle tecnologie cloud. Gli oggetti sono “*bundled data*” ossia dati che hanno senso solo se stanno insieme (*ad esempio un file*) con i corrispondenti metadati.

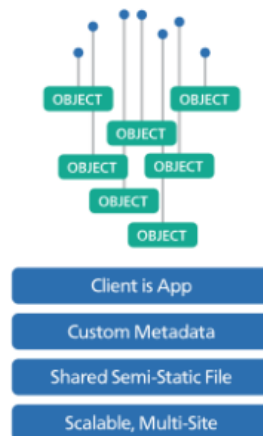


Figure 11569 - Object storage.

Ogni oggetto ha un identificatore univoco calcolato con una certa politica; le applicazioni accedono agli oggetti utilizzando l'identificatore.

Immutabilità

Ogni oggetto memorizzato è immutabile: **qualsiasi** cambiamento produce un'altra versione dell'oggetto in questione e quindi viene memorizzato ex-novo sovrascrivendo quello precedente.

RAID – Redundant Array of Independent Disks

Nel paradigma classico: tutto è memorizzato in una singola macchina server e l'applicazione è in esecuzione sullo stesso server o su un altro.

Il problema di questo modello è dato dal fatto che gli hard drive possono guastarsi: principalmente a causa dell'età (*gli SSD durano circa cinque anni, invece gli HDD sono molto più duraturi*) e ciò può causare la perdita totale dei dati se non replicati. Serve un meccanismo di replicazione automatico e veloce per garantire il recupero dei dati. Per questo motivo è stato introdotto RAID.

RAID è una tecnica nel quale i dati sono replicati in modo automatico su diversi hard drive, seguendo uno tra molti set di schemi che si possono scegliere in base ai nostri requisiti.

Serve a garantire buone prestazioni oppure aggiungere ridondanza, ovviamente questa tecnica è limitata dalla quantità di hard drive installati in un singolo server.

Più è rigorosa la politica di RAID e meno spazio di memorizzazione sarà disponibile ma avrò più garanzie. Ogni schema ha il suo rate ridondanza-capacità disponibile, sta a noi progettisti scegliere il più adatto.

Le tecniche di RAID sono di solito implementate in hardware (*velocissimo e trasparente*) tramite un RAID Controller che si occupa di replicare i dati su diversi hard drive. A volte RAID può essere implementato dal sistema operativo stesso.

RAID 0

Punta ad avere le migliori performance possibili, ossia minimo tempo di lettura e massima capacità disponibile. Supponendo di avere due dischi: I dati sono divisi in questi due dischi senza essere replicati e senza avere bit di parità.

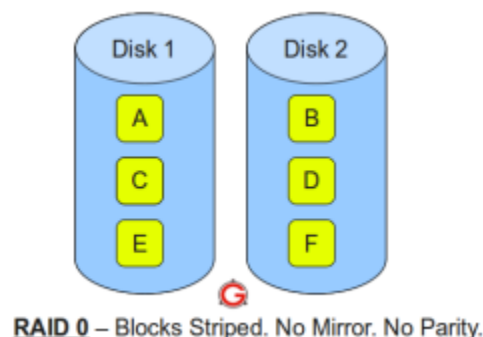


Figure 170 - Schema logico di RAID 0.

Essenzialmente RAID 0 non introduce nulla di nuovo.

RAID 1

Si ha la massima ridondanza, quindi ogni disco è replicato almeno una volta su un altro disco.

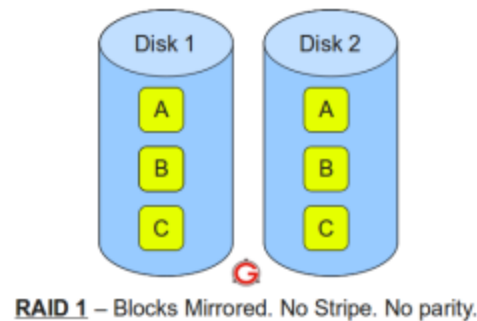


Figure 171 - Schema logico di RAID 1.

Non si hanno bit di parità ma il grado di replicazione è massimo.

RAID 5

Compromesso tra RAID 0 e 1: i blocchi sono sparsi su diversi dischi e non sono replicati ma in aggiunta si ha un blocco di parità. Il blocco di parità serve per aggiungere tolleranza ai guasti, ossia per recuperare i dati in caso di perdita di un disco.

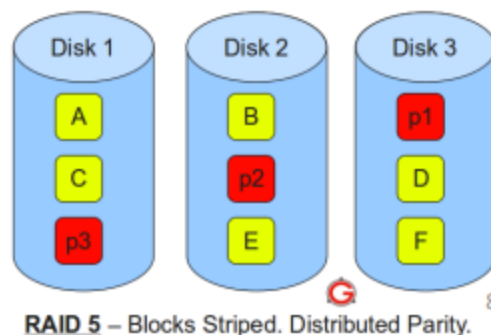


Figure 172 - Schema logico di RAID 5.

RAID 10

Questa configurazione è responsabile del mantenimento di un minimo di quattro dischi e dello stripping dei dati completi tra coppie mirrored.

In una tale configurazione di coppie e dischi mirrored, è evidente che i dati possono essere recuperati da questa disposizione anche se una coppia mirrored è funzionale. Tuttavia, dovrebbe essere chiaro che due dischi che falliscono su una coppia mirrored simile porterà alla perdita di dati poiché non ci sono parità tra i gruppi striped. RAID 10 è indicato per fornire un sistema migliore e sicuro rispetto ad altre configurazioni presenti sul mercato.

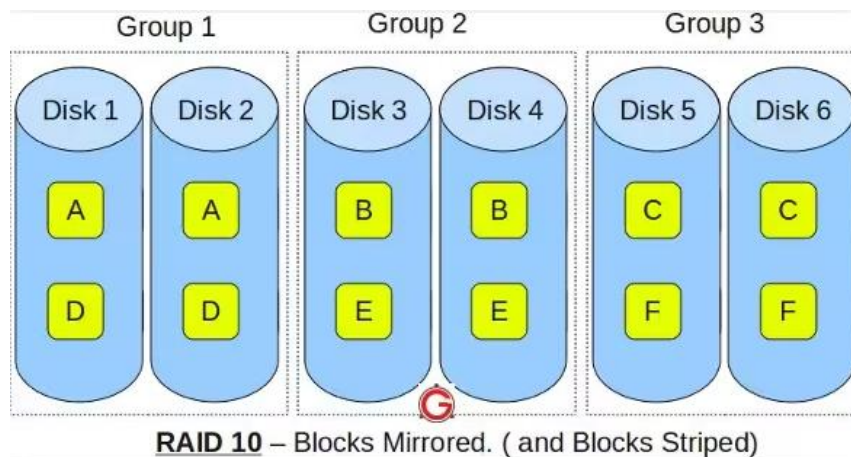


Figure 173 - Schema logico di RAID 10.

DFS - File System Distribuito

Serve ad evitare i limiti dati dal modello a singolo server.

Un DFS permette di aumentare le capacità di memorizzazione di un server distribuendo i file su molteplici server fisici connessi con una LAN; i server sincronizzandosi tra loro utilizzando la LAN, nascondono questa complessità e fanno sembrare che si usi un'unica macchina di memorizzazione.

NFS – Network File System

Esempio di DFS per Linux che utilizza un approccio client-server.

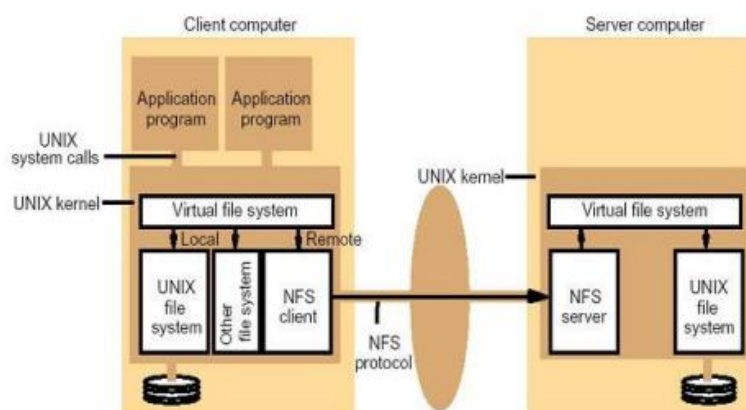


Figure 174 - Schema di un DFS.

NFS definisce un protocollo di comunicazione tra il client e il server: in particolare si ha uno scambio di messaggi sincrono (*per assicurare consistenza e per semplificare l'implementazione*). Le funzionalità di NFS sono inserite nel kernel di Linux e sono del tutto trasparenti alle applicazioni.

Server NFS

Il server NFS (ha una grande capacità) esporta (*rende disponibili*) una o più directory del proprio file system agli altri computer della rete.

Client NFS

I client (hanno meno capacità) richiedono l'accesso ai file e alle directory dei server NFS della rete. Per fare ciò, montano le directory esportate dal server nel proprio file system locale (*come se creassero un collegamento diretto tra di esse, una specie di portale*).

Quando una directory è montata, appare per il client come se fosse una parte del proprio file system locale, anche se fisicamente risiede sul server remoto.

Comunicazione di NFS

Questo protocollo consente ai client di inviare richieste di lettura, scrittura, creazione, eliminazione e altre operazioni sui file e sulle directory al server NFS in modo trasparente, come se il file fosse nel file system del client.

Il server risponde a queste richieste e gestisce gli accessi ai dati in base alle autorizzazioni configurate.

GlusterFS

GlusterFS propone una soluzione più distribuita, è un'altra implementazione di NFS che propone un'architettura decentralizzata. GlusterFS crea un DFS utilizzando le capacità di memorizzazione disponibile su tutti i nodi. Non esistono client o server ma solo nodi che offrono il loro spazio di memorizzazione.

Configurazioni di GlusterFS

Il livello di ridondanza e la modalità di replicazione può essere facilmente configurato. Ci sono tre modalità base:

- **Replicated volumes:** i blocchi di ogni file vengono replicati in tutti i nodi del sistema. Questo comporta la massima data redundancy (se tutti i server falliscono tranne uno, i dati sono al sicuro).
- **Distributed volumes:** i file sono suddivisi in blocchi, i blocchi di un file sono memorizzati in un singolo server, si può avere una politica di balancing per distribuire i file tra i server. Se un nodo fallisce non vengono persi tutti i file, come nel caso striped, ma vengono persi i file su quel singolo server.
- **Striped volumes:** equivalente di RAID 0, il file è suddiviso in blocchi che sono distribuiti tra server differenti. Comporta le massime prestazioni, però se un nodo fallisce i dati vengono persi.

È possibile avere combinazioni delle configurazioni precedenti, come ad esempio striped e distributed assieme. È l'equivalente del RAID 5.

SAN - Storage Area Network

Il requisito fondamentale del cloud storage è la scalabilità: la richiesta di memoria varia con le macchine virtuali e il carico a cui sono sottoposte. Le soluzioni viste in precedenza non hanno questa proprietà.

Soluzione “*Hypervisor*”

L’hypervisor crea degli hard drive virtuali basandosi sugli hard drive fisici installati sulla macchina e assegna questi hard drive virtuali alle macchine virtuali ospitate su quella macchina fisica.

Questa soluzione però ha una scarsa scalabilità e mantiene gli stessi limiti del modello tradizionale. È limitata dalla capacità della singola macchina.

Il problema della live migration

La strategia di prima non permette una funzionalità fondamentale in un ambiente cloud: la migrazione delle macchine virtuali.

La live migration è una tecnica che consiste nello spostare la macchina virtuale da un compute node ad un altro in maniera dinamica e veloce.

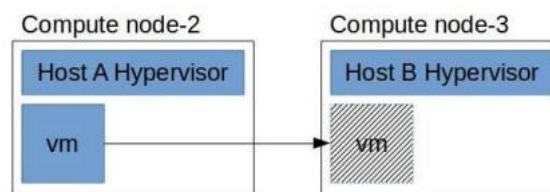


Figure 175 - La live migration.

Perché la tecnica di prima non è compatibile? Perché il contenuto della memoria secondaria della macchina virtuale è in locale, quindi per poter migrare una macchina virtuale devo anche inviare il contenuto dell’hard disk virtuale, ciò comporterebbe un traffico e una latenza inaccettabile per i nostri scopi.

Per risolvere questo problema viene introdotto il common storage. La SAN è una infrastruttura dedicata alla memorizzazione; ossia una parte del datacenter con macchine totalmente dedicate a memorizzare dati e renderli accessibili a tutte le altre macchine del datacenter con connessioni ad altissima velocità (*fibra ottica*).

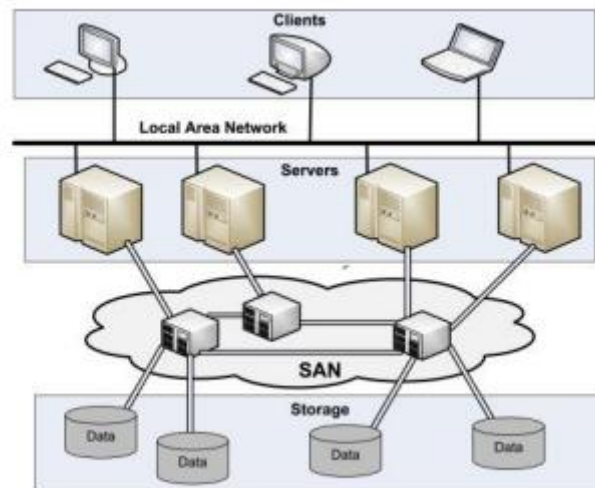


Figure 116 - SAN

Queste macchine sono specializzate in questo campo; ossia ottimizzate per massimizzare la velocità delle operazioni I/O da e verso la memoria secondaria.

I pro della SAN

La memorizzazione a questo punto avverrà grazie alla piattaforma che permetterà alle macchine virtuali di creare e gestire il loro spazio di memoria all'interno della SAN; quindi, in caso di migrazione il loro hard disk virtuale non verrà toccato poiché memorizzato nella SAN e non nella macchina fisica precedente.

La comunicazione dalle macchine virtuali verso la SAN verrà regolata grazie a protocolli ad hoc come iCSI, ossia un protocollo standard basato su TCP.

I contro della SAN

Queste macchine sono estremamente complesse da realizzare e richiedono hardware molto potente e dedicato; quindi, risultano incredibilmente costose.

Per risolvere questo problema sono stati introdotti i sistemi di storage distribuiti come alternativa non costosa, i quali offrono una soluzione scalabile, ad alte prestazioni, senza single point of failure basata su commodity hardware general purpose. Una di queste soluzioni è Ceph.

Ceph – Software Storage Solution

Ceph è una soluzione open-source puramente software che propone un'applicazione distribuita per creare un cloud storage e renderlo disponibile ai compute node della piattaforma cloud. Ceph deve essere installato al di sopra di un sistema operativo Linux e non ha particolari richieste riguardo le prestazioni dell'hardware.

È un'applicazione distribuita il cui servizio è quello di creare il cloud storage e renderlo disponibile ai client.

Ceph è altamente distribuito e non espone single point of failure. L'infrastruttura risultante mostrerà alcuni server con Ceph installato con la responsabilità di gestire il cloud storage.

Caratteristiche di Ceph

- Soluzione software-based: non è richiesto hardware speciale.
- Soluzione open source.
- Può essere eseguito su commodity hardware.
- Ogni cosa deve essere auto gestibile il più possibile.
- Soluzione deve essere scalabile e non deve essere un single point of failure.

Ceph e RAID

Le tecniche di RAID non sono necessarie, poiché la replicazione e la distribuzione ed il recovery dei dati è gestita direttamente da Ceph via software.

Comunque, RAID 0 è l'unica tecnica consigliata perché Ceph ha già politiche di replicazione ma comunque fa comodo spartire porzioni di dati su più dischi.

Comunque, è possibile usarle assieme, ma è importante capire come ciascuna tecnologia gestisce la ridondanza e la distribuzione dei dati per fare una scelta informata su come combinarle; in tal caso ci sono due possibilità:

- **RAID sotto Ceph:** Creare volumi RAID sui singoli nodi di storage e poi aggiungerli al cluster Ceph come OSD.
- **Ceph su RAID:** Utilizzare i dischi singoli direttamente in Ceph senza RAID, lasciando che Ceph gestisca la ridondanza e la distribuzione dei dati.

Mentre è tecnicamente possibile combinare Ceph e RAID, nella maggior parte dei casi potrebbe non essere necessario e potrebbe aggiungere complessità senza offrire benefici significativi.

Servizi di Ceph

Tutte le macchine con Ceph installato compongono il “*Ceph Cluster*” e rendono disponibile i loro hard drive. Ceph è un object storage anche se consente altre tipologie di memorizzazione sopra la memorizzazione ad oggetti: memorizzazione a blocchi o tramite un file system.

Ogni client può interagire con i server Ceph per memorizzare e recuperare i dati.

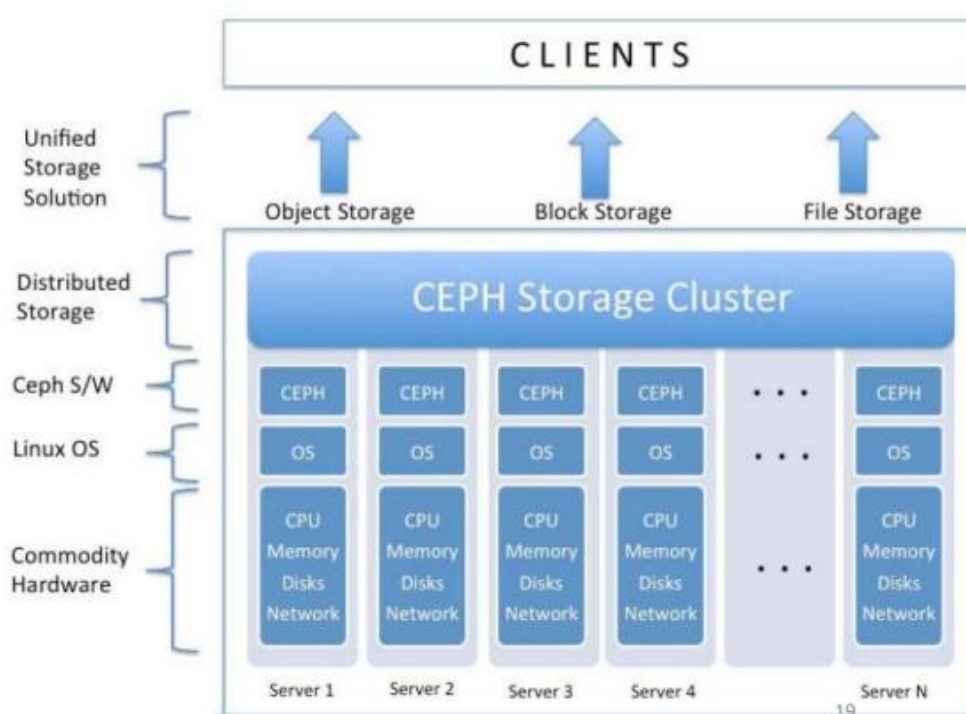


Figure 11777 - Servizi di Ceph

Architettura di Ceph

Ceph è composto da vari servizi software costruiti sopra altri servizi, ognuno con una certa funzionalità: i client comunicano con questi moduli per ricevere i servizi di memorizzazione.



Figure 11878 - Architettura di Ceph.

Ceph ha una struttura modulare come OpenStack. Ogni modulo ha una funzionalità separata dagli altri.

Oggetto di Ceph

Ogni oggetto di Ceph è una stringa (*senza limiti di grandezza*) con associati dei metadati. L'identificatore è calcolato tramite una funzione di hash che ha come input la stringa e i metadati relativi all'oggetto, ed è unico per ogni oggetto.

ID	Binary Data	Metadata
1234	0101010101010100110101010010 0101100001010100110101010010 0101100001010100110101010010	name1 value1 name2 value2 nameN valueN

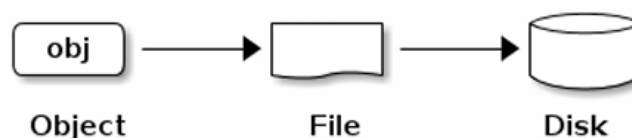


Figure 11979 - Oggetto di Ceph.

Gli oggetti quando vengono creati vengono diretti ad un OSD, questo OSD memorizza l'oggetto nell'hard drive che controlla (*per avere una elevata affidabilità*). Ogni hard drive solitamente memorizza gli oggetti locali utilizzando il file system, il file system è memorizzato nell'hard drive.

Dopo aver memorizzato un oggetto, Ceph può essere configurato per memorizzare più istanze (repliche) di un oggetto.

Quando Ceph riceve un inserimento da parte di un client, memorizza i dati associati alla richiesta in un oggetto e poi inserisce l'oggetto in un file di un OSD secondo una certa politica che vedremo a breve.

RADOS - Reliable Autonomic Distributed Object Store

Ogni cosa in Ceph è memorizzata come un oggetto e RADOS ha il compito di memorizzare questi oggetti a prescindere della tipologia di dato. Le altre tecniche di memorizzazione sono costruite al di sopra della memorizzazione ad oggetti (*come RBD*).

RADOS si assicura che tutti i dati rimangano in uno stato consistente e sempre disponibili: perciò gestirà la replicazione, rilevazione di guasti, migrazione dei dati e bilanciamento sui nodi del cluster.

RADOS implementa tutte le funzionalità del cluster.

I client che vogliono usare le funzionalità di Ceph comunicheranno con RADOS tramite la libreria LIBRADOS (possono accedere al file system e agli oggetti); disponibile per vari linguaggi di programmazione.

Le funzionalità di RADOS sono realizzate tramite i servizi MON e OSD.

OSD - Object Storage Device

Dispositivo di memorizzazione ad oggetti, è il basic building block che regola come deve essere fatta una lettura o una scrittura su un hard drive gestito da Ceph.

Responsabile di scrivere e leggere i dati all'interno del disco. Di solito è presente un'istanza di OSD per ogni hard drive fisico.

Supponiamo che ogni nodo ha due hard drive, allora ogni nodo ha due istanze del servizio OSD, una per gestire operazioni di lettura e scrittura nel disco 1 e una per gestire operazioni di lettura e scrittura nel disco 2.

MON - MONitor

Ha il compito di monitorare la condizione del cluster e di gestire le informazioni più sensibili per gestire tutte le operazioni di Ceph (*status e configurazione*). I monitor possono essere visti come i controller dell'infrastruttura.

MON è un'applicazione distribuita su varie istanze in modo da assicurare la resistenza ai guasti. Ovviamente ogni istanza di MON comunica con le altre istanze e per ogni decisione riguardo lo status del cluster di solito la decisione è presa tramite un sistema simile ad un'elezione. Per poter effettuare l'elezione in caso di desincronizzazione tra i monitor, il

numero di monitor deve essere dispari (se fosse pari, l'elezione potrebbe generare un pareggio).

Le istanze di MON sono installate su alcune macchine del Ceph cluster, non su tutte.

LIBRADOS

Serie di servizi creati sfruttando le primitive di RADOS.

Ceph File System – CephFS

File system distribuito che permette di accedere agli oggetti di Ceph.

RADOS Block Device – RBD

Fornisce il block storage. Permette una visione a blocchi degli oggetti. Questa funzionalità è utile quando framework come OpenStack richiedono un file system distribuito a blocchi.

Algoritmo CRUSH

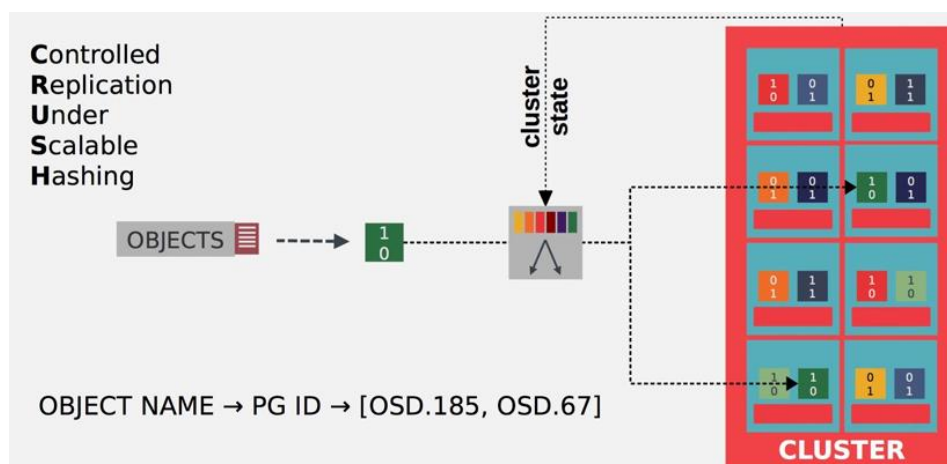


Figure 12080 - Workflow di CRUSH.

Tramite CRUSH sia il client che gli OSD possono calcolare la locazione di un oggetto (*sia per le letture che per le scritture*) senza avere una tabella di lookup.

Per evitare colli di bottiglia, Ceph adotta una procedura distribuita in cui il client stesso può decidere in quale OSD un nuovo oggetto dovrebbe essere inserito oppure il client può recuperare l'OSD che memorizza un oggetto che il client vuole.

L'input di questo algoritmo è l'identificatore dell'oggetto e lo stato corrente del cluster, fornisce come output una lista di ID degli OSD.

L'algoritmo viene eseguito nel client, questo rende Ceph scalabile perché non c'è un single point of failure.

Vediamo ora la procedura di locazione di un oggetto.

1. Il client contatta una delle istanze del MON per ricevere lo status del cluster.

Il MON invia al client la Map che contiene le seguenti informazioni:

- Il timestamp della creazione della Map e dell'ultimo aggiornamento di questa Map.
- Mappa degli OSD, dove per ogni OSD si ha: lista dei pool, lista delle repliche di questo OSD e lo status.
- PG Map, che contiene i dettagli di ogni placement group.
- Cluster MAP: Lo status del cluster è rappresentato da un Cluster MAP (*struttura tenuta aggiornata da un sottogruppo delle istanze di MON*). Fornisce una lista di dispositivi di storage e regole per attraversare la gerarchia quando vengono memorizzare i dati.

La Map è utilizzata per recuperare un placement group che è una lista ordinata di OSD determina il vero OSD che deve memorizzare un oggetto o che sta memorizzando correntemente un oggetto.

2. La cluster MAP e l'identificatore dell'oggetto vengono passati come input all'algoritmo CRUSH, il quale fornisce una lista ordinata di OSD.

Il primo OSD della lista è il primario ed è responsabile di avviare la procedura di replicazione.

3. Il primo OSD della lista viene contattato per ricevere l'oggetto (*lettura*) o modificarlo (*scrittura*).
4. Il primario riflette l'update sugli altri OSD (*secondario, terziario, ...*).

Piazzamento di un oggetto

1. Il client contatta un MON per ottenere la cluster Map più aggiornata.
2. Il client dà come input all'algoritmo CRUSH, la cluster Map assieme all'ID dell'oggetto.
3. Se vogliamo inserire un oggetto, l'ID dell'oggetto viene calcolato applicando la funzione di hash al nuovo oggetto che vogliamo memorizzare più i suoi metadati. Se vogliamo recuperare un oggetto, l'ID dell'oggetto dobbiamo conoscerlo.

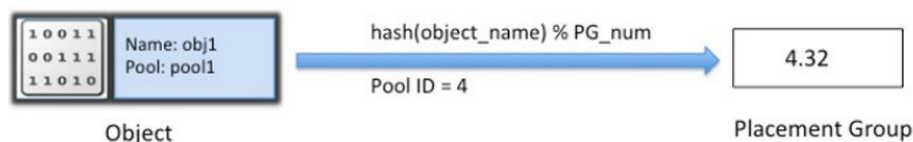


Figure 12181 - Calcolo del placement group.

4. Il client esegue l'algoritmo CRUSH per determinare il OSD in cui memorizzare l'oggetto.



Figure 12282 - Esecuzione di CRUSH.

5. CRUSH restituisce il placement group, ovvero una lista ordinata di OSD, dove il primo è il primario e il secondo è il secondario (*dove piazzare la replica*).
6. Il client scrive l'oggetto nello OSD primario nel caso di scrittura. Se vuole leggere l'oggetto lo chiede al primario.
7. Lo OSD primario in background si occupa della replicazione, memorizza l'oggetto localmente e con la sua copia di CRUSH Map identifica lo OSD secondario, lo OSD terziario ecc, fino a raggiungere il grado di replicazione voluto.
8. A quel punto seguendo la stessa procedura di prima, replica l'oggetto nello OSD secondario e terziario ecc.

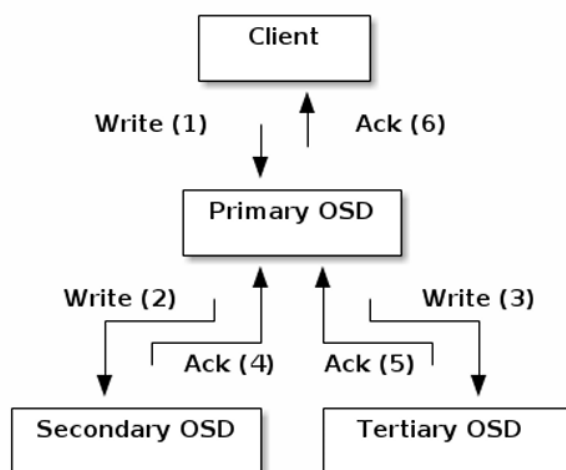


Figure 12383 - Replicazione di Ceph.

9. Finita la replicazione invia il messaggio di successo al client (*alta consistenza*).

Se un OSD fallisce, il sistema lo riconosce ed inizia un processo per controllare il grado di replicazione di ogni oggetto memorizzato nell'hard drive guasto, se non è sufficiente, allora quell'oggetto viene replicato in altri OSD fino a raggiungerlo.

Pools

Gli oggetti possono essere organizzati in pool, un pool è un insieme di oggetti. Partizioni logiche per memorizzare gli oggetti. I client scrivono gli oggetti nei pools, al resto ci pensa Ceph. Gli oggetti appartenenti ad applicazioni differenti sono separati in pool diversi.

Placement Group

Ogni pool ha un numero arbitrario di placement group. Un placement group determina in quale OSD è memorizzato un determinato oggetto.

Ridondanza dei MON

Operare con un solo monitor è possibile ma espone un single point of failure. Quindi si può replicare il MONitor facendo in modo che tutti i MONitor restituiscano sempre gli stessi risultati.

Per avere il consenso tra i monitor si utilizza l'algoritmo di Paxos.

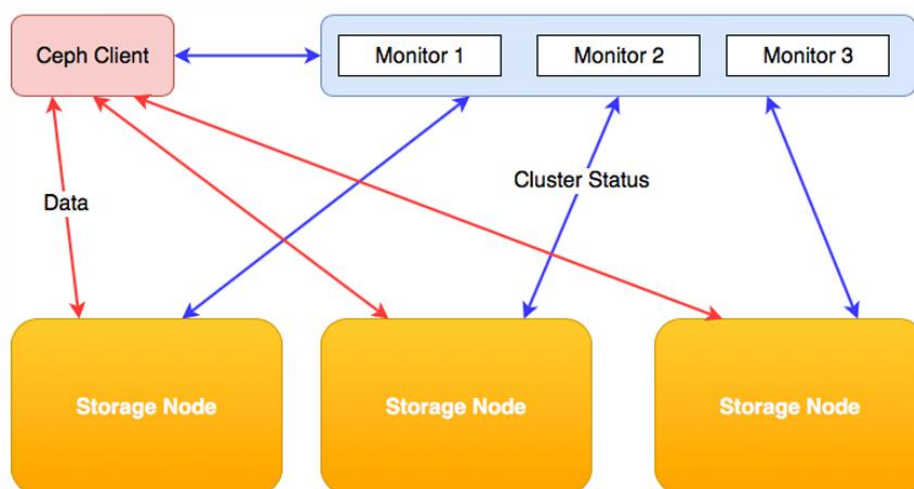


Figure 12484 - Ridondanza dei MON.

Calcolo dei Placement Groups

Ogni pool ha un determinato numero di placement group e l'algoritmo CRUSH mappa un placement group in uno o più OSD in modo dinamico.

Questo mapping dinamico è eseguito seguendo la condizione attuale del cluster in modo che i dati siano considerati al sicuro, disponibili anche in caso di guasto e ovviamente seguendo anche una logica di load balancing. La cluster Map viene creata dai Mon tenendo conto lo stato del sistema in modo da evitare colli di bottiglia.

Cluster Map che produce i placement group tiene conto dove gli OSD vengono messi in modo da evitare che due repliche differenti dello stesso oggetto vengano messe nello stesso nodo, ad esempio nel caso in cui il nodo abbia due o più hard drive. Quindi vogliamo evitare bottleneck e single point of failure

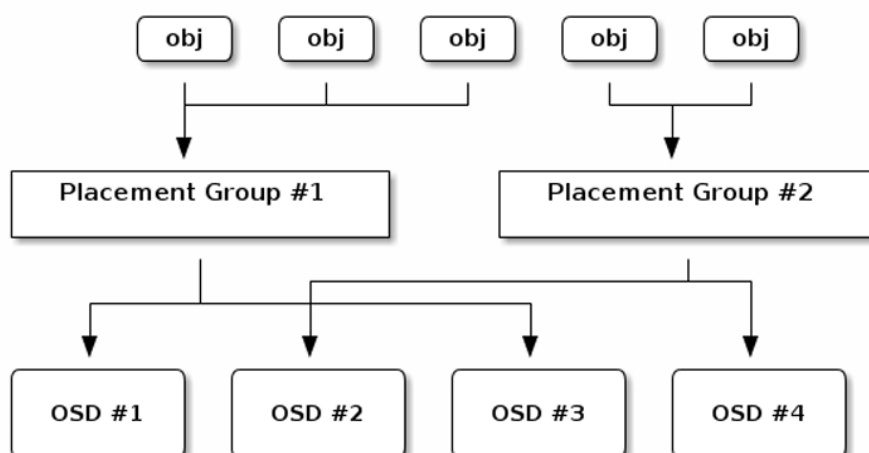


Figure 12585 - Mappatura dei placement groups negli OSD.

Recovering e Rebalancing

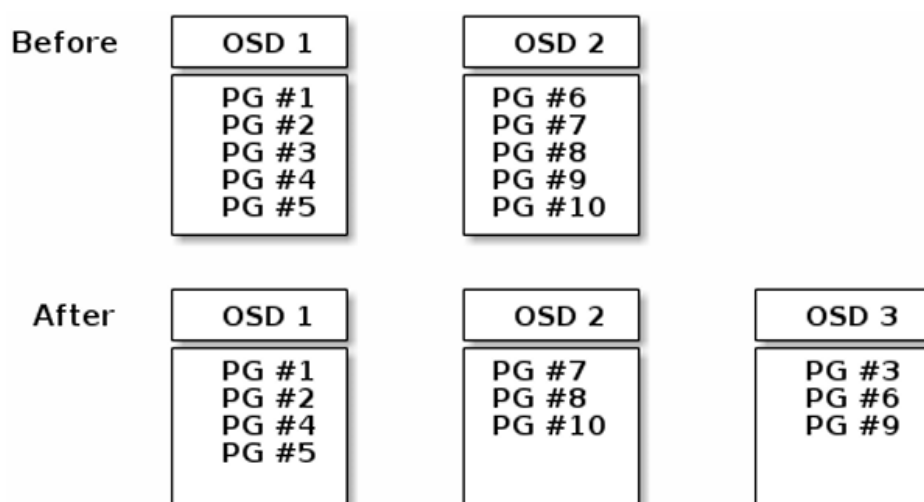


Figure 126 - Rimappatura di alcuni placement groups.

Grazie al livello di indirectione dato dai placement group tra il client e gli OSD, Ceph può spostare i placement group in altri OSD senza che il client se ne accorga.

Detto ciò, il cluster muta nel tempo e quando accade alcuni oggetti devono essere spostati senza danneggiare il servizio. Questo è possibile perché l'algoritmo CRUSH restituisce un risultato basato sulla situazione attuale e quindi basta ricalcolare CRUSH e poi spostare l'oggetto.

Ceph e OpenStack

Ceph è normalmente usato da OpenStack come servizio block storage (*per questo esiste RBD, che fornisce una visione a blocchi di Ceph*): ogni blocco è un virtual hard drive, il virtual hard drive è sicuro in Ceph. Quando un oggetto viene modificato, abbiamo lo stato iniziale

dell'hard drive, viene creato un nuovo oggetto che non include una nuova replica dell'hard drive, include solo le differenze con lo stato iniziale.

STaaS – Storage as a Service

Un'alternativa a Ceph è quella di sfruttare un servizio già esistente. I servizi STaaS adottano una memorizzazione ad oggetti e sfruttano un'interfaccia REST per memorizzare e leggere oggetti.

Spesso sono usati come backup a lungo termine.

Laboratorio 0 – Architetture Parallele

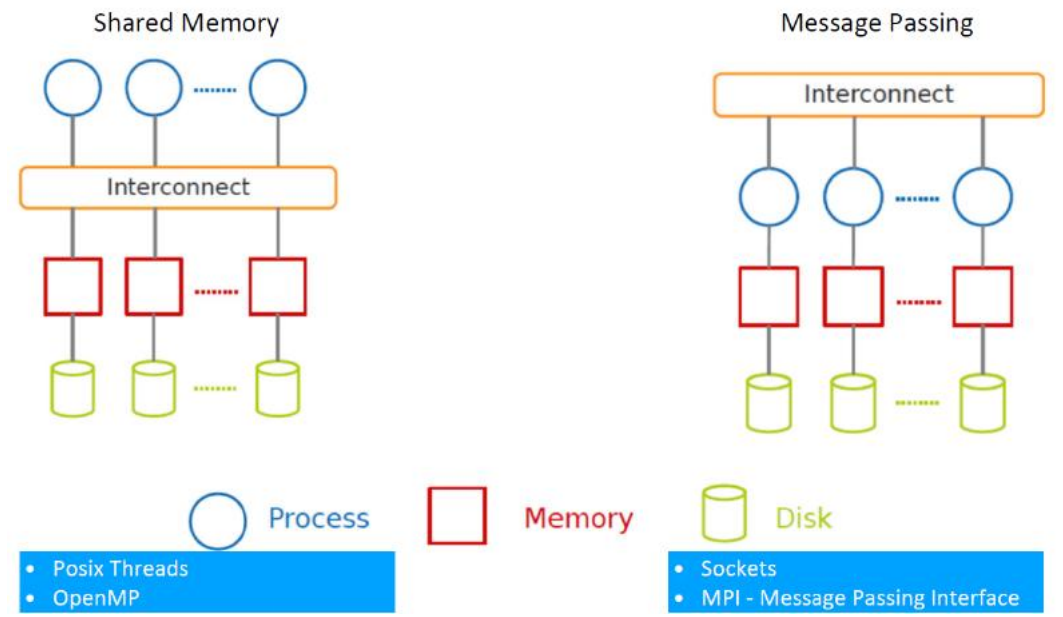


Figure 1277 – Tipi di architetture parallele.

Le architetture parallele possono comunicare tramite due metodologie:

- Scambio di messaggi.
- Memoria condivisa.

Gli step tipici per un'esecuzione in una architettura parallela sono i seguenti:

- **Identificare** quali parti del lavoro possono essere eseguite contemporaneamente.
- **Partizionare** il lavoro concorrente su processori indipendenti.
- **Distribuire** i dati di input, output e intermedi di un programma.
- **Coordinare** gli accessi ai dati condivisi, questo per evitare conflitti.
- **Assicurare** il corretto ordine di lavoro utilizzando la **sincronizzazione**.

Alcuni dei precedenti passaggi possono essere omessi:

- In un modello di programmazione parallela a memoria condivisa non c'è nessuna necessità di distribuire i dati.
- Nel modello di programmazione parallela con **message passing**, non c'è bisogno di coordinare i dati condivisi (perché non ci sono dati condivisi).
- La partizione dei processori potrebbe essere fatta automaticamente.

Task Dependency Graph

Come costruire un task dependency graph?

- Il primo passo nello sviluppo di un algoritmo parallelo è scomporre il problema in compiti che possono essere eseguiti concorrentemente.
- Un determinato problema può essere scomposto in task in molti modi diversi.
 - I task possono essere di dimensioni/granularità uguali, diverse o anche indeterminate.
- La scomposizione è modellata/illustrata sotto forma di Task Dependency Graph (TDG) il quale comprende:
 1. **Direct Acyclic Graph(DAG)**
 2. **Nodi** = task
 3. **Archi diretti** = dipendenza di controllo tra i task
 4. **Etichette dei nodi** = dimensione computazionale/peso del task

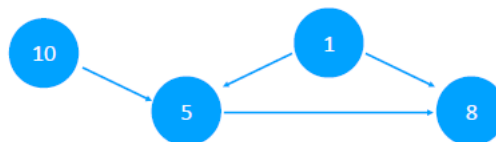


Figure 1288 – Task dependency graph.

Esempio

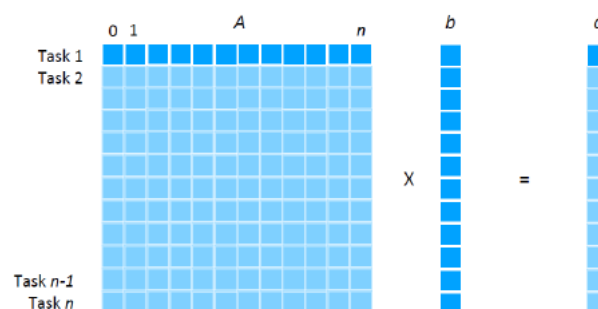


Figure 1299 – Moltiplicazione matrice per vettore.

Il calcolo di ogni elemento del vettore di output c è indipendente dalle altre righe in A . In base a ciò, un prodotto denso matrice-vettore può essere scomposto in n task.

- La figura evidenzia la porzione della matrice e del vettore accessibile dal task 1.
- Sebbene i task condividano dati (il vettore b), non hanno dipendenze.
- Nessun task deve aspettare il (parziale) completamento di un altro task.
- Tutti i task hanno la stessa dimensione in termini di numero di operazioni.

Consideriamo la seguente query sulla seguente tabella:

```
SELECT * FROM T WHERE
Model = "CIVIC" AND Year = "2001" AND
(Color = "GREEN" OR Color = "WHITE")
```

ID#	Model	Year	Color	Dealer	Price
4523	Civic	2002	Blue	MN	\$18,000
3476	Corolla	1999	White	IL	\$15,000
7623	Camry	2001	Green	NY	\$21,000
9834	Prius	2001	Green	CA	\$18,000
6734	Civic	2001	White	OR	\$17,000
5342	Altima	2001	Green	FL	\$19,000
3845	Maxima	2001	Blue	NY	\$22,000
8354	Accord	2000	Green	VT	\$18,000
4395	Civic	2001	Red	CA	\$17,000
7352	Civic	2002	Red	WA	\$18,000

Figure 190 – Esempio query.

L'esecuzione della query può essere divisa in sotto-task in vari modi.

Ogni task può essere considerato come la generazione di una tabella intermedia con entrate che soddisfano una particolare clausola.

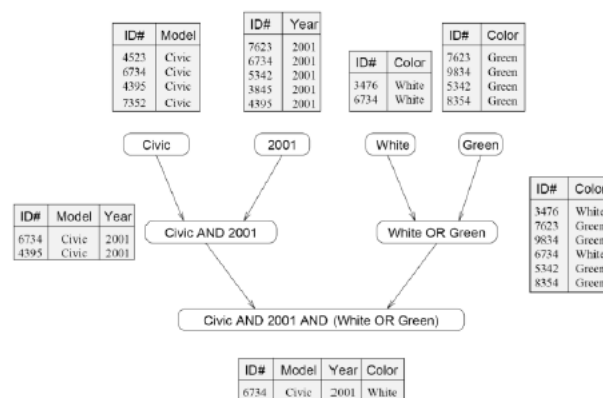


Figure 191 – Suddivisione query in sottotask.

Lo stesso problema può essere decomposto in sotto-task in un altro modo equivalente.

Questo perché l'operazione AND è associativa.

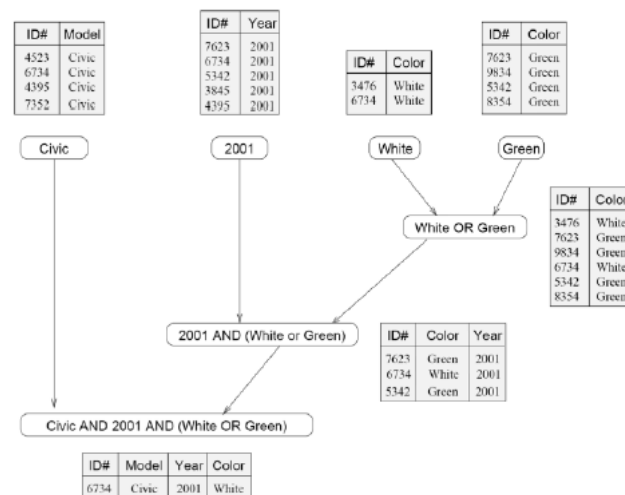


Figure 192 – Scomposizione equivalente.

Granularità dei task

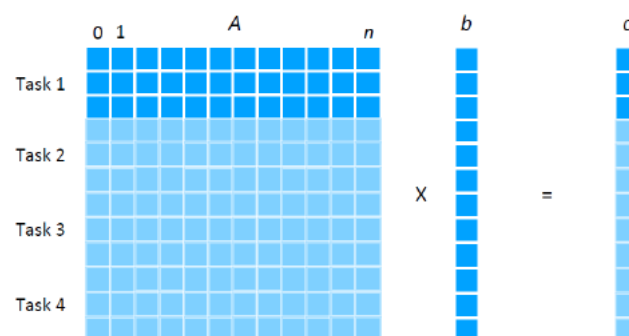


Figure 193 – Granularità dei task.

- Il numero di task in cui un problema è scomposto determina la sua granularità.
- La scomposizione in un gran numero di task risulta in una scomposizione **fine-grained**.
- La scomposizione in un piccolo numero di compiti risulta in una scomposizione **coarse-grained**.

Granularità dei task: il numero di task in cui un problema è scomposto determina la sua granularità.

- Grande numero di task per un problema: scomposizione fine-grained.
- Piccolo numero di task per un problema: scomposizione coarse-grained.

Grado di concorrenza: il numero di task che possono essere eseguiti in parallelo.

Grado massimo di concorrenza: il numero massimo di task che possono essere eseguiti in parallelo durante l'esecuzione di un programma.

Grado medio di concorrenza: numero medio di task che possono essere elaborati in parallelo durante l'esecuzione del programma.

Il sistema parallelo è utilizzato in modo efficiente quando il grado medio di concorrenza è simile al grado massimo di concorrenza.

Il grado di concorrenza aumenta quando la granularità diventa fine-grained.

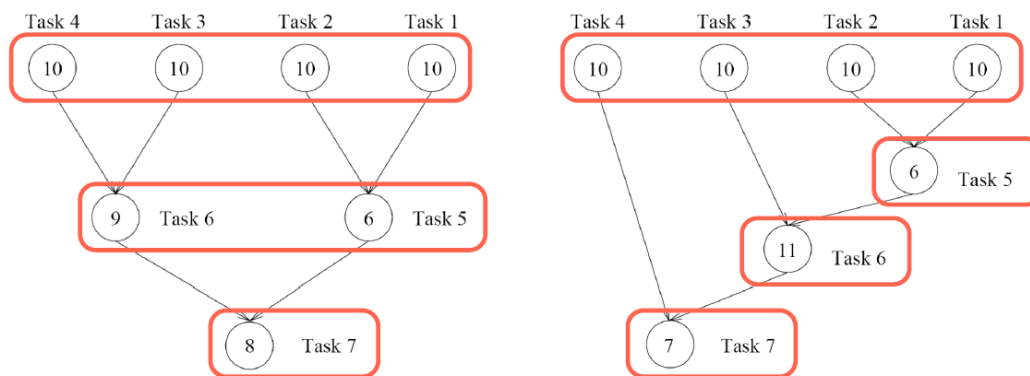


Figure 195 – Grado concorrenza.

Ogni percorso diretto nel TDG rappresenta una sequenza di task che devono essere elaborati uno dopo l'altro.

Il percorso diretto nel TDG può essere misurato sia sul numero di task sia come la somma dei pesi dei task coinvolti.

Lunghezza del Percorso Critico: la lunghezza del percorso più lungo in un grafico di dipendenza dei compiti. Corrisponde al tempo minimo di esecuzione di un programma parallelo.

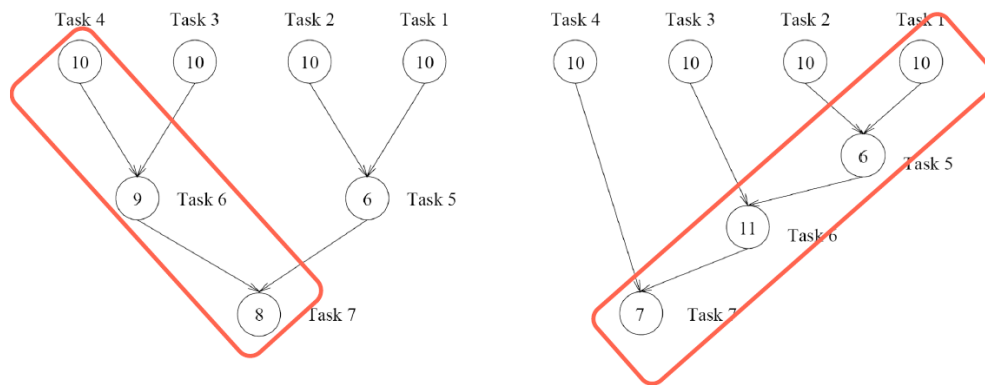


Figure 196 – Percorso critico.

Limitazioni delle Prestazioni Parallele: aumentando il numero di task della scomposizione (passando a una scomposizione fine-grained) possiamo ridurre il tempo parallelo, ma ci sono limitazioni a seconda del problema.

Limiti intrinseci sulla granularità della computazione, es.: nel prodotto matrice-vettore si basa sul numero di celle della matrice (non più di n^2 task).

Aumentando la concorrenza aumentiamo anche la quantità di dati che devono essere scambiati con altri compiti.

Ciò comporta un sovraccarico di comunicazione che aumenta insieme all'aumento del grado di parallelismo.

C'è un compromesso tra la granularità di una scomposizione e i sovraccarichi associati.

I sotto-task di solito devono scambiarsi dati tra loro per eseguire il lavoro in una scomposizione.

Task Interaction Graph (TIG): Il grafico dei task come nodi e lo scambio di dati (cioè le interazioni) come archi è indicato come TIG.

Nel TIG gli archi sono solitamente non diretti, gli archi hanno un'etichetta che rappresenta la quantità di dati scambiati.

Nodi = compiti, Etichette dei nodi = peso computazionale dei compiti, Archi = interazione o scambio di dati.

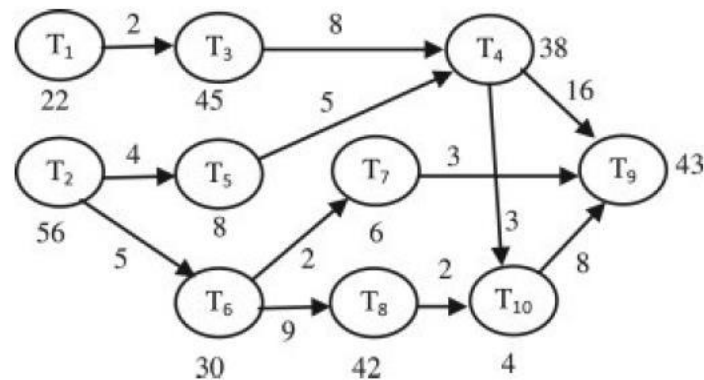


Figure 197 – Task interaction graph.

Laboratorio 1 - MapReduce

Per aumentare l'utilizzazione e il numero di jobs eseguiti al secondo dobbiamo cercare di parallelizzare le computazioni che eseguiamo, altrimenti a causa della mole dei dati la latenza può essere molto grande.

Parallelizzare i processi

Noi abbiamo sempre programmato in questo modo: per ottenere O partendo da I utilizzo una serie di m funzioni in cascata.

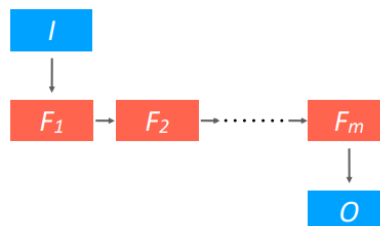


Figure 13098 - Programma puramente sequenziale.

Ma a volte alcune di queste m funzioni sono parallelizzabili: ossia cambiando l'ordine di esecuzione ed eseguendo alcune funzioni contemporaneamente ottengo lo stesso risultato.

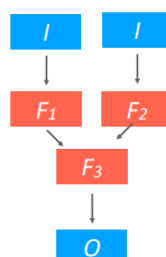


Figure 199 - Programma parallelizzato.

L'elaborazione parallela e distribuita si basa sul concetto di "Divide Et Impera": dividere il job in tanti piccoli task ed eseguirli in parallelo.

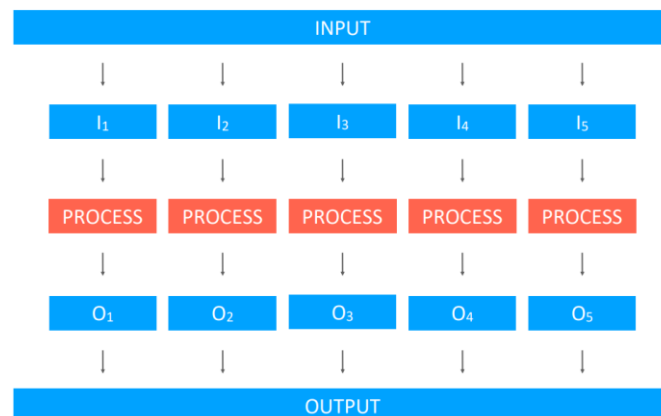


Figure 200131 – Il concetto di Divide et Impera

Questo tipo di coordinazione si può ottenere con una implementazione di tipo "Master-Slave".

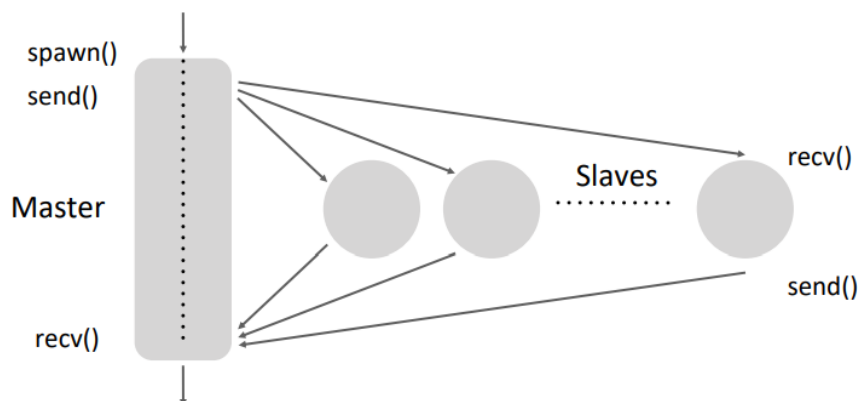


Figure 201 - Creazione degli slaves.

Il master crea N slave in modo dinamico e ad ognuno di essi è assegnata una porzione dei dati da elaborare.

Gli svantaggi del master-slave sono i seguenti:

- Non ha buone performance quando i task assegnati non sono bilanciati.
- Effettua partizionamento statico sui dati in input.
- N slave sono creati dinamicamente dal master.
- Una partizione dell'input è assegnata ad ogni slave.

Ma questo porta a vari problemi da risolvere:

- Come dividere i dati?
- Come distribuire le porzioni di dati ai processi?
- Come collezionare i risultati dei processi?

- Come unire i dati parziali?
- Come coordinare l'accesso ai dati?
- Cosa succede se ho più porzioni che processo?
- Cosa succede se devo fare in modo che porzioni siano accedute da più processi insieme?
- Cosa succede se una porzione diventa inaccessibile?

MapReduce utilizza i seguenti principi:

- **Right level of abstraction:** la maggior parte della complessità del sistema non è gestita (è nascosta) dallo sviluppatore dell'applicazione. Lo sviluppatore scrive poche linee di codice. Lo svantaggio consiste nel dover rispettare certe regole definite dal framework per implementare un'applicazione.
- **Scale “out”, not “up”:** abbiamo un cluster composto da commodity hardware, macchine comuni ed economiche e non macchine potenti e molto costose. Lo svantaggio è che queste macchine sono soggette maggiormente a guasti, le macchine costose sono soggette a meno guasti perché sono di meno, non perché sono più costose.
- **Move processing to the data:** evitiamo di trasferire i dati tra i nodi in cui risiedono i task e spostiamo anzi i task verso i nodi in cui risiedono i dati, questo perché i dati sono molti e costa molto di più spostarli. Con questo approccio viene ridotta la quantità di dati che viaggia in rete. Lo svantaggio è che il codice deve essere portabile perché nel cluster possono esserci macchine differenti con architetture diverse.

Il framework di MapReduce

MapReduce risponde a tutte queste domande. MapReduce è un framework costruito su principi consolidati e con tecniche di programmazione funzionali. MapReduce garantisce:

Purezza

Dato un input, il sistema che utilizza MapReduce produrrà sempre lo stesso output. Lavorano solo sui loro parametri, non hanno effetti collaterali.

```
int pure(int a, int b)
{
    return a + b;
}
```

```
const notpure(const moonLandingTime)
{
    return Date.now - moonLandingTime;
}
```

Figure 202 - Differenza tra una funzione pura (sinistra) e una funzione non pura (destra).

Immutabilità

Non ci sono variabili nella programmazione funzionale, ma solo delle costanti; se il valore cambia allora si crea una nuova struttura dati con il nuovo valore che va a sostituire quella vecchia.

Questa cosa permette ai processi di ignorare gli altri processi, perché ogni struttura può essere solo letta o inserita ex-novo.

High-order functions

Funzioni che operano utilizzando altre funzioni come parametri (*ad esempio una funzione di callback*) o restituendole come output. Ciò rende il codice più facile da creare e correggere.

Pro di MapReduce

Fornisce dei chiari limiti a cosa uno sviluppatore può e non può fare.

Contro di MapReduce

Il programmatore deve dividere l'algoritmo in un piccolo numero di frammenti che devono incastrarsi in maniera particolare. Non sempre è facile/possibile fare una cosa del genere.

MapReduce Programming model

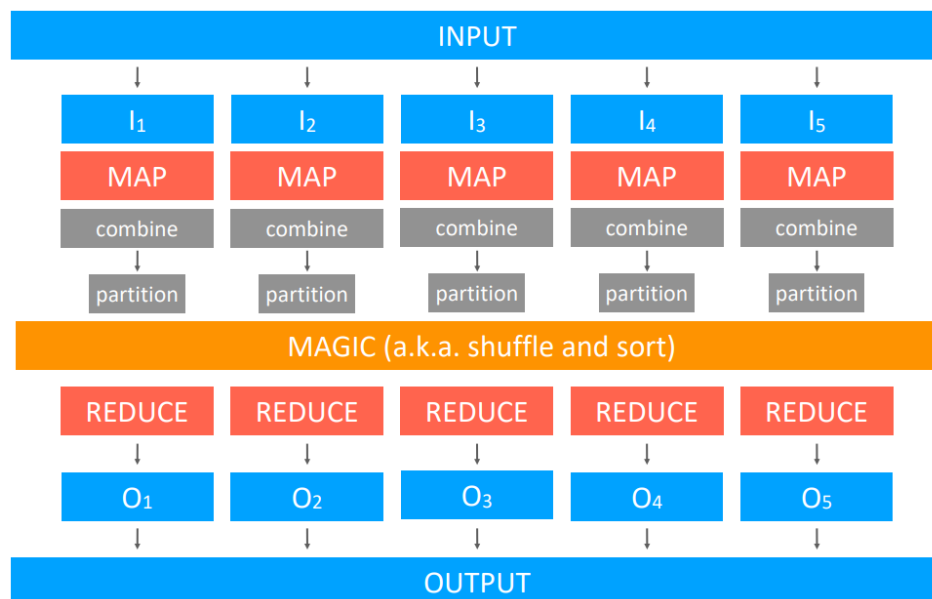


Figure 203 - Ordine di lavoro dei moduli di MapReduce.

Split

L'input del mapper viene prelevato dal DFS e viene visto come una singola e grande coppia chiave-valore. Solitamente, la chiave viene ignorata dato che spesso l'input è un grande documento di testo che deve semplicemente essere diviso in chunk.

Un modo standard di divisione dell'input in chunk è quello di dividere il documento per linee (*ossia fino al ritorno carrello*).

Gli split sono partizioni logiche dei dati, create automaticamente da Hadoop, non sono i dati reali, vengono riportati solo i metadati; quindi, la grandezza dello split e le location (un blocco è replicato) del blocco a cui quello split appartiene.

Map

Modulo che normalmente implementa una funzione stateless. Il mapper riceve una porzione dei dati di input (una coppia chiave-valore) da elaborare e restituisce un set (*0 o più*) di coppie chiave-valore.

Ogni mapper dovrebbe essere eseguito su un nodo che ha una sua porzione di dati localmente per evitare traffico inutile; inoltre, tutti i processi mapper vengono eseguiti in parallelo.

Il mapper legge l'input sottoforma di coppie chiave-valore, queste coppie sono lette dal file system distribuito, potrebbe utilizzare o ignorare la chiave di input.

Ad esempio, viene letta una linea di un file alla volta, la chiave è l'offset in byte a cui inizia la linea, il valore è il contenuto della linea stessa, la chiave solitamente è irrilevante.

Reduce

Modulo che implementa una funzione stateless. Il reducer riceve i risultati di elaborazione parziali da alcuni dei mapper (lista di valori associati ad una chiave) e produce un frammento dell'output finale. Tipicamente qualche coppia chiave-valore (*molte meno di quelle in uscita dal mapper*).

Il numero di reducer è un parametro deciso dal progettista, può essercene 1 o più.

Dopo il termine della fase di map, tutti i valori intermedi relativi ad una chiave vengono combinati in una lista, ogni lista è inviata ad un reducer (shuffle). Le liste sono inviate al reducer ordinate per valore della chiave (sort).

La cosa importante da ricordare è che tutti i valori associati ad una determinata chiave andranno sempre dallo stesso reducer poiché la relazione chiave-reducer è deterministica.

Le coppie chiave-valore in output vengono scritte nel file system distribuito.

Map e Reduce sono all'interno di due high-level function il mapper ed il reducer.

Partitioners (shuffle)

Ha il compito di distribuire il carico prodotto dai mapper ai reducers in modo uniforme. Di default il partitioner è molto semplice: le coppie restituite dai mapper sono distribuite ai reducers tramite una funzione di hash o comunque tramite un metodo deterministico. Ignora i valori associati alle chiavi, considera solo le chiavi.

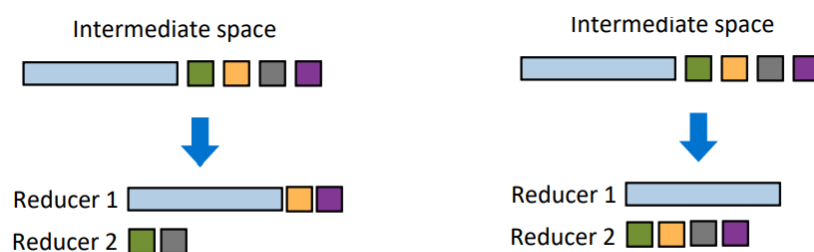


Figure 204 - Esempio di lavoro svolto dal partitioners.

Ignorare i valori è un problema perché a volte alcune chiavi hanno un peso maggiore (*hanno associata una maggiore quantità di dati*) e quindi si può considerare un partitioner più sofisticato che tiene conto anche il carico di lavoro previsto in ogni reducer, in modo da bilanciarne il carico.

Questo partitioner più sofisticato deve essere fornito dal progettista.

Combiners

Componente opzionale, servono per ottimizzare il processo dei moduli successivi ai mapper e per ridurre il traffico.

Sono detti mini-reducer, sono quasi sempre uguali ai reducer, la differenza è che il reducer è una fase diversa rispetto al map, ogni reducer lavora sul pool di risultati prodotti da ogni mapper, mentre il combiner lavora solo sull'output di un solo mapper.

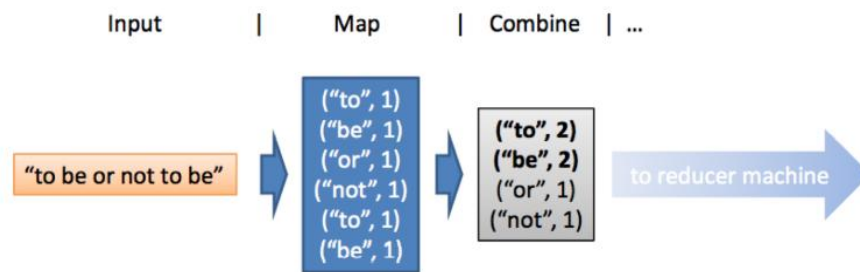


Figure 205 - Esempio del lavoro svolto dai combiners.

Posizionati dopo ogni mapper, eseguono un'aggregazione locale direttamente sull'output di ogni mapper, in modo da ridurre la quantità di informazioni che viaggerà nella rete e anche per ridurre il carico di lavoro dei reducers.

Per utilizzare i combiner, il reduce deve essere associativo e commutativo.

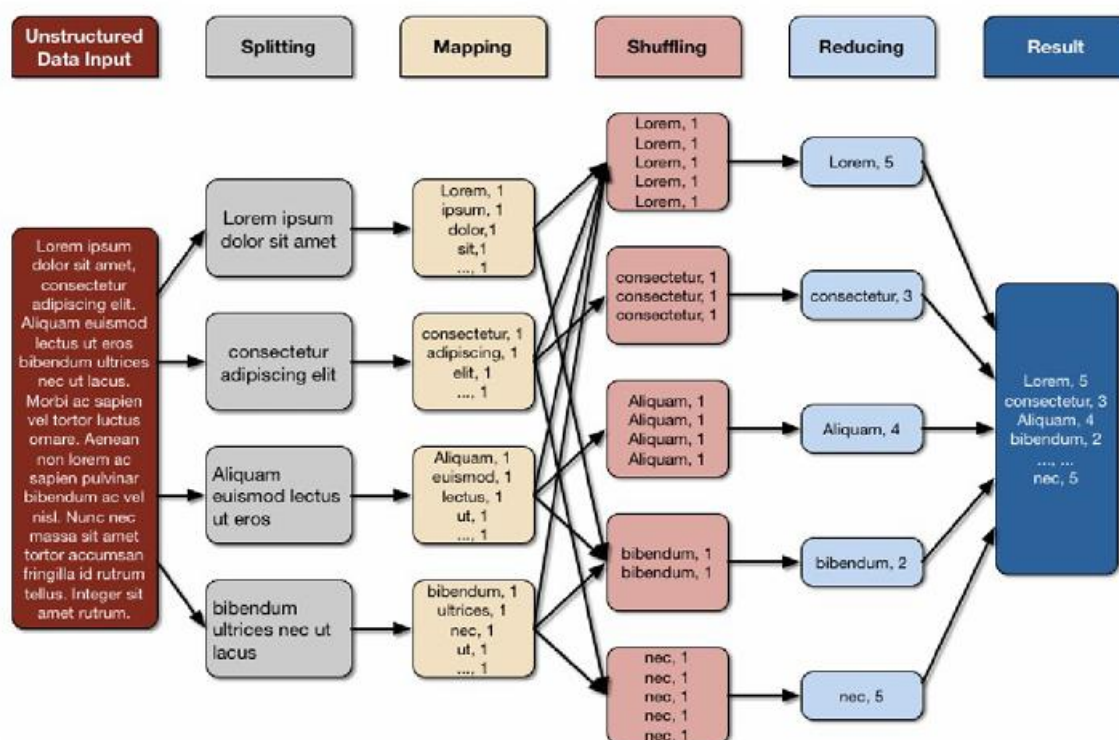


Figure 206 - esempio di utilizzo di MapReduce per un contatore di parole.

Sort

La fase di sort avviene sia sul lato del mapper sia sul lato del reducer:

- **Sort lato mapper:** Prima di inviare i dati attraverso la rete, ogni mapper ordina i dati all'interno di ciascuna partizione per chiave. Questo ordinamento locale facilita la fase successiva e rende il trasferimento dei dati più efficiente.
- **Sort lato reducer:** Quando i reducers ricevono i dati dalle varie partizioni, ordinano ulteriormente i dati per chiave. Questo passaggio garantisce che i dati siano ordinati in modo corretto prima di essere passati alla funzione reduce.

Hadoop

Il job MapReduce viene sviluppato localmente, successivamente il codice viene compilato, viene prodotto un eseguibile jar, questo file viene inserito tramite SSH nel cluster Hadoop, il debug ed esecuzione avvengono nel cluster Hadoop.

Hadoop può essere installato secondo tre modalità:

- **Modalità local:** Hadoop utilizza il file system locale come sostituto di HDFS.
- **Modalità pseudo-distributed:** viene utilizzato HDFS nella macchina localmente.
- **Modalità fully-distributed:** viene utilizzato HDFS nel cluster.

Hadoop types

- **Map function:** prende in input una coppia (K1, V1) e restituisce una lista di coppie (K2, V2).
- **Combiner function:** prende in input K2 e una lista di valori V2 (K2 e V2 sono intermedi) e restituisce una lista di coppie (K2, V2).
- **Reduce function:** prende in input K2 e una lista di valori V2 e restituisce una lista di coppie (K3, V3).
- **Partitioner function:** prende in input una copia (K2, V2) e restituisce un intero (ignora il valore, utilizza solo K).

I tipi di input sono impostati dal formato di input, ad esempio un *TextInputFormat* genera chiavi del tipo *LongWritable* e valori di tipo *Text*.

I tipi di K2, V2, K3, V3 sono decisi di default dal *TextOutputFormat* e sono *LongWritable* per le chiavi e *Text* per i valori.

Il numero di reducer task di default è uno, però è possibile crearne di più, mentre il numero di mapper può essere deciso però Hadoop fa come vuole, decide lui se crearne quella quantità o meno, in genere crea tanti mapper quanti sono gli input split.

Serializzazione in Hadoop

Serializzazione è il processo di convertire un oggetto ad una sequenza di byte, mentre deserializzazione è il processo inverso.

Serializzazione viene utilizzata nella comunicazione tra processi e per la memorizzazione persistente.

Hadoop utilizza il suo formato di serializzazione detto **writable**, è un wrapper per un java built in type (ad esempio *intWritable* è il writable per *int*).

L'obiettivo è quello di rendere i processi di serializzazione e deserializzazione più semplici, veloci e più efficienti.

Funzione di setup

Funzione opzionale eseguita prima della funzione map e serve per eseguire alcune operazioni preliminari prima del mapper. Prende il contesto come input e serve per il setup dell'ambiente in cui la logica del mapper è in esecuzione.

Funzione di cleanup

Funzione opzionale eseguita prima che la funzione map e che la funzione reduce terminino. Prende il contesto come input e serve per deallocare risorse, ripulire memoria ecc.

Design patterns per MapReduce

Vediamo ora alcune pratiche per implementare un programma in MapReduce secondo certe metodologie di progettazione. Un algoritmo complesso deve essere gestito in modo tale che il job MapReduce possa essere eseguito.

MapReduce richiede che noi sviluppatori implementiamo e decidiamo:

- Input data.
- Configurazione (*numero di reducers, output path, ...*).
- Implementazione del **mapper** e del **reducer**.

E opzionalmente:

- Implementazione delle funzioni di **setup**, **cleanup**.
- Implementazione dei moduli **combiner** e **partitioner**.

```

1: class MAPPER
2:   method MAP(docid a, doc d)
3:     for all term t ∈ doc d do
4:       EMIT(term t, count 1)

1: class REDUCER
2:   method REDUCE(term t, counts [c1, c2, ...])
3:     sum ← 0
4:     for all count c ∈ counts [c1, c2, ...] do
5:       sum ← sum + c
6:     EMIT(term t, count sum)

```

Figure 207 - Pseudocodice di un mapper e di un reducer classico che conta le parole in un testo.

In-Mapper Combining

Tecnica di realizzazione del mapper dove i dati intermedi (*output dei mappers*) sono scritti in locale e trasferiti ai reducers attraverso la rete del cluster. Essenzialmente abbiamo unito mapper e combiner in un solo modulo in modo da semplificare il tutto.

```

1: class MAPPER
2:   method MAP(docid a, doc d)
3:     H ← new ASSOCIATIVEARRAY
4:     for all term t ∈ doc d do
5:       H{t} ← H{t} + 1
6:     for all term t ∈ H do
7:       EMIT(term t, count H{t})

```

Figure 208 - Stateless In-Mapper combining.

Il termine “*stateless*” in questo contesto ha lo stesso significato che ha nel protocollo http, ossia significa che non c'è uno stato condiviso tra due esecuzioni diverse della funzione map. Quindi due esecuzioni differenti della funzione map che contano il numero di lettere “a” in un testo restituiranno la prima ad esempio (a, 7) e la seconda (a, 5) e non (a, 12).

```

1: class MAPPER
2:   method INITIALIZE
3:      $H \leftarrow \text{new ASSOCIATIVEARRAY}$ 
4:   method MAP(docid  $a$ , doc  $d$ )
5:     for all term  $t \in \text{doc } d$  do
6:        $H\{t\} \leftarrow H\{t\} + 1$ 
7:   method CLOSE
8:     for all term  $t \in H$  do
9:       EMIT(term  $t$ , count  $H\{t\}$ )

```

Figure 209 - Stateful In-Mapper combining, il “vero” in-mapper combining.

Nel caso stateful è il caso vero) il mapper si ricorda di ciò che ha fatto prima e quando deve eseguire di nuovo la funzione continua ad aggiornare ciò che ha fatto prima; infatti, prima di emettere il risultato fa svariate passate della funzione map ed infine emette il risultato aggregato finale.

Il metodo *initialize* viene utilizzato nella funzione *setup()*.

Il risultato viene emesso solo una volta al termine di tutte le computazioni che deve fare quel mapper. Questo permette di trasferire molti meno dati.

Contro di questo approccio

Colli di bottiglia: Dovuti alle scritture sul disco locale e al trasferimento lungo la rete, i risultati intermedi devono essere scritti da qualche parte.

Bug di ordinamento: L'output finale dipende dalla storia dei singoli output del mapper. Se i singoli output non venissero uniti nell'output finale in ordine, in alcuni algoritmi potrebbe può dare problemi e rendere la funziona non pura (risultato non deterministico) e va sempre evitata.

Problemi di scalabilità: dovuti al consumo di memoria primaria per memorizzare lo stato di ogni mapper: i combiner in-mapper utilizzano un array associativo per rappresentare lo stato.

Questo problema può essere parzialmente evitato emettendo l'array associativo periodicamente (ossia si emette un risultato intermedio superata una certa soglia) in modo da non farlo diventare troppo grande, ovviamente seguendo un certo criterio.

Pro di questo approccio

Siamo sicuri che il combining verrà eseguito: Il combiner classico (*esterno*) è opzionale e scritto dal programmatore; nonostante sia fornito, Hadoop potrebbe anche non eseguirlo per dare priorità ad altri task; mentre il combiner in-mapper è sicuro che sia eseguito poiché è incluso nel mapper.

Meno traffico in rete: Il combiner serve a diminuire la quantità di dati che viaggerà nella rete ma non riduce la quantità di dati che esce dai mapper, i trasferimenti da mapper a combiners possono rappresentare un problema. I combiner in-mapper diminuiscono la quantità di dati che esce dai mapper eliminando dati inutili o aggregandoli.

Pairs and stripes

Dato un input testuale di N parole, il sistema genera una matrice co-occurrence quadrata composta da $N \times N$ celle. Una matrice co-occurrence è una struttura dati costruita partendo da un set di frasi. La cella m_{ij} contiene il numero di volte che la parola w_i viene prima della parola w_j .

Ci sono due approcci:

- **Pairs** Genera $O(N^2)$ dati utilizzando uno spazio $O(1)$; più efficiente in memoria dato che non usa un array associativo.
- **Stripes** Genera $O(N)$ dati utilizzando uno spazio $O(N)$; meno efficiente in memoria dato che usa un array associativo.

Pairs

1. Per ogni termine w , controllo i suoi vicini.
2. Per ogni vicino u emetto $\{ \langle u, w \rangle, 1 \}$

Il reducer riceverà una serie di dati del tipo: $\{ \langle u, w \rangle, 1 \}$

3. Sommo tutti i contatori a chiave comune (*la chiave è la coppia*).
4. Per ogni coppia $\langle u, w \rangle$ emette un dato del tipo $\{ \langle u, w \rangle, k \}$ ossia una singola cella della matrice.

```

1: class MAPPER
2:   method MAP(docid  $a$ , doc  $d$ )
3:     for all term  $w \in \text{doc } d$  do
4:       for all term  $u \in \text{NEIGHBORS}(w)$  do
5:         EMIT(pair ( $w, u$ ), count 1)           ▷ Emit count for each co-occurrence
1: class REDUCER
2:   method REDUCE(pair  $p$ , counts [ $c_1, c_2, \dots$ ])
3:      $s \leftarrow 0$ 
4:     for all count  $c \in \text{counts } [c_1, c_2, \dots]$  do
5:        $s \leftarrow s + c$                        ▷ Sum co-occurrence counts
6:     EMIT(pair  $p$ , count  $s$ )

```

Figure 210- Pseudocodice pairs.

Stripes

1. Per ogni termine w , definisco un array associativo H dedicato solo a w
2. Per ogni termine u vicino a w incremento $H(u)$
3. Il mapper emette $H(u)$

Il reducer riceverà dati del tipo: $\{ w, \text{stripe } H \}$

4. Il reducer alloca un altro array associativo H_f , che conterrà la somma di tutti gli array associativi di w .
5. Sommo gli array associativi con chiave comune.
6. Emette un dato del tipo $\{ w, \text{stripe } H_f \}$.

Ossia una intera riga della matrice.

```

1: class MAPPER
2:   method MAP(docid  $a$ , doc  $d$ )
3:     for all term  $w \in \text{doc } d$  do
4:        $H \leftarrow \text{new ASSOCIATIVEARRAY}$ 
5:       for all term  $u \in \text{NEIGHBORS}(w)$  do
6:          $H\{u\} \leftarrow H\{u\} + 1$                                 ▷ Tally words co-occurring with  $w$ 
7:       EMIT(Term  $w$ , Stripe  $H$ )

1: class REDUCER
2:   method REDUCE(term  $w$ , stripes  $[H_1, H_2, H_3, \dots]$ )
3:      $H_f \leftarrow \text{new ASSOCIATIVEARRAY}$ 
4:     for all stripe  $H \in \text{stripes } [H_1, H_2, H_3, \dots]$  do
5:       SUM( $H_f, H$ )                                              ▷ Element-wise sum
6:     EMIT(term  $w$ , stripe  $H_f$ )

```

Figure 211 - Pseudocodice stripes.

Schemi per operazioni ad algebra relazionale

A_1	A_2	A_3	A_4	A_5	Schema
					Tuple t

Relation R

Figure 212 - Tabella dove individuiamo una tupla t .

Un caso d'uso in cui le operazioni di algebra relazionale vengono utilizzate è quello dei motori di ricerca.

L'input è sempre la tabella R , dove ogni riga è una tupla.

Selezione

Map: Emette ogni tupla t in R che soddisfa la selezione.

Il mapper emette come risultato coppie del tipo $\{t, -\}$ con $-$ valori null.

Reducer: Per ogni coppia $\{t, -, -, -\}$ restituisce $\{t, -\}$.

Proiezione

Map: Per ogni tupla t in R , crea una nuova tupla t' che contiene solo gli attributi proiettati in coppie $\{t', -\}$.

Reducer: Per ogni $\{t', -, -, -\}$ in input, restituisce $\{t', -\}$.

Unione

Map: Per ogni tupla t in R e S restituisce coppie $\{t, -\}$.

Reducer: Per ogni input key t , ci saranno 1 o 2 valori uguali a $-$; li unisce in un unico output $\{t, -\}$ (ossia quella tupla deve esserci in R o in S).

Intersezione

Map: Per ogni tupla t in R e S restituisce coppie $\{t, -\}$.

Reducer: Per ogni input key t , ci saranno 1 o 2 valori uguali a $-$; li unisce in un unico output $\{t, -\}$ solo se ce ne sono 2 (ossia quella tupla deve esserci sia in R che in S), altrimenti non fa niente.

Differenza

Map:

- Per ogni tupla t in R restituisce $(t, \mathbf{R})$ (R indica che t è in R)
- Per ogni tupla t in S restituisce $(t, \mathbf{S})$

Reducer: Per ogni input key t , ci saranno 1 o 2; se c'è un valore uguale a $\{t, \mathbf{R}\}$ (ossia quella tupla deve esserci solo in R), restituisce $\{t, -\}$, altrimenti non fa niente.

Join naturale

Supponiamo di avere due relazioni $R(A,B)$ e $S(B,C)$; l'attributo comune è B . Si deve unire le tuple che condividono il valore dell'attributo B .

Map:

- Per ogni tupla (a, b) di R , restituisce $(b, (\mathbf{R}, a))$.
- Per ogni tupla (b, c) di S , restituisce $(b, (\mathbf{S}, c))$.

Reducer: Per ogni input key b , ci sarà una lista di valori nella forma $(\mathbf{R}, a)$ e $(\mathbf{S}, c)$. Fa il prodotto cartesiano tra queste due liste e restituisce.

Raggruppamento e aggregazione

Supponiamo di avere una relazione $R(A,B,C)$ e noi vogliamo raggruppare A e aggregare su B , senza considerare C .

Map: Per ogni tupla (a, b, c) di R , restituisce (a, b) , ogni valore di a è un gruppo.

Reducer: Applica l'operatore di aggregazione alla lista di valori b associati al gruppo a e ottieni x , l'output è (a,x) .

Stage chaining

Spesso non si può eseguire l'intero algoritmo in unica esecuzione MapReduce poiché le funzioni di Map e di Reduce risulterebbero troppo complesse. Quindi possiamo concatenare due o più esecuzioni di MapReduce in cascata, quindi l'output del primo job è l'input del secondo job.

Schema per moltiplicazione tra matrice e vettore

In questo caso assumiamo che la matrice (*non quadrata* $m > n$) sia troppo grande rispetto alla memoria della macchina, ad esempio venti miliardi di righe e il vettore v può entrare o meno nella macchina (*è grande quanto una colonna della matrice*).

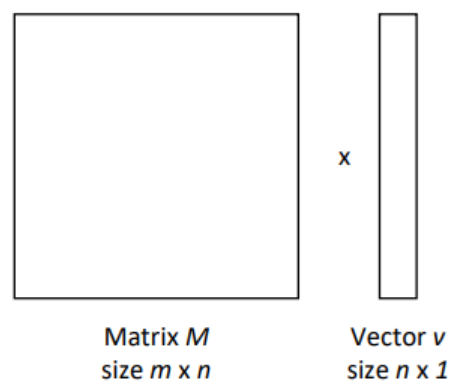


Figure 213 - La matrice e il vettore da moltiplicare.

Il nostro obiettivo è moltiplicare la matrice per questo vettore. Questo problema è risolto in due modi, dipendentemente dalla dimensione della matrice.

Il vettore entra in memoria

La matrice è in un DFS, memorizzata come una lunghissima lista di tuple: ogni cella è una tupla di tre elementi { `indice riga` , `indice colonna` , `valore` }.

Il vettore è presente nelle memorie locali di tutti i nodi e quindi disponibile a tutti i mappers (*questa cosa è possibile perché il vettore viene inserito nell'oggetto di configurazione e quindi recuperato dal driver durante la fase di setup*).

Il mapper processerà riga per riga i dati di input e calcolerà il prodotto tra: $((i, j), m_{ij})$ e (j, v_j) .

Il reducer prenderà tutti i risultati con chiave i e sommerà i prodotti.

Quindi il reducer restituisce una roba del tipo $(i, m_{i1}v_1 + m_{i2}v_2 + \dots + m_{in}v_n)$.

Il vettore non entra in memoria

Non può essere condiviso con il configuration object ma va condiviso attraverso HDFS come la matrice.

L'intera colonna della matrice verrà moltiplicata solo per una piccola parte del vettore.

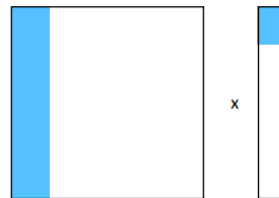


Figure 214 - La prima moltiplicazione parziale.

Notare che le moltiplicazioni blu e verde sono indipendenti; quindi dividendo il vettore in piccole parti, abbastanza piccole da essere contenute in memoria allora tutto ciò è fattibile.



Figure 215 - Tutte le moltiplicazioni parziali.

Schema per moltiplicazione tra matrice e matrice

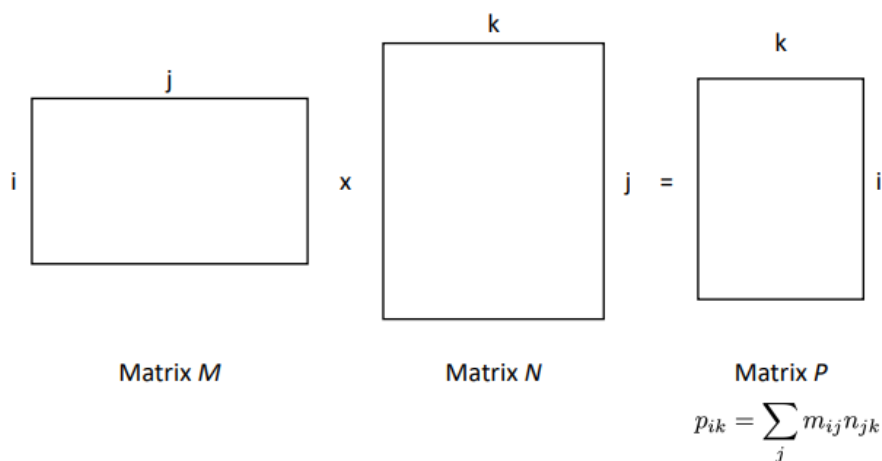


Figure 216 - l'operazione da eseguire, un prodotto tra matrici.

Si segue la stessa filosofia di prima. Il numero di righe della prima matrice deve essere uguale al numero di colonne della seconda matrice. In questo caso una matrice può essere vista come una tabella di 4 attributi: (*nome matrice*, *indice riga*, *indice colonna*, *valore*).

Molto spesso con matrici molto grandi, queste matrici sono sparse, ovvero contengono molti valori uguali a 0, in questo caso gli elementi corrispondenti a 0 vengono omessi nella rappresentazione a tuple, perché gli elementi 0 nella moltiplicazione non interferiscono.

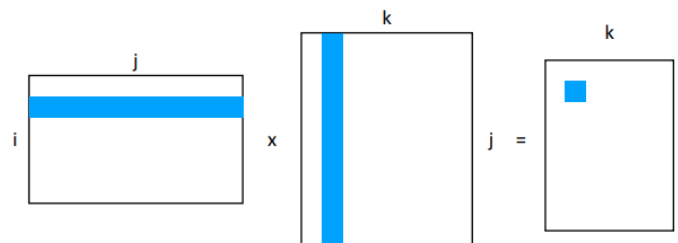


Figure 217 - Moltiplicazione parziale.

Algorithm 1: The Map Function

```

1 for each element  $m_{ij}$  of  $M$  do
2   produce  $(key, value)$  pairs as  $((i, k), (M, j, m_{ij}))$ , for  $k = 1, 2, 3, \dots$  up
   to the number of columns of  $N$ 
3 for each element  $n_{jk}$  of  $N$  do
4   produce  $(key, value)$  pairs as  $((i, k), (N, j, n_{jk}))$ , for  $i = 1, 2, 3, \dots$  up
   to the number of rows of  $M$ 
5 return Set of  $(key, value)$  pairs that each key,  $(i, k)$ , has a list with
   values  $(M, j, m_{ij})$  and  $(N, j, n_{jk})$  for all possible values of  $j$ 

```

Algorithm 2: The Reduce Function

```

1 for each key  $(i, k)$  do
2   sort values begin with  $M$  by  $j$  in  $list_M$ 
3   sort values begin with  $N$  by  $j$  in  $list_N$ 
4   multiply  $m_{ij}$  and  $n_{jk}$  for  $j_{th}$  value of each list
5   sum up  $m_{ij} * n_{jk}$ 
6 return  $(i, k), \sum_{j=1} m_{ij} * n_{jk}$ 

```

Figure 218 – Pseudocodice moltiplicazione.

Fuori da ogni singolo elemento produciamo numerose coppie chiave-valore: (i, k) sono le chiavi, sono indici del prodotto e il valore (M, j, m_{ij}) e li calcoliamo per ogni valore di k . Ripetiamo in modo equivalente per N considerando i e non j .

Nella funzione reduce, consideriamo le liste come hashmap, utilizziamo put e get, memorizziamo l'hashmap senza ordinamenti. Alla fine, abbiamo due hashmap differenti, prendiamo gli elementi da esse e calcoliamo tutti i prodotti e infine sommiamo i prodotti.

$(i, k), (M, j, m_{ij})$		
$m_{11} = 1$		
$(1, 1), (M, 1, 1)k = 1$		
$(1, 2), (M, 1, 1)k = 2$		
$m_{12} = 2$	$(i, k), (N, j, n_{jk})$	
$(1, 1), (M, 2, 2)k = 1$	$n_{11} = a$	
$(1, 2), (M, 2, 2)k = 2$	$(1, 1), (N, 1, a)i = 1$	
.....	$(2, 1), (N, 1, a)i = 2$	
$m_{23} = 6$	$n_{21} = c$	
$(2, 1), (M, 3, 6)k = 1$	$(1, 1), (N, 2, c)i = 1$	
$(2, 2), (M, 3, 6)k = 2$	$(2, 1), (N, 2, c)i = 2$	
	$n_{31} = c$	
	$(1, 1), (N, 3, c)i = 1$	
	$(2, 1), (N, 3, c)i = 2$	
		
	$n_{32} = f$	
	$(1, 2), (N, 3, f)i = 1$	
	$(2, 2), (N, 3, f)i = 2$	
		$((i, k), [(M, j, m_{ij}), (M, j, m_{ij}), \dots, (N, j, n_{jk}), (N, j, n_{jk}), \dots])$
		$(1, 1), [(M, 1, 1), (M, 2, 2), (M, 3, 3), (N, 1, a), (N, 2, c), (N, 3, e)]$
		$(1, 2), [(M, 1, 1), (M, 2, 2), (M, 3, 3), (N, 1, b), (N, 2, d), (N, 3, f)]$
		$(2, 1), [(M, 1, 4), (M, 2, 5), (M, 3, 6), (N, 1, a), (N, 2, c), (N, 3, e)]$
		$(2, 2), [(M, 1, 4), (M, 2, 5), (M, 3, 6), (N, 1, b), (N, 2, d), (N, 3, f)]$
$[(M, 1, 1), (M, 2, 2), (M, 3, 3), (N, 1, a), (N, 2, c), (N, 3, e)]$		
$list_M = [(M, 1, 1), (M, 2, 2), (M, 3, 3)]$		
$list_N = [(N, 1, a), (N, 2, c), (N, 3, e)]$		
	$P(1, 1) = 1a + 2c + 3e$	
	$P(1, 1) = 1a + 2c + 3e$	
	$P(1, 2) = 1b + 2d + 3f$	
	$P(2, 1) = 4a + 5c + 6e$	
	$P(2, 2) = 4b + 5d + 6f$	

Figure 219 – Esempio moltiplicazione.

Laboratorio 2 - Hadoop

Hadoop è un framework basato sul modello di programmazione MapReduce. È composto da due componenti HDFS e YARN.

MapReduce job

Unità di lavoro che il client vuole sia eseguita. Consiste in:

- Input.
- Programma di MapReduce.

- Configurazione.

Hadoop esegue i MapReduce Job dividendoli in **task**.

Hadoop task

Unità di lavoro di Hadoop. Consiste in:

- Map task.
- Reduce task.

Questi task sono schedulati tramite un componente chiamato YARN e sono eseguiti sui nodi di un cluster. Se un task fallisce, sarà schedulato automaticamente su un altro nodo.

Input splits

Hadoop divide l'input di porzioni di lunghezza fissa chiamati input splits. Hadoop crea un map task per ogni split.

HDFS – Hadoop Distributed File System

Il file system distribuito di Hadoop, progettato per essere eseguito su molte macchine (*organizzate in cluster*) che singolarmente hanno prestazioni mediocri o comunque non eccezionali (commodity hardware).

Questo file system distribuito deve essere altamente fault tolerant, i dati sono replicati tra i nodi per implementare questo aspetto.

Qualsiasi file system distribuito si basa su:

- **Divisione dei dati in chunk** (*tipicamente 64 megabytes nella version 1.0, mentre 128 megabyte dalle versioni 2.0 in poi*)
- **Ogni chunk è replicato circa tre volte** su compute nodes diversi. HDFS farà in modo che due repliche di un chunk non siano mai nello stesso rack (*un rack è un gruppo di macchine server messi dentro una specie di armadio*).
- **La posizione dei chunk è decisa da un nodo master** (*anch'esso replicato*).

Blocchi di memoria (*chunk*)

In un normale file system il blocco è la minima unità di scrittura/lettura, di solito un blocco del file system ha dimensioni di qualche kilobyte. I blocchi dei dischi sono solitamente di 512 byte.

Un blocco di un DFS è molto più grande (*appunto 128 megabyte*). Il motivo per cui sono più grossi è quello di minimizzare il costo della ricerca di un chunk. Grazie all'astrazione a blocchi, un file HDFS può occupare più chunk e può essere memorizzato in blocchi non contigui, può essere più grande di un singolo disco, utilizzando i blocchi come unità di astrazione migliora il sottosistema di storage.

Ogni blocco poi viene anche replicato e distribuito tra i vari host del cluster. Tipicamente i chunk non vengono replicati più volte nello stesso rack, se tale rack fallisce, tutte le repliche vengono perse. È possibile decidere il grado di replicazione e la dimensione dei blocchi.

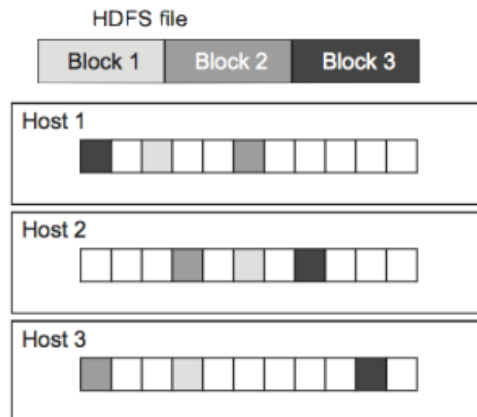


Figure 220 - Come sono disposti i record in un HDFS.

Architettura di HDFS

È una architettura master/slave composta da un namenode e una serie di compute node.

Master (*namenode*)

Ce ne è uno solo in tutto il cluster HDFS, è un server master che gestisce il file system e regola l'accesso dei client verso i file.

- Gestisce il mapping dei file nei blocchi.
- Mantiene aggiornato l'albero del filesystem.
- Conosce le informazioni e i metadati di ogni file.
- Conosce la locazione di ogni blocco.
- Gestisce i permessi di accesso per ogni blocco.

Slaves (*datanode*)

Di solito ogni nodo nel cluster è datanode (*spesso il namenode è anche datanode*). Essi sono quelli che effettivamente gestiscono i blocchi fisicamente.

Al momento della memorizzazione il file viene diviso in blocchi e spartito tra i vari datanode secondo una politica decisa dal namenode.

I datanode gestiscono le richieste di lettura/scrittura, creazione, cancellazione e replicazione sotto istruzioni del master.

Per verificarne l'attività ogni datanode invia periodicamente un heartbeat (*segnale utilizzato sia in HDFS che in YARN*) al namenode e viceversa.

In figura il client è un'interfaccia, un qualsiasi componente software per interagire con HDFS.

Replicazione viene gestita direttamente tra i datanode, il namenode non viene coinvolto.

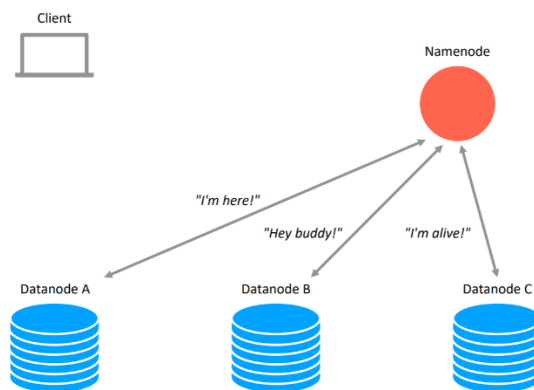


Figure 221 - Heartbeat tra i datanode e il namenode.

Lettura nel HDFS

1. Il client chiede al master che intende leggere un file.
2. Il master dice al client (*dopo eventuali verifiche di permessi*) le locazioni dei blocchi da leggere (vengono inviati i metadati).
3. Il client legge i blocchi voluti (*se un datanode è occupato, proverà a leggere dalla replica*).

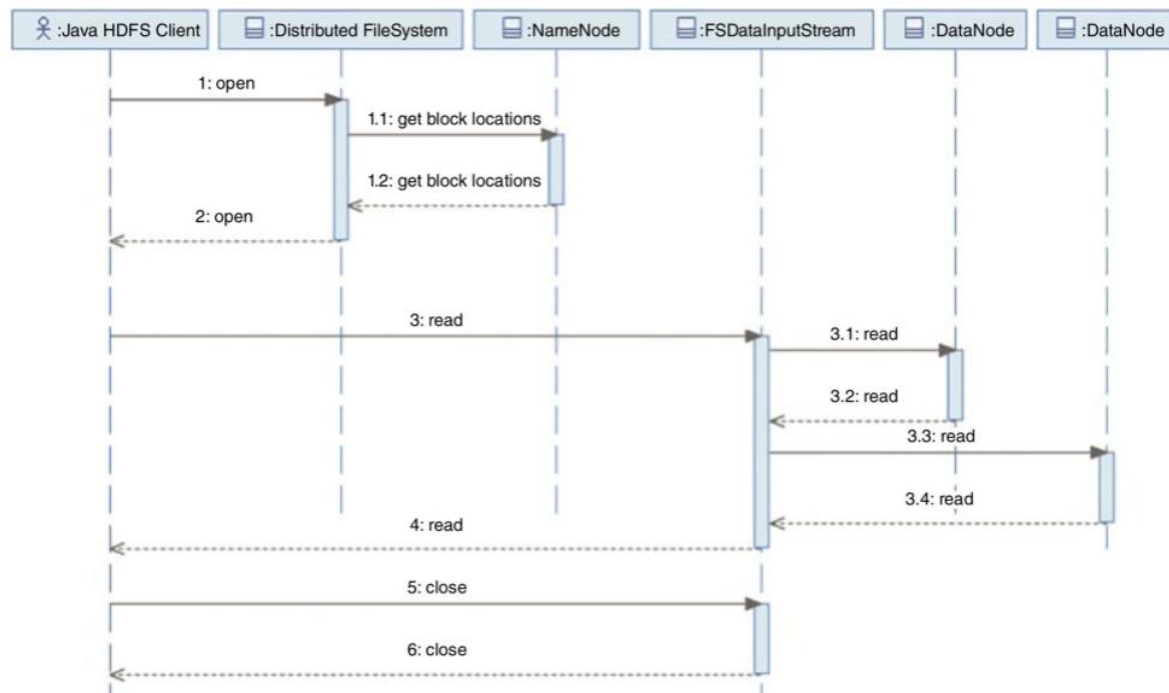


Figure 222 - Workflow di una lettura di un file.

Come determinare la replica più vicina al client (*in modo da ridurre la quantità di dati in movimento*)?

Il client può essere un map task o un reduce task, in questo caso è una lettura, quindi è un map task. Il client utilizza una API per richiedere una lettura, questa richiesta viene gestita da un'interfaccia di Hadoop (Distributed FileSystem).

1. Client → Distributed FileSystem: Richiesta lettura di un file
2. Distributed FileSystem → NameNode: Richiesta delle posizioni dei blocchi che devono essere letti
3. NameNode → Distributed FileSystem: Risposta di tutte le possibili posizioni dei blocchi di quel file. Le posizioni vengono inviati in un ordine specifico, ovvero in ordine in base alla vicinanza rispetto a chi vuole leggere il blocco.
4. Client → FSDataInputStream: Richiesta lettura blocchi del file.
5. FSDataInputStream → DataNode1: Richiesta di blocchi.
6. DataNode1 → FSDataInputStream: Trasferimento dei blocchi.
7. FSDataInputStream → DataNode2: Richiesta di blocchi.
8. DataNode2 → FSDataInputStream: Trasferimento dei blocchi.
9. FSDataInputStream → Client: il client legge il file composto da blocchi consecutivi (non conosce divisione in blocchi del file).
10. Client → FSDataInputStream: Chiusura della connessione una volta letto il file intero.
11. FSDataInputStream → Client: Chiusura della connessione.

Scrittura nel HDFS

1. Il client chiede al master che intende creare un file (*specificando grandezza e altro*).

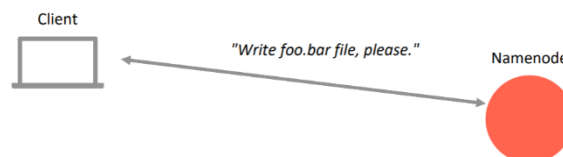


Figure 223132 - Il client chiede al master di scrivere un file.

2. Il namenode calcola quanti blocchi sono necessari e attraverso una politica di scheduling decide i datanode su cui scrivere i blocchi del nuovo file.

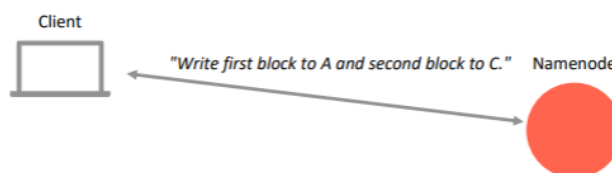


Figure 224 - Il namenode da istruzioni al client su cosa deve fare.

3. Il namenode li comunica al client i datanode su cui scrivere i blocchi.
4. Il client scrive i blocchi sui relativi nodi.

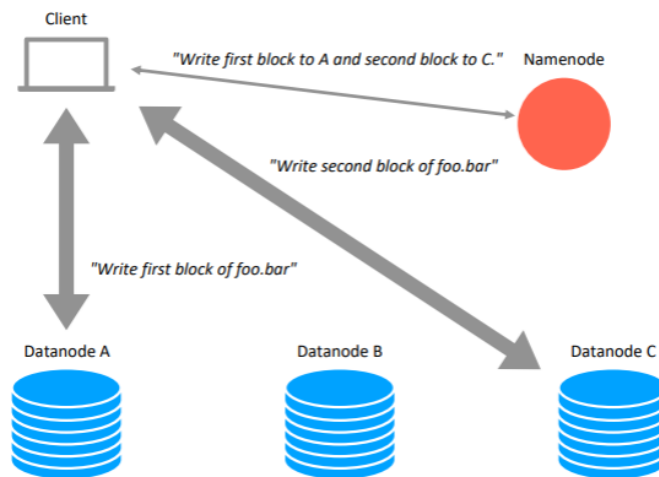


Figure 133 - Il client effettua le scritture.

5. Il primo datanode inoltra il pacchetto (*replica*) al secondo datanode designato.
6. Il secondo datanode inoltra il pacchetto (*replica*) al terzo datanode designato.

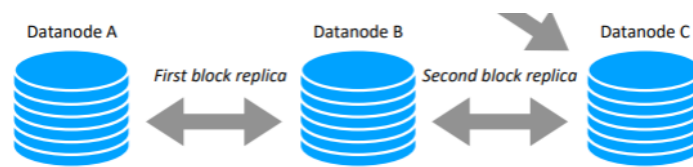


Figure 226 - Processo di replicazione.

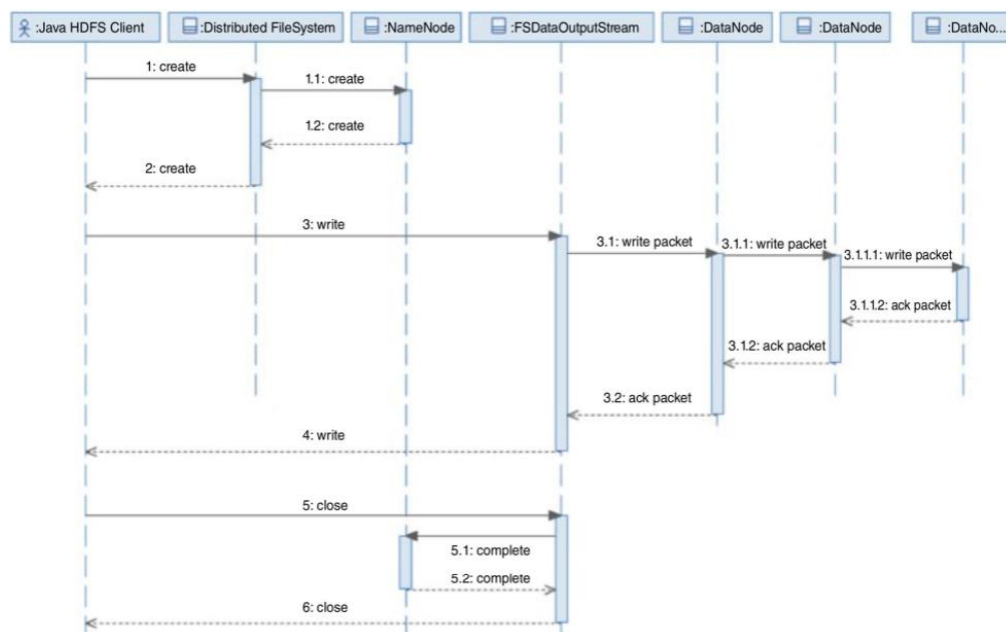


Figure 227 - Flusso di esecuzione di una scrittura.

In questo esempio abbiamo un file di un solo blocco.

1. Client → Distributed FileSystem: Richiesta creazione di un file.
2. Distributed FileSystem → NameNode: Il namenode decide di quanti blocchi il file deve essere composto, dove devono essere memorizzati e anche informazioni sulla replicazione dei blocchi.
3. NameNode → Distributed FileSystem: Risposta con lista di DataNode per i blocchi (*dove bisogna scrivere i blocchi*).
4. Client → FSDataOutputStream: Richiesta scrittura blocchi del file.
5. FSDataOutputStream → DataNode1: Inizio della scrittura del blocco.
6. DataNode1 → DataNode2: Replica dei dati.
7. DataNode2 → DataNode3: Replica dei dati.
8. DataNode3 → DataNode2: Dati scritti.
9. DataNode2 → DataNode1: Dati scritti.
10. DataNode1 → FSDataOutputStream: Scrittura file completata.
11. Client → FSDataOutputStream: Chiusura della connessione una volta scritto il file intero.
12. FSDataOutputStream → Client: Chiusura della connessione.

Replicazione nel HDFS

Per ogni blocco da inserire, il NameNode seleziona una serie di DataNode per ospitare le repliche.

Prima Replica - Località dei Dati

Il NameNode cerca di ridurre la latenza e ottimizzare le performance scegliendo DataNode che sono fisicamente vicini al client che effettua la scrittura.

Quindi se il client è parte del cluster HDFS, il primo DataNode scelto sarà quello stesso nodo.

Se il client è fuori dal cluster HDFS, il primo datanode è scelto randomicamente.

Seconda replica – Random su un altro rack

Su un datanode casuale su un rack diverso di quello dove è presente la prima replica.

Terza replica – Random sullo stesso rack della seconda

La terza replica è posizionata sullo stesso rack della seconda replica ma su un nodo diverso e scelto casualmente.

Ulteriori repliche - Random

Ulteriori repliche sono posizionate su nodi scelti randomicamente nel cluster facendo attenzione al load balancing.

Il sistema cerca di evitare di inserire troppe repliche nello stesso rack/nodo.

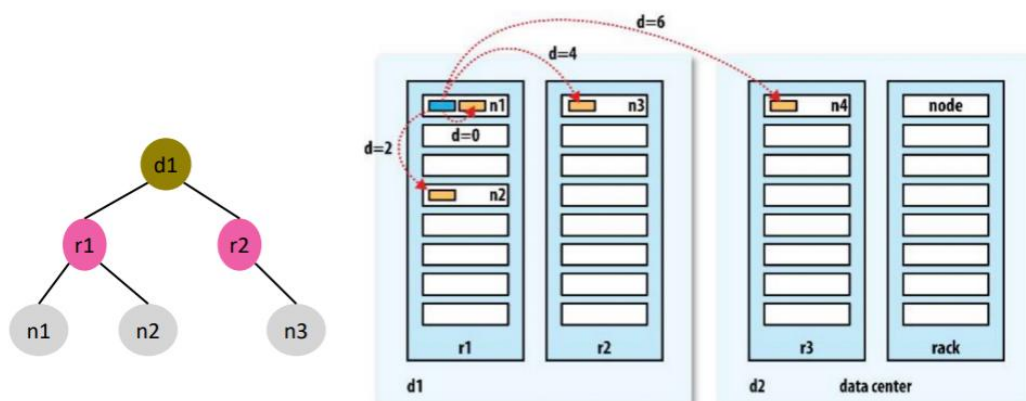


Figure 228 - Criterio di selezione dei nodi in cui mettere le repliche.

Come si vede in figura sopra, un cluster di Hadoop è rappresentato come un albero dove d1 e d2 sono due datacenter, i dati in ogni datacenter sono organizzati in rack, in d1 abbiamo r1 e r2, per capire le distanze tra i nodi, i datacenter, rack e server vengono rappresentati come un grafo, per capire la distanza tra due server (ad esempio tra n1 ed n2) bisogna scorrere il

grafo verso l'alto e contare i nodi incontrati finché non viene raggiunto il primo antenato in comune.

Nell'esempio n1 e n2 distano entrambi 1 dal nodo comune r1, quindi la loro distanza è 2.

Hadoop 1.0

La versione 1.0 di Hadoop è completamente diversa dalle versioni successive. In Hadoop 1.0 ho il HDFS (*data storage*) e un'implementazione di MapReduce (*data processing*). L'implementazione MapReduce è quella che gestisce le risorse, per eliminare le limitazioni di questo approccio, Hadoop 2.0 e 3.0 hanno introdotto un terzo livello YARN inserito tra HDFS e MapReduce.

Quali erano i limiti di Hadoop 1.0?

- **Poca flessibilità:** i task erano solo di due tipi (*map* e *reduce*), eventuali task di reduce non potevano usare le risorse dei task map e viceversa.

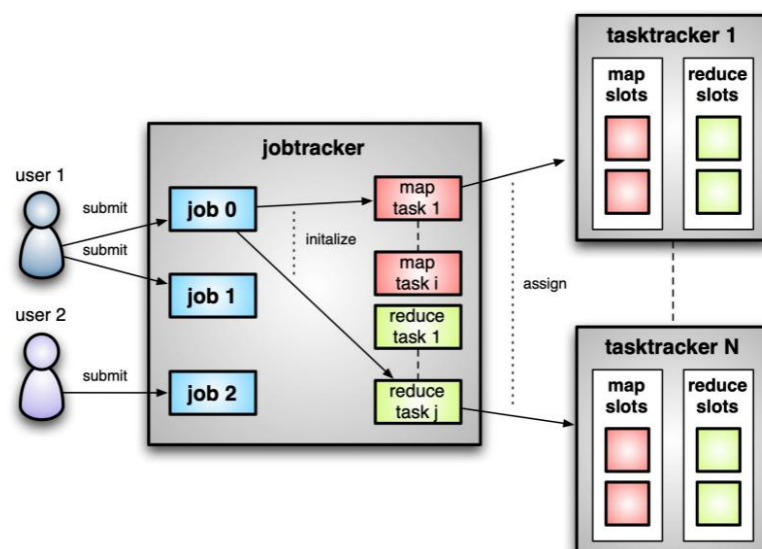


Figure 229 – Architettura di Hadoop 1.0

Un **job** è un lavoro che il client vuole sia eseguito (è un programma MapReduce).

Un **task** è un processo di map reduce che viene eseguito su un nodo del cluster.

Uno **slot** è un elemento di processazione per i task rimossi in hadoop 2.0 e 3.0. uno slot è un insieme di risorse hardware (CPU, memoria ecc) utilizzato per eseguire un map task o reduce task.

Un problema in Hadoop 1.0 è che uno slot ha una quantità di risorse fissa; quindi, se uno slot non ha abbastanza risorse per un task, inevitabilmente questo task rallenta, mentre se il task richiede meno risorse di quelle assegnate stiamo sprecando risorse, perché non possono essere assegnate ad altri task.

Un secondo problema degli slot è che ognuno di loro ha associato una label, che può essere map o reduce; quindi, gli slot che hanno map come label possono essere utilizzati per eseguire solo map task, mentre gli slot che hanno reduce come label possono essere utilizzati per eseguire solo reduce task.

Questo è un grande limite, se una macchina ha solo map task, non è possibile eseguire reduce task su essa.

Un altro problema in Hadoop 1.0 sono i jobtracker. Hadoop 1.0 utilizza il metodo master-slave, il master è il jobtracker, gli slave sono i tasktracker,

Jobtracker

Processo che continuamente accetta i job inviati dai client, crea i task MapReduce dai job, assegna i task ai tasktracker (uno per ogni macchina fisica) e continuamente traccia lo stato di esecuzione dei task e dei task tracker.

Il jobtracker è un elemento di centralizzazione (*single point of failure*) che limita la scalabilità del sistema. Aggiungendo nuovi job, più componenti ci saranno nel cluster e questo approccio non scala, è limitato dal numero di nodi nel cluster per il monitoraggio ed è limitato dal numero di job, perché il jobtracker deve gestirli. Replicarlo è molto complicato, c'è solo una replica del jobtracker, se fallisce, l'intero cluster è down.

Il jobtracker ha una quantità di lavoro eccessiva, è un grande bottleneck perché deve tenere traccia dello stato di tutti i task e tasktracker che in un sistema distribuito possono essere centinaia.

Tasktracker

Esegue dei task MapReduce su istruzione del jobtracker. Gestisce memorizzazione e trasmissione di output intermedi, invia report sullo stato di esecuzione del task al jobtracker.

Hadoop 1.0 supporta un massimo di 4000 nodi e 40000 task concorrenti.

Hadoop 2.0 – YARN - Yet Another Resource Negotiator

Framework di gestione delle risorse introdotto con Hadoop 2.0. mentre in Hadoop 2.0 e 3.0 questi numeri possono raggiungere rispettivamente i 10000 e i 100000.

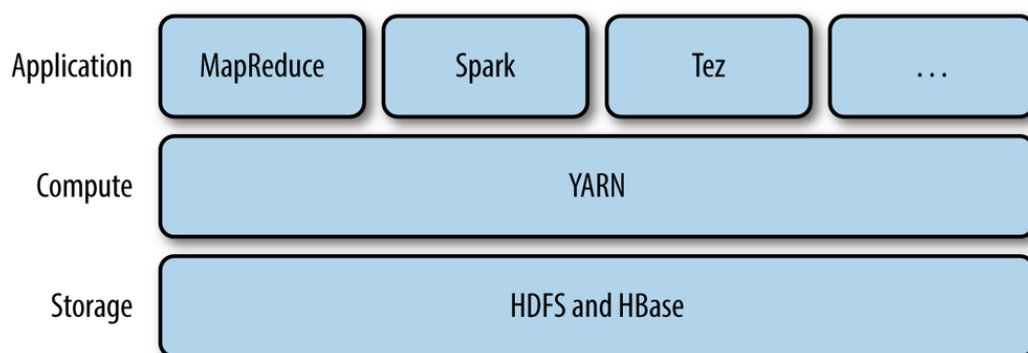


Figure 230 - Architettura di Hadoop 2.0

Con l'aumentare della complessità e della varietà dei carichi di lavoro gestiti da Hadoop, c'era bisogno di un sistema più flessibile e scalabile per gestire le risorse del cluster.

Con YARN, le risorse di calcolo e di archiviazione del cluster Hadoop possono essere sfruttate in modo più efficiente e possono essere eseguite più applicazioni contemporaneamente nello stesso cluster.

In YARN non ci sono più jobtracker e tasktracker.

YARN fornisce utili servizi per processi a lunga durata come ad esempio:

- **ResourceManager** (*uno per cluster, però non è detto*) per gestire le risorse del cluster.
- **NodeManager** (*uno per ogni nodo del cluster*) per lanciare i container e monitorare l'uso delle risorse del container.

YARN risolve le limitazioni degli slot introducendo i container. Un container è un set di risorse computazionali allocate per eseguire uno specifico processo. Le risorse assegnate ad un container possono anche essere cambiate dinamicamente, non ha un insieme fisso di risorse.

Inoltre, un container non ha label, quindi può eseguire qualsiasi task map o reduce.

Il problema del bottleneck del jobtracker viene risolto nel seguente modo: il resource manager gestisce solo le risorse, non monitora l'esecuzione dei task.

Il node manager invia heartbeat al resource manager, però i map task e reduce task non inviano gli heartbeat al resource manager, li inviano ad un altro componente, l'application master. Ogni job ha un application master, i task del job comunicano con l'application master di quel job.

ResourceManager (RM)

Eseguito solo su un nodo del cluster, è responsabile della gestione delle risorse globali del cluster e della pianificazione delle risorse per le applicazioni.

Il ResourceManager è composto da due parti:

- **ApplicationManager** che accetta richieste di risorse dagli utenti.
- **Scheduler** che assegna risorse alle applicazioni.

NodeManager (NM)

Eseguito su ciascun nodo del cluster ed è responsabile della gestione delle risorse disponibili sul singolo nodo, monitorando l'utilizzo delle risorse da parte delle applicazioni e comunicando con il ResourceManager.

ApplicationMaster (AM)

Componente cruciale che gestisce l'esecuzione di un'applicazione specifica all'interno del cluster.

Ogni applicazione in esecuzione su YARN ha il proprio Application Master, il quale è responsabile di diverse funzioni chiave per garantire che l'applicazione venga eseguita correttamente ed efficientemente.

L'Application Master può essere personalizzato per diverse tipologie di applicazioni (es., *MapReduce*, *Spark*, *Tez*), rendendo YARN una piattaforma versatile.

In caso di fallimento dell'Application Master, YARN può rilanciare l'Application Master su un altro nodo del cluster per riprendere l'esecuzione dell'applicazione.

Gestione delle Risorse

Richiesta di Risorse: L'Application Master richiede risorse specifiche (*CPU*, *memoria*) al Resource Manager di YARN per eseguire i container necessari per l'applicazione.

Assegnazione delle Risorse: Una volta che le risorse sono allocate, l'Application Master avvia i container sui nodi del cluster.

Monitoraggio

Stato dell'Applicazione: Monitora lo stato dei tasks eseguiti nei container, inclusi il loro progresso e la loro completamento.

Gestione degli Errori: Rileva e gestisce i fallimenti dei tasks, eventualmente richiedendo nuove risorse per rilanciare i tasks falliti.

Coordinamento

Coordinazione dei Tasks: Coordina l'esecuzione di tasks paralleli, assicurandosi che vengano eseguiti nell'ordine corretto e gestendo le dipendenze tra tasks.

Comunicazione: Comunica con i mappers e i reducers (nel caso di applicazioni MapReduce) o altri tipi di tasks in base al framework specifico utilizzato.

Gestione del Ciclo di Vita

Inizializzazione: Avvia i tasks necessari all'inizio dell'applicazione.

Completamento: Gestisce la chiusura dell'applicazione e il rilascio delle risorse una volta che l'applicazione è completata.

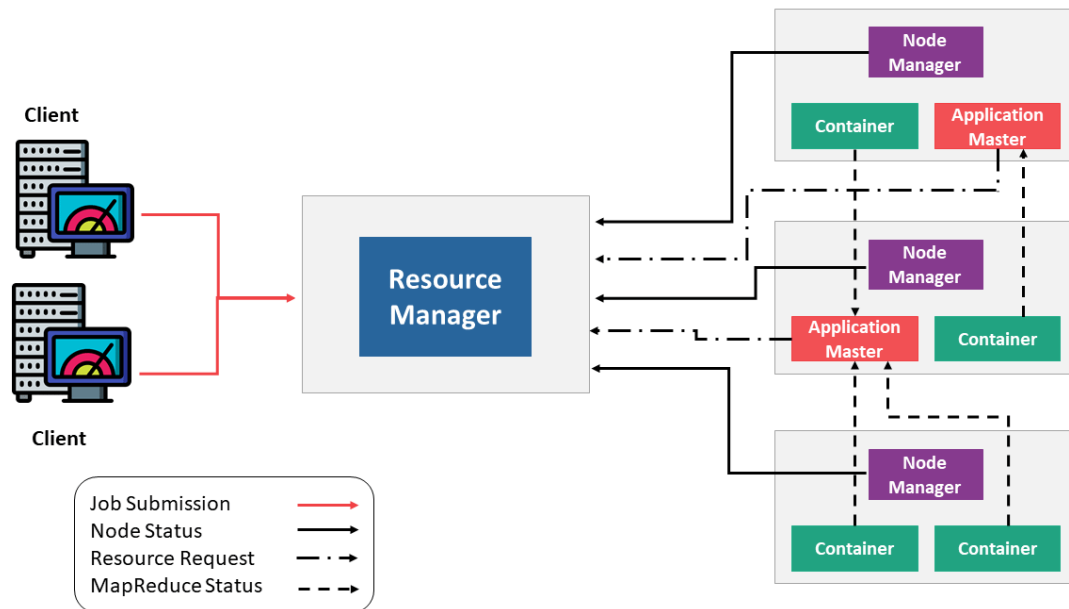


Figure 231 – Diagramma che mostra il workflow e l'architettura di YARN.

Esecuzione di un'Applicazione

In figura i componenti blu sono quelli forniti da Hadoop, tutti tranne Job appartengono a YARN, Job appartiene a MapReduce. Nuvola è HDFS. I componenti arancioni invece sono quelli sviluppati dallo sviluppatore. Notiamo che i componenti già forniti da Hadoop sono in maggioranza rispetto a quelli da sviluppare, facilitando il lavoro agli sviluppatori.

1 - Inizio dell'Applicazione

L'utente esegue il job interagisce con il modulo Job che espone un'interfaccia verso HDFS e il resource manager. Le informazioni che vengono scritte in HDFS sono il jar file che include i task map e reduce, gli input split e le configurazioni (object configuration).

Quando un utente sottomette un'applicazione a YARN, il Resource Manager alloca un container su un nodo del cluster per eseguire l'application Master dell'applicazione.

2 - Richiesta di Risorse

L'Application Master, una volta avviato effettua alcune azioni preliminari e recupera gli input split da HDFS (tra gli split ci sono altre informazioni come dimensione degli split e location dei blocchi relativi a tali split).

Una volta ricevute tali informazioni comunica con il Resource Manager per richiedere ulteriori risorse necessarie per eseguire i tasks dell'applicazione.

3 - Esecuzione dei Tasks

Le risorse allocate vengono utilizzate per eseguire i tasks nei container distribuiti sui nodi del cluster. Un task può essere eseguito completamente all'interno della JVM dell'application master (uber mode), ciò non avviene quasi mai perché solitamente viene eseguito nel nodo in cui l'application master conosce che c'è una copia dei dati che richiede.

A questo punto se il task viene assegnato ad un nodo, l'application master comunica direttamente con il node manager del nodo. Il node manager esegue una JVM composta da YarnChild e il task map o reduce da eseguire, lo YarnChild contiene le principali funzioni della JVM, chiede ad HDFS il jar file e le informazioni di configurazione.

4 - Monitoraggio e Coordinamento

L'Application Master monitora e coordina l'esecuzione dei tasks, gestendo errori e ripetizioni se necessario.

5 - Completamento

Una volta che tutti i tasks sono completati, l'Application Master comunica al Resource Manager che l'applicazione è terminata e rilascia le risorse allocate.

Inoltre, vengono scambiati gli heartbeat per conoscere l'utilizzazione delle risorse, il progresso dei task e per fault tolerance.

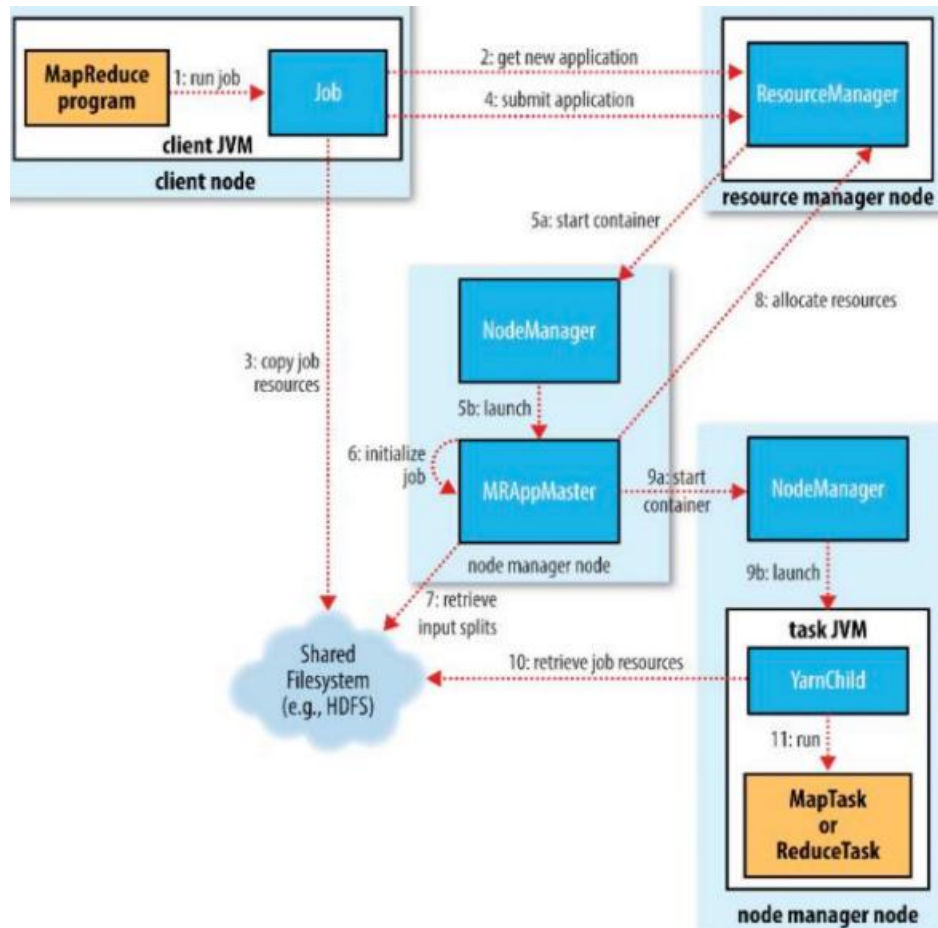


Figure 232 – Diagramma che mostra il workflow dell'esecuzione di un'applicazione con YARN.

Lo scheduler di YARN

Le risorse del cluster sono limitate, quindi serve un sistema che ne regoli l'utilizzo.

Scheduler FIFO

Lo scheduler può adottare una logica FIFO, ossia le applicazioni sono messe in una coda e messe in esecuzione secondo l'ordine di avvio.

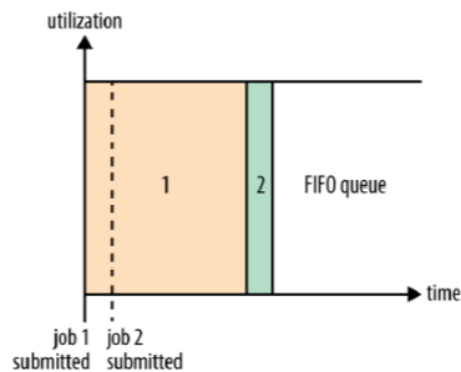


Figure 233 - Esempio di scheduling FIFO.

Appena arriva la prima richiesta vengono soddisfatte le sue richieste (*quindi si allocano queste risorse*), appena la richiesta è soddisfatta le risorse vengono liberate e la prossima applicazione viene servita.

Semplice e non richiede configurazioni, ma una schedulazione di questo tipo è inadatta ad un sistema con vari cluster poiché alcuni job potrebbero soffrire di starvation: ad esempio il job 2 dovrà aspettare molto prima di andare in esecuzione poiché il job 1 è molto grande.

Scheduler con politica “Capacity”

Si hanno varie code (*ognuna gestita con politica FIFO*).

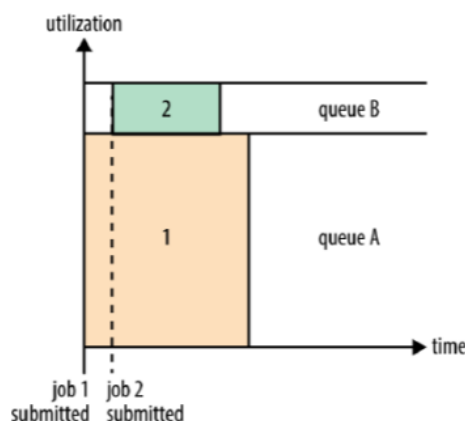


Figure 234 - Esempio di capacity scheduling.

Il cluster è suddiviso in diverse code, ciascuna delle quali ha una capacità configurata in termini di percentuale delle risorse totali del cluster.

Ad esempio, si possono avere code A, B e C che ricevono rispettivamente il 50%, 30% e 20% delle risorse del cluster.

Le risorse sono allocate alle applicazioni basandosi sulla capacità configurata per ogni coda. Se una coda ha una capacità del 50%, può utilizzare fino al 50% delle risorse del cluster.

All'interno di ogni coda i job vengono schedulati secondo una politica FIFO.

In questo caso un job grande impiega più tempo per essere eseguito rispetto allo scheduler FIFO.

Fair Scheduler

Ho una sola coda ma è gestita in maniera non FIFO.

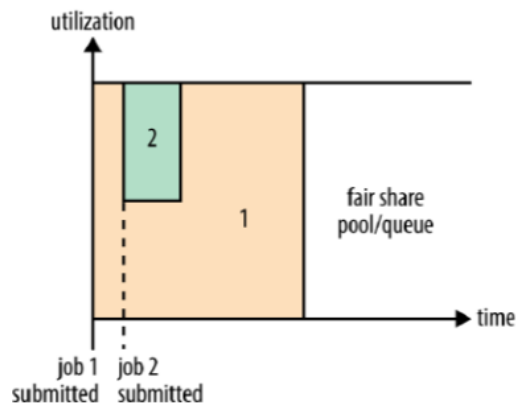


Figure 235 - Esempio di Fair scheduling.

Le risorse non sono riservate in maniera fissa ma sono dinamicamente assegnate in modo bilanciato a tutti i job in esecuzione.

Dall'esempio nell'immagine sopra:

1. Quando il primo job inizia prende tutte le risorse del cluster.
2. Quando il secondo job inizia le risorse del cluster vengono divise a metà tra il primo e il secondo job.
3. Quando il secondo job finisce, il primo si riprende tutte le risorse.

Questo scheduler è per-user, ovvero se un utente A crea due job e l'utente B crea un job, ogni job di A ottiene il 25% delle risorse, mentre al job di B viene assegnato il 50% delle risorse.

Delay scheduling

Tecnica usata insieme allo scheduler per migliorare le prestazioni delle applicazioni. Il delay scheduling è supportato sia dal fair che dal capacity scheduler. L'obiettivo principale del delay scheduling è quello di aumentare la probabilità che i tasks vengano eseguiti sui nodi dove i dati di input sono già presenti, riducendo così la necessità di trasferire i dati attraverso il cluster.

È stato dimostrato che invece di terminare un task perché non ci sono nodi disponibili, se aspettiamo 2 secondi è molto probabile che un nodo ottenga le risorse sufficienti per eseguire il nostro task in esso.

Il node manager invia periodicamente degli heartbeat al resource manager, in questi messaggi ci sono anche informazioni riguardo al nodo, ovvero relative alle risorse disponibili sul nodo. Il resource manager può effettuare il delay scheduling fino a quando non riceve un certo numero di heartbeat prestabilito, quando supera questa soglia, il delay scheduling non può essere più attuato.

1. **Località dei Dati:** Ogni task in un job ha preferenze di località dei dati, cioè nodi sui quali sarebbe preferibile eseguire il task per minimizzare i trasferimenti di dati.
2. **Assegnazione Iniziale:** Quando un nodo diventa disponibile, YARN tenta di assegnare tasks che hanno dati localizzati su quel nodo.
3. **Attesa (Delay):** Se quel nodo per cui il task con preferenze di località dei dati non è disponibile, invece di assegnare immediatamente il task ad un nodo senza preferenza di località (*che causerebbe trasferimenti di dati*) oppure senza terminare il task, YARN può aspettare per un breve periodo (*delay*).
4. **Assegnazione dopo il Ritardo:** Dopo il periodo di attesa, se quel nodo per cui il task con preferenze di località non è ancora disponibile, YARN assegnerà il task ad un nodo disponibile, anche se questo comporta il trasferimento dei dati.

Tolleranza ai guasti e ai fallimenti

Tutti i nodi del cluster di Hadoop inviano periodicamente segnali di heartbeat al NodeManager e al ResourceManager. I seguenti fallimenti sono ordinati dal meno critico al più critico.

Task Failure

Se un task non risponde o non completa il lavoro entro un tempo specifico è considerato fallito, oppure quando il codice del mapper o del reduce lancia un'eccezione in runtime, viene considerato fallito se avviene un'uscita improvvisa dalla JVM, hanging task, ovvero task che non sono falliti, però non inviano update, quindi YARN attende per 10 minuti, se trascorrono senza ricevere heartbeat, allora il task è considerato fallito.

L'application master libera il container e marca il task come fallito e cercherà di riprovare l'esecuzione del task su un nodo diverso, in preferibilmente in un nodo che ha tutte le repliche dei dati utili a quel task.

Application Master Failure

L'application master invia periodicamente un heartbeat al resource manager. Quest'ultimo rileverà un fallimento del primo e nel caso creerà un'altra istanza di application master su un nuovo container gestito da un altro node manager.

Il resource manager libera il container e cercherà di riavviarlo 2 o 3 volte su un nodo diverso del cluster. Ogni volta l'application master riprenderà il lavoro da dove era stato interrotto.

Node Manager Failure

Heartbeat periodico al resource manager; dopo dieci minuti di nessun contatto il resource manager rimuove il container dal pool; tutte i task che erano in esecuzione su tale container vengono marcate come failed e verranno schedulati in un altro nodo. Possibilmente l'application master può includere nella blacklist tale node manager.

Per ritentare i task, l'application master deve ripetere la procedura di richiesta di risorse al resource manager.

Resource manager failure

Senza di esso nessun job o task può essere lanciato; per questo è un single point of failure.

La soluzione è replicarlo e lasciare le repliche in stand-by fino al momento in cui il resource manager fallisce; questa transizione è guidata da un controller dedicato chiamato "*failover controller*" che rileva il fallimento del resource manager e manda in esecuzione una delle copie sospese.

Clients e node managers devono essere configurati per gestire un fallimento di questo tipo: provano in maniera round-robin a connettersi a tutti i resource manager fino a che non ne trovano uno attivo.

Esecuzione speculativa

I task molto lenti possono allungare di molto il termine di un job e a volte questa lentezza dipende da fenomeni temporanei e imprevedibili; quindi, è ragionevole tentare lo stesso task da un'altra parte. Per questo è stata introdotta questa tecnica:

1. L'application master tiene traccia del progresso di ogni task in un job.
2. L'application master lancia in modo speculativo due volte lo stesso task che sta andando più lento degli altri.
3. Quello che finisce prima "vince", l'altro viene terminato.

Shuffle and Sort

Livello intermedio tra il map e il reduce, il map task inserisce i risultati in un buffer in-memory (RAM), quando il buffer raggiunge la soglia dell'80%, c'è un thread nel map task (spilling thread) che prende le coppie key-value dal buffer in-memory e le copia nel disco local della macchina.

Prima avviene lo shuffle dei dati, dopo partizionati e infine ordinati. Mentre lo spilling thread ottiene il contenuto dalla memoria, il map task scrive contenuto in memoria a meno che la memoria non sia al 100% piena, in questo caso si ferma, attende che lo spilling thread svuota l'intero buffer per poter riscrivere.

Ogni volta che lo spilling thread va in azione, scriverà in un altro file nel disco locale. Infine, avviene il merge dei dati nel disco locale per creare le combinazioni di output unificate, ognuna delle partizioni andrà ad un reducer.

Il map task invia gli heartbeat all'application master, l'application master conosce quando il mapper ha dei dati pronti, i reduce task inviano heartbeat all'application master riescono a capire quando le partizioni create dal map task sono pronte.

Nel reducer c'è il copying thread che prende il contenuto dal disco e lo copia localmente nel nodo in cui il reducer è in esecuzione in un misto tra il disco locale e RAM.

Li copia in questo modo perchè inizia a copiare in RAM, quando la RAM non è disponibile inizia a copiare nel disco. Infine, tutti i pezzi sono uniti e vengono inviati in input al reduce che effettua le varie operazioni, produce il suo output che viene scritto nella directory specificata nella command line quando il job Hadoop viene eseguito.

L'output directory include l'output proveniente dai reducer ed è in HDFS non sul disco locale.

I limiti di MapReduce

MapReduce è idonea solo per computazioni one-pass

MapReduce è ottimo per computazioni di tipo “*one-pass*”, ovvero: il processo di MapReduce viene eseguito attraversando i dati solo una volta per ciascuna fase del processo.

MapReduce risulta inefficiente per computazioni “*multi-pass*”, ovvero: un processo in cui i dati devono essere attraversati e processati più volte per completare una serie di operazioni o trasformazioni. In altre parole, il set di dati viene letto e manipolato in diverse fasi o passaggi separati. Ogni passaggio può implicare una lettura e una scrittura dei dati intermedi prima che il prossimo passaggio possa iniziare.

MapReduce non supporta la condivisione dei dati

Infatti, non esistono primitive per la condivisione di dati tra diversi mapper o diversi reducer senza utilizzare il disco (o il DFS).

Gli stati tra gli step vanno in HDFS, mentre gli stati durante shuffle and sort vanno nel disco locale.

Quindi comunicazioni e I/O creando un grosso collo di bottiglia.

Laboratorio 3 – Spark

Spark è un framework che si differenzia con Hadoop per le tecniche di memorizzazione, Spark utilizza tecniche in-memory (memoria RAM) a differenza di Hadoop che utilizza il disco. Grazie a queste tecniche Spark è molto più efficiente.

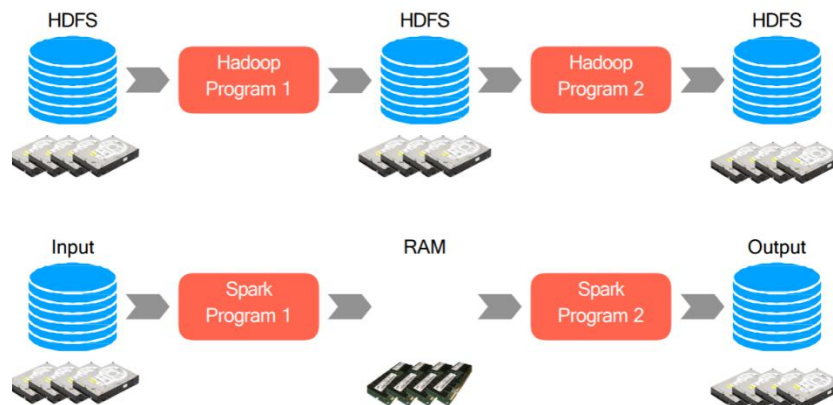


Figure 236 - Differenza tra Hadoop (sopra) e Spark (sotto), da notare come Spark utilizzi tecniche In-Memory.

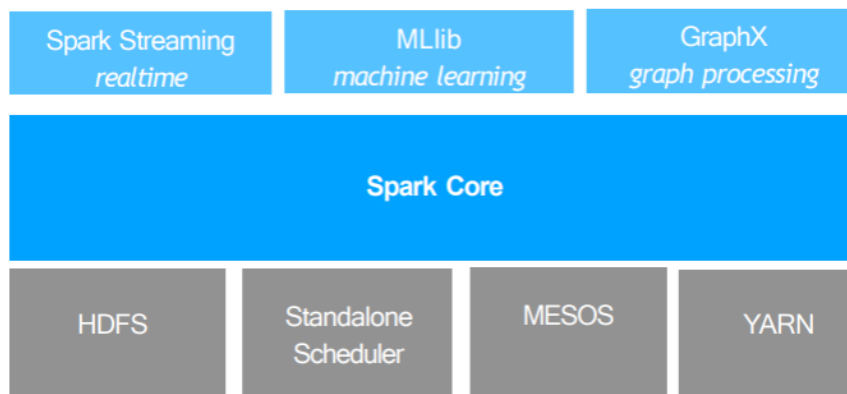


Figure 237 – Diagramma che mostra la composizione in moduli di Spark.

Spark offre le seguenti funzionalità di base per:

- Scheduling dei task.
- Memory management.
- Fault recovery.
- Interazione con sistemi di memorizzazione.

Spark restringe l'interfaccia di programmazione, esprime calcoli complessi come grafi di operazioni ad alto livello. Il sistema sceglie come dividere (split) ogni operatore in task e dove eseguire ogni task, le varie parti vengono eseguite più volte per il fault recovery.

Resilient Distributed Dataset – RDD

Una delle strutture dati fondamentali di Spark, progettata per gestire e processare grandi quantità di dati in modo distribuito e fault-tolerant.

Un RDD è una collezione di oggetti immutabili distribuita nel cluster che può essere manipolata in maniera parallela e resiliente.

Il core di Spark è incentrato nel fornire funzionalità (API) che vano a manipolare queste collezioni.

Gli RDD sono distribuiti

Gli RDD sono distribuiti su molteplici compute node di un cluster. Ciò significa che i dati sono suddivisi in partizioni e ciascuna partizione è memorizzata e processata su nodi diversi. Le partizioni sono un pezzo atomico d'informazione.

Il numero di partizioni può essere specificato dall'utente o determinato automaticamente da Spark in base alla sorgente dei dati e alle risorse disponibili nel cluster.



Figure 238 - Esempio di collezione di dati distribuiti.

Una volta creato, un RDD non può essere modificato. Tuttavia, è possibile creare nuovi RDD applicando transformation all'RDD originale.

```
# Leggere il file di testo e creare un RDD
lines = sc.textFile("example.txt")
```

Figure 239 - Creazione di un RDD.

Molteplici RDD possono essere creati partendo da un dispositivo di memoria permanente.

Resilienza di un RDD

Gli RDD sono resilienti perché Spark memorizza il DAG (Directed Acyclic Graph) delle transformation utilizzate per creare l'RDD. In caso di fallimento di un nodo, Spark può ricostruire l'RDD riapplicando queste transformation ai dati di origine.

Lineage

In figura vediamo che in pochissime righe possiamo creare un programma di MapReduce, il grafo in figura è il grafo relativo al codice, nel quale abbiamo le operazioni di map che prendono il file di input e ogni colonna bianca è un RDD, mentre ogni pallino blu è una partizione di un RDD.

Ogni map task effettua calcoli in una specifica partizione di un RDD, ogni map produce in output una partizione di un RDD. Il reducer effettua delle operazioni sull'RDD ricevuto dai mapper e produce un altro RDD e lo invia in input al filter.

Alla fine, vogliamo conoscere la lineage (i discendenti) di ogni partizione di ogni RDD, quindi se selezioniamo una partizione di un RDD (ad esempio l'ultima in alto) conoscere la sua lineage significa conoscere quali partizioni degli RDD che la precedono, hanno contribuito a creare quella partizione.

```
file.map(lambda rec: (rec.type, 1))
    .reduceByKey(lambda x, y: x+y)
    .filter(lambda (type, count): count > 10)
```

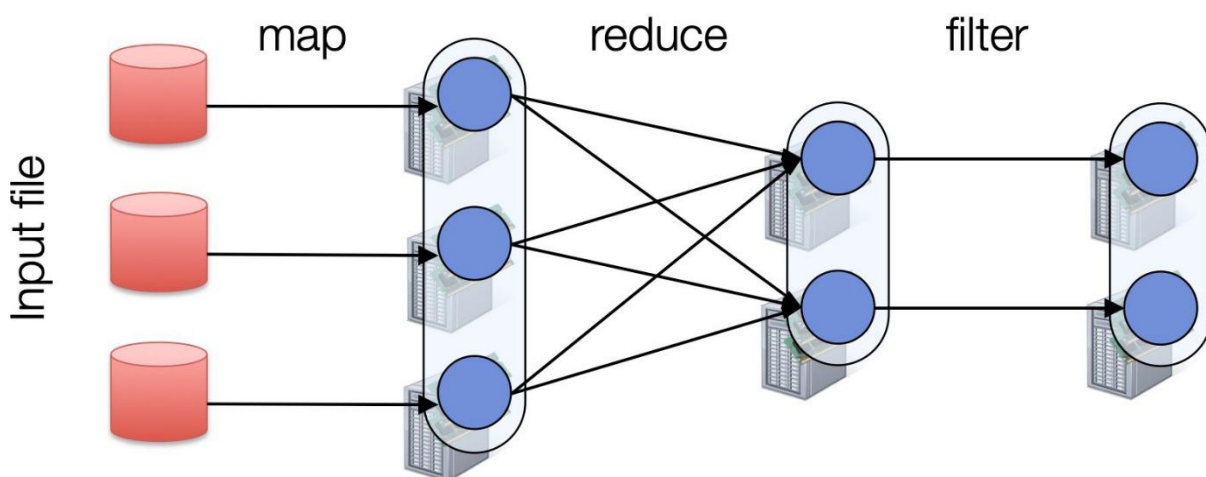


Figure 240 - Lineage.

Conoscere il lineage è importante perché se una partizione viene persa è possibile andare indietro fino alla partizione precedente che può essere utilizzata per ricalcolare la partizione persa. Per questo ogni RDD deve essere immutabile, altrimenti non si potrebbero ricostruire partizioni perse.

In alcuni casi una partizione può essere calcolata utilizzando solo una partizione, in altri servono più partizioni RDD (ad esempio nella fase reduce servono tutte le partizioni dell'RDD).

Quindi alcune operazioni sono più veloci, altre sono più lente.

Architettura di Spark

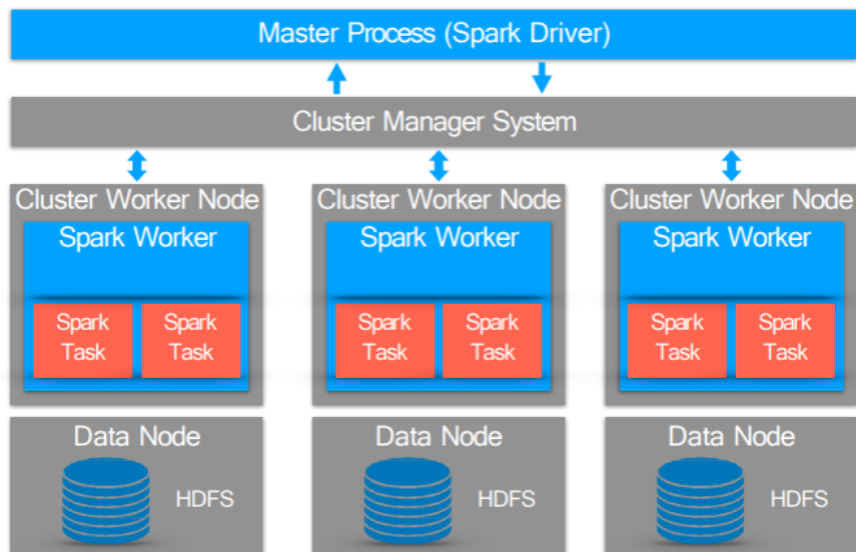


Figure 241 - Architettura di Spark in un cluster con tre nodi worker.

Lo Spark driver interagisce con il Cluster Manager System (YARN) per richiedere risorse.

In figura gli worker corrispondono ai container in YARN, sono ambienti che possono includere uno o più Spark task, i quali sono le istanze delle nostre operazioni.

Un'applicazione Spark consiste in:

- Un solo processo driver.
- Alcuni processi esecutori.

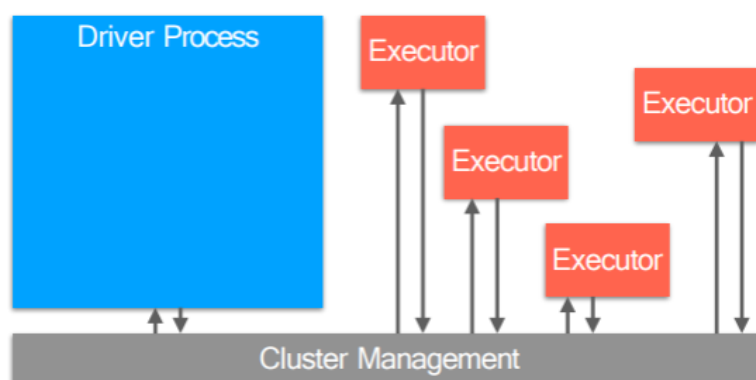


Figure 242 – Il processo driver e quattro processi esecutori.

Processo Driver

Il cuore di una applicazione Spark; viene eseguito su uno specifico nodo del cluster ed è associato al codice `main()`. I suoi compiti sono molteplici:

- Mantenere le informazioni dell'applicazione Spark.
- Interazione con l'utente.
- Analizzare, distribuire e schedare il lavoro tra i processi esecutori.

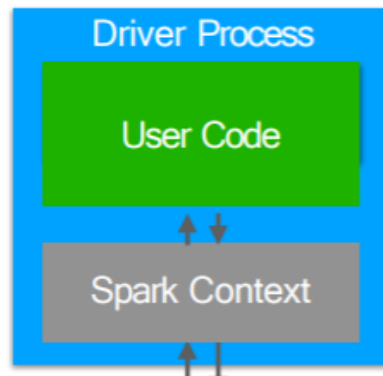


Figure 243 - Il processo Driver.

Il processo driver è composto da due componenti:

Codice utente – User code

Il codice `main()` dell'applicazione.

Contesto – Spark context

Oggetto che rappresenta la connessione con il cluster. Nella shell `pyspark` è creato automaticamente all'avvio ed è sempre accessibile tramite un'apposita variabile chiamata `sc`.

Se invece creiamo uno script Python questo oggetto deve essere creato all'inizio (*il prima possibile*) come in figura.

Lo Spark context crea il DAG in automatico dal codice e conosce la sequenza di operazioni e su quali dati ogni operazione opererà, inoltre interagisce con YARN perché in stage specifici del DAG, lo Spark context vede che serve un'operazione `map` su certi dati e quindi chiede a YARN di avviare i task nei nodi in cui ci sono i dati necessari a quei task.

```
# import spark
from pyspark import SparkConf, SparkContext
# initialize SparkConf
conf = SparkConf()
# set the application name
conf.setAppName("My Spark App")
# initialize a new SparkContext with Spark configuration
sc = SparkContext(conf=conf)
```

Figure 244 - Programma Python per inizializzare un'applicazione Spark.

Spark-submit e master flag

Per effettuare il submit di job tra tutti i cluster manager è possibile utilizzare il tool *spark-submit*.

Utilizzando il flag `--master` è possibile specificare il node manager che vogliamo esegua la nostra applicazione.

Questo flag può assumere valori differenti:

- **local**: esegue in locale con un solo core;
- **local[N]**: esegue in locale con N core;
- **local[*]**: esegue in locale con il numero massimo di core che ha la macchina;
- **yarn**: utilizza l'intero cluster tramite YARN.

Processo Esecutore

Esegue il codice assegnatogli dal processo driver e infine invia periodicamente un report sullo stato della computazione al driver.

Modello di programmazione di Spark

Spark si basa su:

- Operatori parallelizzabili.
- Funzioni di alto livello che eseguono funzioni definite dall'utente in parallelo.

Data flow – DAG

Rappresentazione delle operazioni di trasformazione che devono essere eseguite su un RDD. Il DAG descrive come le trasformazioni sono collegate tra loro e l'ordine in cui devono essere eseguite. Il concetto di DAG è fondamentale per comprendere come Spark ottimizza e pianifica l'esecuzione dei job in modo efficiente.

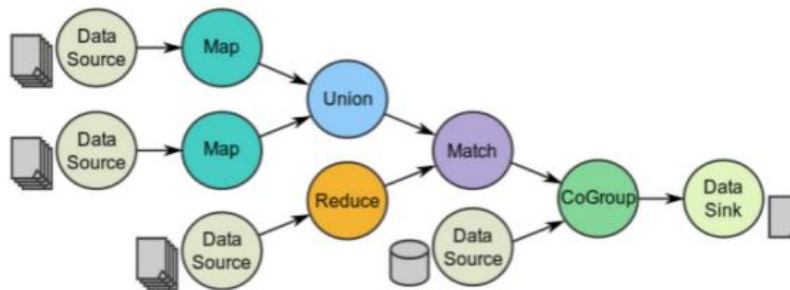


Figure 245 - Esempio di Data flow.

Un data flow è composto da molteplici sorgenti di dati, operatori, “pozzi” di dati (*data sink*) tutti connessi tramite input e output. Un data flow è un grafo aciclico (*DAG – Directed Acyclic Graph*) che rappresenta la descrizione di un job.

Ogni pallino rappresenta un'operazione, è un higher order function

Un RDD supporta due tipologie di operazione:

- **Trasformazione:** Permette di costruire un piano logico.
- **Azione:** Permette di attivare la computazione.

Trasformazione di un RDD

Inizialmente i dati in HDFS devono essere convertiti in RDD, il primo metodo per farlo è utilizzare il metodo *parallelize* che prende come argomento una struttura dati contenuta localmente in-memory nello Spark driver.

Permettono di creare un piano logico di esecuzione; nel concreto una transformation è un'operazione su un RDD che produce un nuovo RDD. Le trasformazioni consentono allo Spark context di capire il DAG che rappresenta l'applicazione.

Le transformation sono valutate in modo “*pigro*”, ovvero una trasformazione non viene eseguita immediatamente ma si costruisce un DAG e l'esecuzione verrà effettivamente effettuata a seguito di una Action.

La computazione a seguito di un'azione può essere la computazione dell'intero grafo computazionale oppure un sottoinsieme di trasformazioni in base a dove l'azione eseguita nel DAG. Alcune azioni lavorano solo su un sottoinsieme di trasformazioni e partizioni di un RDD.

Esempi di transformation includono `map`, `filter`, `flatMap`, `groupByKey` e `reduceByKey`.

```
numbers = [1 , 2 , 3 , 4]

rdd = sc.parallelize(numbers) # creazione di un RDD partendo da una variabile

rdd2 = sc.textFile("path/to/file") # creazione di un RDD partendo da un file

words = "I love Spark".split(" ")

Rdd_words = sc.parallelize(words, 2) # creazione di un RDD partendo da una
variabile e indicando anche il numero di partizioni volute (2). Spark in
questo caso prende il 2 come consiglio, se però preferisce crearne di più o
di meno non segue il consiglio

# vediamo ora qualche esempio di trasformazione sul primo rdd

rdd2 = rdd.map(lambda x: x * 2) # RDD contenente [2, 4, 6, 8]

rdd3 = rdd.filter(lambda x: x % 2 == 0) # RDD contenente [2, 4]


rdd = sc.parallelize(["hello world", "apache spark"])

rdd2 = rdd.flatMap(lambda x: x.split(" ")) # RDD contenente ["hello",
"world", "apache", "spark"]


rdd = sc.parallelize([('a', 1), ('b', 2), ('a', 3)])
```



```

rdd2 = rdd.groupByKey() # RDD contenente [('a', [1, 3]), ('b', [2])]

rdd = sc.parallelize([('a', 1), ('b', 2), ('a', 3)])

rdd2 = rdd.reduceByKey(lambda x, y: x + y) # RDD contenente [('a', 4),
('b', 2)]

```

La funzione lambda in Python è una funzione anonima che utilizziamo per definire la logica di un operatore.

Le trasformazioni si dividono in due categorie:

Wide transformation

Quelle in cui i dati devono essere riorganizzati tra le partizioni. Ciò significa che una partizione dell'RDD di output può dipendere da molte partizioni dell'RDD di input.

Lavorano su diverse partizioni RDD.

Queste trasformazioni richiedono uno shuffle dei dati, che implica un movimento dei dati tra i nodi del cluster, rendendo queste operazioni **più costose in termini di tempo di esecuzione e utilizzo delle risorse** rispetto alle trasformazioni narrow.

Ad esempio, la funzione `groupByKey` raggruppa i dati basandosi sulle chiavi, causando uno shuffle per distribuire le chiavi tra le partizioni.

```

val rdd = sc.parallelize(Seq((1, "a"), (2, "b"), (1, "c"), (2, "d")))

val result = rdd.groupByKey() // result: [(1, [a, c]), (2, [b, d])]

```

Oppure:

```

val rdd = sc.parallelize(Seq((1, 1), (2, 2), (1, 3), (2, 4)))

val result = rdd.reduceByKey((x, y) => x + y) // result: [(1, 4), (2, 6)]

```

Quindi le trasformazioni wide:

- Richiedono uno shuffle.
- I dati sono riorganizzati tra i nodi del cluster.
- Meno efficienti in termini di tempo di esecuzione a causa del sovraccarico dello shuffle.

Narrow transformation

Quelle in cui ciascuna partizione dell'RDD di input è utilizzata da al massimo una partizione dell'RDD di output. Quindi le narrow transformation lavorano solo su una partizione RDD.

Questo significa che non è necessaria una riorganizzazione dei dati tra i nodi del cluster, e quindi non c'è bisogno di uno shuffle dei dati.

Le narrow transformations sono efficienti perché possono essere eseguite in modo pipelined (senza necessità di salvare dati intermedi su disco). Le narrow transformations sono molto efficienti in termini di esecuzione perché non richiedono il movimento dei dati tra i nodi. Tutte le operazioni avvengono localmente all'interno delle partizioni.

Ad esempio, la funzione `map` applica una funzione a ciascun elemento dell'RDD e restituisce un nuovo RDD con i risultati. Ogni partizione di output corrisponde a una singola partizione di input.

```
val rdd = sc.parallelize(Seq(1, 2, 3, 4))  
  
val result = rdd.map(x => x * 2) // result: [2, 4, 6, 8]
```

oppure:

```
val rdd = sc.parallelize(Seq(1, 2, 3, 4))  
  
val result = rdd.filter(x => x % 2 == 0) // result: [2, 4]
```

Quindi le trasformazioni narrow:

- Non richiedono shuffle.
- Sono eseguite in modo pipelined.
- Più efficienti in termini di tempo di esecuzione.

Action

Da istruzione a Spark di calcolare un risultato da una serie di trasformazioni.

Ci sono tre tipologie di azione:

- **View Data:** Azione di vedere dati dalla console.
- **Collect Data:** Azione di collezionare oggetti nativi nel loro rispettivo linguaggio (Python nel nostro caso).
- **Write to Output:** Azione di scrivere l'output su un data sink.

Esempi di action

L'azione *collect* è l'opposto della *parallelize*, prende un RDD e lo converte in una struttura dati locale nello Spark driver. L'azione *first* è l'esempio che un'azione non per forza calcola l'intero DAG, restituisce solo il primo elemento di un RDD, serve quindi solo la prima partizione dell'RDD e non l'intero RDD.

L'azione *reduce* non è l'equivalente di *reduce* in MapReduce, l'equivalente è una trasformazione *reduceByKey*.

- `collect` returns all the elements of the RDD as an **array** at the driver
- `first` returns the first **value** in the RDD
- `take` returns an **array** with the first *n* elements of the RDD
 - Variations on this function: `takeOrdered` and `takeSample`
- `count` returns the **number** of elements in the dataset
- `max` and `min` return the **maximum** and **minimum** values, respectively.
- `reduce` aggregates the elements of the dataset using a given **function**.
 - The given function should be **commutative** and **associative** so that it can be computed correctly in parallel.
- `saveAsTextFile` writes the elements of an RDD as a **text file**.
 - Local filesystem, HDFS or any other Hadoop-supported file system.

Figure 246 - Piccola lista di possibili action di Spark.

In figura l'attributo `withReplacement` è un booleano, se vale `true` indica che se estraiamo un elemento dall'RDD corrisponde ad estrarre l'elemento e rimetterlo nell'RDD; quindi, quell'elemento può essere nuovamente estratto dall'RDD.

Se vale `false`, un elemento estratto dall'RDD non può essere estratto nuovamente dall'RDD.

```
numbers = sc.parallelize([1, 2, 2, 2, 1, 1, 4, 3, 3, 5, 5])
numbers.collect()# triggers execution on ALL elements, takes time
# list [1, 2, 2, 2, 1, 1, 4, 3, 3, 5, 5]
numbers.first()
# int 1
numbers.take(4) # triggers execution on 4 elements, good for debug
# list [1, 2, 2, 2]
numbers.takeOrdered(4)
# list [1, 1, 1, 2]
withReplacement= True
numberToTake = 4
randomSeed = 123456
numbers.takeSample(withReplacement, numberToTake, randomSeed)
# list [1, 5, 2, 5]
```

Figure 247 - Esempio di RDD actions (1).

```

numbers = sc.parallelize([1, 2, 2, 2, 1, 1, 4, 3, 3, 5, 5])
numbers.count()
# int 11
numbers.countByValue()
# dict {1: 3, 2: 3, 4: 1, 3: 2, 5: 2}
numbers.max()
# int 5
numbers.min()
# int 1
numbers.reduce(lambda x, y: x + y)
# int 29
numbers.saveAsTextFile("numbers.txt")
# then, you may check contents of file 'numbers.txt'

```

Figure 248 - Esempio di RDD actions (2).

Le trasformazioni sono di due tipi:

- **Generic transformation:** lavorano su un RDD generico.
- **Key-value transformation:** lavorano su RDD composti da elementi key-value.

Esempi di trasformazioni generiche RDD

- **distinct** removes duplicates from the RDD
- **filter** returns the RDD records that match some **predicate function**

```

numbers = sc.parallelize([1,2,2,2,3,3,4,5,5,5,5])
distinct_numbers = numbers.distinct()
print(distinct_numbers.collect()) # this is an action
[2, 4, 1, 3, 5]
even_numbers = distinct_numbers.filter(lambda x: x % 2 == 0)
print(even_numbers.collect()) # this is an action
[2, 4]

```

Figure 249 - Esempio di trasformazione generica RDD (1).

La trasformazione *map* è l'equivalente di map di MapReduce in Hadoop, prende l'RDD, processa elemento per elemento dell'RDD e per ogni elemento dell'RDD produce solo un elemento in output.

Mentre *flatMap* processa elemento per elemento dell'RDD e può produrre un RDD di qualsiasi numero di elementi.

- `map` and `flatMap` apply a given function to each RDD element **independently**
- `map` transforms an RDD of **length n** into another RDD of **length n** .
- `flatMap` allows returning **0, 1 or more elements** from map function.

```
data = sc.parallelize(range(10))
squared_data = data.map(lambda x: x * x)
print(squared_data.collect()) # this is an action
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

squared_cubed_data_1 = data.map(lambda x: (x * x, x * x * x))
print(squared_cubed_data_1.collect()) # this is an action
[(0, 0), (1, 1), (4, 8), (9, 27), (16, 64), (25, 125), (36, 216), (49, 343),
(64, 512), (81, 729)]

squared_cubed_data_2 = data.flatMap(lambda x: (x * x, x * x * x))
print(squared_cubed_data_2.collect()) # this is an action
[0, 0, 1, 1, 4, 8, 9, 27, 16, 64, 25, 125, 36, 216, 49, 343, 64, 512, 81,
729]
```

Figure 250 - Esempio di trasformazione RDD (2)

- `sortBy` sorts an RDD
- `union` performs the **merging** of RDDs
- `intersection` performs the **set intersection** of RDD

```
words = sc.parallelize("nel mezzo del cammin di nostra vita".split(" "))
sorted_words = words.sortBy(lambda w: len(w))
print(sorted_words.collect()) # this is an action
['di', 'nel', 'del', 'vita', 'mezzo', 'cammin', 'nostra']

data1 = sc.parallelize(range(0,7))
data2 = sc.parallelize(range(3,10))
union = data1.union(data2)
print(union.collect()) # this is an action
[0, 1, 2, 3, 4, 5, 6, 3, 4, 5, 6, 7, 8, 9]

intersection = data1.intersection(data2)
print(intersection.collect()) # this is an action
[3, 4, 5, 6]
```

Figure 251 - Esempio di Trasformazione RDD (3)

Esempi di trasformazioni key-value RDD

- In a **(k, v)** pair, **k** is the **key**, and **v** is the **value**
- To create a key-value RDD:
 - **map** over your current RDD to a basic key-value structure.
 - Use the **keyBy** to create a key from the current value.
 - Use the **zip** to zip together two RDDs.

```
words = sc.parallelize("nel mezzo del cammin di nostra vita".split(" "))
keywords1 = words.map(lambda w: (w, 1)) print(keywords1.collect())
[('nel', 1), ('mezzo', 1), ('del', 1), ('cammin', 1), ('di', 1), ('nostra', 1), ('vita', 1)]

keywords2 = words.keyBy(lambda w: w[0].upper())
print(keywords2.collect()) # this is an action
[('N', 'nel'), ('M', 'mezzo'), ('D', 'del'), ('C', 'cammin'), ('D', 'di'), ('N', 'nostra'),
 ('V', 'vita')]

numbers = sc.parallelize(range(7))
keywords3 = words.zip(numbers)
print(keywords3.collect()) # this is an action
[('Nel', 0), ('mezzo', 1), ('del', 2), ('cammin', 3), ('di', 4), ('nostra', 5), ('vita', 6)]
```

Figure 252 - Esempio di Trasformazione RDD key-value (1)

- **keys** and **values** extract keys and values from the RDD, respectively
- **lookup** looks up the **list of values** for a particular key in an RDD

```
words = sc.parallelize("nel mezzo del cammin di nostra vita".split(" "))
keywords = words.keyBy(lambda w: w[0])
# [('n', 'nel'), ('m', 'mezzo'), ('d', 'del'), ('c', 'cammin'), ('d', 'di'), ('n',
'nostra'), ('v', 'vita')]
k = keywords.keys()
# ['n', 'm', 'd', 'c', 'd', 'n', 'v']
v = keywords.values()
# ['nel', 'mezzo', 'del', 'cammin', 'di', 'nostra', 'vita']
look = keywords.lookup("n")
print(look)
['nel', 'nostra']
```

Figure 253 - Esempio di Trasformazione RDD key-value (2)

- **reduceByKey** combines values with the same key
 - Takes a **function** as input and uses it to **combine values** of the same key
- **sortByKey** returns an RDD sorted by the key

```
words = sc.parallelize("fare o non fare non esiste provare".split(" "))
wordcount = words.map(lambda w: (w, 1)).reduceByKey(lambda x, y: x + y)
print(wordcount.collect()) # this is an action
[('provare', 1), ('fare', 2), ('non', 2), ('esiste', 1), ('o', 1)]

sorted_wordcount = wordcount.sortByKey()
print(sorted_wordcount.collect()) # this is an action
[('esiste', 1), ('fare', 2), ('non', 2), ('o', 1), ('provare', 1)]
```

Figure 253 - Esempio di Trasformazione RDD key-value (3)

- **join** performs an inner-join on the key
- Other types of join:
 - **fullOuterJoin**
 - **leftOuterJoin, rightOuterJoin**
 - **cartesian**

```
cars = sc.parallelize(["Ferrari", "Porsche", "Mercedes"])
colors = sc.parallelize(["red", "black", "pink"])
joined = cars.cartesian(colors)
print(joined.collect())
[('Ferrari', 'red'), ('Ferrari', 'black'), ('Ferrari', 'pink'), ('Porsche',
'black'), ('Porsche', 'pink'), ('Mercedes', 'red'),
('Mercedes', 'black'), ('Mercedes', 'pink')]

cars = sc.parallelize([(1, "Ferrari"), (1, "Porsche"), (2, "Mercedes")])
colors = sc.parallelize([(1, "red"), (2, "black"), (3, "pink")])
joined = cars.join(colors)
print(joined.collect())
[(1, ('Ferrari', 'red')), (1, ('Porsche', 'red')), (2, ('Mercedes', 'black'))]
```

Figure 254 - Esempio di Trasformazione RDD key-value (4)

Spark Job

Un'applicazione in Spark è un programma completo che viene eseguito su un cluster. Ogni applicazione è gestita da un driver program che contiene il codice principale e utilizza l'API di Spark per creare e gestire RDD.

Un'applicazione è composta da uno o più job.

Un Job in Spark è un'unità di lavoro che viene generata ogni volta che viene chiamata un'azione (come `collect`, `count`, `saveAsTextFile`) su un RDD. Per ogni azione abbiamo un job, un job è una sequenza di trasformazioni.

Ogni job è suddiviso in uno o più stage e rappresenta il lavoro necessario per eseguire una singola azione su un dataset. Spark decide quante stage compongono un job in base al tipo di trasformazioni che compongono quel job (narrow transformation o wide transformation).

Uno stage inizia quando c'è una wide transformation all'interno del job, le trasformazioni all'interno di ogni stage ci sono solo trasformazioni narrow.

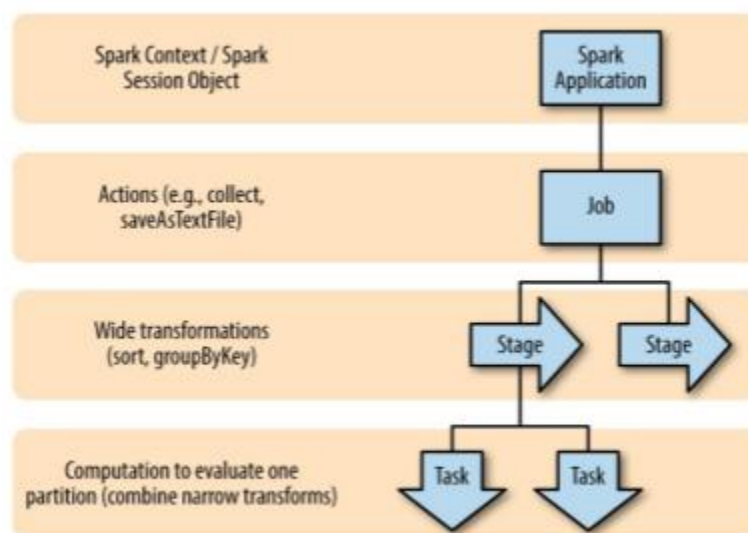


Figure 255 – Uno spark job, diviso in stage e in task.

Una volta invocata un'azione, Spark costruisce un Directed Acyclic Graph (DAG) delle trasformazioni che descrivono come ottenere i dati richiesti partendo dagli RDD di origine.

Spark stage

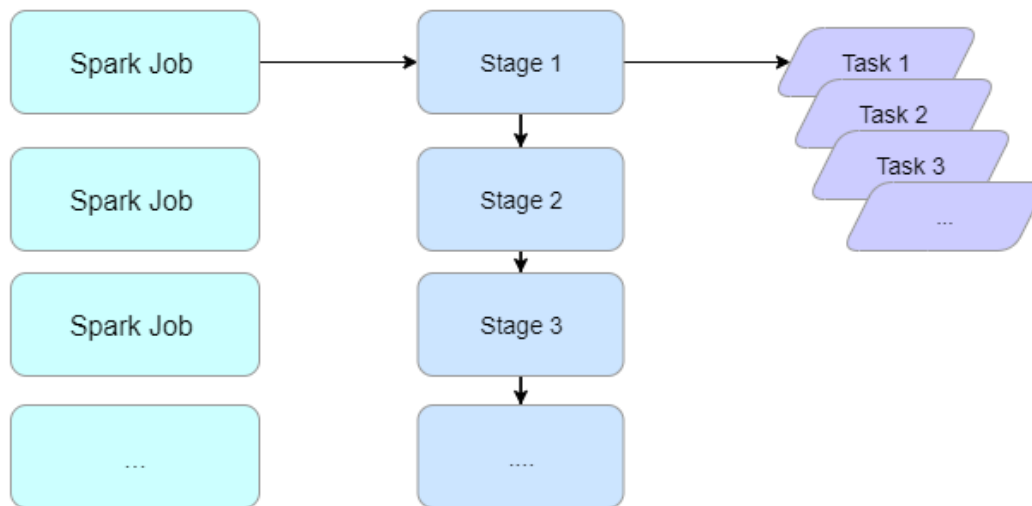


Figure 256 - Suddivisione di uno Spark job, da notare come i task sono parallelizzati e invece gli stage sono eseguiti in sequenza.

Ogni job è suddiviso in uno o più stage, che rappresentano una sequenza di operazioni che possono essere eseguite senza interruzioni dovute a operazioni di shuffling. Uno stage termina quando si verifica un'operazione di shuffling (ad esempio, una riduzione) che richiede la ridistribuzione dei dati tra le partizioni.

Le wide transformations richiedono uno shuffle, che spezza l'esecuzione in più stage. Uno stage viene eseguito prima dello shuffle, e un altro stage dopo lo shuffle.

Il primo stage crea task che eseguono le trasformazioni fino al punto in cui è necessario lo shuffle. Questi task producono dati intermedi che vengono distribuiti tra i nodi.

Il secondo stage crea task che leggono i dati riorganizzati dallo shuffle e completano le trasformazioni restanti.

Uno stage è un insieme di task che possono essere eseguiti senza necessità di shuffle dei dati. Le dipendenze tra gli RDD determinano dove gli stages vengono separati. Ogni stage è costituito da una serie di trasformazioni narrow (*quindi che da una partizione RDD generano solo una partizione RDD di output*) che possono essere eseguite in parallelo.

Il numero di task all'interno di uno stage è deciso in base al numero di partizioni all'interno dello stage, quindi se abbiamo N partizioni RDD, avremo N task all'interno dello stage.

Minimizzare il numero di wide transformations (e quindi di shuffle) può migliorare significativamente l'efficienza dell'esecuzione, poiché gli shuffle sono operazioni costose in termini di tempo e risorse.

Spark task

Un Task è l'unità di esecuzione più piccola in Spark. Ogni stage è suddiviso in uno o più task, che vengono eseguiti su partizioni specifiche dei dati. I task sono eseguiti in parallelo su diversi nodi del cluster. Un task esegue la stessa operazione (come una funzione `map` o `reduce`) su una singola partizione di dati.

Un task è composto da tutte le narrow transformations di quello stage e viene eseguita su una singola partizione di dati.

Esecuzione pipelined (non spiegato)

Poiché non è necessario uno shuffle, le narrow transformations possono essere eseguite in modo pipelined e localmente in ogni partizione. Questo significa che Spark può combinare più narrow transformations in un unico stage e creare un insieme di task che eseguono queste trasformazioni in una sequenza continua senza interruzioni.

I task sono distribuiti ai worker del cluster e ogni task esegue le trasformazioni su una singola partizione. Il numero di task è generalmente uguale al numero di partizioni dell'RDD.

Quando tutti i task di tutti gli stages sono completati, il job è considerato concluso e il risultato dell'azione viene restituito al driver o salvato nel sistema di storage designato.

```
val rdd = sc.textFile("data.txt")

val result = rdd.map(line => line.split(","))

                    .filter(fields => fields.length > 2)

                    .map(fields => (fields(0), fields(1).toInt))
```

In questo esempio, `map`, `filter` e il secondo `map` sono tutte narrow transformations. Spark esegue queste operazioni in un unico stage, creando task che applicano tutte e tre le trasformazioni a ciascuna partizione dell'RDD.

Esempio di uno Spark Job

```
rdd = sc.textFile("hdfs://path/to/file.txt")

lineCount = rdd.count()
```

L'azione `count()` viene invocata.

Spark crea un DAG per rappresentare le operazioni necessarie per leggere il file di testo e contare il numero di righe.

Il DAG può essere suddiviso in due stage:

1. **Stage 1:** Leggere i dati dal file di testo.

2. **Stage 2:** Contare le righe.

Esecuzione dei task:

- **Stage 1:** Spark legge il file di testo e crea task per ogni partizione del file.
- **Stage 2:** Spark esegue i task per contare le righe in ogni partizione e aggrega i risultati.

Persistenza degli RDD

Di default ogni RDD su cui è stata applicata una trasformazione può essere ricalcolato ogni volta che un'azione viene eseguita su di esso.

Quando viene eseguita la prima azione, le partizioni vengono rimosse per poter eseguire la seconda azione, però la seconda azione ricalcolerà le stesse partizioni iniziali della prima azione (quelle in comune) e questo non è efficiente.

Spark offre un controllo sulle partizioni che necessitano di persistere in memoria tra diverse azioni.

Spark supporta il caching delle partizioni RDD in memoria RAM per velocizzarne l'accesso.

Spark fornisce i seguenti metodi per questo scopo:

- **persist():** consente di specificare dove memorizzare l'RDD (MEMORY_ONLY, MEMORY_AND_DISK, DISK_ONLY).
- **cache():** equivalente di chiamare persist() con l'opzione MEMORY_ONLY.

Quando un RDD è memorizzato, ogni nodo memorizza qualche partizione di quel RDD.

Quando un'azione termina, successivamente può esserci un'altra azione, all'inizio del grafo le due azioni potrebbero essere uguali e ad un certo punto la prima azione seguirà un ramo del grafo, mentre la seconda ne seguirà un altro.

In caso di guasto, un RDD può essere riottenuto ricalcolando tutte le trasformazioni che sono state effettuate fino a quel momento (*essenzialmente un recovery log*).

Laboratorio 4 - Interfaccia REST

In generale non esistono linee guida per creare una interfaccia REST, però esistono alcune buone regole che è bene seguire.

Risorsa

Oggetto o rappresentazione di qualcosa. Ha dei dati associati e un set di metodi che si possono utilizzare.

Ad esempio, un impiegato di una compagnia avrà la corrispondente risorsa “*Impiegato*”.

Collezione

Un set di risorse. Ad esempio, “*Impiegati*” è la collezione di tutte le risorse “*Impiegato*”.

URL

Path univoco corrispondente alla risorsa.

Endpoint

Ogni interfaccia REST si basa sugli endpoint (nome dell'URL associato ad un metodo) il cui nome dovrebbe riflettere la risorsa associata ad esso e non ad un'azione. Questo perché le azioni devono essere definite tramite il metodo di richiesta e non col nome della risorsa.

Ad esempio, i seguenti nomi non sono adatti:

- `/getAllEmployees` → Lista dei dipendenti di una compagnia.
- `/addNewEmployee` → Aggiunge un nuovo dipendente.

Invece nomi più consoni sono:

- `GET /employees` → Lista dei dipendenti.
- `GET /employees/ID` → Restituisce un dipendente.
- `DELETE /employees` → Elimina tutti i dipendenti.
- `DELETE /employees/ID` → Rimuove un dipendente.
- `POST /employees` → Crea un nuovo dipendenti.

Regole generali

- Metodo GET deve essere utilizzato per richiedere dei dati dalla sorgente senza modificarla.
- Metodo POST deve essere utilizzato per creare una nuova risorsa.
- Metodo PUT deve essere utilizzato per richiedere l'aggiornamento della risorsa o la creazione di una nuova.
- Metodo DELETE deve essere utilizzato per richiedere la cancellazione di una risorsa.

Una collezione può essere rappresentata con il suo nome, mentre una risorsa della collezione con *nome collezione/nome risorsa* oppure *nome collezione/ID univoco*.

Design della API

Ci sono vari tool per progettare una interfaccia REST. Questi tool permettono di specificare la definizione della API, la documentazione e di creare in modo automatico uno scheletro per l'applicazione partendo dalla definizione delle API, che comprende funzioni/handler vuoti che possono essere implementati.

Uno di questi strumenti è Swagger.

Swagger

Swagger è un framework ampiamente utilizzato nel contesto del cloud computing per la progettazione, la costruzione, la documentazione e la manutenzione di API (*Application Programming Interface*) RESTful.

Originariamente sviluppato da SmartBear Software, Swagger è ora parte del progetto open-source OpenAPI, gestito dalla Linux Foundation.

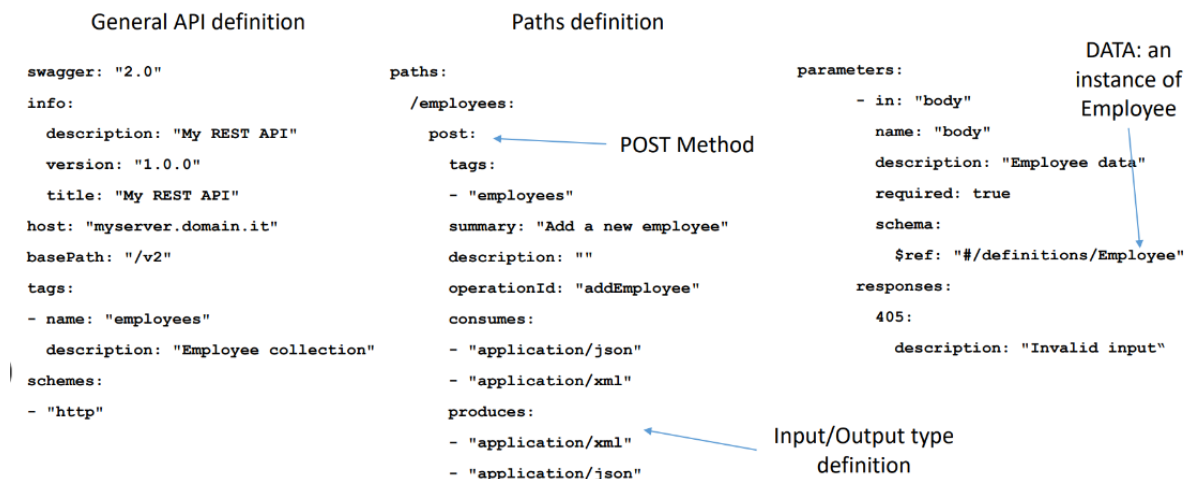


Figure 134 - Esempio di definizione con Swagger.

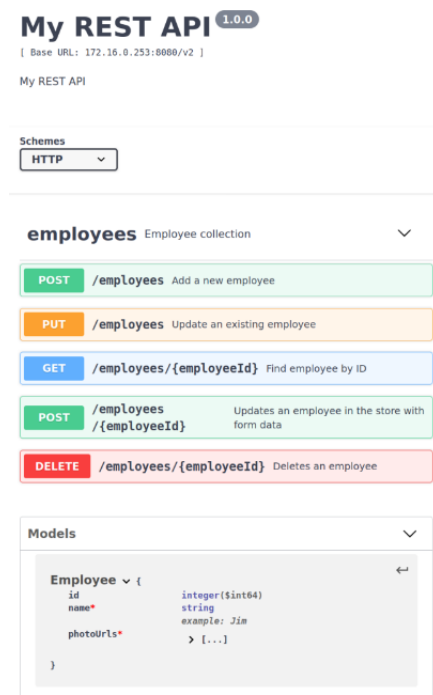


Figure 135 - Ci sono anche delle GUI per utilizzare Swagger in modo più comodo.

Data la descrizione della API, Swagger auto genera lo scheletro del codice con un set di server REST per Tomcat pronti a eseguire lo scheletro fornito.

Funzionalità e Componenti di Swagger

Swagger fornisce una serie di strumenti e specifiche che aiutano gli sviluppatori a creare e gestire API. Le principali funzionalità e componenti includono:

OpenAPI Specification (OAS)

Questa è una specifica standard che descrive le API RESTful. Un file di specifica OpenAPI è solitamente scritto in JSON o YAML e contiene tutte le informazioni necessarie per comprendere e interagire con l'API, inclusi gli endpoint, i metodi HTTP, i parametri, i tipi di dati e le risposte.

Swagger Editor

Un editor online che consente di scrivere e modificare file di specifica OpenAPI. Fornisce un'interfaccia user-friendly per creare documentazioni di API, con funzionalità di anteprima e validazione in tempo reale.

Swagger UI

Un'interfaccia grafica che visualizza la documentazione dell'API in modo interattivo. Consente agli utenti di esplorare e testare le API direttamente dal browser, senza la necessità di altri

strumenti. Questo è particolarmente utile per sviluppatori e tester che devono capire e utilizzare l'API.

Swagger Codegen

Uno strumento che genera automaticamente codice client e server in vari linguaggi di programmazione a partire da una specifica OpenAPI. Questo può accelerare lo sviluppo, riducendo la quantità di codice boilerplate che gli sviluppatori devono scrivere manualmente.

SwaggerHub

Una piattaforma collaborativa per la progettazione e la documentazione delle API. SwaggerHub combina Swagger Editor e Swagger UI in un unico ambiente collaborativo, permettendo ai team di lavorare insieme sulla creazione e gestione delle API.

Utilizzo di Swagger nel Cloud Computing

Nel contesto del cloud computing, Swagger è particolarmente utile per diverse ragioni:

Standardizzazione

Swagger fornisce una specifica standardizzata per descrivere le API, facilitando l'interoperabilità tra diversi servizi e piattaforme nel cloud.

Documentazione e Test

Con Swagger UI, è possibile generare documentazione interattiva per le API, rendendo più facile per gli sviluppatori capire e utilizzare le API offerte da vari servizi cloud. Inoltre, la possibilità di testare le API direttamente dalla documentazione è un grande vantaggio.

Automazione

Con Swagger Codegen, è possibile automatizzare la generazione di SDK (Software Development Kit) e client per API, semplificando l'integrazione delle API cloud nelle applicazioni.

Collaborazione

Strumenti come SwaggerHub supportano la collaborazione tra team distribuiti, che è una caratteristica comune nei progetti di cloud computing. I team possono lavorare insieme per progettare, documentare e mantenere le API in modo efficace.

Compatibilità con DevOps e CI/CD

Swagger si integra bene con i processi DevOps e CI/CD (*Continuous Integration/Continuous Deployment*), permettendo l'automazione dei test e delle distribuzioni delle API, migliorando l'efficienza e la qualità del software sviluppato.

Esempio di Specifica OpenAPI

Un semplice esempio di una specifica OpenAPI in YAML che descrive un'API per la gestione degli utenti potrebbe essere il seguente file yaml.

```
openapi: 3.0.0
info:
  title: User API
  version: 1.0.0
paths:
  /users:
    get:
      summary: Retrieve a list of users
      responses:
        '200':
          description: A list of users
          content:
            application/json:
              schema:
                type: array
                items:
                  $ref: '#/components/schemas/User'
  /users/{id}:
    get:
      summary: Retrieve a single user
      parameters:
        - name: id
          in: path
          required: true
          schema:
            type: string
      responses:
        '200':
          description: A single user
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/User'
components:
  schemas:
    User:
      type: object
      properties:
        id:
          type: string
        name:
```



```

    type: string
  email:
    type: string

```

In questo esempio, la specifica descrive due endpoint (`/users` e `/users/{id}`) e il formato della risposta JSON per un utente.

Apache Tomcat

Implementazione open-source di Java Servlet, ovvero componente software che estende le funzionalità di un server web permettendogli di eseguire codice Java a seguito di una richiesta http (*notare che le REST sono fatte in codice Java*).

Schierare una interfaccia REST

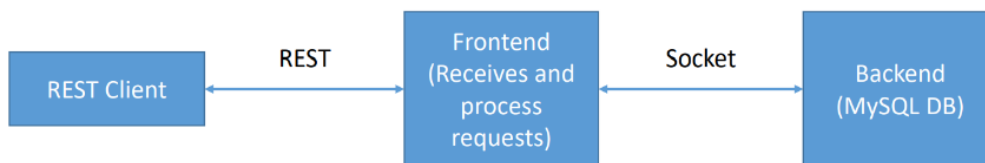


Figure 259 - Il nostro obiettivo.

Esempio sviluppo REST API:

- Lo Swagger Editor è basato su un linguaggio di markup detto **yaml**.
- Incolliamo il file `employees.yaml` nello swagger editor, il quale contiene la struttura per gli impiegati di un'azienda.
- Nel campo `host` del codice incollato cambiamo l'indirizzo con l'indirizzo della propria VM.
- Si genera il server premendo in alto nell'editor su *generate server* e si seleziona *python-flask*.
- Carichiamo il file scaricato nella VM tramite *WinSCP*.
- Accediamo alla VM e unzippiamo il file.
- Apriamo la cartella `python-flask-server` e creiamo ed eseguiamo il container `rest-server` (vedi sotto).
 - apriamo il file `requirements.txt` tramite *nano requirements.txt* e cambiamo la prima riga in `connexion>=2, <3`.
 - per provare nel browser utilizziamo un comando che contiene l'indirizzo della VM.
 - per l'ultimo comando (`curl`) apriamo un altro terminale della VM e lo eseguiamo inserendo l'indirizzo della propria VM nel comando.

1 - Generare il container

```
docker build -t rest-server
```

Generiamo un container in cui mettere il server web con Tomcat.

2 – Eseguire il container

```
docker run -p 8080:8080 -it rest-server
```

3 – Testare il container

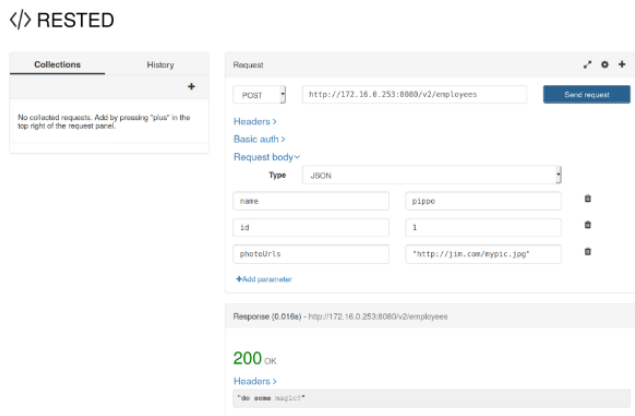
Aprire il browser e andare all'indirizzo: `http://10.1.Y.X:8080/v2/employees/`

Poi eseguire da linea di comando:

```
curl -X POST "http://10.1.Y.X:8080/v2/employees" -H
"accept: application/json" -H "Content-Type:
application/json" -d "{ \"id\": 0, \"name\": \"Jim\",
\"photoUrls\": \"http://mywebsite.org/mypic.jpg\"}"
```

- To test REST interfaces install on your browser a REST extension (e.g. RESTED in Firefox or Chrome)
- Test a POST to add a new employee
- The payload must contain the data associated with the employee, e.g.:


```
{
  "id": 0,
  "name": "Jim",
  "photoUrls":
    "http://jim.com/mypic.jpg",
}
```
- The response is “do some magic!” as the implementation is missing



4 – Implementare la funzione associata alla richiesta REST

Nel file `swagger_server/controllers/employees_controller.py`.

```
def add_employee(body):
    if connexion.request.is_json:
```

```
body = Employee.from_dict(connexion.request.get_json())

return 'do some magic!'
```

5 – Creare il database di backend

- Aggiungiamo il database MySQL con i seguenti comandi (creiamo un altro container per MySQL):
 - con il terzo comando è possibile leggere la password ad esempio (X2^PU/01kd_6S#tNr&7sO782s_:DsU@O).

- Instantiate and additional container with MySQL

- Fetch a docker image with MySQL preinstalled

```
docker pull mysql/mysql-server
```

- Instantiate the container

```
docker run --name=mysql -d mysql/mysql-server
```

- Recover root DB password

```
docker logs mysql 2>&1 | grep GENERATED
```

```
root@HAJJVX80PD7M5Q0:~/python-flask-server# docker logs mysql 2>&1 | grep GENERATED
[Entrypoint] GENERATED ROOT PASSWORD: 4DuDUPuSi$AHuk0bUcopfuLujUS
```

- Cambiamo la password con i seguenti comandi:
 - dopo aver eseguito i primi due occorre incollare la password precedentemente letta.
 - nel terzo comando cambiamo 'password' con una personalizzata.

- Change the password

```
docker exec -it mysql bash
```

```
mysql -uroot -p
```

- Type the new password

```
ALTER USER 'root'@'localhost' IDENTIFIED BY 'password';
```

- Allow root connections from other hosts

```
use mysql;
```

```
update user set host='%' where host='localhost' AND
user='root';
```

```
FLUSH PRIVILEGES;
```

- Si crea un nuovo database:

- Create a new database

```
CREATE DATABASE company;
```

- Create a new table employees

```
CREATE TABLE IF NOT EXISTS `company`.`employees` (
  `id` INT AUTO_INCREMENT ,
  `name` VARCHAR(150) NOT NULL ,
  `picUrl` VARCHAR(150) NOT NULL ,
  PRIMARY KEY (`id`) )
ENGINE = InnoDB;
```

- Usciamo da mySQL e accediamo a python-flask-server con *cd python-flask-server*.
- Apriamo il Dockerfile e inseriamo le seguenti righe subito prima di *RUN pip3 install*.
- Apriamo il file requirements.txt (file che definisce l'ambiente Python in cui le librerie devono essere installate) ed aggiungiamo in fondo al file la riga di codice in fondo in figura sotto.

- Install at container bootstrap the libraries to implement MySQL client functionalities (add before RUN pip3 install ...)

```
RUN apk add --update --no-cache mariadb-connector-c-dev \
    && apk add --no-cache --virtual .build-deps \
        mariadb-dev \
        gcc \
        musl-dev \
    && pip install mysqlclient==1.4.6 \
    && apk del .build-deps
```

- Add among requirements the python MySQL libraries, add the following line in the file requirements.txt:

```
mysqlclient
```

- Eseguiamo nuovamente il comando build, *docker build -t rest-server*.
- Eseguiamo il comando *docker network inspect bridge* per trovare l'ip del container mySQL (ad esempio 172.17.0.2/16).

- Retrieve the IP of the DB container and connect to the DB on the python program

```
import MySQLdb
```

```
...
```

```
mydb = MySQLdb.connect(host="10.1.1.X",  
user="root",passwd="password", db="company")
```

- Store the data in the database

```
mycursor = mydb.cursor()
```

```
sql = "INSERT INTO employees (id, name, picUrl) VALUES (%s, %s,  
%s)"
```

```
val = (body.id, body.name, body.photo_urls)
```

```
mycursor.execute(sql, val)
```

```
mydb.commit()
```